

Db2 12 for z/OS

Introduction to Db2 for z/OS
Last updated: 2026-01-27



Notes

Before using this information and the product it supports, be sure to read the general information under "Notices" at the end of this information.

Subsequent editions of this PDF will not be delivered in IBM Publications Center. Always download the latest edition from [IBM Documentation](#).

2026-01-27 edition

This edition applies to Db2[®] 12 for z/OS[®] (product number 5650-DB2), Db2 12 for z/OS Value Unit Edition (product number 5770-AF3), and to any subsequent releases until otherwise indicated in new editions. Make sure you are using the correct edition for the level of the product.

Specific changes are indicated by a vertical bar to the left of a change. A vertical bar to the left of a figure caption indicates that the figure has changed. Editorial changes that have no technical significance are not noted.

© **Copyright International Business Machines Corporation 2001, 2026.**

US Government Users Restricted Rights – Use, duplication or disclosure restricted by GSA ADP Schedule Contract with IBM Corp.

Contents

About this information.....	ix
Who should read this information.....	x
Db2 Utilities Suite for z/OS.....	x
Terminology and citations.....	x
Accessibility features for Db2 for z/OS.....	xi
How to send comments.....	xii
 Chapter 1. What is Db2 for z/OS?.....	 1
 Chapter 2. Db2 for z/OS and SQL concepts.....	 3
SQL: The language of Db2.....	3
Submitting SQL statements to Db2.....	4
Static SQL statements.....	6
Embedded dynamic SQL statements.....	6
Deferred embedded SQL.....	6
Interactive SQL statements.....	7
SQL Call Level Interface and Open Database Connectivity.....	7
Java database connectivity and embedded SQL for Java.....	8
Submitting SQL statements with QMF for Workstation.....	8
Db2 database objects overview.....	9
Db2 schemas and schema qualifiers.....	10
Db2 tables.....	11
Db2 keys.....	14
Constraints.....	16
Db2 table columns.....	20
Db2 indexes.....	33
Db2 views.....	34
Aliases.....	36
Triggers.....	37
User-defined types.....	39
Routines in Db2 for z/OS: functions and procedures	40
Sequences.....	41
Db2 system objects.....	42
Db2 catalog.....	42
Db2 directory.....	43
Active and archive logs.....	44
Bootstrap data set.....	45
Buffer pools.....	45
Data definition control support database.....	46
The resource limit facility.....	46
Work file database.....	46
Subsystem parameters.....	47
Application processes, transactions, concurrency, and recovery.....	47
Locking, commit, and rollback.....	48
Unit of work.....	49
Unit of recovery.....	50
Rolling back work.....	50
Packages and application plans.....	51
Storage structures.....	52
Db2 databases.....	53

Db2 table spaces.....	55
Db2 index spaces.....	56
Db2 hash spaces (deprecated).....	56
Db2 storage groups.....	57
Accessing Db2 data by using SQL statements.....	58
Ways to select data from columns.....	59
How a SELECT statement works.....	61
SQL functions and expressions.....	62
Ways to filter the number of returned rows.....	69
Ways to order rows.....	76
Ways to summarize group values.....	79
Ways to merge lists of values.....	80
Ways to specify search conditions.....	81
Ways to join data from more than one table.....	82
Subqueries.....	84
Ways to access Db2 data that is not in a table.....	86
Modifying Db2 data by using SQL statements.....	86
Inserting Db2 data by using INSERT statements.....	87
Updating Db2 data by using UPDATE statements.....	91
Modifying Db2 data by using MERGE statements.....	94
Deleting data by using DELETE statements.....	97
Deleting data by using TRUNCATE statements.....	98
Defining Db2 objects.....	99
Creation of tables.....	99
Creation of table spaces.....	107
Creation of indexes.....	111
Creation of views.....	128
Creating large objects.....	131
Creation of databases.....	132
Creation of relationships with referential constraints.....	133
Creation of triggers.....	137
Creation of user-defined functions.....	138
Character conversion.....	139
Character sets and code pages.....	142
Coded character sets and CCSIDS.....	143
Encoding scheme and CCSID rules for strings.....	144
Expanding conversions.....	149
Contracting conversions.....	150
Distributed relational databases.....	150
Connections.....	150
Distributed unit of work.....	151
Remote unit of work.....	154
Distributed data.....	156
Remote servers.....	156
Connectivity in distributed environments.....	157
Distributed data access.....	157
Support for high availability.....	166
Db2 data sharing.....	167
Advantages of Db2 data sharing.....	167
How Db2 protects data consistency in a data sharing environment.....	173
How updates are made in a data sharing environment.....	174
How Db2 writes changed data to disk in a data sharing environment.....	177
Ways that other tasks are affected by data sharing.....	178
Ways that availability is affected by data sharing.....	178
Continuous delivery in Db2 12.....	179
pureXML.....	180

Chapter 3. Management of Db2 operations.....	183
Common ways to interact with Db2 for z/OS.....	183
The DB2I primary option menu.....	185
Commands for controlling Db2 and related facilities.....	186
Db2 utilities.....	187
Management of data sets.....	193
Authorization and security mechanisms for data access.....	193
How authorization IDs control data access.....	194
How authorization IDs hold privileges and authorities.....	194
Ways to control access to Db2 subsystems.....	196
Ways to control access to data.....	197
Ways to control access to Db2 objects through explicit privileges and authorities.....	198
Row-level and column-level access control.....	200
Use of multilevel security to control access.....	201
Use of views to control access.....	201
Use of grant and revoke privileges to control access.....	202
Backup, recovery, and restart.....	203
Backup and recovery resources and tools.....	205
Db2 restart.....	207
Regular backups and data checks.....	207
Control of database changes and data consistency.....	207
Events in the recovery process.....	209
Optimization of availability during backup and recovery.....	210
Data encoding basics.....	211
Introduction to troubleshooting problems in Db2 for z/OS.....	213
Chapter 4. Db2 performance management.....	219
Initial steps for performance management.....	219
Performance objectives.....	219
Application design for performance.....	220
Origin of performance problems.....	220
Tools for performance analysis.....	221
Ways to move data efficiently through the system.....	222
The role of buffer pools in caching data.....	222
The effect of data compression on performance.....	224
How data organization can affect performance.....	225
Ways to improve performance for multiple users.....	227
Improved performance through the use of locks.....	228
Improved performance through concurrency control.....	231
Ways to improve query performance.....	233
Tools for improving query performance.....	233
Query and application performance analysis.....	235
Using EXPLAIN to understand the access path.....	238
Hash access paths (deprecated).....	240
Chapter 5. Programming for Db2 for z/OS.....	243
Tools and IDEs for developing Db2 applications.....	247
Programming languages and methods for developing application programs that access Db2 for z/OS.....	249
Performance information for SQL application programming.....	250
Preparation process for an application program.....	252
Application programming default values.....	254
Static SQL applications.....	255
Declaration of table and view definitions.....	256
Data access with host variables.....	257
Data access with host-variable arrays.....	258

Data access with host structures.....	258
Row retrieval with a cursor.....	259
Ways to check the execution of SQL statements.....	261
Dynamic SQL applications.....	262
Use of ODBC to execute dynamic SQL.....	263
Use of Java to execute static and dynamic SQL.....	264
SQLJ support.....	265
JDBC support.....	266
Use of an application program as a stored procedure.....	267
Languages used to create stored procedures.....	267
Stored procedure processing.....	268
Use of the SQL procedural language to create a stored procedure.....	270
Use of development tools to create a stored procedure.....	270
Setup of the stored procedure environment.....	271
Preparation of a stored procedure.....	271
How applications can call stored procedures.....	271
Web application environment.....	272
Components of web-based applications.....	272
Architectural characteristics of web-based applications.....	273
Benefits of Db2 for z/OS as a server.....	275
Web-based applications and WebSphere Studio Application Developer.....	275
Processing XML data with Db2 pureXML.....	277
Chapter 6. Db2 database design.....	281
Db2 objects and their relationships.....	281
Logical database design using entity-relationship modeling.....	282
Logical database design with Unified Modeling Language.....	292
Physical database design.....	293
Row design for effective space use.....	297
XML and Db2.....	298
Chapter 7. Db2 for z/OS architecture.....	301
IBM zSystems architecture and the z/OS operating system.....	301
Db2 address spaces in the z/OS environment.....	303
Db2 internal resource lock manager.....	304
Db2 and the z/OS Security Server.....	305
Db2 and DFSMS.....	305
Db2 attachment facilities.....	305
CICS attachment facility.....	306
IMS attachment facility.....	307
TSO attachment facility.....	308
Call attachment facility.....	309
Resource Recovery Services attachment facility.....	309
Distributed data facility.....	309
Db2 in a Parallel Sysplex environment.....	310
Chapter 8. An overview of Db2 for z/OS and related products.....	313
Scenarios for using Db2.....	313
Availability and scalability for large businesses.....	313
Critical business information for decision makers.....	316
Data distribution and Web access.....	316
Db2 data servers and environments.....	317
Enterprise servers.....	318
Db2 distributed editions.....	318
Db2 on smaller-scale servers.....	318
Personal, mobile, and pervasive environments.....	318
Multiple transaction and application environments.....	319

Db2 and network communication.....	319
Db2 Connect.....	319
Clients supported by Db2 data servers.....	320
Sources of data.....	321
Tools for managing Db2 for z/OS.....	321
IBM Db2 Administration Foundation for z/OS.....	322
IBM Data Server Manager.....	324
IBM Data Studio.....	325
Db2 Administration Tool for z/OS.....	326
The Db2 command line processor.....	327
Tools and IDEs for developing Db2 applications.....	327
Middleware components.....	329
Chapter 9. Db2 sample tables.....	331
Activity table (DSN8C10.ACT).....	331
Department table (DSN8C10.DEPT).....	332
Employee table (DSN8C10.EMP).....	334
Employee photo and resume table (DSN8C10.EMP_PHOTO_RESUME).....	338
Project table (DSN8C10.PROJ).....	339
Project activity table (DSN8C10.PROJACT).....	340
Employee-to-project activity table (DSN8C10.EMPPROJACT).....	341
Unicode sample table (DSN8C10.DEMO_UNICODE).....	342
Relationships among the sample tables.....	343
Views on the sample tables.....	344
Storage of sample application tables.....	348
Storage group for sample application data.....	349
Databases for sample application data.....	349
Table spaces for sample application data.....	350
SYSDUMMYx tables.....	351
Chapter 10. Db2 basics tutorial: Working with Db2 for z/OS interactively.....	353
Module 1: Querying and modifying data.....	354
Lesson 1.1: Querying data interactively.....	355
Lesson 1.2: Creating a table.....	360
Lesson 1.3: Modifying data interactively.....	362
Module 2: Running Db2 utilities.....	366
Lesson 2.1: Running the UNLOAD utility from DB2I.....	366
Lesson 2.2: Running the LOAD utility from DB2I.....	373
Lesson 2.3: Running the COPY utility from a JCL job.....	376
Information resources for Db2 for z/OS and related products.....	383
Notices.....	385
Programming interface information.....	386
Trademarks.....	386
Terms and conditions for product documentation.....	387
Privacy policy considerations.....	387
Glossary.....	389
Index.....	391

About this information

This information provides a comprehensive introduction to IBM® Db2 for z/OS. It explains the basic concepts that are associated with relational database management systems in general, and with Db2 for z/OS in particular.

After reading this information, you will understand basic concepts about Db2.

Throughout this content, "Db2" means "Db2 12 for z/OS." References to other Db2 products use complete names or more-specific abbreviations.

Important: To find the most up to date content for Db2 12 for z/OS, always use [IBM Documentation](#) or download the latest PDF file from [PDF format manuals for Db2 12 for z/OS \(Db2 for z/OS in IBM Documentation\)](#).

This Db2 12 for z/OS product documentation generally assumes that the highest available function level is activated and that your applications are running with the highest available application compatibility level, with the following exceptions:

- The following documentation sections describe the Db2 12 migration process and how to activate new capabilities in function levels:
 - [Migrating to Db2 12 \(Db2 Installation and Migration\)](#)
 - [What's new in Db2 12 \(Db2 for z/OS What's New?\)](#)
 - [Adopting new capabilities in Db2 12 continuous delivery \(Db2 for z/OS What's New?\)](#)
- [FL 501](#) Labels like this one usually mark new content or changes for Db2 function levels. The label is also a link to the description of the function level that introduces the changes. For more information, see [How Db2 function levels are documented \(Db2 for z/OS What's New?\)](#).

Continuous delivery in Db2 12

The availability of most new-function application capabilities in Db2 12 depends on the type of enhancement, the activated function level, and the application compatibility levels of each application. For a list of all available function levels that are currently available in Db2 12, see [Db2 12 function levels \(Db2 for z/OS What's New?\)](#).

Many new-function enhancements take effect at any function level when you apply a PTF in each Db2 subsystem or data sharing member. For more information, see [New-function APARs for Db2 12 \(Db2 for z/OS What's New?\)](#).

Virtual storage enhancements

Virtual storage enhancements become available at the activation of the function level that introduces them or higher. Activation of function level 100 introduces all virtual storage enhancements in the initial Db2 12 release. That is, activation of function level 500 introduces no virtual storage enhancements.

Subsystem parameters

New subsystem parameter settings are in effect only when the function level that introduced them or a higher function level is activated. Many subsystem parameter changes in the initial Db2 12 release take effect in function level 500. For more information about subsystem parameter changes in Db2 12, see [Subsystem parameter changes in Db2 12 \(Db2 for z/OS What's New?\)](#).

Optimization enhancements

Optimization enhancements become available after the activation of the function level that introduces them or higher, and full prepare of the SQL statements. When a full prepare occurs depends on the statement type:

- For static SQL statements, after bind or rebind of the package
- For non-stabilized dynamic SQL statements, immediately, unless the statement is in the dynamic statement cache

- For stabilized dynamic SQL statements, after invalidation, free, or changed application compatibility level

Activation of function level 100 introduces all optimization enhancements in the initial Db2 12 release. That is, function level 500 introduces no optimization enhancements.

SQL capabilities

New SQL capabilities become available after the activation of the function level that introduces them or higher, for applications that run at the equivalent application compatibility level or higher. New SQL capabilities in the initial Db2 12 release become available in function level 500 for applications that run at the equivalent application compatibility level or higher. You can continue to run SQL statements compatibly with lower function levels, or previous Db2 releases, including Db2 11 and DB2® 10. For details, see [Application compatibility \(APPLCOMPAT\) levels in Db2 12 \(Db2 Application programming and SQL\)](#)

Who should read this information

If you are new to Db2 for z/OS, this information is for you.

Perhaps you have worked with other Db2 family products, such as Db2 for Linux®, UNIX, and Windows, Db2 for i, or Db2 server for VSE and VM. Perhaps you have worked on non-IBM database management systems (DBMSs) or on the IBM hierarchic DBMS, which is called Information Management System (IMS). Perhaps you have never worked with DBMSs, but you want to work with this product, which many companies use for mission-critical data and application programs. Regardless of your background, if you want to learn about Db2 for z/OS, this information can help you.

If you will be working with Db2 for z/OS and already know what specific job you will have, begin by reading the first three chapters. Then, you can consider what your role will be when you choose to read all or only a subset of the remaining chapters. For example, assume that you know you will be a database administrator (DBA) for an organization that has some distributed applications and is beginning to plan for on demand business. In this case you would probably want to read the chapters about designing objects and data, implementing your database design, Db2 and the Web, and accessing distributed data.

This information is written with the assumption that most readers are data processing professionals.

Db2 Utilities Suite for z/OS

Important: Db2 Utilities Suite for z/OS is available as an optional product. You must separately order and purchase a license to such utilities, and discussion of those utility functions in this publication is not intended to otherwise imply that you have a license to them.

Db2 12 utilities can use the DFSORT program regardless of whether you purchased a license for DFSORT on your system. For more information about DFSORT, see <https://www.ibm.com/support/pages/dfsor>.

Db2 utilities can use IBM Db2 Sort for z/OS as an alternative to DFSORT for utility SORT and MERGE functions. Use of Db2 Sort for z/OS requires the purchase of a Db2 Sort for z/OS license. For more information about Db2 Sort for z/OS, see [Db2 Sort for z/OS documentation](#).

Related concepts

[Db2 utilities packaging \(Db2 Utilities\)](#)

Terminology and citations

When referring to a Db2 product other than Db2 for z/OS, this information uses the product's full name to avoid ambiguity.

The following terms are used as indicated:

Db2

Represents either the Db2 licensed program or a particular Db2 subsystem.

IBM rebranded DB2 to Db2, and Db2 for z/OS is the new name of the offering that was previously known as "DB2 for z/OS". As a result, you might sometimes still see references to the original names,

such as "DB2 for z/OS" and "DB2", in different IBM web pages and documents. If the PID, Entitlement Entity, version, modification, and release information match, assume that they refer to the same product.

IBM OMEGAMON® for Db2 Performance Expert on z/OS

Refers to any of the following products:

- IBM IBM OMEGAMON for Db2 Performance Expert on z/OS
- IBM Db2 Performance Monitor on z/OS
- IBM Db2 Performance Expert for Multiplatforms and Workgroups
- IBM Db2 Buffer Pool Analyzer for z/OS

C, C++, and C language

Represent the C or C++ programming language.

CICS®

Represents CICS Transaction Server for z/OS.

IMS

Represents the IMS Database Manager or IMS Transaction Manager.

MVS

Represents the MVS element of the z/OS operating system, which is equivalent to the Base Control Program (BCP) component of the z/OS operating system.

RACF®

Represents the functions that are provided by the RACF component of the z/OS Security Server.

Accessibility features for Db2 for z/OS

Accessibility features help a user who has a physical disability, such as restricted mobility or limited vision, to use information technology products successfully.

Accessibility features

The following list includes the major accessibility features in z/OS products, including Db2 for z/OS. These features support:

- Keyboard-only operation.
- Interfaces that are commonly used by screen readers and screen magnifiers.
- Customization of display attributes such as color, contrast, and font size

Tip: IBM Documentation (which includes information for Db2 for z/OS) and its related publications are accessibility-enabled for the IBM Home Page Reader. You can operate all features using the keyboard instead of the mouse.

Keyboard navigation

For information about navigating the Db2 for z/OS ISPF panels using TSO/E or ISPF, refer to the *z/OS TSO/E Primer*, the *z/OS TSO/E User's Guide*, and the *z/OS ISPF User's Guide*. These guides describe how to navigate each interface, including the use of keyboard shortcuts or function keys (PF keys). Each guide includes the default settings for the PF keys and explains how to modify their functions.

Related accessibility information

IBM and accessibility

For more information about the commitment that IBM has to accessibility, see the *IBM Accessibility Center* at <http://www.ibm.com/able>.

How to send your comments about Db2 for z/OS documentation

Your feedback helps IBM to provide quality documentation.

Send any comments about Db2 for z/OS and related product documentation by email to db2zinfo@us.ibm.com.

To help us respond to your comment, include the following information in your email:

- The product name and version
- The address (URL) of the page, for comments about online documentation
- The book name and publication date, for comments about PDF manuals
- The topic or section title
- The specific text that you are commenting about and your comment

Related concepts

[About Db2 12 for z/OS product documentation \(Db2 for z/OS in IBM Documentation\)](#)

Related reference

[PDF format manuals for Db2 12 for z/OS \(Db2 for z/OS in IBM Documentation\)](#)

Chapter 1. What is Db2 for z/OS?

Db2 for z/OS is a relational database management system that runs on the IBM zSystems™ platform.

A *relational database* is a database in which all of the data is logically contained in tables. These databases are organized according to the relational model, where data is perceived to exist in one or more tables. Each table contains a specific number of columns and a number of unordered rows. Each column in a table is related in some way to the other columns. In a relational database, *referential integrity* ensures data integrity by enforcing rules with referential constraints, check constraints, and triggers. You can rely on constraints and triggers to ensure the integrity and validity of your data, rather than relying on individual applications to do that work.

Db2 is software that you can use to manage relational databases. IBM offers a family of Db2 products that run on a range of operating systems, including Linux, UNIX, Windows, IBM i, VSE, VM, and z/OS.

z/OS is the main operating system for IBM's most robust hardware platform, IBM zSystems. Db2 for z/OS is the enterprise data server for IBM zSystems. It manages core business data across an enterprise and supports key business applications. Db2 for z/OS supports thousands of customers and millions of users. It remains continuously available, scalable, and highly secure.

With Db2 for z/OS and other Db2 family products, you can define and manipulate your data by using structured query language (SQL). SQL is the standard language for accessing data in relational databases.

Db2 product names

Throughout this content, "Db2" means "Db2 12 for z/OS." References to other Db2 products use complete names or more-specific abbreviations.

This information sometimes refers to the Db2 12 for z/OS product as "Db2 12" or "version 12". Unless a different product, platform, or release is specified, you can assume that they refer to the same product.

Blogs for Db2 for z/OS: See the [IBM Community blog for Db2 for z/OS and its ecosystem](#) for the latest news about new capabilities and enhancements in Db2 for z/OS continuous delivery, from the IBM experts who design, build, test, and support Db2

[Subscribe today!](#)

Related concepts

[Scenarios for using Db2](#)

Scenarios can illustrate how some organizations might successfully use Db2.

[SQL: The language of Db2 \(Db2 SQL\)](#)

Related reference

[Introduction to the New Mainframe: z/OS Basics \(IBM Redbooks\)](#)

Related information

[IBM Db2 12 for z/OS Technical Overview \(IBM Redbooks\)](#)

[IBM Db2 for z/OS database software](#)

Chapter 2. Db2 for z/OS and SQL concepts

Many structures and processes are associated with a relational database. The structures are the key components of a Db2 database system, and the processes are the interactions that occur when applications access the database system.

In a relational database, data is perceived to exist in one or more *tables*. Each table contains a specific number of *columns* and a number of unordered *rows*. Each column in a table is related in some way to the other columns. Thinking of the data as a collection of tables gives you an easy way to visualize the data that is stored in a Db2 database.

Tables are at the core of a Db2 database. However, a Db2 database involves more than just a collection of tables; a Db2 database also involves other objects, such as views and indexes, and larger data containers, such as table spaces.

With Db2 for z/OS and the other Db2 products, you can define and manipulate your data by using structured query language (SQL). SQL is the standard language for accessing data in relational databases.

SQL: The language of Db2

The language that you use to access the data in Db2 tables is the *structured query language (SQL)*. SQL is a standardized language for defining and manipulating data in a relational database.

The SQL language consists of SQL statements.

By using SQL statements, you can accomplish activities in the following categories, among others:

- Define, modify, or drop data objects such as tables by using various forms of CREATE, ALTER, and DROP statements. These statements are formally categorized as *SQL schema statements*, and sometimes informally as *Data Definition Language (DDL)* statements.
- Retrieve, insert, update, or delete data in tables by using SELECT, INSERT, UPDATE, and MERGE statements. These statements are formally categorized as *SQL data statements*, and sometimes informally as *Data Manipulation Language (DML)* statements.
- Authorize users to access specific resources such as tables or views by using various forms of GRANT, REVOKE, and TRANSFER OWNERSHIP statements. These statements are formally categorized as SQL schema statements, but they are also sometimes informally categorized as *Data Control Language (DCL)* statements.

For a list of supported SQL statements by category, see [Categories of SQL statements \(Db2 SQL\)](#).

When you write an SQL statement, you specify what you want done, not how to do it. To access data for example, you need only to name the tables and columns that contain the data. You do not need to describe how to get to the data.

In accordance with the relational model of data, the database uses these principles:

- The database is perceived as a set of tables.
- Relationships are represented by values in tables.
- Data is retrieved by using SQL to specify a *result table* that can be derived from one or more tables.

Db2 transforms each SQL statement, that is, the specification of a result table, into a sequence of operations that optimize data retrieval. This transformation occurs when the SQL statement is *prepared*. This transformation is also known as *binding*.

All executable SQL statements must be prepared before they can run. The result of preparation is the executable or *operational form* of the statement.

As the following example illustrates, SQL is generally intuitive.



Example

Assume that you are shopping for shoes and you want to know what shoe styles are available in size 8. The SQL query that you need to write is similar to the question that you might ask a salesperson, "What shoe styles are available in size 8?" Just as the salesperson checks the shoe inventory and returns with an answer, Db2 retrieves information from a table (SHOES) and returns a result table. The query looks like this:

```
SELECT STYLE
FROM SHOES
WHERE SIZE = 8;
```

Assume that the answer to your question is that two shoe styles are available in a size 8: loafers and sandals. The result table looks like this:

```
STYLE
=====
LOAFERS
SANDALS
```



Syntax diagrams for SQL statements

For more information about the conventions for syntax diagrams in the Db2 product documentation, see [How to read syntax diagrams \(Db2 for z/OS in IBM Documentation\)](#).

Related concepts

[Programming for Db2 for z/OS](#)

You use application programs that contain SQL statements to perform data definition or data manipulation operations on Db2 for z/OS objects.

Related tasks

[Programming applications for performance \(Db2 Performance\)](#)

Related reference

[Statements \(Db2 SQL\)](#)

Submitting SQL statements to Db2

You can use several different methods to submit SQL statements to Db2 interactively or through application programs.

You can issue SQL statements interactively from a keyboard at a terminal or through a terminal emulator, or by using various developer and administrator tools.

SQL statements can also be issued through application programs. The application programs can contain SQL statements that are statically embedded in the application. Alternatively, application programs can create their SQL statements dynamically, such as in response to information that a user provides by filling in a form. The method of preparing an SQL statement for execution and the persistence of its operational form distinguish static SQL from dynamic SQL.

Interactive SQL

Interactive SQL refers to SQL statements that you submit to Db2 by using SPUFI (SQL processor using file input), the Db2 command line processor, or developer tools such as IBM Db2 for z/OS Developer Extension and Db2 Query Management Facility (QMF). For more information, see the following topics:

- [“Interactive SQL statements” on page 7](#)
- [Executing SQL by using SPUFI \(Db2 Application programming and SQL\)](#)
- [The Db2 command line processor \(Db2 Commands\)](#)
- [Introduction to Db2 for z/OS Developer Extension](#)

Static SQL

The source form of a *static SQL statement* is embedded within an application program written in a host language such as COBOL. The statement is prepared before the program is executed and the operational form of the statement persists beyond the execution of the program. For more information, see the following topics:

- [“Static SQL statements” on page 6](#)
- [“Static SQL applications” on page 255](#)

Dynamic SQL

You can use *dynamic SQL* when you do not know the content of an SQL statement when you write a program or before you run it. You can issue dynamic SQL statements interactively, by embedding the SQL statements in application programs. For more information, see the following topics:

- [Interactively SQL through SPUFI or tools such as QMF](#)
- [Embedded dynamic SQL](#)
- [Deferred embedded SQL](#)
- [“Dynamic SQL applications” on page 262](#)
- [Including dynamic SQL in your program \(Db2 Application programming and SQL\)](#)

Db2 for z/OS ODBC

Db2 for z/OS ODBC (Open Database Connectivity) is an application programming interface (API) that enables C and C++ application programs to access relational databases. The ODBC interface offers an alternative to using embedded static SQL and a different way of performing dynamic SQL. Through the interface, an application invokes a C function at execution time to connect to a data source, to dynamically issue SQL statements, and to retrieve data and status information. For more information, see the following topics:

- [“SQL Call Level Interface and Open Database Connectivity” on page 7](#)
- [Introduction to Db2 ODBC \(Db2 Programming for ODBC\)](#)
- [ODBC functions \(Db2 Programming for ODBC\)](#)

Db2 access for Java: SQLJ, JDBC and pureQuery

SQLJ, JDBC, and pureQuery are methods for accessing Db2 data from applications that are written in the Java programming language. In general, Java applications use SQLJ for static SQL, and they use JDBC for dynamic SQL. IBM pureQuery provides benefits to both static and dynamic SQL.

For more information, see:

- [“Use of Java to execute static and dynamic SQL” on page 264](#)
- [Java application development for IBM data servers \(Db2 Application Programming for Java\)](#)
- [Supported drivers for JDBC and SQLJ \(Db2 Application Programming for Java\)](#)

Related concepts

[How SQL statements are invoked \(Db2 SQL\)](#)

Related tasks

[Overview of programming applications that access Db2 for z/OS data \(Db2 Application programming and SQL\)](#)

Related reference

[Statements \(Db2 SQL\)](#)

[Categories of SQL statements \(Db2 SQL\)](#)

Static SQL statements

The source form of a *static SQL statement* is embedded within an application program written in a host language such as COBOL. The statement is prepared before the program is executed and the operational form of the statement persists beyond the execution of the program.

Static SQL statements in a source program must be processed before the program is compiled. This processing can be accomplished through the Db2 coprocessor or Db2 precompiler. The Db2 coprocessor or Db2 precompiler checks the syntax of the SQL statements, turns them into host language comments, and generates host language statements to invoke Db2.

The preparation of an SQL application program includes processing of the SQL statements, the preparation of its static SQL statements, and compilation of the modified source program.

Related concepts

[Static SQL applications](#)

For most Db2 users, static SQL provides a straightforward, efficient path to Db2 data.

[How SQL statements are invoked \(Db2 SQL\)](#)

Related tasks

[Overview of programming applications that access Db2 for z/OS data \(Db2 Application programming and SQL\)](#)

[Processing SQL statements for program preparation \(Db2 Application programming and SQL\)](#)

Embedded dynamic SQL statements

Programs that contain *embedded dynamic SQL* statements must be precompiled like those that contain static SQL, but unlike static SQL, the dynamic statements are constructed and prepared at run time.

The source form of a dynamic statement is a character string that is passed to Db2 by the program using the static SQL PREPARE or EXECUTE IMMEDIATE statement. A statement that is prepared using the PREPARE statement can be referenced in a DECLARE CURSOR, DESCRIBE, or EXECUTE statement. Whether the operational form of the statement is persistent depends on whether dynamic statement caching is enabled.

SQL statements embedded in a REXX application are dynamic SQL statements. SQL statements submitted to an interactive SQL facility and to the Call Level Interface (CLI) are also dynamic SQL.

Related concepts

[Embedding SQL statements in application programs \(Db2 SQL\)](#)

[Dynamic SQL statement preparation and execution \(Db2 SQL\)](#)

[Dynamic SQL applications](#)

With dynamic SQL, Db2 prepares and executes the SQL statements within a program while the program is running. Dynamic SQL is a good choice when you do not know the format of an SQL statement before you write or run a program.

[Embedded dynamic SQL statements](#)

Programs that contain *embedded dynamic SQL* statements must be precompiled like those that contain static SQL, but unlike static SQL, the dynamic statements are constructed and prepared at run time.

Related tasks

[Including dynamic SQL in your program \(Db2 Application programming and SQL\)](#)

Deferred embedded SQL

A *deferred embedded SQL* statement is neither fully static nor fully dynamic.

Like a static statement, it is embedded within an application, but like a dynamic statement, it is prepared during the execution of the application. Although prepared at run time, a deferred embedded SQL statement is processed with bind-time rules such that the authorization ID and qualifier determined at bind time for the plan or package owner are used.

Interactive SQL statements

Interactive SQL refers to SQL statements that you submit to Db2 by using SPUFI (SQL processor using file input), the Db2 command line processor, or by using developer tools such as IBM Db2 for z/OS Developer Extension or QMF for Workstation.

You can enter and run SQL statements at a TSO terminal by using the SPUFI (SQL processor using file input) facility. SPUFI prepares and executes these statements dynamically. The easiest and most efficient way to run SQL is to use developer tools from a workstation, such as IBM Db2 for z/OS Developer Extension or QMF for Workstation. All of these tools prepare and dynamically execute the SQL statements.

IBM Db2 for z/OS Developer Extension for Microsoft Visual Studio Code

IBM Db2 for z/OS Developer Extension for Visual Studio Code (Db2 Developer Extension) is a free downloadable extension that provides SQL language support to a modern and widely adopted IDE. Db2 Developer Extension delivers capabilities that help developers to write syntactically correct Db2 for z/OS SQL quickly. For more information, see [Introduction to Db2 for z/OS Developer Extension](#).

Coupled with [IBM Z Open Editor for Visual Studio Code](#), Db2 Developer Extension delivers a comprehensive set of tools for developing applications that run on the IBM zSystems platform. Both IBM Z® Open Editor and Db2 Developer Extension are also supported by the [Eclipse Theia](#) IDE platform.

QMF for Workstation

QMF for Workstation is a query tool that lets you enter and run your SQL statements easily. This topic acquaints you with using QMF for Workstation to create and run SQL statements. QMF for Workstation simplifies access to Db2 from a workstation. In fact, was QMF for Workstation built for Db2.

You can use QMF for WebSphere® to enter and run SQL statements from your web browser or use QMF for TSO/CICS to enter and run SQL statements from TSO or CICS. For more information, see [“Submitting SQL statements with QMF for Workstation”](#) on page 8.

Related concepts

[Interactive invocation of SQL statements \(Db2 SQL\)](#)

[Db2 basics tutorial: Working with Db2 for z/OS interactively](#)

This tutorial shows you how to use Db2 for z/OS in the z/OS terminal interface to complete some basic database administration tasks

[The Db2 command line processor \(Db2 Commands\)](#)

[Submitting SQL statements with QMF for Workstation](#)

QMF for Workstation is a tool that helps you build and manage powerful queries without requiring previous experience with SQL.

Related tasks

[Executing SQL by using SPUFI \(Db2 Application programming and SQL\)](#)

Related information

[Introduction to Db2 for z/OS Developer Extension](#)

SQL Call Level Interface and Open Database Connectivity

The Db2 Call Level Interface (CLI) is an application programming interface in which functions are provided to application programs to process dynamic SQL statements.

Db2 CLI allows users to access SQL functions directly through a call interface. CLI programs can also be compiled using an Open Database Connectivity (ODBC) Software Developer's Kit, available from Microsoft or other vendors, enabling access to ODBC data sources. Unlike using embedded SQL, no precompilation is required. Applications developed using this interface can be executed on a variety of databases without being compiled against each of databases. Through the interface, applications use procedure calls at execution time to connect to databases, to issue SQL statements, and to get returned data and status information.

Related concepts

[Introduction to Db2 ODBC \(Db2 Programming for ODBC\)](#)

[Conceptual view of a Db2 ODBC application \(Db2 Programming for ODBC\)](#)

Related information

[ODBC functions \(Db2 Programming for ODBC\)](#)

Java database connectivity and embedded SQL for Java

Db2 provides two standards-based Java programming APIs: Java Database Connectivity (JDBC) and embedded SQL for Java (SQL/OLB or SQLJ). Both can be used to create Java applications and applets that access Db2.

Static SQL cannot be used by JDBC. SQLJ applications use JDBC as a foundation for such tasks as connecting to databases and handling SQL errors, but can contain embedded static SQL statements in the SQLJ source files. An SQLJ file has to be translated with the SQLJ translator before the resulting Java source code can be compiled.

Related concepts

[Java application development for IBM data servers \(Db2 Application Programming for Java\)](#)

[Supported drivers for JDBC and SQLJ \(Db2 Application Programming for Java\)](#)

Submitting SQL statements with QMF for Workstation

QMF for Workstation is a tool that helps you build and manage powerful queries without requiring previous experience with SQL.

With the query-related features of Db2 Query Management Facility (QMF) and QMF for Workstation in particular, you can perform the following tasks:

- Build powerful queries without knowing SQL
- Analyze query results online, including OLAP analysis
- Edit query results to update Db2 data
- Format traditional text-based reports and reports with rich formatting
- Display charts and other complex visuals
- Send query results to an application of your choice
- Develop applications using robust API commands

How SQL statements are entered and processed

You can create your SQL statements using QMF for Workstation in several ways:

- Use the Database Explorer window to easily find and run saved queries that everyone at the same database server can share.
- If you know SQL, type the SQL statement directly in the window.
- If you don't know SQL, use the prompted or diagram interface to build the SQL statement.

The Database Explorer presents the objects that are saved on a server in a tree structure. By expanding and collapsing branches, you can easily locate and use saved queries. You can open the selected query and see the SQL statements or run the query.

If you need to build a new query, you can enter the SQL statements directly in the query window, or you can create the SQL statements using diagrams or prompts. As you build a query by using diagrams or prompts, you can open a view to see the SQL that is being created.

How you can work with query results

When you finish building the query, you can click the **Run Query** button to execute the SQL statements. After you run the query, QMF for Workstation returns the query results in an interactive window.

The query results are formatted by the comprehensive formatting options of QMF for Workstation. A robust expression language lets you conditionally format query results by retrieved column values. You can add calculated columns to the query results and group data columns on both axes with or without summaries. You can also use extensive drag-and-drop capabilities to easily restructure the appearance of the query results.

In addition to formatting the query results, you can perform the following actions:

- Create traditional text-based reports or state-of-the-art reports with rich formatting.
- Display query results by using charts and other complex visuals.
- Share reports by storing them on the database server.
- Send query results to various applications such as Microsoft Excel or Lotus 1-2-3.

Related reference

[Db2 Query Management Facility \(QMF\) information](#)

Db2 database objects overview

In Db2 for z/OS, you use database objects, such as tables, table spaces, indexes, index spaces, keys, views, and databases to organize and access your data.

The brief descriptions here show how the structures fit into an overall view of Db2. The following figure shows how some Db2 structures contain others. To some extent, the notion of "containment" provides a hierarchy of structures.

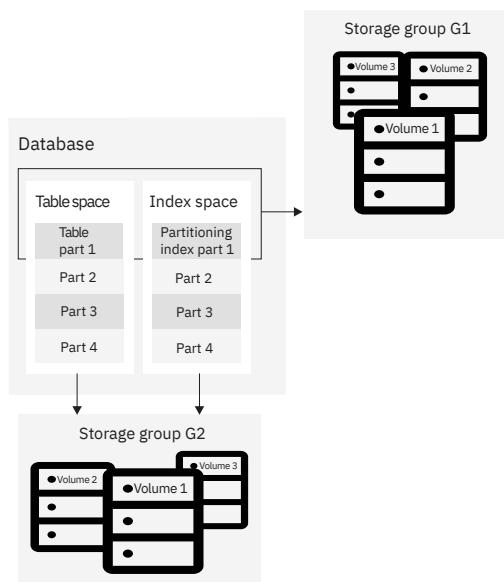


Figure 1. A hierarchy of Db2 structures

The Db2 structures from the most to the least inclusive are:

Databases

A set of Db2 structures that include a collection of tables, their associated indexes, and the table spaces in which they reside.

Storage groups

A set of volumes on disks that hold the data sets in which tables and indexes are stored.

Table spaces

A logical unit of storage in a database. A table space is a page set. The recommended partition-by-growth and partition-by-range table space types always contain data for only a single table. However, the segmented (non-UTS) and simple table space types, which are deprecated, can each contain one or more tables. For more information, see [Table space types and characteristics in Db2 for z/OS \(Db2 Administration Guide\)](#).

Deprecated function: FL 504 Non-UTS table spaces for base tables are deprecated. CREATE TABLESPACE statements that run at application compatibility level V12R1M504 or higher always create a partition-by-growth or partition-by-range table space, and CREATE TABLE statements that specify a non-UTS table space (including existing multi-table segmented table spaces) return an error. However, you can use a lower application compatibility level to create table spaces of the deprecated types if needed, such as for recovery situations. For instructions, see [Creating non-UTS table spaces \(deprecated\)](#) (Db2 Administration Guide).

Tables

All data in a Db2 database is presented in *tables*, which are collections of rows all having the same columns. A table that holds persistent user data is a *base table*. A table that stores data temporarily is a *temporary table*.

Views

A *view* is an alternative way of representing data that exists in one or more tables. A view can include all or some of the columns from one or more base tables.

Indexes

An *index* is an ordered set of pointers to the data in a Db2 table. The index is stored separately from the table. An index is either a simple index or an extended index. An extended index is one of the following objects:

- An expression-based index
- A spatial index
- An XML index

Related concepts

Db2 system objects

Unlike the Db2 data structures that users create and access, Db2 controls and accesses system objects.

[Implementing your Db2 for z/OS database design \(Db2 Administration Guide\)](#)

Storage structures

In Db2, a *storage structure* is a set of one or more VSAM data sets that hold Db2 tables or indexes. A storage structure is also called a *page set*.

Db2 schemas and schema qualifiers

The objects in a relational database are organized into sets called schemas. A *schema* is a collection of named objects that provides a logical classification of objects in the database. The *schema name* of the object determines the schema that the object belongs to.

Objects are assigned to schemas when they are created. That is, when certain types of database objects are created, they are given qualified two-part names. The first part is the schema name (or the qualifier), which is either implicitly or explicitly specified. The second part is the name of the object. The default schema is the authorization ID of the owner of the plan or package. The objects that a schema can contain include tables, indexes, table spaces, distinct types, functions, stored procedures, and triggers.

User objects, such as a distinct types, functions, procedures, sequences, or triggers should not be created in a *system schema*, which is any one of a set of schemas that are reserved for use by the Db2 subsystem. For more information, see [Reserved schema names in Db2 for z/OS \(Db2 SQL\)](#).

How CURRENT SCHEMA and CURRENT SQLID affect object schemas

In earlier versions of Db2 for z/OS, CREATE statements had certain restrictions when the value of CURRENT SCHEMA was different from CURRENT SQLID value. Although those restrictions no longer exist, you must consider how to determine the qualifier and owner when CURRENT SCHEMA and CURRENT SQLID contain different values. The rules for how the owner is determined depend on the type of object being created.

CURRENT SCHEMA and CURRENT SQLID affect only dynamic SQL statements. Static CREATE statements are not affected by either CURRENT SCHEMA or CURRENT SQLID.

The following table summarizes the effect of CURRENT SCHEMA in determining the schema qualifier and owner for these objects:

- Aliases
- Auxiliary tables
- Created global temporary tables
- Indexes
- Tables
- Views

Table 1. Schema qualifier and owner for objects

Specification of name for new object being created	Schema qualifier of new object	Owner of new object
<i>name</i> (no qualifier)	value of CURRENT SCHEMA	value of CURRENT SQLID
<i>abc.name</i> (single qualifier)	abc	abc
<i>.....abc.name</i> (multiple qualifiers)	abc	abc

The following table summarizes the effect of CURRENT SCHEMA in determining the schema qualifier and owner for these objects:

- User-defined type
- User-defined function
- Procedure
- Sequence
- Trigger

Table 2. Schema qualifier and owner for additional objects

Specification of name for new object being created	Schema qualifier of new object	Owner of new object
<i>name</i> (no qualifier)	value of CURRENT SCHEMA	value of CURRENT SQLID
<i>abc.name</i> (single qualifier)	abc	value of CURRENT SQLID
<i>.....abc.name</i> (multiple qualifiers)	abc	value of CURRENT SQLID

Related reference

[Reserved schema names in Db2 for z/OS \(Db2 SQL\)](#)

Db2 tables

Tables are logical structures that Db2 maintains. Db2 supports several different types of tables.

Tables are made up of columns and rows. The rows of a relational table have no fixed order. The order of the columns, however, is always the order in which you specified them when you defined the table.

At the intersection of every column and row is a specific data item, which is called a *value*. A *column* is a set of values of the same type. A *row* is a sequence of values such that the *n*th value is a value of the *n*th column of the table. Every table must have one or more columns, but the number of rows can be zero.

Db2 accesses data by referring to its content instead of to its location or organization in storage.

Db2 supports the following types of tables:

accelerator-only table

A table that stores rows only in the accelerator, not in Db2. The table and column definition of the accelerator-only table is contained in Db2 catalog tables. Any queries that reference the accelerator-only table, must be executed in the accelerator. If a query that references an accelerator-only table is not eligible for query acceleration, an error is issued. To change the contents of an accelerator-only table, the data change statement must be executed in the accelerator.

archive table

A table that stores rows that are deleted from another table.

archive-enabled table

A table that has an associated archive table. When rows are deleted from an archive-enabled table, Db2 can automatically insert those rows into an archive table.

auxiliary table

A table created with the SQL statement CREATE AUXILIARY TABLE and used to hold the data for a column that is defined in a base table.

base table

The most common type of table in Db2. You create a base table with the SQL CREATE TABLE statement. The Db2 catalog table, SYSIBM.SYSTABLES, stores the description of the base table. The table description and table data are persistent. All programs and users that refer to this type of table refer to the same description of the table and to the same instance of the table.

clone table

A table that is structurally identical to a base table. You create a clone table by using an ALTER TABLE statement for the base table that includes an ADD CLONE clause. The clone table is created in a different instance of the same table space as the base table, is structurally identical to the base table in every way, and has the same indexes, before triggers, and LOB objects. In the Db2 catalog, the SYSTABLESPACE table indicates that the table space has only one table in it, but SYSTABLESPACE.CLONE indicates that a clone table exists. Clone tables can be created only in a partition-by range or partition-by-growth table space that is managed by Db2. The base and clone table each have separate underlying VSAM data sets (identified by their data set instance numbers) that contain independent rows of data.

empty table

A table with zero rows.

history table

A table that is used to store historical versions of rows from the associated system-period temporal table.

materialized query table

A table, which you define with the SQL CREATE TABLE statement, that contains materialized data that is derived from one or more source tables. Materialized query tables are useful for complex queries that run on large amounts of data. Db2 can precompute all or part of such queries and use the precomputed, or materialized, results to answer the queries more efficiently. Materialized query tables are commonly used in data warehousing and business intelligence applications.

Several Db2 catalog tables, including SYSIBM.SYSTABLES and SYSIBM.SYSVIEWS, store the description of the materialized query table and information about its dependency on a table, view, or function. The attributes that define a materialized query table tell Db2 whether the table is:

- System-maintained or user-maintained.
- Refreshable: All materialized tables can be updated with the REFRESH TABLE statement. Only user-maintained materialized query tables can also be updated with the LOAD utility and the UPDATE, INSERT, and DELETE SQL statements.
- Enabled for query optimization: You can enable or disable the use of a materialized query table in automatic query rewrite.

Materialized query tables can be used to improve the performance of dynamic SQL queries. If Db2 determines that a portion of a query could be resolved using a materialized query table, the query might be rewritten by Db2 to use the materialized query table. This decision is based in part on

the settings of the CURRENT REFRESH AGE and the CURRENT MAINTAINED TABLE TYPES FOR OPTIMIZATION special registers.

result table

A table that contains a set of rows that Db2 selects or generates, directly or indirectly, from one or more base tables in response to an SQL statement. Unlike a base table or a temporary table, a result table is not an object that you define using a CREATE statement.

sample table

One of several tables shipped with the Db2 licensed program that contains sample data. Many examples in this information are based on sample tables.

temporal table

A table that records the period of time when a row is valid.

Db2 supports two types of periods, which are the system period (specified by a SYSTEM_TIME clause) and the application period (specified by a BUSINESS_TIME clause). The system period consists of a pair of columns with system-maintained values that indicates the period of time when a row is valid. The application period consists of a pair of columns with application-maintained values that indicates the period of time when a row is valid.

system-period temporal table

A system-period temporal table is a base table that is defined with system-period data versioning. You can modify an existing table to become a system-period temporal table by specifying the ADD PERIOD SYSTEM_TIME clause on the ALTER TABLE statement. After creating a history table that corresponds to the system-period temporal table, you can define system-period data versioning on the table by issuing the ALTER TABLE ADD VERSIONING statement with the USE HISTORY table clause.

application-period temporal table

An application-period temporal table is a base table that includes an application period (specified by a BUSINESS_TIME clause). You can modify an existing table to become an application-period temporal table by specifying the ADD PERIOD BUSINESS_TIME clause on the ALTER TABLE statement.

bitemporal table

A bitemporal table is a table that is both a system-period temporal table and an application-period temporal table. You can use a bitemporal table to keep application period information and system-based historical information. Therefore, you have a lot of flexibility in how you query data, based on periods of time.

temporary table

A table that is defined by the SQL statement CREATE GLOBAL TEMPORARY TABLE or DECLARE GLOBAL TEMPORARY TABLE to hold data temporarily. Temporary tables are especially useful when you need to sort or query intermediate result tables that contain many rows, but you want to store only a small subset of those rows permanently.

created global temporary table

A table that you define with the SQL CREATE GLOBAL TEMPORARY TABLE statement. The Db2 catalog table, SYSIBM.SYSTABLES, stores the description of the created temporary table. The description of the table is persistent and shareable. However, each individual application process that refers to a created temporary table has its own distinct instance of the table. That is, if application process A and application process B both use a created temporary table named TEMPTAB:

- Each application process uses the same table description.
- Neither application process has access to or knowledge of the rows in the other application instance of TEMPTAB.

declared global temporary table

A table that you define with the SQL DECLARE GLOBAL TEMPORARY TABLE statement. The Db2 catalog does not store a description of the declared temporary table. Therefore, the description and the instance of the table are not persistent. Multiple application processes can refer to the same declared temporary table by name, but they do not actually share the same

description or instance of the table. For example, assume that application process A defines a declared temporary table named TEMP1 with 15 columns. Application process B defines a declared temporary table named TEMP1 with five columns. Each application process uses its own description of TEMP1; neither application process has access to or knowledge of rows in the other application instance of TEMP1.

XML table

A special table that holds only XML data. When you create a table with an XML column, Db2 implicitly creates an XML table space and an XML table to store the XML data.

Related concepts

[Types of tables \(Db2 Administration Guide\)](#)

Related reference

[CREATE TABLE statement \(Db2 SQL\)](#)

[ALTER TABLE statement \(Db2 SQL\)](#)

Db2 keys

A *key* is a column or an ordered collection of columns that is identified in the description of a table, an index, or a referential constraint. Keys are crucial to the table structure in a relational database.

Keys are important in a relational database because they ensure that each record in a table is uniquely identified, they help establish and enforce referential integrity, and they establish relationships between tables. The same column can be part of more than one key.

A *composite key* is an ordered set of two or more columns of the same table. The ordering of the columns is not constrained by their actual order within the table. The term *value*, when used with respect to a composite key, denotes a composite value. For example, consider this rule: "The value of the foreign key must be equal to the value of the primary key." This rule means that each component of the value of the foreign key must be equal to the corresponding component of the value of the primary key.

Db2 supports several types of keys.

Unique keys

A *unique constraint* is a rule that the values of a key are valid only if they are unique. A key that is constrained to have unique values is a *unique key*. Db2 uses a *unique index* to enforce the constraint during the execution of the LOAD utility and whenever you use an INSERT, UPDATE, or MERGE statement to add or modify data. Every unique key is a key of a unique index. You can define a unique key by using the UNIQUE clause of either the CREATE TABLE or the ALTER TABLE statement. A table can have any number of unique keys.

The columns of a unique key cannot contain null values.

Primary keys

A *primary key* is a special type of unique key and cannot contain null values. For example, the DEPTNO column in the DEPT table is a primary key.

A table can have no more than one primary key. Primary keys are optional and can be defined in CREATE TABLE or ALTER TABLE statements.

The unique index on a primary key is called a *primary index*. When a primary key is defined in a CREATE TABLE statement or ALTER TABLE statement, Db2 automatically creates the primary index.

If a unique index already exists on the columns of the primary key when it is defined in the ALTER TABLE statement, this unique index is designated as the primary index when Db2 is operating in new-function mode and implicitly created the table space.

Parent keys

A *parent key* is either a primary key or a unique key in the parent table of a referential constraint. The values of a parent key determine the valid values of the foreign key in the constraint.

Foreign keys

A *foreign key* is a key that is specified in the definition of a referential constraint in a CREATE or ALTER TABLE statement. A foreign key refers to or is related to a specific parent key.

Unlike other types of keys, a foreign key does not require an index on its underlying column or columns. A table can have zero or more foreign keys. The value of a composite foreign key is null if any component of the value is null.

The following figure shows the relationship between some columns in the DEPT table and the EMP table.

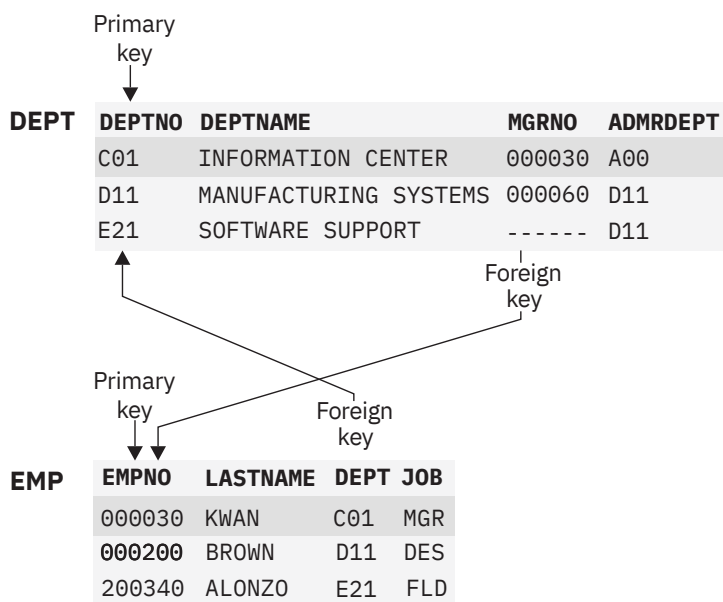


Figure 2. Relationship between DEPT and EMP tables

Figure notes: Each table has a primary key:

- DEPTNO in the DEPT table
- EMPNO in the EMP table

Each table has a foreign key that establishes a relationship between the tables:

- The values of the foreign key on the DEPT column of the EMP table match values in the DEPTNO column of the DEPT table.
- The values of the foreign key on the MGRNO column of the DEPT table match values in the EMPNO column of the EMP table when an employee is a manager.

To see a specific relationship between rows, notice how the shaded rows for department C01 and employee number 000030 share common values.

Related concepts

[Referential constraints](#)

Db2 ensures referential integrity between your tables when you define referential constraints.

Constraints

A *constraint* is a rule that Db2 enforces for column values to prevent duplicate values or to set restrictions on data that is added to a table.

Db2 uses the following types of constraints.

Related concepts

[Ways to maintain data integrity \(Db2 Application programming and SQL\)](#)

Unique constraints

A *unique constraint* is a rule that the values of a key are valid only if they are unique in a table.

Unique constraints are optional and can be defined in the CREATE TABLE or ALTER TABLE statements with the PRIMARY KEY clause or the UNIQUE clause. The columns specified in a unique constraint must be defined as NOT NULL. A unique index enforces the uniqueness of the key during changes to the columns of the unique constraint.

A table can have an arbitrary number of unique constraints, with at most one unique constraint defined as a primary key. A table cannot have more than one unique constraint on the same set of columns.

A unique constraint that is referenced by the foreign key of a referential constraint is called the *parent key*.

Referential constraints

Db2 ensures referential integrity between your tables when you define referential constraints.

Referential integrity is the state in which all values of all foreign keys are valid. Referential integrity is based on *entity integrity*. Entity integrity requires that each entity have a unique key. For example, if every row in a table represents relationships for a unique entity, the table should have one column or a set of columns that provides a unique identifier for the rows of the table. This column (or set of columns) is called the parent key of the table. To ensure that the parent key does not contain duplicate values, a unique index must be defined on the column or columns that constitute the parent key. Defining the parent key is called entity integrity.

A *referential constraint* is the rule that the nonnull values of a foreign key are valid only if they also appear as values of a parent key. The table that contains the parent key is called the *parent table* of the referential constraint, and the table that contains the foreign key is a *dependent* of that table.

The relationship between some rows of the DEPT and EMP tables, shown in the following figure, illustrates referential integrity concepts and terminology. For example, referential integrity ensures that every foreign key value in the DEPT column of the EMP table matches a primary key value in the DEPTNO column of the DEPT table.

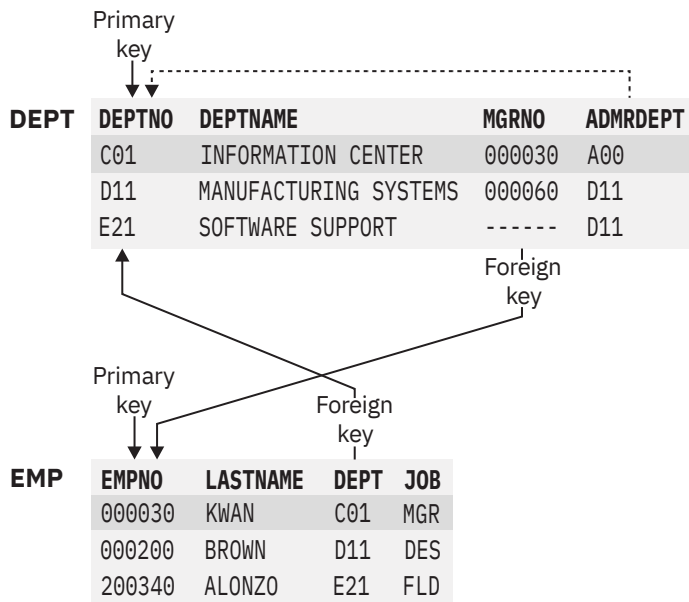


Figure 3. Referential integrity of DEPT and EMP tables

Two parent and dependent relationships exist between the DEPT and EMP tables.

- The foreign key on the DEPT column establishes a parent and dependent relationship. The DEPT column in the EMP table depends on the DEPTNO in the DEPT table. Through this foreign key relationship, the DEPT table is the parent of the EMP table. You can assign an employee to no department (by specifying a null value), but you cannot assign an employee to a department that does not exist.
- The foreign key on the MGRNO column also establishes a parent and dependent relationship. Because MGRNO depends on EMPNO, EMP is the parent table of the relationship, and DEPT is the dependent table.

You can define a primary key on one or more columns. A primary key that includes two or more columns is called a *composite key*. A foreign key can also include one or more columns. When a foreign key contains multiple columns, the corresponding primary key must be a composite key. The number of foreign key columns must be the same as the number of columns in the parent key, and the data types of the corresponding columns must be compatible. (The sample project activity table, DSN8C10.PROJACT, is an example of a table with a primary key on multiple columns, PROJNO, ACTNO, and ACSTDAT.)

A table can be a dependent of itself; this is called a *self-referencing table*. For example, the DEPT table is self-referencing because the value of the administrative department (ADMRDEPT) must be a department ID (DEPTNO). To enforce the self-referencing constraint, Db2 requires that a foreign key be defined.

Similar terminology applies to the rows of a parent-and-child relationship. A row in a dependent table, called a *dependent row*, refers to a row in a parent table, called a *parent row*. But a row of a parent table is not always a parent row—perhaps nothing refers to it. Likewise, a row of a dependent table is not always a dependent row—the foreign key can allow null values, which refer to no other rows.

Referential constraints are optional. You define referential constraints by using CREATE TABLE and ALTER TABLE statements.

Db2 enforces referential constraints when the following actions occur:

- An INSERT statement is applied to a dependent table.
- An UPDATE statement is applied to a foreign key of a dependent table or to the parent key of a parent table.
- A MERGE statement that includes an insert operation is applied to a dependent table.
- A MERGE statement that includes an update operation is applied to a foreign key of a dependent table or to the parent key of a parent table.

- A DELETE statement is applied to a parent table. All affected referential constraints and all delete rules of all affected relationships must be satisfied in order for the delete operation to succeed.
- The LOAD utility with the ENFORCE CONSTRAINTS option is run on a dependent table.
- The CHECK DATA utility is run.

Another type of referential constraint is an *informational referential constraint*. This type of constraint is not enforced by Db2 during normal operations. An application process should verify the data in the referential integrity relationship. An informational referential constraint allows queries to take advantage of materialized query tables.

The order in which referential constraints are enforced is undefined. To ensure that the order does not affect the result of the operation, there are restrictions on the definition of delete rules and on the use of certain statements. The restrictions are specified in the descriptions of the SQL statements CREATE TABLE, ALTER TABLE, INSERT, UPDATE, MERGE, and DELETE.

The rules of referential integrity involve the following concepts and terminology:

parent key

A primary key or a unique key of a referential constraint.

parent table

A table that is a parent in at least one referential constraint. A table can be defined as a parent in an arbitrary number of referential constraints.

dependent table

A table that is a dependent in at least one referential constraint. A table can be defined as a dependent in an arbitrary number of referential constraints. A dependent table can also be a parent table.

descendent table

A table that is a dependent of another table or a table that is a dependent of a descendent table.

referential cycle

A set of referential constraints in which each associated table is a descendent of itself.

parent row

A row that has at least one dependent row.

dependent row

A row that has at least one parent row.

descendent row

A row that is dependent on another row or a row that is a dependent of a descendent row.

self-referencing row

A row that is a parent of itself.

self-referencing table

A table that is both parent and dependent in the same referential constraint. The constraint is called a *self-referencing constraint*.

A temporal referential constraint can be defined for a table that contains a BUSINESS_TIME period. The PERIOD BUSINESS_TIME clause is used in both the FOREIGN KEY clause and the REFERENCES clause to indicate that there must not be a row in the child table for which the period of time represented by the BUSINESS_TIME period value for that row is not contained in the BUSINESS_TIME period of one or more corresponding rows in the parent table. Unlike normal referential constraints, it is not necessary that there be exactly one corresponding row in the parent table where the BUSINESS_TIME period contains the BUSINESS_TIME period of the child row. As long as the BUSINESS_TIME period of a row in the child table is contained in the union of the BUSINESS_TIME periods of two or more contiguous matching rows in the parent table, the temporal referential constraint is satisfied.

Additionally, the following indexes must be defined:

- A unique index on the parent table with the BUSINESS_TIME WITHOUT OVERLAPS clause.
- A non-unique index on the child table with the BUSINESS_TIME WITH OVERLAPS clause. Alternatively, the index on the child table, can be defined without the BUSINESS_TIME WITH OVERLAPS clause if the

end of the index key includes the end column followed by the begin column of the BUSINESS_TIME period (both in ascending order).

The following rules provide referential integrity:

insert rule

A nonnull insert value of the foreign key must match some value of the parent key of the parent table. The value of a composite foreign key is null if any component of the value is null.

update rule

A nonnull update value of the foreign key must match some value of the parent key of the parent table. The value of a composite foreign key is treated as null if any component of the value is null.

delete rule

Controls what happens when a row of the parent table is deleted. The choices of action, made when the referential constraint is defined, are RESTRICT, NO ACTION, CASCADE, or SET NULL. SET NULL can be specified only if some column of the foreign key allows null values.

More precisely, the delete rule applies when a row of the parent table is the object of a delete or propagated delete operation and that row has dependents in the dependent table of the referential constraint. A *propagated delete* refers to the situation where dependent rows are deleted when parent rows are deleted. Let P denote the parent table, let D denote the dependent table, and let p denote a parent row that is the object of a delete or propagated delete operation. If the delete rule is:

- RESTRICT or NO ACTION, an error occurs and no rows are deleted.
- CASCADE, the delete operation is propagated to the dependent rows of p in D .
- SET NULL, each nullable column of the foreign key of each dependent row of p in D is set to null.

Each referential constraint in which a table is a parent has its own delete rule, and all applicable delete rules are used to determine the result of a delete operation. Thus, a row cannot be deleted if it has dependents in a referential constraint with a delete rule of RESTRICT or NO ACTION or the deletion cascades to any of its descendents that are dependents in a referential constraint with the delete rule of RESTRICT or NO ACTION.

The deletion of a row from parent table P involves other tables and can affect rows of these tables:

- If D is a dependent of P and the delete rule is RESTRICT or NO ACTION, D is involved in the operation but is not affected by the operation and the deletion from the parent table P does not take place.
- If D is a dependent of P and the delete rule is SET NULL, D is involved in the operation and rows of D might be updated during the operation.
- If D is a dependent of P and the delete rule is CASCADE, D is involved in the operation and rows of D might be deleted during the operation. If rows of D are deleted, the delete operation on P is said to be propagated to D . If D is also a parent table, the actions described in this list apply, in turn, to the dependents of D .

Any table that can be involved in a delete operation on P is said to be *delete-connected* to P . Thus, a table is delete-connected to table P if it is a dependent of P or a dependent of a table to which delete operations from P cascade.

Related concepts

[Referential constraints \(Db2 Application programming and SQL\)](#)

Related reference

[Department table \(DSN8C10.DEPT\)](#)

The department table describes each department in the enterprise and identifies its manager and the department to which it reports.

[Employee table \(DSN8C10.EMP\)](#)

The sample employee table identifies all employees by an employee number and lists basic personnel information.

[Project activity table \(DSN8C10.PROJECT\)](#)

The sample project activity table lists the activities that are performed for each project.

Check constraints

A *check constraint* is a rule that specifies the values that are allowed in one or more columns of every row of a base table.

Like referential constraints, check constraints are optional and you define them by using the CREATE TABLE and ALTER TABLE statements. The definition of a check constraint restricts the values that a specific column of a base table can contain.

A table can have any number of check constraints. Db2 enforces a check constraint by applying the restriction to each row that is inserted, loaded, or updated. One restriction is that a column name in a check constraint on a table must identify a column of that table.

For example, you can create a check constraint to ensure that all employees earn a salary of \$30,000 or more:

```
CHECK (SALARY >= 30000)
```

Related concepts

[Check constraints \(Db2 Application programming and SQL\)](#)

Db2 table columns

A column definition has two basic components, the column name and the data type.

A *column* contains values that have the same data type. If you are familiar with the concepts of records and fields, you can think of a *value* as a field in a record. A value is the smallest unit of data that you can manipulate with SQL. For example, in the EMP table, the EMPNO column identifies all employees by a unique employee number. The HIREDATE column contains the hire dates for all employees. You cannot overlap columns.

Online schema enhancements provide flexibility that lets you change a column definition. Carefully consider the decisions that you make about column definitions. After you implement the design of your tables, you can change a column definition with minimal disruption of applications.

Throughout the implementation phase of database design, refer to the complete descriptions of SQL statement syntax and usage for each SQL statement that you work with.

Column names

Following column naming guidelines that are developed for your organization ensures that you make good choices that are consistent.

Generally, the database administrator (DBA) is involved in determining the names of attributes (or columns) during the physical database design phase. To make the right choices for column names, DBAs follow the guidelines that the data administrators developed.

Sometimes columns need to be added to the database after the design is complete. In this case, Db2 rules for unique column names must be followed. Column names must be unique within a table, but you can use the same column name in different tables. Try to choose a meaningful name to describe the data in a column to make your naming scheme intuitive.

For more information, see [Column names \(Db2 SQL\)](#) and [Naming conventions in SQL \(Db2 SQL\)](#).

Related concepts

[Data types in Db2 for z/OS \(Db2 SQL\)](#)

Related tasks

[Altering the data type of a column \(Db2 Administration Guide\)](#)

Related reference

[Employee table \(DSN8C10.EMP\)](#)

The sample employee table identifies all employees by an employee number and lists basic personnel information.

Data types of columns

Every column in every Db2 table has a data type. The data type influences the range of values that the column can have and the set of operators and functions that apply to it.

You specify the data type of each column at the time that you create the table. You can also change the data type of a table column. The new data type definition is applied to all data in the associated table when the table is reorganized.

Some data types have parameters that further define the operators and functions that apply to the column. Db2 supports both IBM-supplied data types and user-defined data types. The data types that IBM supplies are sometimes called *built-in data types*.

In Db2 for z/OS, user-defined data types are called *distinct types*.

How Db2 compares values of different data types

Db2 compares values of different types and lengths. A comparison occurs when both values are numeric, both values are character strings, or both values are graphic strings. Comparisons can also occur between character and graphic data or between character and datetime data if the character data is a valid character representation of a datetime value. Different types of string or numeric comparisons might have an impact on performance.

Related concepts

[Entity attributes in database design](#)

When you define attributes for entities, you generally work with the data administrator to decide on names, data types, and appropriate values for the attributes.

[Data types of columns \(Db2 Application programming and SQL\)](#)

[Assignment and comparison \(Db2 SQL\)](#)

[Casting between data types \(Db2 SQL\)](#)

[Rules for result data types \(Db2 SQL\)](#)

[Distinct types](#)

A *distinct type* is a user-defined data type that is based on existing built-in Db2 data types.

String data types

Db2 supports several types of string data: character strings, graphic strings, and binary strings.

Character strings contain text and can be either a fixed-length or a varying-length. *Graphic strings* contain graphic data, which can also be either a fixed-length or a varying-length. *Binary strings* contain strings of binary bytes and can be either a fixed-length or a varying-length. All of these types of string data can be represented as large objects.

The following table describes the different string data types and indicates the range for the length of each string data type.

Table 3. String data types

Data type	Denotes a column of...
CHARACTER(<i>n</i>)	Fixed-length character strings with a length of <i>n</i> bytes. <i>n</i> must be greater than 0 and not greater than 255. The default length is 1.
VARCHAR(<i>n</i>)	Varying-length character strings with a maximum length of <i>n</i> bytes. <i>n</i> must be greater than 0 and less than a number that depends on the page size of the table space. The maximum length is 32704.
CLOB(<i>n</i>)	Varying-length character strings with a maximum of <i>n</i> characters. <i>n</i> cannot exceed 2,147,483,647. The default length is 1M.

Table 3. String data types (continued)

Data type	Denotes a column of...
GRAPHIC(<i>n</i>)	Fixed-length graphic strings that contain <i>n</i> double-byte characters. <i>n</i> must be greater than 0 and less than 128. The default length is 1.
VARGRAPHIC(<i>n</i>)	Varying-length graphic strings. The maximum length, <i>n</i> , must be greater than 0 and less than a number that depends on the page size of the table space. The maximum length is 16352.
DBCLOB(<i>n</i>)	Varying-length strings of double-byte characters with a maximum of <i>n</i> double-byte characters. <i>n</i> cannot exceed 1,073,741,824. The default length is 1M.
BINARY(<i>n</i>)	Fixed-length or varying-length binary strings with a length of <i>n</i> bytes. <i>n</i> must be greater than 0 and not greater than 255. The default length is 1.
VARBINARY(<i>n</i>)	Varying-length binary strings with a length of <i>n</i> bytes. The length of <i>n</i> must be greater than 0 and less than a number that depends on the page size of the table space. The maximum length is 32704.
BLOB(<i>n</i>)	Varying-length binary strings with a length of <i>n</i> bytes. <i>n</i> cannot exceed 2,147,483,647. The default length is 1M.

In most cases, the content of the data that a column is to store dictates the data type that you choose.

Example

For example, assume that the DEPT table has a column, DEPTNAME. The data type of the DEPTNAME column is VARCHAR(36). Because department names normally vary considerably in length, the choice of a varying-length data type seems appropriate. If you choose a data type of CHAR(36), for example, the result is a lot of wasted, unused space. In this case, Db2 assigns all department names, regardless of length, the same amount of space (36 bytes). A data type of CHAR(6) for the employee number (EMPNO) is a reasonable choice because all values are fixed-length values (6 bytes).

Fixed-length and variable-length character strings

Using VARCHAR saves disk space, but it incurs a 2-byte overhead cost for each value. Using VARCHAR also requires additional processing for varying-length rows. Therefore, using CHAR is preferable to VARCHAR, unless the space that you save by using VARCHAR is significant. The savings are not significant if the maximum column length is small or if the lengths of the values do not have a significant variation.

Recommendation: Generally, do not define a column as VARCHAR(*n*) or CLOB(*n*) unless *n* is at least 18 characters.

String subtypes

If an application that accesses your table uses a different encoding scheme than your DBMS uses, the following string subtypes can be important:

BIT

Does not represent characters.

SBCS

Represents single-byte characters.

MIXED

Represents single-byte characters and multibyte characters.

String subtypes apply only to CHAR, VARCHAR, and CLOB data types. However, the BIT string subtype is not allowed for the CLOB data type.

Graphic and mixed data

When columns contain *double-byte character set* (DBCS) characters, you can define them as either graphic data or mixed data.

Graphic data can be either GRAPHIC, VARGRAPHIC, or DBCLOB. Using VARGRAPHIC saves disk space, but it incurs a 2-byte overhead cost for each value. Using VARGRAPHIC also requires additional processing for varying-length rows. Therefore, using GRAPHIC data is preferable to using VARGRAPHIC unless the space that you save by using VARGRAPHIC is significant. The savings are not significant if the maximum column length is small or if the lengths of the values do not vary significantly.

Recommendation: Generally, do not define a column as VARGRAPHIC(*n*) unless *n* is at least 18 double-byte characters (which is a length of 36 bytes).

Mixed-data character string columns can contain both *single-byte character set* (SBCS) and DBCS characters. You can specify the mixed-data character string columns as CHAR, VARCHAR, or CLOB with MIXED DATA.

Recommendation: If all of the characters are DBCS characters, use the graphic data types. (Kanji is an example of a language that requires DBCS characters.) For SBCS characters, use mixed data to save 1 byte for every single-byte character in the column.

Encoding schemes for string data

For string data, all characters are represented by a common encoding representation (Unicode, ASCII, or EBCDIC). Encoding schemes apply to string data types and to distinct types that are based on string types.

Multinational companies that engage in international trade often store data from more than one country in the same table. Some countries use different coded character set identifiers. Db2 for z/OS supports the Unicode encoding scheme, which represents many different geographies and languages. If you need to perform character conversion on Unicode data, the conversion is more likely to preserve all of your information.

In some cases, you might need to convert characters to a different encoding representation. The process of conversion is known as *character conversion*. Most users do not need a knowledge of character conversion. When character conversion does occur, it does so automatically and a successful conversion is invisible to the application and users.

Related concepts

[Distinct types](#)

A *distinct type* is a user-defined data type that is based on existing built-in Db2 data types.

[Character strings \(Db2 SQL\)](#)

[Graphic strings \(Db2 SQL\)](#)

[Binary strings \(Db2 SQL\)](#)

[International data and character conversion in Db2 for z/OS \(Db2 Internationalization Guide\)](#)

Related reference

[Department table \(DSN8C10.DEPT\)](#)

The department table describes each department in the enterprise and identifies its manager and the department to which it reports.

Numeric data types

Db2 supports several types of numeric data types, each of which has its own characteristics.

For numeric data, use numeric columns rather than string columns. Numeric columns require less space than string columns, and Db2 verifies that the data has the assigned type.

For example, assume that Db2 calculates a range between two numbers. If the values have a string data type, Db2 assumes that the values can include all combinations of alphanumeric characters. In contrast, if the values have a numeric data type, Db2 can calculate a range between the two values more efficiently.

The following table describes the numeric data types.

Table 4. Numeric data types

Data type	Denotes a column of...
SMALLINT	Small integers. A <i>small integer</i> is binary integer with a precision of 15 bits. The range is -32768 to +32767.
INTEGER or INT	Large integers. A <i>large integer</i> is binary integer with a precision of 31 bits. The range is -2147483648 to +2147483647.
BIGINT	Big integers. A <i>big integer</i> is a binary integer with a precision of 63 bits. The range of big integers is -9223372036854775808 to +9223372036854775807.
DECIMAL or NUMERIC	A <i>decimal</i> number is a packed decimal number with an implicit decimal point. The position of the decimal point is determined by the precision and the scale of the number. The scale, which is the number of digits in the fractional part of the number, cannot be negative or greater than the precision. The maximum precision is 31 digits. All values of a decimal column have the same precision and scale. The range of a decimal variable or the numbers in a decimal column is $-n$ to $+n$, where n is the largest positive number that can be represented with the applicable precision and scale. The maximum range is $1 - 10^{31}$ to $10^{31} - 1$.
DECFLOAT	A <i>decimal floating-point</i> value is an IEEE 754r number with a decimal point. The position of the decimal point is stored in each decimal floating-point value. The maximum precision is 34 digits. The range of a decimal floating-point number is either 16 or 34 digits of precision; the exponent range is respectively 10-383 to 10+384 or 10-6143 to 10+6144.
REAL	A <i>single-precision floating-point</i> number is a short floating-point number of 32 bits. The range of single-precision floating-point numbers is approximately $-7.2E+75$ to $7.2E+75$. In this range, the largest negative value is about $-5.4E-79$, and the smallest positive value is about $5.4E-79$.
DOUBLE	A <i>double-precision floating-point</i> number is a long floating-point number of 64-bits. The range of double-precision floating-point numbers is approximately $-7.2E+75$ to $7.2E+75$. In this range, the largest negative value is about $-5.4E-79$, and the smallest positive value is about $5.4E-79$.

Note: IBM zSystems and z/Architecture® use the System/390® format and support IEEE floating-point format.

For integer values, use SMALLINT, INTEGER, or BIGINT (depending on the range of the values). Do not use DECIMAL for integer values.

You can define an exact numeric column as an identity column. An *identity column* has an attribute that enables Db2 to automatically generate a unique numeric value for each row that is inserted into the table. Identity columns are ideally suited to the task of generating unique primary-key values. Applications that use identity columns might be able to avoid concurrency and performance problems that sometimes occur when applications implement their own unique counters.

Date, time, and timestamp data types

Although storing dates and times as numeric values is possible, using datetime data types is recommended. The datetime data types are DATE, TIME, and TIMESTAMP.

The following table describes the data types for dates, times, and timestamps.

Table 5. Date, time, and timestamp data types

Data type	Denotes a column of...
DATE	A <i>date</i> is a three-part value representing a year, month, and day in the range of 0001-01-01 to 9999-12-31.
TIME	A <i>time</i> is a three-part value representing a time of day in hours, minutes, and seconds, in the range of 00.00.00 to 24.00.00.
TIMESTAMP	A <i>timestamp</i> is a seven-part value representing a date and time by year, month, day, hour, minute, second, and microsecond, in the range of 0001-01-01-00.00.00.000000000 to 9999-12-31-24.00.00.000000000 with nanosecond precision. Timestamps can also hold timezone information.

Db2 stores values of datetime data types in a special internal format. When you load or retrieve data, Db2 can convert it to or from any of the formats in the following table.

Table 6. Date and time format options

Format name	Abbreviation	Typical date	Typical time
International Standards Organization	ISO	2003-12-25	13.30.05
IBM USA standard	USA	12/25/2003	1:30 PM
IBM European standard	EUR	25.12.2003	13.30.05
Japanese Industrial Standard Christian Era	JIS	2003-12-25	13:30:05

GUIP

Example 1

The following query displays the dates on which all employees were hired, in IBM USA standard form, regardless of the local default:

```
SELECT EMPNO, CHAR(HIREDATE, USA) FROM EMP;
```

Example 2

When you use datetime data types, you can take advantage of Db2 built-in functions that operate specifically on datetime values, and you can specify calculations for datetime values. Assume that a manufacturing company has an objective to ship all customer orders within five days. You define the SHIPDATE and ORDERDATE columns as DATE data types. The company can use datetime data types and the DAYS built-in function to compare the shipment date to the order date. Here is how the company might code the function to generate a list of orders that have exceeded the five-day shipment objective:

```
DAYS(SHIPDATE) - DAYS(ORDERDATE) > 5
```

As a result, programmers do not need to develop, test, and maintain application code to perform complex datetime arithmetic that needs to allow for the number of days in each month.

GUIP

You can use the following sample user-defined functions (which come with Db2) to modify the way dates and times are displayed.

- **ALTDAT** returns the current date in a user-specified format or converts a user-specified date from one format to another.
- **ALTTIME** returns the current time in a user-specified format or converts a user-specified time from one format to another.

At installation time, you can also supply an exit routine to make conversions to and from any local standard.

When loading date or time values from an outside source, Db2 accepts any of the date and time format options that are listed in this information. Db2 converts valid input values to the internal format. For retrieval, a default format is specified at Db2 installation time. You can subsequently override that default by using a precompiler option for all statements in a program or by using the scalar function CHAR for a particular SQL statement and by specifying the format that you want.

Related concepts

[Datetime constants \(Db2 SQL\)](#)

XML data type

The XML data type is used to define columns of a table that store XML values. This pureXML® data type provides the ability to store well-formed XML documents in a database.

All XML data is stored in the database in an internal representation. Character data in this internal representation is in the UTF-8 encoding scheme.

XML values that are stored in an XML column have an internal representation that is not a string and not directly comparable to string values. An XML value can be transformed into a serialized string value that represents the XML document by using the XMLSERIALIZE function or by retrieving the value into an application variable of an XML, string, or binary type. Similarly, a string value that represents an XML document can be transformed to an XML value by using the XMLPARSE function or by storing a value from a string, binary, or XML application data type in an XML column.

The size of an XML value in a Db2 table has no architectural limit. However, serialized XML data that is stored in or retrieved from an XML column is limited to 2 GB.

Validation of an XML document against an XML schema, typically performed during INSERT or UPDATE into an XML column, is supported by the XML schema repository (XSR). If an XML column has an XML type modifier, documents that are inserted into the column or updated in the column are automatically validated against an XML schema.

Large object data types

You can use large object data types to store audio, video, images, and other files that are larger than 32 KB.

The VARCHAR, VARGRAPHIC, and VARBINARY data types have a storage limit of 32 KB. However, applications often need to store large text documents or additional data types such as audio, video, drawings, images, and a combination of text and graphics. For data objects that are larger than 32 KB, you can use the corresponding large object (LOB) data types to store these objects.

Db2 provides three data types to store these data objects as strings of up to 2 GB in size:

Character large objects (CLOBs)

Use the CLOB data type to store SBCS or mixed data, such as documents that contain single character set. Use this data type if your data is larger (or might grow larger) than the VARCHAR data type permits.

Double-byte character large objects (DBCLOBs)

Use the DBCLOB data type to store large amounts of DBCS data, such as documents that use a DBCS character set.

Binary large objects (BLOBs)

Use the BLOB data type to store large amounts of noncharacter data, such as pictures, voice, and mixed media.

If your data does not fit entirely within a data page, you can define one or more columns as LOB columns. An advantage to using LOBs is that you can create user-defined functions that are allowed only on LOB data types.

Related concepts

[Large objects \(LOBs\) \(Db2 SQL\)](#)

Related tasks

[Creating large objects](#)

Defining large objects to Db2 is different than defining other types of data and objects.

ROWID data type

You use the ROWID data type to uniquely identify rows in a Db2 subsystem.

Db2 can generate a value for the column when a row is added, depending on the option that you choose (GENERATED ALWAYS or GENERATED BY DEFAULT) when you define the column. You can use a ROWID column in a table for several reasons.

- You can define a ROWID column to include LOB data in a table.
- You can use direct-row access so that Db2 accesses a row directly through the ROWID column. If an application selects a row from a table that contains a ROWID column, the row ID value implicitly contains the location of the row. If you use that row ID value in the search condition of subsequent SELECT statements, Db2 might be able to navigate directly to the row.

Requirement: To use direct row access, you must use a retrieved ROWID value before you commit. When your application commits, it releases its claim on the table space. After the commit, if a REORG is run on your table space, the physical location of the rows might change.

- You can define a ROWID column with the IMPLICITLY HIDDEN attribute. With *implicitly hidden ROWID columns*, the result of a select statement includes the column only when the select-clause explicitly names the column. That is, the result of an SQL statement that only implies selection of the column, such as when SELECT * is used, does not return the implicitly hidden ROWID column.

Related concepts

[Row ID values \(Db2 SQL\)](#)

[Direct row access \(PRIMARY_ACESSTYPE='D'\) \(Db2 Performance\)](#)

Related tasks

[Specifying direct row access by using row IDs \(Db2 Application programming and SQL\)](#)

Related reference

[ROWID scalar function \(Db2 SQL\)](#)

Distinct types

A *distinct type* is a user-defined data type that is based on existing built-in Db2 data types.

A distinct type is internally the same as a built-in data type, but Db2 treats them as a separate and incompatible type for semantic purposes. Defining your own distinct type ensures that only functions that are explicitly defined on a distinct type can be applied to its instances.

Examples

GUI

Example 1

You might define a US_DOLLAR distinct type that is based on the Db2 DECIMAL data type to identify decimal values that represent United States dollars. The US_DOLLAR distinct type does not automatically acquire the functions and operators of its source type, DECIMAL.

Although you can have different distinct types that are based on the same built-in data types, distinct types have the property of *strong typing*. With this property, you cannot directly compare instances of a distinct type with anything other than another instance of that same type. Strong typing prevents semantically incorrect operations (such as explicit addition of two different currencies) without first undergoing a conversion process. You define which types of operations can occur for instances of a distinct type.

If your company wants to track sales in many countries, you must convert the currency for each country in which you have sales.

Example 2

You can define a distinct type for each country. For example, to create US_DOLLAR types and CANADIAN_DOLLAR types, you can use the following CREATE DISTINCT TYPE statements:

```
CREATE DISTINCT TYPE US_DOLLAR AS DECIMAL (9,2);
CREATE DISTINCT TYPE CANADIAN_DOLLAR AS DECIMAL (9,2);
```

Example 3

After you define distinct types, you can use them in your CREATE TABLE statements:

```
CREATE TABLE US_SALES
  (PRODUCT_ITEM_NO INTEGER,
   MONTH            INTEGER,
   YEAR             INTEGER,
   TOTAL_AMOUNT     US_DOLLAR);
CREATE TABLE CANADIAN_SALES
  (PRODUCT_ITEM_NO INTEGER,
   MONTH            INTEGER,
   YEAR             INTEGER,
   TOTAL_AMOUNT     CANADIAN_DOLLAR);
```

GUIP

User-defined functions support the manipulation of distinct types.

Related concepts

String data types

Db2 supports several types of string data: character strings, graphic strings, and binary strings.

Distinct types (Db2 SQL)

User-defined type assignments (Db2 SQL)

User-defined type comparisons (Db2 SQL)

Null values in table columns

Some columns cannot have a meaningful value in every row. Db2 uses a special value indicator, the *null value*, to stand for an unknown or missing value. A null value is a special value that Db2 interprets to mean that no data is present.

If you do not specify otherwise, Db2 allows any column to contain null values. Users can create rows in the table without providing a value for the column.

Using the NOT NULL clause enables you to disallow null values in the column. Primary keys must be defined as NOT NULL.

Example

GUIP

For example, The table definition for the DEPT table specifies when you can use a null value. Notice that you can use nulls for the MGRNO column only:

```
CREATE TABLE DEPT
  (DEPTNO    CHAR(3)          NOT NULL,
   DEPTNAME  VARCHAR(36)     NOT NULL,
   MGRNO     CHAR(6)          ,
   ADMRDEPT  CHAR(3)          NOT NULL,
   PRIMARY KEY (DEPTNO)
)
IN MYDB.MYTS;
```

GUIP

Before you decide whether to allow nulls for unknown values in a particular column, you must be aware of how nulls affect results of a query:

Nulls in application programs

Nulls do not satisfy any condition in an SQL statement other than the special IS NULL predicate. Db2 sorts null values differently than non-null values. Null values do not behave like other values. For example, if you ask Db2 whether a null value is larger than a given known value, the answer is UNKNOWN. If you then ask Db2 whether a null value is smaller than the same known value, the answer is still UNKNOWN.

If getting a value of UNKNOWN is unacceptable for a particular column, you could define a default value instead. Programmers are familiar with the way default values behave.

Nulls in join operations

Nulls need special handling in join operations. If you perform a join operation on a column that can contain null values, consider using an outer join.

Related concepts

[When to use null or default values](#)

Using a null value is easier and better than using a default value in some situations.

[Data types in Db2 for z/OS \(Db2 SQL\)](#)

[Ways to join data from more than one table](#)

When you want to see information from multiple tables, you can use a SELECT statement. SELECT statements can retrieve and join column values from two or more tables into a single row. The retrieval is based on a specified condition, typically of matching column values.

[Default values for table columns](#)

Db2 defines some default values, and you define others (by using the DEFAULT clause in the CREATE TABLE or ALTER TABLE statement).

[Entity attributes in database design](#)

When you define attributes for entities, you generally work with the data administrator to decide on names, data types, and appropriate values for the attributes.

Related reference

[CREATE TABLE statement \(Db2 SQL\)](#)

[ALTER TABLE statement \(Db2 SQL\)](#)

Default values for table columns

Db2 defines some default values, and you define others (by using the DEFAULT clause in the CREATE TABLE or ALTER TABLE statement).

If a column is defined as NOT NULL WITH DEFAULT or if you do not specify NOT NULL, Db2 stores a default value for a column whenever an insert or load does not provide a value for that column. If a column is defined as NOT NULL, Db2 does not supply a default value.

Default values defined by Db2

Db2 generates a default value for ROWID columns. Db2 also determines default values for columns that users define with NOT NULL WITH DEFAULT, but for which no specific value is specified, as shown in the following table.

Table 7. Db2-defined default values for data types

For columns of...	Data types	Default
Numbers	SMALLINT, INTEGER, BIGINT, DECIMAL, NUMERIC, REAL, DOUBLE, DECFLOAT, or FLOAT	0
Fixed-length strings	CHAR or GRAPHIC	Blanks
	BINARY	Hexadecimal zeros

Table 7. Db2-defined default values for data types (continued)

For columns of...	Data types	Default
Varying-length strings	VARCHAR, CLOB, VARGRAPHIC, DBCLOB, VARBINARY, or BLOB	Empty string
Dates	DATE	CURRENT DATE
Times	TIME	CURRENT TIME
Timestamps	TIMESTAMP	CURRENT TIMESTAMP
ROWIDs	ROWID	Db2-generated

User-defined default values

You can specify a particular default value, such as:

```
DEFAULT 'N/A'
```

When you choose a default value, you must be able to assign it to the data type of the column. For example, all string constants are VARCHAR. You can use a VARCHAR string constant as the default for a CHAR column even though the type isn't an exact match. However, you could not specify a default value of 'NA' for a column with a numeric data type.

In the next example, the columns are defined as CHAR (fixed length). The special registers (USER and CURRENT SQLID) that are referenced contain varying length values.

For example, if you want a record of each user who inserts any row of a table, define the table with two additional columns:

```
PRIMARY_ID    CHAR(8)    WITH DEFAULT USER,
SQL_ID        CHAR(8)    WITH DEFAULT CURRENT SQLID,
```

You can then create a view that omits those columns and allows users to update the view instead of the base table. Db2 then adds, by default, the primary authorization ID and the SQLID of the process.

When you add columns to an existing table, you must define them as nullable or as not null with default. Assume that you add a column to an existing table and specify not null with default. If Db2 reads from the table before you add data to the column, the column values that you retrieve are the default values. With few exceptions, the default values for retrieval are the same as the default values for insert.

Default values for ROWID

Db2 always generates the default values for ROWID columns.

Related concepts

[When to use null or default values](#)

Using a null value is easier and better than using a default value in some situations.

[Authorization and security mechanisms for data access](#)

Authorization is an important part of controlling Db2. The security and authorization mechanisms that control access to Db2 data are both direct and indirect.

[Null values in table columns](#)

Some columns cannot have a meaningful value in every row. Db2 uses a special value indicator, the *null value*, to stand for an unknown or missing value. A null value is a special value that Db2 interprets to mean that no data is present.

Related reference

[CREATE TABLE statement \(Db2 SQL\)](#)

[ALTER TABLE statement \(Db2 SQL\)](#)

Related information

[Implementing Db2 tables \(Db2 Administration Guide\)](#)

When to use null or default values

Using a null value is easier and better than using a default value in some situations.

Suppose that you want to find out the average salary for all employees in a department. The salary column does not always need to contain a meaningful value, so you can choose between the following options:

- Allowing null values for the SALARY column
- Using a non-null default value (such as, 0)

By allowing null values, you can formulate the query easily, and Db2 provides the average of all known or recorded salaries. The calculation does not include the rows that contain null values. In the second case, you probably get a misleading answer unless you know the nonnull default value for unknown salaries and formulate your query accordingly.

The following figure shows two scenarios. The table in the figure excludes salary data for employee number 200440, because the company just hired this employee and has not yet determined the salary. The calculation of the average salary for department E21 varies, depending on whether you use null values or nonnull default values.

- The left side of the figure assumes that you use null values. In this case, the calculation of average salary for department E21 includes only the three employees (000320, 000330, and 200340) for whom salary data is available.
- The right side of the figure assumes that you use a nonnull default value of zero (0). In this case, the calculation of average salary for department E21 includes all four employees, although valid salary information is available for only three employees.

As you can see, only the use of a null value results in an accurate average salary for department E21.

```
SELECT DEPT, AVG (SALARY)
FROM EMP
GROUP BY DEPT;
```

With null value

EMPNO	DEPT	SALARY
000320	E21	19950.00
000330	E21	25370.00
200340	E21	23840.00
200440	E21	-----

↓

DEPT	AVG (SALARY)
====	=====
.	.
.	.
E21	23053.33
	(Average of nonnull salaries)

With default value of 0

EMPNO	DEPT	SALARY
000320	E21	19950.00
000330	E21	25370.00
200340	E21	23840.00
200440	E21	0.00

↓

DEPT	AVG (SALARY)
====	=====
.	.
E21	17290.00

Figure 4. When nulls are preferable to default values

Null values are distinct in most situations so that two null values are not equal to each other.

Example

GUIP

The following example shows how to compare two columns to see if they are equal or if both columns are null:

```
WHERE E1.DEPT IS NOT DISTINCT FROM E2.DEPT
```

GUIP

Related concepts

[Null values in table columns](#)

Some columns cannot have a meaningful value in every row. Db2 uses a special value indicator, the *null value*, to stand for an unknown or missing value. A null value is a special value that Db2 interprets to mean that no data is present.

[Default values for table columns](#)

Db2 defines some default values, and you define others (by using the DEFAULT clause in the CREATE TABLE or ALTER TABLE statement).

[Assignment and comparison \(Db2 SQL\)](#)

Check constraints for column values

You can use check constraints to ensure that only values from the domain for the column or attribute are allowed.

By using check constraints, programmers can avoid developing, testing, and maintaining application code that performs these checks.

You can choose to define check constraints by using the SQL CREATE TABLE statement or ALTER TABLE statement. For example, you might want to ensure that each value in the SALARY column of the EMP table contains more than a certain minimum amount.

Db2 enforces a check constraint by applying the relevant search condition to each row that is inserted, updated, or loaded. An error occurs if the result of the search condition is false for any row.

Check constraints that insert table rows

When you use the INSERT statement or the MERGE statement to add a row to a table, Db2 automatically enforces all check constraints for that table. If the data violates any check constraint that is defined on that table, Db2 does not insert the row.

GUIP

Example 1

Assume that the NEWEMP table has the following two check constraints:

- Employees cannot receive a commission that is greater than their salary.
- Department numbers must be between '001' to '100,' inclusive.

Consider this INSERT statement, which adds an employee who has a salary of \$65,000 and a commission of \$6000:

```
INSERT INTO NEWEMP  
(EMPNO, FIRSTNAME, LASTNAME, DEPT, JOB, SALARY, COMM)  
VALUES ('100125', 'MARY', 'SMITH', '055', 'SLS', 65000.00, 6000.00);
```

The INSERT statement in this example succeeds because it satisfies both constraints.

Example 2

Consider this INSERT statement:

```
INSERT INTO NEWEMP
(EMPNO, FIRSTNAME, LASTNAME, DEPT, JOB, SALARY, COMM)
VALUES ('120026', 'JOHN', 'SMITH', '055', 'DES', 5000.00, 55000.00 );
```

The INSERT statement in this example fails because the \$55,000 commission is higher than the \$5,000 salary. This INSERT statement violates a check constraint on NEWEMP.



Check constraints that update tables

Db2 automatically enforces all check constraints for a table when you use the UPDATE statement or the MERGE statement to change a row in the table. If the intended update violates any check constraint that is defined on that table, Db2 does not update the row.



Example 1

Assume that the NEWEMP table has the following two check constraints:

- Employees cannot receive a commission that is greater than their salary.
- Department numbers must be between '001' to '100,' inclusive.

Consider this UPDATE statement:

```
UPDATE NEWEMP
SET DEPT = '011'
WHERE FIRSTNAME = 'MARY' AND LASTNAME= 'SMITH';
```

This update succeeds because it satisfies the constraints that are defined on the NEWEMP table.

Example 2

Consider this UPDATE statement:

```
UPDATE NEWEMP
SET DEPT = '166'
WHERE FIRSTNAME = 'MARY' AND LASTNAME= 'SMITH';
```

This update fails because the value of DEPT is '166,' which violates the check constraint on NEWEMP that DEPT values must be between '001' and '100.'



Related concepts

[Check constraints \(Db2 Application programming and SQL\)](#)

Related tasks

[Adding or dropping table check constraints \(Db2 Administration Guide\)](#)

Related reference

[CREATE TABLE statement \(Db2 SQL\)](#)

Db2 indexes

An *index* is an ordered set of pointers to rows of a table. Db2 can use indexes to improve performance and ensure uniqueness. Understanding the structure of Db2 indexes can help you achieve the best performance for your system.

Conceptually, you can think of an index to the rows of a Db2 table like you think of an index to the pages of a book. Each index is based on the values of data in one or more columns of a table.

The main purpose of an index is to improve performance for access to the data. In most cases, access to data is faster with an index than with a scan of the data. For example, you can create an index on the DEPTNO column of the sample DEPT table so that Db2 can easily locate a specific department and avoid reading through each row of, or *scanning*, the table.

An index is stored separately from the data in the table. Each index is physically stored in its own index space. When you define an index by using the CREATE INDEX statement, Db2 builds this structure and maintains it automatically. However, you can perform necessary maintenance such as reorganizing it or recovering the index.

Another purpose of an index is to ensure uniqueness. For example, a unique index on the employee table ensures that no two employees have the same employee number.

In most cases, the users of an index are unaware that it is being used because Db2 decides whether to use the index to access the table.

Db2 supports simple indexes and extended indexes. An extended index is one of the following objects:

- An expression-based index
- A spatial index
- An XML index

Be aware that indexes have both benefits and disadvantages. A greater number of indexes can simultaneously improve the access performance of a particular transaction and require additional processing for inserting, updating, and deleting index keys.

Related concepts

Creation of indexes

Indexes provide efficient access to table data, but can require additional processing when you modify data in a table.

Related reference

CREATE INDEX statement (Db2 SQL)

Db2 views

A *view* is an alternative way of representing data that exists in one or more tables. A view can include all or some of the columns from one or more base tables.

A view is a named specification of a result table. Conceptually, creating a view is somewhat like using binoculars. You might look through binoculars to see an entire landscape or to look at a specific image within the landscape, such as a tree.

You can create Db2 views that achieve the following goals:

- Combines data from different base tables
- Based on other views or on a combination of views and tables
- Omit certain data, thereby shielding some table data from users

In fact, these are common underlying reasons to use a view. Combining information from base tables and views simplifies retrieving data for a user, and limiting the data that a user can see is useful for security. You can use views for a number of different purposes. A view can:

- Control access to a table
- Make data easier to use
- Simplify authorization by granting access to a view without granting access to the table
- Show only portions of data in the table
- Show summary data for a given table
- Combine two or more tables in meaningful ways
- Show only the selected rows that are pertinent to the process that uses the view

How to access data in Db2 views

To retrieve or access information from a view, you use views like you use base tables. You can use a SELECT statement to show the information from the view. The SELECT statement can name other views

and tables, and it can use the WHERE, GROUP BY, and HAVING clauses. It cannot use the ORDER BY clause or name a host variable.

Whether a view can be used in an insert, update, or delete operation depends on its definition. For example, if a view includes a foreign key of its base table, INSERT and UPDATE operations that use the view are subject to the same referential constraint as the base table. Likewise, if the base table of a view is a parent table, DELETE operations that use the view are subject to the same rules as DELETE operations on the base table. *Read-only* views cannot be used for insert, update, and delete operations.

How to create Db2 views

To define a view, you use the CREATE VIEW statement and assign a name (up to 128 characters in length) to the view. Specifying the view in other SQL statements is effectively like running an SQL SELECT statement. At any time, the view consists of the rows that would result from the SELECT statement that it contains. You can think of a view as having columns and rows just like the base table on which the view is defined.

You also can specify a period specification for a view, subject to certain restrictions.

Examples of creating Db2 views

GUI

Example 1

The following figure shows a view of the EMP table that omits sensitive employee information and renames some of the columns.

Base table, **EMP**:

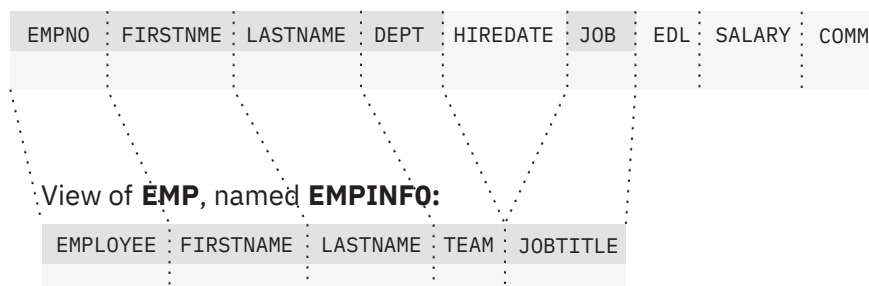


Figure 5. A view of the EMP table

Figure note: The EMPINFO view represents a table that includes columns named EMPLOYEE, FIRSTNAME, LASTNAME, TEAM, and JOBTITLE. The data in the view comes from the columns EMPNO, FIRSTNAME, LASTNAME, DEPT, and JOB of the EMP table.

Example 2

The following CREATE VIEW statement defines the EMPINFO view that is shown in the preceding figure:

```
CREATE VIEW EMPINFO (EMPLOYEE, FIRSTNAME, LASTNAME, TEAM, JOBTITLE)
AS SELECT EMPNO, FIRSTNAME, LASTNAME, DEPT, JOB
FROM EMP;
```

When you define a view, Db2 stores the definition of the view in the Db2 catalog. However, Db2 does not store any data for the view itself, because the data exists in the base table or tables.

Example 3

You can narrow the scope of the EMPINFO view by limiting the content to a subset of rows and columns that includes departments A00 and C01 only:

```
CREATE VIEW EMPINFO (EMPLOYEE, FIRSTNAME, LASTNAME, TEAM, JOBTITLE)
AS SELECT EMPNO, FIRSTNAME, LASTNAME, WORKDEPT, JOB
FROM EMP
WHERE WORKDEPT = 'A00' OR WORKDEPT = 'C01';
```

GUPI

In general, a view inherits the attributes of the object from which it is derived. Columns that are added to the tables after the view is defined on those tables do not appear in the view.

Restriction: You cannot create an index for a view. In addition, you cannot create any form of a key or a constraint (referential or otherwise) on a view. Such indexes, keys, or constraints must be built on the tables that the view references.

Related concepts

[Creation of views](#)

When you design your database, you might need to give users access to only certain pieces of data. You can give users controlled access by designing and using views.

Related reference

[Employee table \(DSN8C10.EMP\)](#)

The sample employee table identifies all employees by an employee number and lists basic personnel information.

[CREATE VIEW statement \(Db2 SQL\)](#)

[Views on the sample tables](#)

Db2 creates a number of views on the sample tables for use in the sample applications.

Related information

[Implementing Db2 views \(Db2 Administration Guide\)](#)

Aliases

An *alias* is a substitute for the three-part name of a table or view.

An alias can be defined at a local server and can refer to a table or view that is at the current server or a remote server. The alias name can be used wherever the table name or view name can be used to refer to the table or view in an SQL statement.

Suppose that data is occasionally moved from one Db2 subsystem to another. Ideally, users who query that data are not affected when this activity occurs. They always want to log on to the same system and access the same table or view, regardless of where the data resides. You can achieve this result by using an alias for an object name.

An alias can be a maximum of 128 characters, qualified by an owner ID. You use the CREATE ALIAS and DROP ALIAS statements to manage aliases.

GUPI For example, assume that you create an alias with the following statement:

```
CREATE ALIAS TESTTAB FOR USIBMST0DB22.IDEMP01.EMP;
```

If a user with the ID JONES dynamically creates the alias, JONES owns the alias, and you query the table like this:

```
SELECT SUM(SALARY), SUM(BONUS), SUM(COMM)
FROM JONES.TESTTAB;
```

GUPI

The object for which you are defining an alias does not need to exist when you execute the CREATE ALIAS statement. However, the object must exist when a statement that refers to the alias executes.

When you want an application to access a server other than the server that is specified by a location name, you do not need to change the location name. Instead, you can use a location alias to override the location name that an application uses to access a server. As a result, a Db2 for z/OS requester can access multiple Db2 databases that have the same name but different network addresses. Location aliases allow easier migration to a Db2 server and minimize application changes.

After you create an alias, anyone who has authority over the object that the alias is referencing can use that alias. A user does not need a separate privilege to use the alias.

Related reference

[CREATE ALIAS statement \(Db2 SQL\)](#)

Triggers

A *trigger* defines a set of actions that are executed when a delete, insert, or update operation occurs on a specified table or view. When such an operation is executed, the trigger is said to be *activated*.

Db2 supports two types of triggers, basic and advanced:

- *Basic triggers* support a limited set of SQL statements, and require the MODE DB2SQL clause on the CREATE TRIGGER statement. You can identify basic triggers by querying the SYSIBM.SYSTRIGGERS catalog table. Blank values in the SQLPL column identify basic triggers. For more information, see [CREATE TRIGGER statement \(basic trigger\) \(Db2 SQL\)](#).
- *Advanced triggers* support a larger set of SQL statements, including SQL procedure language (SQL PL). A CREATE TRIGGER statement for an advanced trigger must not specify the MODE DB2SQL clause. Advanced triggers are supported at application compatibility level V12R1M500 or higher. You can identify advanced triggers by querying the SYSIBM.SYSTRIGGERS catalog table. 'Y' values in the SQLPL column identify advanced triggers. For more information about advanced triggers, see [CREATE TRIGGER statement \(advanced trigger\) \(Db2 SQL\)](#).

For a detailed comparison of the types, see [“Basic and advanced triggers” on page 38](#).

You can use triggers along with referential constraints and check constraints to enforce data integrity rules. Triggers are more powerful than constraints because you can use them to do the following things:

- Update other tables
- Automatically generate or transform values for inserted or updated rows
- Invoke functions that perform operations both inside and outside of Db2

For example, assume that you need to prevent an update to a column when a new value exceeds a certain amount. Instead of preventing the update, you can use a trigger. The trigger can substitute a valid value and invoke a procedure that sends a notice to an administrator about the attempted invalid update.

You define triggers with the CREATE TRIGGER statement.

Triggers move the business rule application logic into the database, which results in faster application development and easier maintenance. The business rule is no longer repeated in several applications, and the rule is centralized to the trigger. For example, Db2 can check the validity of the changes that any application makes to a salary column, and you are not required to change application programs when the logic changes.

There are a number of criteria that are defined when creating a trigger, which are used to determine when a trigger should be activated.

- The *subject table* (also known as the triggering table) defines the table or view for which the trigger is defined.
- The *trigger event* defines a specific SQL operation that modifies the subject table. The operation could be a delete, insert, or update.
- The *trigger activation time* defines whether the trigger should be activated before or after the trigger event is performed on the subject table.

The statement that causes a trigger to be activated includes a set of *affected rows*. These are the rows of the subject table that are being deleted, inserted or updated. The *trigger granularity* defines whether the actions of the trigger are to be performed once for the statement, or once for each of the rows in the set of affected rows.

The *trigger action* consists of an optional search condition and a set of SQL statements that are executed whenever the trigger is activated. The SQL statements are only executed if no search condition is specified, or the specified search condition evaluates to true.

The triggered action can refer to the values in the set of affected rows. This is supported through the use of *transition variables*. Transition variables use the names of the columns in the subject table, qualified by a specified name that identifies whether the reference is to the old value (prior to the update) or the new value (after the update). The new value can also be changed using an assignment in a before update or insert trigger.

Another means of referring to the values in the set of affected rows is by using *transition tables*. Transition tables also use the names of the columns of the subject table, but have a name specified that allows the complete set of affected rows to be treated as a table. Transition tables can only be used in after triggers, and cannot be modified. Separate transition tables can be defined for old and new values.

Multiple triggers can be specified for a combination of table, event, or activation time. The order in which the triggers are activated is the same as the order in which they were created. Thus, the most recently created trigger is the last trigger that is activated.

The activation of a trigger might cause *trigger cascading*. This is the result of the activation of one trigger that executes SQL statements that cause the activation of other triggers, or even the same trigger again. The triggered actions might also cause updates as a result of the original modification, which might result in the activation of additional triggers. With trigger cascading, a significant chain of triggers might be activated, causing significant change to the database as a result of a single delete, insert or update statement.

The actions that are performed in the trigger are considered to be part of the operation that caused the trigger to be executed.

- The database manager ensures that the operation and the triggers that are executed as a result of that operation either all complete or are all backed out. Operations that occurred prior to the triggering operation are not affected.
- The database manager effectively checks all constraints (except for a constraint with a RESTRICT delete rule) after the operation and the associated triggers have been executed.

Basic and advanced triggers

Basic and advanced triggers offer different functionality. See the corresponding syntax diagrams for the CREATE TRIGGER (basic) and CREATE TRIGGER (advanced) SQL statements for more information. The following table identifies some behavioral differences between the two types of triggers.

Table 8. Behavioral differences of basic and advanced triggers

Behavior	Basic Trigger	Advanced Trigger
CREATE or ALTER TRIGGER statement invocation	CREATE or ALTER TRIGGER (basic) statement can be embedded in an application program or issued interactively. It is an executable statement that can be dynamically prepared only if DYNAMICRULES run behavior is implicitly or explicitly specified.	CREATE or ALTER TRIGGER (advanced) statement can be issued interactively. It is an executable statement that can be dynamically prepared only if DYNAMICRULES run behavior is implicitly or explicitly specified.
Authorization requirement	The privilege set must include SYSADM or the SELECT privilege on the table or view on which the trigger is defined, if the REFERENCING clause is included in the trigger definition.	The privilege set must include SYSADM or the SELECT privilege for the triggering table or view on which the trigger is defined.

Table 8. Behavioral differences of basic and advanced triggers (continued)

Behavior	Basic Trigger	Advanced Trigger
Default encoding scheme	The default encoding scheme is Unicode.	The default encoding scheme is determined from the value of the DEFAULT APPLICATION ENCODING SCHEME field on the installation panel DSNTIPF.
Null attribute for transition variables	A transition variable is defined with the same null attribute as the column it is associated with.	All transition variables are nullable.
Unhandled warnings at completion of a trigger	Unhandled warnings are not returned to the statement that activated a trigger. When processing in a trigger completes with a warning, the warning is not returned to the statement that activated the trigger.	Unhandled warnings are returned to the statement that activated a trigger. When processing in a trigger completes with a warning, the warning is returned to the statement that activated the trigger.
Transition variables passed as OUT or INOUT arguments to a procedure	Changes to transition variables in a procedure are not visible on return to the invoking environment, and changes to transition variables (for an AFTER trigger) are not disallowed.	Changes to transition variables in a procedure are visible on return to the invoking environment, and changes to transition variables (for an AFTER trigger) are disallowed.
Stand-alone <i>fullselect</i> and VALUES statements	Supported.	Not supported. Use a SELECT INTO statement or VALUES INTO statement instead.

Related concepts

Creation of triggers

You can use triggers to define and enforce business rules that involve different states of the data. Triggers automatically execute a set of SQL statements whenever a specified event occurs. These statements validate and edit database changes, read and modify the database, and invoke functions that perform various operations.

User-defined types

A *user-defined data type* is a data type that is defined to the database using a CREATE statement.

A user-defined data type is a distinct type or an array type.

A *distinct type* is a user-defined type that shares its internal representation with a built-in data type (its source type), but is considered to be a separate and incompatible data type for most operations. A distinct type is created with an SQL CREATE TYPE (distinct) statement. A distinct type can be used to define a column of a table, or a parameter of a routine.

An *array type* is a user-defined data type that consists of an ordered set of elements of a single built-in data type. Elements can be accessed and modified by their index position. An array type is created with an SQL CREATE TYPE (array) statement. An array type can be used as a parameter of an SQL routine, as a variable in an SQL routine, and as a global variable.

Related concepts

[Array types and values \(Db2 SQL\)](#)

[Distinct types \(Db2 SQL\)](#)

Related reference

[CREATE TYPE statement \(Db2 SQL\)](#)

Routines in Db2 for z/OS: functions and procedures

A *routine* is an executable SQL object. The two types of routines in Db2 for z/OS are functions and stored procedures.

Functions

A *function* is a routine that can be invoked from within other SQL statements and that returns a value or a table.

Functions are classified as either SQL functions or external functions. SQL functions are written using SQL statements, including SQL procedural language (SQL PL). External functions reference a host language program. The host language program can contain SQL, but does not require SQL.

You define functions by using the CREATE FUNCTION statement. You can classify functions as built-in functions, user-defined functions, or cast functions that are generated for distinct types. Functions can also be classified as aggregate, scalar, or table functions, depending on the input data values, result values, and the context in which they can be invoked.

For more information, see [Functions \(Db2 SQL\)](#).

Procedures

A procedure, also known as a stored procedure, is a routine that you can call to perform operations that can include SQL statements.

Procedures are classified as either SQL procedures or external procedures. SQL procedures contain only SQL statements, including SQL procedural language (SQL PL). External procedures reference a host language program that might or might not contain SQL statements.

Db2 for z/OS supports the following types of procedures:

Native SQL procedures

The procedure body is written exclusively in SQL statements, including SQL procedural language (SQL PL) statements. The procedure body is contained and specified in the procedure definition along with various attributes of the procedure. A package is generated for a native SQL procedure. It contains the procedure body, including control statements. It might sometimes also include statements generated by Db2. Each time that the procedure is invoked, the package executes one or more times.

All SQL procedures that are created with a CREATE PROCEDURE statement that does not specify the FENCED or EXTERNAL options are native SQL procedures. More capabilities are supported for native SQL procedures, they usually perform better than external SQL procedures, and no associated C program is generated for them.

For more information, see [Creating native SQL procedures \(Db2 Application programming and SQL\)](#).

External stored procedures

The procedure body is an external program that is written in a programming language such as C, C++, COBOL, or Java and it can contain SQL statements. The source code for an external stored procedure is separate from the procedure definition and is bound into a package. The name of the external executable is specified as part of the procedure definition along with various attributes of the procedure. All programs must be designed to run using Language Environment. Your COBOL and C++ stored procedures can contain object-oriented extensions. Each time that the stored procedure is invoked, the logic in the procedure controls whether the package executes and how many times.

For more information, see [Creating external stored procedures \(Db2 Application programming and SQL\)](#).

External SQL procedures (deprecated)

The procedure body is written exclusively in SQL statements, including SQL procedural language (SQL PL) statements. The procedure body is specified in the procedure definition along with various

attributes of the procedure. A C program and an associated package are generated for an external SQL procedure. It contains the procedure body, including control statements. It might sometimes also include statements generated by Db2. Each time that the procedure is invoked, the package executes one or more times.

Native SQL procedures are more fully supported, easier to maintain, and typically perform better than external SQL procedures, which are deprecated.

For more information, see [Creating external SQL procedures \(deprecated\) \(Db2 Application programming and SQL\)](#).

SQL control statements are supported in SQL procedures. Control statements are SQL statements that allow SQL to be used in a manner similar to writing a program in a structured programming language. SQL control statements provide the capability to control the logic flow, declare and set variables, and handle warnings and exceptions. Some SQL control statements include other nested SQL statements.

SQL procedures provide the same benefits as procedures in a host language. That is, a common piece of code needs to be written and maintained only once and can be called from several programs.

SQL procedures provide additional benefits when they contain SQL statements. In this case, SQL procedures can reduce or eliminate network delays that are associated with communication between the client and server and between each SQL statement. SQL procedures can improve security by providing a user the ability to invoke only a procedure instead of providing them with the ability to execute the SQL that the procedure contains.

You define procedures by using the CREATE PROCEDURE statement.

Related concepts

[Functions \(Db2 SQL\)](#)

[Creation of user-defined functions](#)

You can create your own functions in Db2 to simplify your queries.

[Use of an application program as a stored procedure](#)

A stored procedure is a compiled program that can execute SQL statements.

[External stored procedures \(Db2 Application programming and SQL\)](#)

[SQL control statements for external SQL procedures \(Db2 SQL\)](#)

[SQL procedural language \(SQL PL\) \(Db2 SQL\)](#)

Related tasks

[Creating a user-defined function \(Db2 Application programming and SQL\)](#)

[Implementing Db2 stored procedures \(Stored procedures provided by Db2\)](#)

Related reference

[Procedures that are supplied with Db2 \(Db2 SQL\)](#)

Sequences

A *sequence* is a stored object that simply generates a sequence of numbers in a monotonically ascending (or descending) order. A sequence provides a way to have Db2 automatically generate unique integer primary keys and to coordinate keys across multiple rows and tables.

A sequence can be used to exploit parallelization, instead of programmatically generating unique numbers by locking the most recently used value and then incrementing it.

Sequences are ideally suited to the task of generating unique key values. One sequence can be used for many tables, or a separate sequence can be created for each table requiring generated keys. A sequence has the following properties:

- Guaranteed, unique values, assuming that the sequence is not reset and does not allow the values to cycle
- Monotonically increasing or decreasing values within a defined range
- Can increment with a value other than 1 between consecutive values (the default is 1).

- Recoverable. If Db2 should fail, the sequence is reconstructed from the logs so that Db2 guarantees that unique sequence values continue to be generated across a Db2 failure.

Values for a given sequence are automatically generated by Db2. Use of Db2 sequences avoids the performance bottleneck that results when an application implements sequences outside the database. The counter for the sequence is incremented (or decremented) independently of the transaction. In some cases, gaps can be introduced in a sequence. A gap can occur when a given transaction increments a sequence two times. The transaction might see a gap in the two numbers that are generated because there can be other transactions concurrently incrementing the same sequence. A user might not realize that other users are drawing from the same sequence. Furthermore, it is possible that a given sequence can appear to have generated gaps in the numbers, because a transaction that might have generated a sequence number might have rolled back or the Db2 subsystem might have failed. Updating a sequence is not part of a transaction's unit of recovery.

A sequence is created with a CREATE SEQUENCE statement. A sequence can be referenced using a *sequence-reference*. A sequence reference can appear most places that an expression can appear. A sequence reference can specify whether the value to be returned is a newly generated value, or the previously generated value.

Although there are similarities, a sequence is different than an identity column. A sequence is an object, whereas an identity column is a part of a table. A sequence can be used with multiple tables, but an identity column is tied to a single table.

Related reference

[CREATE SEQUENCE statement \(Db2 SQL\)](#)

Db2 system objects

Unlike the Db2 data structures that users create and access, Db2 controls and accesses system objects.

Db2 has a comprehensive infrastructure that enables it to provide data integrity, performance, and the ability to recover user data. In addition, Parallel Sysplex® data sharing uses shared system objects.

Related concepts

[Db2 database objects overview](#)

In Db2 for z/OS, you use database objects, such as tables, table spaces, indexes, index spaces, keys, views, and databases to organize and access your data.

Db2 catalog

Db2 maintains a set of tables that contain information about the data that Db2 controls. These tables are collectively known as the *Db2 catalog*. The Db2 catalog consists of tables of data about everything defined to the Db2 system, including table spaces, indexes, tables, copies of table spaces and indexes, and storage groups.

The catalog tables contain information about Db2 objects such as tables, views, and indexes. When you create, alter, or drop an object, Db2 inserts, updates, or deletes rows of the catalog that describe the object.

The DSNDB06 system database contains the Db2 catalog.

When you create, alter, or drop any structure, Db2 inserts, updates, or deletes rows in the catalog table that describe the structure and how it relates to other structures. For example, SYSIBM.SYSTABLES is one catalog table that records information when a table is created. Db2 inserts a row into SYSIBM.SYSTABLES that includes the table name, its owner, its creator, and the name of its table space and its database.

To understand the role of the catalog, consider what happens when the EMP table is created. Db2 records the following data:

Table information

To record the table name and the name of its owner, its creator, its type, the name of its table space, and the name of its database, Db2 inserts a row into the catalog.

Column information

To record information about each column of the table, Db2 inserts the name of the table to which the column belongs, its length, its data type, and its sequence number by inserting a row into the catalog for each column of the table.

Authorization information

To record that the owner of the table has authorization to create the table, Db2 inserts a row into the catalog.

Tables in the catalog are like any other database tables with respect to retrieval. If you have authorization, you can use SQL statements to look at data in the catalog tables in the same way that you retrieve data from any other table in the Db2 database. Db2 ensures that the catalog contains accurate object descriptions. If you are authorized to access the specific tables or views on the catalog, you can use SELECT on the catalog. You can use INSERT, UPDATE, and DELETE statements on updatable catalog tables, but you cannot use TRUNCATE or MERGE on the catalog.

The *communications database* (CDB) is part of the Db2 catalog. The CDB consists of a set of tables that establish conversations with remote database management systems (DBMSs). The distributed data facility (DDF) uses the CDB to send and receive distributed data requests.

Tip: For best results, check the consistency of the Db2 catalog and directory regularly, even outside of the migration process. For detailed instructions, see [Verify the integrity of Db2 table spaces](#) and [Check for consistency between catalog tables](#).

Related reference

[Db2 catalog tables \(Db2 SQL\)](#)

Db2 directory

The Db2 directory contains information that Db2 uses during normal operation.

You can use SQL to retrieve data only from the directory tables that are listed in [Db2 directory tables \(Db2 SQL\)](#). You can submit queries against the Db2 catalog for Db2 directory objects.

The directory consists of a set of Db2 tables that are stored in table spaces in system database DSND01. Each of the table spaces that are listed in the following table is contained in a VSAM linear data set.

Table 9. Directory table spaces

Table space name	Description
SCT02	Contains the internal form of SQL statements that are contained in an application. If you bound a plan with SQL statements in a prior release, Db2 created a structure in SCT02.
SPT01 Skeleton package	Contains the internal form of SQL statements that are contained in a package.
SYSSPUXA	Contains the contents of a package selection.
SYSSPUBX	Contains the contents of a package explain block.
SYSLGRNX Log range	Tracks the opening and closing of table spaces, indexes, or partitions. By tracking this information and associating it with relative byte addresses (RBAs) as contained in the Db2 log, Db2 can reduce recovery time by reducing the amount of log that must be scanned for a particular table space, index, or partition.

Table 9. Directory table spaces (continued)

Table space name	Description
SYSUTILX System utilities	Contains a row for every utility job that is running. The row persists until the utility is finished. If the utility terminates without completing, Db2 uses the information in the row when you restart the utility.
DBD01 Database descriptor (DBD)	Contains internal information, called <i>database descriptors</i> (DBDs), about the databases that exist within the Db2 subsystem. Each database has exactly one corresponding DBD that describes the database, table spaces, tables, table check constraints, indexes, and referential relationships. A DBD also contains other information about accessing tables in the database. Db2 creates and updates DBDs whenever their corresponding databases are created or updated.
SYSDBDXA	Contains the contents of a DBD section.

Tip: For best results, check the consistency of the Db2 catalog and directory regularly, even outside of the migration process. For detailed instructions, see [Verify the integrity of Db2 table spaces](#) and [Check for consistency between catalog tables](#).

Active and archive logs

Db2 records all data changes and other significant events in a log.

If you keep these logs, Db2 can re-create those changes for you in the event of a failure or roll the changes back to a previous point in time.

Db2 writes each log record to a disk data set called the *active log*. When the active log is full, Db2 copies the contents of the active log to a disk or magnetic tape data set called the *archive log*.

You can choose either single logging or dual logging.

- A single active log contains up to 93 active log data sets.
- With dual logging, the active log has twice the capacity for active log data sets, because two identical copies of the log records are kept.

Each Db2 subsystem manages multiple active logs and archive logs. The following facts are true about each Db2 active log:

- Each log can be duplexed to ensure high availability.
- Each active log data set is a VSAM linear data set (LDS).
- Db2 supports striped active log data sets, however striping is generally unnecessary with the latest devices, and is not recommended in most cases.

Important: Do not use striped active logs for disaster recovery.

Related tasks

[Managing the log and the bootstrap data set \(Db2 Administration Guide\)](#)

[Improving Db2 log performance \(Db2 Performance\)](#)

Related information

[Reading log records \(Db2 Administration Guide\)](#)

Bootstrap data set

The *bootstrap data set* (BSDS) is a VSAM key-sequenced data set (KSDS). This KSDS contains information that is critical to Db2, such as the names of the logs. Db2 uses information in the BSDS for system restarts and for any activity that requires reading the log.

Specifically, the BSDS contains:

- An inventory of all active and archive log data sets that are known to Db2. Db2 uses this information to track the active and archive log data sets. Db2 also uses this information to locate log records to satisfy log read requests during normal Db2 system activity and during restart and recovery processing.
- A wrap-around inventory of all recent Db2 checkpoint activity. Db2 uses this information during restart processing.
- The distributed data facility (DDF) communication record, which contains information that is necessary to use Db2 as a distributed server or requester.
- Information about buffer pools.

Because the BSDS is essential to recovery in the event of subsystem failure, during installation Db2 automatically creates two copies of the BSDS and, if space permits, places them on separate volumes.

The BSDS can be duplexed to ensure availability.

For instructions, see [Convert BSDS records to the extended 10-byte format: DSNTIJCB \(Db2 Installation and Migration\)](#).

Related tasks

[Managing the log and the bootstrap data set \(Db2 Administration Guide\)](#)

[Convert BSDS records to the extended 10-byte format: DSNTIJCB \(Db2 Installation and Migration\)](#)

Buffer pools

Buffer pools are areas of virtual storage that temporarily store pages of table spaces or indexes.

When an application program accesses a row of a table, Db2 places the page that contains that row in a buffer. Access to data in this temporary storage is faster than accessing data on a disk. If the required data is already in a buffer, the application program does not need to wait for it to be retrieved from disk, so the time and cost of retrieving the page is reduced.

Buffer pools require monitoring and tuning. Buffer pool sizes are critical to the performance characteristics of an application or group of applications that access data in those buffer pools.

You can specify default buffer pools for user data and for indexes. A special type of buffer pool that is used only in Parallel Sysplex data sharing is the *group buffer pool*, which resides in the coupling facility. Group buffer pools reside in a special PR/SM LPAR logical partition called a *coupling facility*, which enables several Db2 subsystems to share information and control the coherency of data.

Buffer pools reside in the database services address space (*ssnmDBM1*). The maximum size of a buffer pool is 16 TB.

Related concepts

The role of buffer pools in caching data

Buffer pools are a key element of Db2 performance, and help you to avoid delays when retrieving data.

Related tasks

[Tuning database buffer pools \(Db2 Performance\)](#)

[Calculating buffer pool size \(Db2 Installation and Migration\)](#)

[Enabling automatic buffer pool size management \(Db2 Performance\)](#)

Data definition control support database

The *data definition control support* (DDCS) database refers to a user-maintained collection of tables that are used by data definition control support to restrict the submission of specific Db2 DDL (data definition language) statements to selected application identifiers (plans or collections of packages).

This database is automatically created during installation. After this database is created, you must populate the tables to use this facility. The system name for this database is DSNRGFDB.

The resource limit facility

The *resource limit facility* (sometimes abbreviated RLF) enables you to control the amount of processor resources that are used by SQL statements.

GUI

Resource limits apply to the following types of SQL statements:

- SELECT
- INSERT
- UPDATE
- MERGE
- TRUNCATE
- DELETE

You can specify resource limits for dynamic SQL statements and static SQL statements. Resource limits apply to SQL statement regardless of whether they are issued locally or remotely. The resource limit facility does not apply to primary or secondary authorization IDs that have installation SYSADM or installation SYSOPR authority.

You can establish a single limit for all users, different limits for individual users, or both. You can choose to have these limits applied before the statement is executed through *predictive governing*, or while a statement is running, through *reactive governing*. You can also use reactive and predictive governing in combination. You define these limits in one or more *resource limit tables*, named DSNRLSTxx or DSNRLMTxx, depending on the monitoring purpose. **GUI**

Related concepts

[Resource limit facility controls \(Db2 Performance\)](#)

Related tasks

[Setting limits for system resource usage by using the resource limit facility \(Db2 Performance\)](#)

Related reference

[Resource limit facility tables \(Db2 Performance\)](#)

[-START RLIMIT command \(Db2\) \(Db2 Commands\)](#)

Work file database

Use the *work file database* as storage for processing SQL statements that require working space, such as that required for a sort.

The work file database is used as storage for Db2 work files for processing SQL statements that require working space (such as the space that is required for a sort), and as storage for created global temporary tables and declared global temporary tables.

Db2 creates a work file database and some table spaces in it for you at installation time. You can create additional work file table spaces at any time. You can drop, re-create, and alter the work file database or the table spaces in it, or both, at any time.

In a non-data-sharing environment, the work file database is named DSNDB07. In a data sharing environment, each Db2 member in the data sharing group has its own work file database.

You can also use the work file database for all temporary tables.

Related concepts

[How sort work files are allocated \(Db2 Performance\)](#)

Related reference

[SEPARATE WORK FILES field \(WFDBSEP subsystem parameter\) \(Db2 Installation and Migration\)](#)

Subsystem parameters

Subsystem parameters are settings that apply to a Db2 for z/OS subsystem. For example, the AUTH subsystem parameter controls whether Db2 checks authorizations. Many users colloquially refer to subsystem parameters as *zparms*.

Generally, system administrators set the value of subsystem parameters when they install or migrate Db2. After installation or migration, a system administrator can change the values of subsystem parameters if needed. However, because these parameters can affect the entire subsystem, system administrators should first carefully evaluate and plan for the impact of any changed values. In some cases, changing the value of a subsystem parameter requires Db2 to be stopped and restarted.

Most subsystem parameters are listed on installation panels. However, some parameters are not. For a list of subsystem parameters, including those parameters that are not on panels, see [Directory of subsystem parameters, panel fields, and application default values \(Db2 Installation and Migration\)](#). This list includes important characteristics of each parameter, including whether changing a the parameter requires you to stop and restart Db2.

Subsystem parameter values are contained in a load module that is generated by installation job DSNTIJUZ. The name of this subsystem parameters load module is specified in the PARAMETER MODULE field of installation panel DSNTIPO3. The default name is DSNZPARM. However, to avoid naming conflicts with other members, you should use a different name. Sometimes the naming convention DSNZPxxx is used. In a data sharing environment, each member must have its own subsystem parameter load module.

One way to check the subsystem parameter values is to call stored procedure SYSPROC.ADMIN_INFO_SYSPARM by running sample job DSNTEJ6Z.

Some Db2 subsystem parameters are considered *security parameters*. These parameters can be updated only by processes with a privilege set that includes installation SYSADM authority or SECADM authority. for more information, see [Security parameters in Db2 for z/OS \(Db2 Installation and Migration\)](#).

Related concepts

[Subsystem parameters and application defaults in a data sharing environment \(Db2 Installation and Migration\)](#)

[Application programming default values](#)

Each Db2 for z/OS subsystem has a set of *application programming default values*. These values are the default settings for SQL applications that run on that subsystem. For example, the ENScheme application programming default value specifies the default encoding scheme for new objects. Some users colloquially refer to these values as *DECP values*.

[The Db2 installation CLIST panel session \(Db2 Installation and Migration\)](#)

Related reference

[PARAMETER MODULE field \(Db2 Installation and Migration\)](#)

Application processes, transactions, concurrency, and recovery

All SQL programs execute as part of an application process, which involves running one or more programs. Different application processes might involve running different programs or running the same program at different times. When an application interacts with a Db2 database, a transaction begins.

Many different types of programs access Db2 data: user-written applications, SQL statements that users enter dynamically, and even utilities. The single term that describes any type of access to Db2 data is called an *application process*. All SQL programs run as part of an application process.

A *transaction* is a sequence of actions between the application and the database; the sequence begins when data in the database is read or written. A transaction is also known as a *unit of work*.

For example, Consider what happens when you access funds in a bank account. A banking transaction might involve the transfer of funds from one account to another. During the transaction, an application program first subtracts the funds from the first account, and then it adds the funds to the second account. Following the subtraction step, the data is inconsistent. Consistency is reestablished after the funds are added to the second account.

To ensure data consistency, Db2 uses a variety of techniques that include a commit operation, a rollback operation, and locking.

When the subtraction and addition steps of the banking transaction are complete, the application can use the commit operation to end the transaction, thereby making the changes available to other application processes. The *commit* operation makes the database changes permanent.

Consider what happens if more than one application process requests access to the same data at the same time. Or, under certain circumstances, an SQL statement might run concurrently with a utility on the same table space. Db2 uses locks to maintain data integrity under these conditions to prevent, for example, two application processes from updating the same row of data simultaneously.

Db2 acquires locks to prevent uncommitted changes that are made by one application process from being perceived by any other. Db2 automatically releases all locks that it has acquired on behalf of an application process when that process ends, but an application process can also explicitly request that locks be released sooner. A commit operation releases locks that an application process has acquired and commits database changes that were made by the same process.

Db2 also provides a way to *back out* uncommitted changes that an application process makes. A back out might be necessary in the event of a failure on the part of an application process or in a *deadlock* situation. Deadlock occurs when contention for the use of a resource, such as a table, cannot be resolved. An application process, however, can explicitly request that its database changes be backed out. This operation is called *rollback*. The interface that an SQL program uses to explicitly specify these commit and rollback operations depends on the environment. For example, in the JDBC environment, applications use commit and rollback methods to commit or roll back transactions.

Related concepts

[Commit and rollback of transactions](#)

At any time, an application process might consist of a single transaction. However the life of an application process can involve many transactions as a result of commit or rollback operations.

Related tasks

[Improving concurrency \(Db2 Performance\)](#)

[Programming for concurrency \(Db2 Performance\)](#)

Locking, commit, and rollback

More than one application process might request access to the same data at the same time. Furthermore, under certain circumstances, an SQL statement can execute concurrently with a utility on the same table space. *Locking* is used to maintain data integrity under such conditions, preventing, for example, two application processes from updating the same row of data simultaneously.

Db2 implicitly acquires locks to prevent uncommitted changes made by one application process from being perceived by any other. Db2 will implicitly release all locks it has acquired on behalf of an application process when that process ends, but an application process can also explicitly request that locks be released sooner. A *commit* operation releases locks acquired by the application process and commits database changes made by the same process.

Db2 provides a way to *back out* uncommitted changes made by an application process. This might be necessary in the event of a failure on the part of an application process, or in a *deadlock* situation. An application process, however, can explicitly request that its database changes be backed out. This operation is called *rollback*.

The interface used by an SQL program to explicitly specify these commit and rollback operations depends on the environment. If the environment can include recoverable resources other than Db2 databases, SQL statements such as the COMMIT and ROLLBACK cannot statements be used. Thus, these statements cannot be used in, CICS, or WebSphere environments.

Related concepts

[Lock duration and release behavior in Db2 for z/OS \(Db2 Performance\)](#)

[Concurrency and locks \(Db2 Performance\)](#)

[Lock contention, suspension, deadlock, and timeout \(Db2 Performance\)](#)

Related reference

[COMMIT statement \(Db2 SQL\)](#)

[ROLLBACK statement \(Db2 SQL\)](#)

Unit of work

A *unit of work* is a recoverable sequence of operations within an application process. A unit of work is sometimes called a *logical unit of work*.

At any time, an application process has a single unit of work, but the life of an application process can involve many units of work as a result of commit or full rollback operations.

A unit of work is initiated when an application process is initiated. A unit of work is also initiated when the previous unit of work is ended by something other than the end of the application process. A unit of work is ended by a commit operation, a full rollback operation, or the end of an application process. A commit or rollback operation affects only the database changes made within the unit of work it ends. While these changes remain uncommitted, other application processes are unable to perceive them unless they are running with an isolation level of uncommitted read. The changes can still be backed out. Once committed, these database changes are accessible by other application processes and can no longer be backed out by a rollback. Locks acquired by Db2 on behalf of an application process that protects uncommitted data are held at least until the end of a unit of work.

The initiation and termination of a unit of work define *points of consistency* within an application process. A point of consistency is a claim by the application that the data is consistent. For example, a banking transaction might involve the transfer of funds from one account to another. Such a transaction would require that these funds be subtracted from the first account, and added to the second. Following the subtraction step, the data is inconsistent. Only after the funds have been added to the second account is consistency reestablished. When both steps are complete, the commit operation can be used to end the unit of work, thereby making the changes available to other application processes. The following figure illustrates this concept.

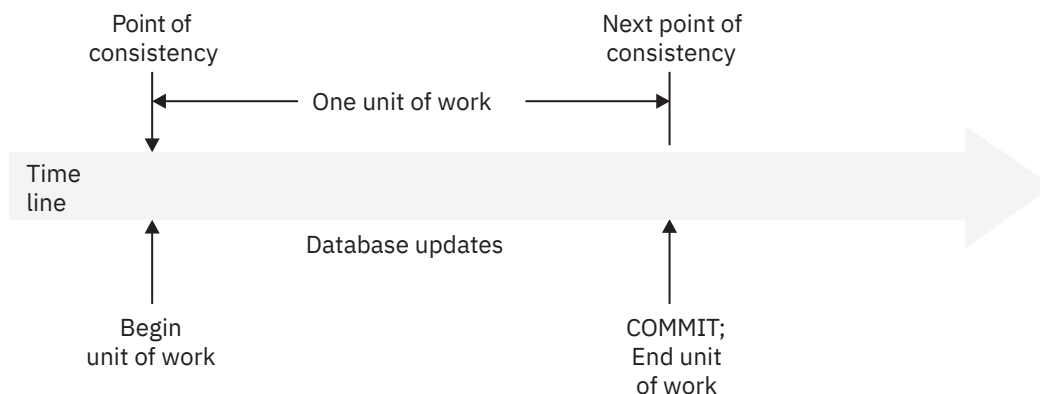


Figure 6. Unit of work with a commit operation

Unit of recovery

A *Db2 unit of recovery* is a recoverable sequence of operations executed by Db2 for an application process.

If a unit of work involves changes to other recoverable resources, the unit of work will be supported by other units of recovery. If relational databases are the only recoverable resources used by the application process, then the scope of the unit of work and the unit of recovery are the same and either term can be used.

Rolling back work

Db2 can back out all changes made in a unit of recovery or only selected changes. Only backing out all changes results in a point of consistency.

Rolling back all changes

The SQL ROLLBACK statement without the TO SAVEPOINT clause specified causes a full rollback operation. If such a rollback operation is successfully executed, Db2 backs out uncommitted changes to restore the data consistency that existed when the unit of work was initiated.

That is, Db2 undoes the work, as shown in the following figure:

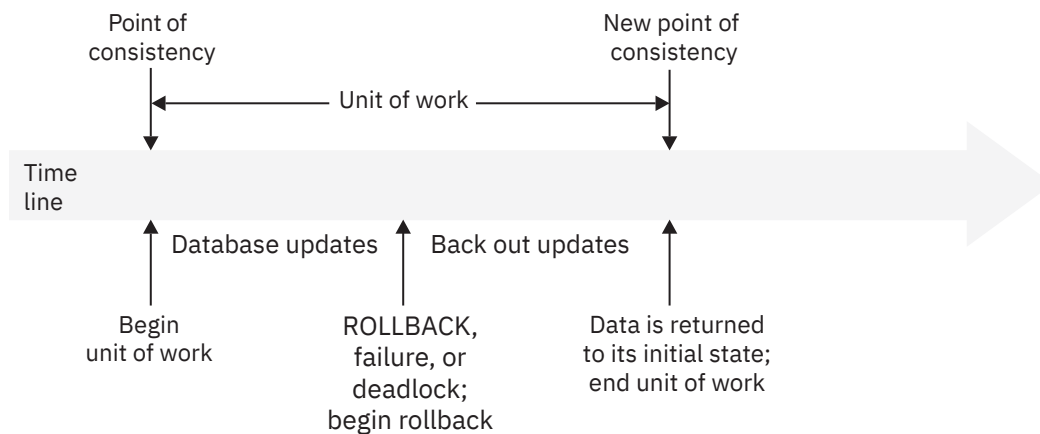


Figure 7. Rolling back all changes from a unit of work

Rolling back selected changes using savepoints

A *savepoint* represents the state of data at some particular time during a unit of work. An application process can set savepoints within a unit of work, and then as logic dictates, roll back only the changes that were made after a savepoint was set.

For example, part of a reservation transaction might involve booking an airline flight and then a hotel room. If a flight gets reserved but a hotel room cannot be reserved, the application process might want to undo the flight reservation without undoing any database changes made in the transaction prior to making the flight reservation. SQL programs can use the SQL SAVEPOINT statement to set savepoints, the SQL ROLLBACK statement with the TO SAVEPOINT clause to undo changes to a specific savepoint or the last savepoint that was set, and the SQL RELEASE SAVEPOINT statement to delete a savepoint. The following figure illustrates this concept.

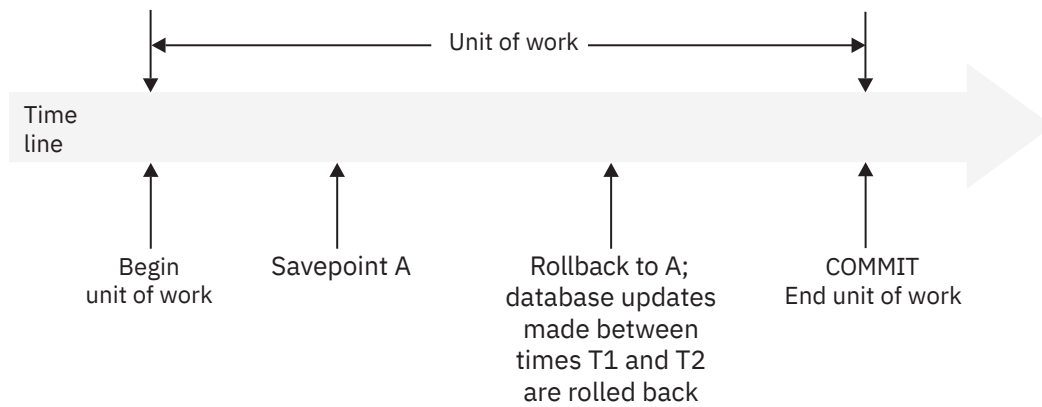


Figure 8. Rolling back changes to a savepoint within a unit of work

Packages and application plans

A *package* contains control structures that Db2 uses when it runs SQL statements. An *application plan* relates an application process to a local instance of Db2 and specifies processing options.

Packages are produced during program preparation. You can think of the control structures as the bound or operational form of SQL statements. All control structures in a package are derived from the SQL statements that are embedded in a single source program.

An application plan contains a list of package names.

Db2 applications require an application plan. Packages make application programs more flexible and easier to maintain.

In general, you create plans and packages by using the Db2 commands BIND PLAN and BIND PACKAGE.

Example

The following figure shows an application plan that contains two packages. Suppose that you decide to change the SELECT statement in package AA to select data from a different table. In this case, you need to bind only package AA again and not package AB.

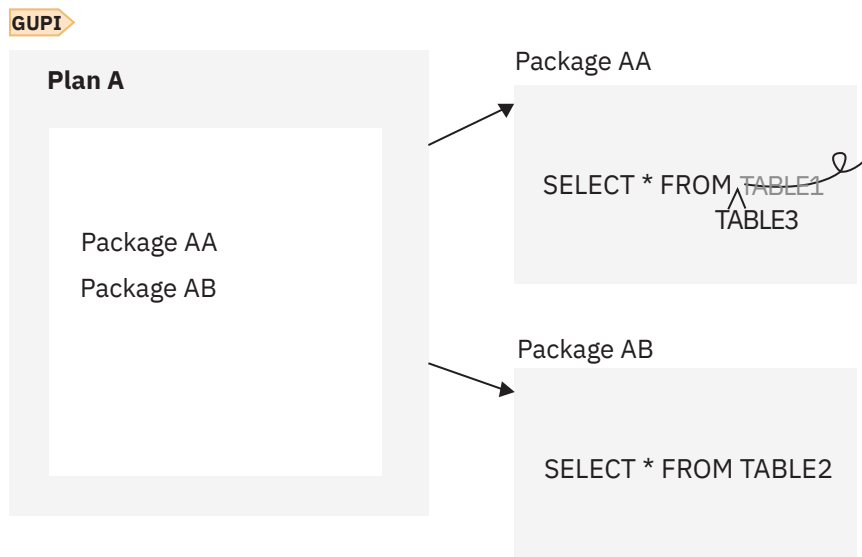


Figure 9. Application plan and packages

GUI

Packages for triggers and routines

A *trigger package* is a special type of package that is created when you issue a CREATE TRIGGER statement. It contains the trigger body and the WHEN clause for the trigger. When the trigger activates, the package executes one or more times.

Packages are also generated, or explicitly created for certain SQL routines, including stored procedures and certain types of user defined functions. The package for a routine contains the routine body, including control statements. The package might also contain statements that are generated by Db2. Each time that the routine is invoked, the package executes one or more times.

Specifically, Db2 generates a package automatically for the following types of SQL routines:

- External scalar functions
- External table functions
- Compiled SQL scalar functions
- Native SQL procedures
- External SQL procedures

For external stored procedures, you explicitly bind the source code for the stored procedure into a package before or you issue the CREATE PROCEDURE statement. Each time that the external procedure is invoked, the package executes one or more times.

Packages for JDBC, SQLJ, and ODBC applications

Packages for JDBC, SQLJ, and ODBC applications serve different purposes. For more information, see [Preparing and running JDBC and SQLJ programs \(Db2 Application Programming for Java\)](#) and [Db2 ODBC run time environment setup \(Db2 Programming for ODBC\)](#).

Related concepts

[Programming for Db2 for z/OS](#)

You use application programs that contain SQL statements to perform data definition or data manipulation operations on Db2 for z/OS objects.

[Preparation process for an application program](#)

How you prepare an application program to run depends on the type of application. The program preparation steps for applications vary based on the type of programming language that is used.

Related reference

[SET CURRENT PACKAGE PATH statement \(Db2 SQL\)](#)

[SET CURRENT PACKAGESET statement \(Db2 SQL\)](#)

[CREATE FUNCTION statement \(overview\) \(Db2 SQL\)](#)

[CREATE PROCEDURE statement \(overview\) \(Db2 SQL\)](#)

[CREATE TRIGGER statement \(basic trigger\) \(Db2 SQL\)](#)

Storage structures

In Db2, a *storage structure* is a set of one or more VSAM data sets that hold Db2 tables or indexes. A storage structure is also called a *page set*.

The two primary types of storage structures in Db2 for z/OS are table spaces and index spaces.

Related concepts

[Db2 database objects overview](#)

In Db2 for z/OS, you use database objects, such as tables, table spaces, indexes, index spaces, keys, views, and databases to organize and access your data.

[Implementing Db2 indexes \(Db2 Administration Guide\)](#)

Related information

[Implementing Db2 table spaces \(Db2 Administration Guide\)](#)

Db2 databases

Db2 *databases* are a set of Db2 structures that include a collection of tables, their associated indexes, and the table spaces in which they reside. You define a database by using the CREATE DATABASE statement.

Whenever a table space is created, it is explicitly or implicitly assigned to an existing database. If you create a table space and do not specify a database name, the table space is created in the default database, DSNDB04. In this case, Db2 implicitly creates a database or uses an existing implicitly created database for the table. All users who have the authority to create table spaces or tables in database DSNDB04 have authority to create tables and table spaces in an implicitly created database. If the table space is implicitly created, and you do not specify the IN clause in the CREATE TABLE statement, Db2 implicitly creates the database to which the table space is assigned.

A single database, for example, can contain all the data that is associated with one application or with a group of related applications. Collecting that data into one database allows you to start or stop access to all the data in one operation. You can also grant authorization for access to all the data as a single unit. Assuming that you are authorized to access data, you can access data that is stored in different databases.

Generally, it is best to minimize the number of table spaces in each database. For the recommended partition-by-range and partition-by-growth table space types, each table space contains only a single table. However, if you do use multi-table segmented (non-UTS) table spaces, which are deprecated, minimize the number of tables in each table space. Too many table spaces and tables in a database can cause decreases in performance and manageability issues. If you reduce the number of table spaces and tables in a database, you improve performance, minimize maintenance, increase concurrency, and decrease log volume.

Deprecated function: [FL 504](#) Non-UTS table spaces for base tables are deprecated. CREATE TABLESPACE statements that run at application compatibility level V12R1M504 or higher always create a partition-by-growth or partition-by-range table space, and CREATE TABLE statements that specify a non-UTS table space (including existing multi-table segmented table spaces) return an error. However, you can use a lower application compatibility level to create table spaces of the deprecated types if needed, such as for recovery situations. For instructions, see [Creating non-UTS table spaces \(deprecated\)](#) (Db2 Administration Guide).

The following figure shows how the main Db2 data structures fit together. Two databases, A and B, are represented as squares. Each database contains a table space and index space. Each index space contains one index. In database B, the table space is partitioned and contains table B1, partitions 1–4. The index space contains one partitioning index, parts 1–4.

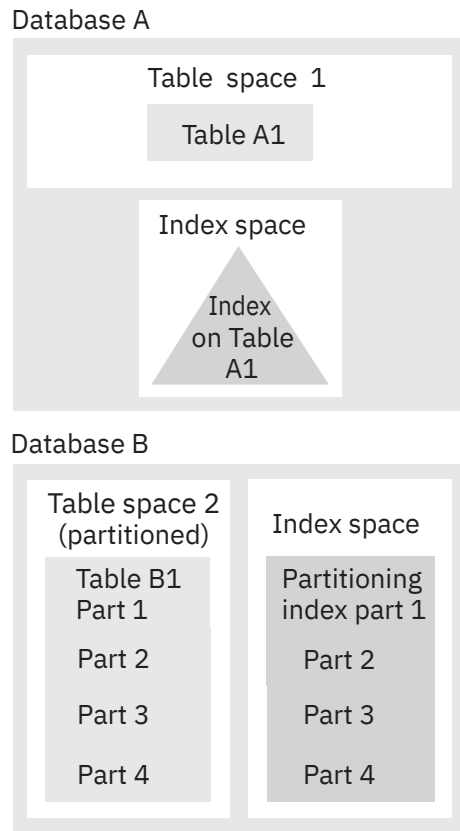


Figure 10. Data structures in a Db2 database

When you migrate Db2 12, Db2 adopts the default database and default storage group that you used in the previous version. You have the same authority for the Db2 12 as you did in the previous version.

Reasons to define a database

In Db2 for z/OS, a database is a logical collection of table spaces and index spaces. Consider the following factors when deciding whether to define a new database for a new set of objects:

- You can start and stop an entire database as a unit; you can display the statuses of all its objects by using a single command that names only the database. Therefore, place a set of tables that are used together into the same database. (The same database holds all indexes on those tables.)
- Some operations lock an entire database. For example, some phases of the LOAD utility prevent some SQL statements (CREATE, ALTER, and DROP) from using the same database concurrently. Therefore, placing many unrelated tables in a single database is often inconvenient.

When one user is executing a CREATE, ALTER, or DROP statement for a table, no other user can access the database that contains that table. QMF users, especially, might do a great deal of data definition; the QMF operations SAVE DATA and ERASE *data-object* are accomplished by creating and dropping Db2 tables. For maximum concurrency, create a separate database for each QMF user.

- The internal database descriptors (DBDs) might become inconveniently large. DBDs grow as new objects are defined, but they do not immediately shrink when objects are dropped; the DBD space for a dropped object is not reclaimed until the MODIFY RECOVERY utility is used to delete records of obsolete copies from SYSIBM.SYSCOPY. DBDs occupy storage and are the objects of occasional input and output operations. Therefore, limiting the size of DBDs is another reason to define new databases.

Related concepts

[Creation of databases](#)

When you define a Db2 database, you name an eventual collection of tables, associated indexes, and the table spaces in which they are to reside.

Db2 table spaces

A Db2 *table space* is a set of volumes on disks that hold the data sets in which tables are actually stored. Every table is stored in table space.

A table space consists of a number of VSAM linear data sets. Table spaces are divided into equal-sized units, called *pages*. Each page is read from disk to the assigned buffer pool, or written from the buffer pool to disk, in a single operation. The page size is controlled by the buffer pool that you assign to the table space (4 KB, 8 KB, 16 KB, or 32 KB in size) for the data. The default page size is 4 KB.

Data in most table spaces can be compressed, which can allow you to store more data on each data page.

You can let Db2 create and manage the table space for you by issuing a CREATE TABLE statement that does not specify an existing table space. Db2 creates a partition-by-growth or partition-by-range table space. For more information, see [“Creation of table spaces” on page 107](#).

You can also explicitly define table spaces by issuing CREATE TABLESPACE statements. You specify the database that the table space belongs to, and the storage group that it uses. When you create a table space, certain options that you specify control the type of table space that is created.

Deprecated function: [FL 504](#) Non-UTS table spaces for base tables are deprecated. CREATE TABLESPACE statements that run at application compatibility level V12R1M504 or higher always create a partition-by-growth or partition-by-range table space, and CREATE TABLE statements that specify a non-UTS table space (including existing multi-table segmented table spaces) return an error. However, you can use a lower application compatibility level to create table spaces of the deprecated types if needed, such as for recovery situations. For instructions, see [Creating non-UTS table spaces \(deprecated\)](#) (Db2 Administration Guide).

For descriptions of the various table space types, see [Table space types and characteristics in Db2 for z/OS](#) (Db2 Administration Guide).

Segmented non-UTS table spaces, which are deprecated, can contain more than one table. However, with the recommended partition-by-growth and partition-by-range table spaces, each table space contains only a single table.

Related tasks

[Creating table spaces explicitly](#) (Db2 Administration Guide)

[Choosing data page sizes](#) (Db2 Performance)

Related reference

[ALTER TABLESPACE statement](#) (Db2 SQL)

[CREATE TABLESPACE statement](#) (Db2 SQL)

Related information

[Implementing Db2 table spaces](#) (Db2 Administration Guide)

[Conversion from index-controlled partitioning to Universal Table Space \(UTS\)](#)

Partitions

A *partition* is a page set that corresponds to a single data set that can be processed or extended independently.

Db2 usually distributes the data evenly across the partitions of a table space when it is created. However, the distribution of the data might become uneven over time, as inserts and deletes occur. You can rebalance data among the partitions by redefining partition boundaries with no impact to availability. You can also add a partition to the table and to each partitioned index on the table; the new partition becomes available immediately.

You can spread the partitions of a large table over several Db2 storage groups or data sets. The partitions of the table do not all need to use the same storage group.

You can also put different partitions on different device types, which means that you can put frequently accessed data in separate partitions and place these partitions on faster devices.

You can use separate jobs for mass update, delete, or insert operations instead of using one large job; each smaller job can work on a different partition. Separating the large job into several smaller jobs that run concurrently can reduce the elapsed time for the whole task.

For certain read-only queries, Db2 can use parallel processing on more than one partition. Parallel processing (for read-only queries) is most efficient when you spread the partitions over different disk volumes and allow each I/O stream to operate on a separate channel. Use the Parallel Sysplex data sharing technology to process a single read-only query across many Db2 subsystems in a data sharing group. You can optimize Parallel Sysplex query processing by placing each Db2 subsystem on a separate central processor complex.

Certain table space types do not have partitions, including segmented (non-UTS) table spaces and simple table spaces. However, all such table spaces are deprecated.

Growth-based partitions

Related concepts

[Partition-by-range table spaces \(Db2 Administration Guide\)](#)

[Partition-by-growth table spaces \(Db2 Administration Guide\)](#)

[Partitioned \(non-UTS\) table spaces \(deprecated\) \(Db2 Administration Guide\)](#)

Segments

A *segment* is a group of pages that holds the rows of a single table. All segments in a table space are the same size.

Related reference

[DEFAULT PARTITION SEGSIZE field \(DPSEGSZ subsystem parameter\) \(Db2 Installation and Migration\)](#)

Db2 index spaces

An *index space* is a Db2 storage structure that contains a single index.

When you create an index by using the CREATE INDEX statement, an index space is automatically defined in the same database as the table. You can define a unique name for the index space, or Db2 can derive a unique name for you. Under certain circumstances, Db2 implicitly creates index spaces.

Db2 hash spaces (deprecated)

A *hash space* is a defined disk space that organizes table data for hash access. Hash-organized table spaces are deprecated and will be unsupported in the future.

Deprecated function:

FL 504 Hash-organized tables are deprecated. Beginning in Db2 12, packages that are bound with APPLCOMPAT(V12R1M504) or higher cannot create hash-organized tables or alter existing tables to use hash-organization. Existing hash-organized tables remain supported, but they will be unsupported in the future.

When you organize a table for hash access, Db2 requires a defined amount of disk space to contain table data. You can specify the amount of disk space to allocate to the hash space when you create a table or alter an existing table. The hash space on a table must be large enough to contain new rows that are added to the table. If a hash space is full, new rows are relocated to the overflow index, which reduces the performance of hash access on that table. Hash spaces can contain only a single table in a partition-by-range or partition-by-growth table space.

Related concepts

[Hash access paths \(deprecated\)](#)

Db2 can use hash access paths to directly access a single row in a table and avoid scanning the table or index. Hash-organized table spaces are deprecated and will be unsupported in the future.

Related tasks

[Managing space and page size for hash-organized tables \(deprecated\) \(Db2 Performance\)](#)

[Monitoring hash access \(deprecated\) \(Db2 Performance\)](#)

Db2 storage groups

Db2 *storage groups* are a set of volumes on disks that hold the data sets in which tables and indexes are stored.

The description of a storage group names the group and identifies its volumes and the VSAM (Virtual Storage Access Method) catalog that records the data sets. The default storage group, SYSDEFLT, is created when you install Db2.

Within the storage group, Db2 does the following actions:

- Allocates storage for table spaces and indexes
- Defines the necessary VSAM data sets
- Extends and deletes VSAM data sets
- Alters VSAM data sets

All volumes of a given storage group must have the same device type. However, parts of a single database can be stored in different storage groups.

Db2 can manage the auxiliary storage requirements of a database by using Db2 storage groups. Data sets in these Db2 storage groups are called "Db2-managed data sets."

These Db2 storage groups are not the same as storage groups that are defined by the DFSMS storage management subsystem (DFSMSsms).

You have several options for managing Db2 data sets:

- Let Db2 manage the data sets. This option means less work for Db2 database administrators.

After you define a Db2 storage group, Db2 stores information about it in the Db2 catalog. (This catalog is not the same as the integrated catalog facility catalog that describes Db2 VSAM data sets). The catalog table SYSIBM.SYSSTOGROUP has a row for each storage group, and SYSIBM.SYSVOLUMES has a row for each volume. With the proper authorization, you can retrieve the catalog information about Db2 storage groups by using SQL statements.

When you create table spaces and indexes, you name the storage group from which space is to be allocated. You can also assign an entire database to a storage group. Try to assign frequently accessed objects (indexes, for example) to fast devices, and assign seldom-used tables to slower devices. This approach to choosing storage groups improves performance.

If you are authorized and do not take specific steps to manage your own storage, you can still define tables, indexes, table spaces, and databases. A default storage group, SYSDEFLT, is defined when Db2 is installed. Db2 uses SYSDEFLT to allocate the necessary auxiliary storage. Information about SYSDEFLT, as with any other storage group, is kept in the catalog tables SYSIBM.SYSSTOGROUP and SYSIBM.SYSVOLUMES.

For both user-managed and Db2-managed data sets, you need at least one integrated catalog facility (ICF) catalog; this catalog can be either a user catalog or a master catalog. These catalogs are created with the ICF. You must identify the catalog of the ICF when you create a storage group or when you create a table space that does not use storage groups.

- Let SMS manage some or all the data sets, either when you use Db2 storage groups or when you use data sets that you have defined yourself. This option offers a reduced workload for Db2 database administrators and storage administrators. You can specify SMS classes when you create or alter a storage group.

- Define and manage your own data sets using VSAM Access Method Services. This option gives you the most control over the physical storage of tables and indexes.

Recommendation: Use Db2 storage groups and whenever you can, either specifically or by default. Also use SMS managed Db2 storage groups whenever you can.

Related tasks

[Choosing data page sizes for LOB data \(Db2 Performance\)](#)

Accessing Db2 data by using SQL statements

You can retrieve data by using the SQL statement `SELECT` to specify a result table that can be derived from one or more tables.

The data that is retrieved through SQL is always in the form of a table. Like the tables from which you retrieve the data, a result table has rows and columns. A program fetches this data one or more rows at a time.

In this information, examples of SQL statements illustrate how to code and use each clause of the `SELECT` statement to query a table. Examples of more advanced queries explain how to fine-tune your queries by using functions and expressions and how to query multiple tables with more complex statements that include unions, joins, and subqueries.

The best way to learn SQL is to develop SQL statements like these examples and then execute them dynamically using a tool such as Db2 Developer Extension. For more information, see [“Tools and IDEs for developing Db2 applications”](#) on page 247.

Example

GUI

Consider this `SELECT` statement:

```
SELECT LASTNAME, FIRSTNAME
FROM EMP
WHERE DEPT = 'D11'
ORDER BY LASTNAME;
```

This `SELECT` statement returns the following result table:

LASTNAME	FIRSTNAME
=====	=====
BROWN	DAVID
LUTZ	JENNIFER
STERN	IRVING

GUI

Many of the examples in this information are based on the sample tables, which represent sample information about a computer company. For more information, see [Chapter 9, “Db2 sample tables,”](#) on page 331.

Related concepts

[Queries \(Db2 SQL\)](#)

Related tasks

[Accessing data in tables from application programs \(Db2 Application programming and SQL\)](#)

Related reference

[select-statement \(Db2 SQL\)](#)

Ways to select data from columns

Several techniques are available for selecting columns from a database for your result tables.

There are several ways to select data from the columns in your table, but you must follow good practices for SELECT statements to guarantee good performance. For best results when you write a SELECT statement, select only rows and columns that your program needs, which reduces your CPU load and memory usage.

Selection of some columns

Select the columns that you want by specifying the name of each column. All columns appear in the order that you specify, not in their order in the table.

GUIP

Example

Notice that the DEPT table contains the DEPTNO column before the MGRNO column. Consider the following query:

```
SELECT MGRNO, DEPTNO
FROM DSN8C10.DEPT
WHERE ADMRDEPT = 'A00';
```

The result table looks like the following example:

MGRNO	DEPTNO
=====	=====
000010	A00
000020	B01
000030	C01
-----	D01
000050	E01

This SELECT statement retrieves data that is contained in the two specified columns of each row in the DEPT table. You can select data from up to 750 columns with a single SELECT statement.

GUIP

Selection of all columns

You do not need to know the column names to select Db2 data. Use an asterisk (*) in the SELECT clause to retrieve all columns from each selected row of the specified table. Db2 selects the columns in the order that the columns are declared in that table. Hidden columns, such as ROWID columns and XML document ID columns, are not included in the result of the SELECT * statement.

GUIP

Example

Consider this query:

```
SELECT *
FROM DSN8A10.DEPT
WHERE ADMRDEPT = 'A00';
```

The result table looks like the following example:

DEPTNO	DEPTNAME	MGRNO	ADMRDEPT	LOCATION
=====	=====	=====	=====	=====
A00	SPIFFY COMPUTER SERVICE DIV.	000010	A00	
B01	PLANNING	000020	A00	
C01	INFORMATION CENTER	000030	A00	

D01	DEVELOPMENT CENTER	-----	A00
E01	SUPPORT SERVICES	000050	A00

This SELECT statement retrieves data from each column of each retrieved row of the DEPT table. Because the example does not specify a WHERE clause, the statement retrieves data from all rows.

In this example, the fifth row contains a null value because no manager is identified for this department. Null values are displayed as dashes.

GUIP

The SELECT * statement is most appropriate when used with dynamic SQL and view definitions. Avoid using SELECT * in static SQL. You write static SQL applications when you know the number of columns that your application returns. That number can change outside your application. If a change occurs to the table, you need to update the application to reflect the changed number of columns in the table.

Use the SELECT * statement only when it is necessary to retrieve all the columns in each retrieved row of your table. Selecting specific columns give your query a higher filter that can retrieve your results more efficiently.

Elimination of duplicate rows

The DISTINCT keyword removes redundant duplicate rows from your result table so that each row contains unique data.

Example

The following query uses the DISTINCT keyword to list the department numbers of the different administrative departments:

GUIP

```
SELECT DISTINCT ADMRDEPT
FROM DSN8C10.DEPT;
```

The result table looks like the following example:

```
ADMRDEPT
=====
A00
D11
E01
```

GUIP

You can use more than one DISTINCT keyword in a single query.

Selection of derived columns and naming the resulting columns

You can select columns that are derived from a constant, an expression, or a function. With the AS clause, you can name resulting columns. This keyword is useful for a column that is derived from an expression or a function.

Example

GUIP

In the following query, the expression SALARY+COMM is named TOTAL_SAL:

```
SELECT EMPNO, (SALARY + COMM) AS TOTAL_SAL
FROM DSN8C10.EMP;
```


The result table looks like the following example:

EMPNO	TOTAL_SAL
=====	=====
000290	16567.00
000310	17172.00
200310	17172.00
000260	18630.00
000300	19170.00
000210	19732.00
:	

This query selects data from all rows in the EMP table, calculates the result of the expression, and returns the columns in the order that the SELECT statement indicates. In the result table, any derived columns, such as (SALARY + COMM) in this example, do not have names. You can use the AS clause to give names to unnamed columns.

To order the rows in the result table by the values in a derived column, specify a name for the column by using the AS clause and use that name in the ORDER BY clause.

GUIP

Related concepts

[Ways to filter the number of returned rows](#)

A variety of different comparison operators in the predicate of a WHERE clause let you filter the number of returned rows.

[Retrieving and excluding rows with null values](#)

A *null value* indicates the absence of a column value in a row. A null value is not the same as zero or all blanks. You can retrieve or exclude rows that contain a null value in a specific row.

[Ways to order rows](#)

You can use the ORDER BY clause to retrieve rows in a specific order.

Related tasks

[Retrieving data by using the SELECT statement \(Db2 Application programming and SQL\)](#)

Related reference

[select-statement \(Db2 SQL\)](#)

[SELECT statement \(Db2 SQL\)](#)

How a SELECT statement works

SQL statements, including SELECT, are made up a series of clauses that are defined by SQL as being executed in a logical order. SELECT statements allow users to definite and organize information that is retrieved from a specified table.

GUIP

The following clause list shows the logical order of clauses in a statement:

1. FROM
2. WHERE
3. GROUP BY
4. HAVING
5. SELECT
6. ORDER BY

GUIP

In addition:

- *Subselects* are processed from the innermost to the outermost subselect. A subselect in a WHERE clause or a HAVING clause of another SQL statement is called a *subquery*.
- The ORDER BY clause can be included in a subselect, a fullselect, or in a SELECT statement.

- If you use an AS clause to define a name in the outermost SELECT clause, only the ORDER BY clause can refer to that name. If you use an AS clause in a subselect, you can refer to the name that it defines outside the subselect.

Examples

GUIP

Example: Invalid SELECT statement

Consider this SELECT statement, which is not valid:

```
SELECT EMPNO, (SALARY + COMM) AS TOTAL_SAL
FROM EMP
WHERE TOTAL_SAL > 50000;
```

The WHERE clause is not valid because Db2 does not process the AS TOTAL_SAL portion of the statement until after the WHERE clause is processed. Therefore, Db2 does not recognize the name TOTAL_SAL that the AS clause defines.

Example:

The following SELECT statement, however, is valid because the ORDER BY clause refers to the column name TOTAL_SAL that the AS clause defines:

```
SELECT EMPNO, (SALARY + COMM) AS TOTAL_SAL
FROM EMP
ORDER BY TOTAL_SAL;
```

GUIP

Related tasks

[Coding SQL statements to avoid unnecessary processing \(Db2 Performance\)](#)

Related reference

[select-statement \(Db2 SQL\)](#)

[SELECT statement \(Db2 SQL\)](#)

SQL functions and expressions

You can use functions and expressions to control the appearance and values of rows and columns in your result tables. Db2 offers many built-in functions, including aggregate functions and scalar functions.

A *built-in function* is a function that is supplied with Db2 for z/OS.

Concatenation of strings

You can concatenate strings by using the CONCAT operator or the CONCAT built-in function.

When the operands of two strings are concatenated, the result of the expression is a string. The operands of concatenation must be compatible strings.

GUIP For example, consider the following query:

```
SELECT LASTNAME CONCAT ' ' CONCAT FIRSTNAME
FROM EMP;
```

This SELECT statement concatenates the last name, a comma, and the first name of each result row. The result table looks like this:

```
=====
HAAS, CHRISTINE
THOMPSON, MICHAEL
KWAN, SALLY
STERN, IRVING
:
```

Alternative syntax for the SELECT statement shown above is as follows:

```
SELECT CONCAT(CONCAT(LASTNAME, ','), FIRSTNAME)
FROM EMP;
```

In this case, the SELECT statement concatenates a comma to the last name, and then concatenates the first name to the result of the first concatenation.

GUIP

Related concepts

[Concatenation operators in expressions \(Db2 SQL\)](#)

Related reference

[CONCAT scalar function \(Db2 SQL\)](#)

[select-statement \(Db2 SQL\)](#)

Calculation of values in one or more columns

You can perform calculations on numeric or datetime data.

The numeric data types are binary integer, floating-point, and decimal. The datetime data types are date, time, and timestamp.

You can retrieve calculated values, just as you display column values, for selected rows.

GUIP

Example 1

Consider this query:

```
SELECT EMPNO,
       SALARY / 12 AS MONTHLY_SAL,
       SALARY / 52 AS WEEKLY_SAL
FROM DSN8C10.EMP
WHERE WORKDEPT = 'A00';
```

The result table looks like the following example:

EMPNO	MONTHLY_SAL	WEEKLY_SAL
000010	4395.83333333	1014.42307692
000110	3875.00000000	894.23076923
000120	2437.50000000	562.50000000
200010	3875.00000000	894.23076923
200120	2437.50000000	562.50000000

The result table displays the monthly and weekly salaries of employees in department A00. If you prefer results with only two digits to the right of the decimal point, you can use the DECIMAL function.

Example 2

To retrieve the department number, employee number, salary, and commission for those employees whose combined salary and commission is greater than \$45,000, write the query as follows:

```
SELECT WORKDEPT, EMPNO, SALARY, COMM
FROM DSN8C10.EMP
WHERE SALARY + COMM > 45000;
```

The result table looks like following example:

DEPT	EMPNO	SALARY	COMM
A00	000010	52750.00	4220.00
A00	000110	46500.00	3720.00
A00	200010	46500.00	4220.00

GUIP

Related concepts

Scalar functions

Db2 offers many different scalar functions, including the CHAR, DECIMAL, and NULLIF scalar functions.

Calculation of aggregate values

You can use the SQL aggregate functions to calculate values that are based on entire columns of data. The calculated values are from only the rows that satisfy the WHERE clause and are therefore selected.

An *aggregate function* is an operation that derives its result by using values from one or more rows. An aggregate function is also known as a *column function*. The argument of an aggregate function is a set of values that are derived from an expression.

You can use the following aggregate functions:

SUM

Returns the total value.

MIN

Returns the minimum value.

AVG

Returns the average value.

MAX

Returns the maximum value.

COUNT

Returns the number of selected rows.

COUNT_BIG

Returns the number of rows or values in a set of rows or values. The result can be greater than the maximum value of an integer.

XMLAGG

Returns a concatenation of XML elements from a collection of XML elements.

You can use SUM and AVG only with numbers. You can use MIN, MAX, COUNT, and COUNT_BIG with any built-in data type.

Examples

GUI

Example 1

This query calculates, for department A00, the sum of employee salaries, the minimum, average, and maximum salary, and the count of employees in the department:

```
SELECT SUM(SALARY) AS SUMSAL,
       MIN(SALARY) AS MINSAL,
       AVG(SALARY) AS AVGSAL,
       MAX(SALARY) AS MAXSAL,
       COUNT(*) AS CNTSAL
FROM EMP
WHERE DEPT = 'A00';
```

The result table looks like this:

SUMSAL	MINSAL	AVGSAL	MAXSAL	CNTSAL
=====	=====	=====	=====	=====
128500.00	29250.00	42833.33333333	52750.00	3

Example 2

You can use (*) in the COUNT and COUNT_BIG functions. In this example, COUNT(*) returns the rows that Db2 processes based on the WHERE clause. This query counts the number of employees that are described in the EMP table:

```
SELECT COUNT(*)
FROM EMP;
```

Example 3

You can use DISTINCT with the SUM, AVG, COUNT, and COUNT_BIG functions. DISTINCT means that the selected function operates on only the unique values in a column. This query counts the different jobs in the EMP table:

```
SELECT COUNT(DISTINCT JOB)
FROM EMP;
```

Aggregate functions like COUNT ignore nulls in the values on which they operate. The preceding example counts distinct job values that are not null.

Note: Do not use DISTINCT with the MAX and MIN functions because using it does not affect the result of those functions.

GUI

Related reference

[DECIMAL or DEC scalar function \(Db2 SQL\)](#)

Scalar functions

Db2 offers many different scalar functions, including the CHAR, DECIMAL, and NULLIF scalar functions.

Like an aggregate function, a *scalar function* produces a single value. Unlike the argument of an aggregate function, the arguments of a scalar function are single values.

GUI

The following example query, which uses the YEAR scalar function, returns the year in which each employee in a particular department was hired:

```
SELECT YEAR(HIREDATE) AS HIREYEAR
FROM EMP
WHERE DEPT = 'A00';
```

The result table looks like this:

```
HIREYEAR
=====
1975
1990
1985
```

The YEAR scalar function produces a single scalar value for each row of EMP that satisfies the search condition. In this example, three rows satisfy the search condition, so YEAR results in three scalar values.

Db2 offers many different scalar functions, including CHAR, DECIMAL, and NULLIF.

CHAR

The CHAR function returns a string representation of the input value. For example, the following SQL statement sets the host variable AVERAGE to the character string representation of the average employee salary:

```
SELECT CHAR(AVG(SALARY))
INTO :AVERAGE
FROM EMP;
```

DECIMAL

The DECIMAL function returns a decimal representation of the input value. For example, assume that you want to change the decimal data type to return a value with a precision and scale that you prefer. The following example represents the average salary of employees as an eight-digit decimal number (the precision) with two of these digits to the right of the decimal point (the scale):

```
SELECT DECIMAL (AVG (SALARY) ,8,2)
FROM EMP;
```

The result table looks like this:

```
=====
32602.30
```

NULLIF

NULLIF returns a null value if the two arguments of the function are equal. If the arguments are not equal, NULLIF returns the value of the first argument. For example, suppose that you want to calculate the average earnings of all employees who are eligible to receive a commission. All eligible employees have a commission of greater than 0, and ineligible employees have a value of 0 for commission:

```
SELECT AVG (SALARY+NULLIF (COMM,0))
AS "AVERAGE EARNINGS"
FROM EMP;
```

The result table looks like this:

```
AVERAGE EARNINGS
=====
35248.8461538
```

GUPI

Specifying a simple expression for the sum of the salary and commission in the select list would include all employees in the calculation of the average. To avoid including those employees who do not earn a commission in the average, you can use the NULLIF function to return a null value instead. The result of adding a null value for the commission to SALARY is itself a null value, and aggregate functions, like AVG, ignore null values. Therefore, this use of NULLIF inside AVG causes the query to exclude each row in which the employee is not eligible for a commission.

Related reference

[Scalar functions \(Db2 SQL\)](#)

Nested functions

Scalar and aggregate functions can be nested in several ways.

You can nest functions in the following ways:

GUPI

Example: Scalar functions within scalar functions

Suppose that you want to know the month and day of hire for a particular employee in department D11. Suppose that you also want the result in USA format (mm/dd/yyyy). Use this query:

```
SELECT SUBSTR((CHAR(HIREDATE, USA)),1,5)
FROM EMP
WHERE LASTNAME = 'BROWN' AND DEPT = 'D11';
```

The result table looks like this:

```
=====
03/03
```

Example: Scalar functions within aggregate functions

In some cases, you might need to invoke a scalar function from within an aggregate function. Suppose that you want to know the average number of years of employment for employees in department A00. Use this query:

```
SELECT AVG(DECIMAL(YEAR(CURRENT DATE - HIREDATE)))
FROM EMP
WHERE DEPT = 'A00';
```

The result table looks like this:

```
=====
20.6666
```

The actual form of the result, 20.6666, depends on how you define the host variable to which you assign the result.

Example: Aggregate functions within scalar functions

Suppose that you want to know the year in which the last employee was hired in department E21. Use this query:

```
SELECT YEAR(MAX(HIREDATE))
FROM EMP
WHERE DEPT = 'E21';
```

The result table looks like this:

```
====
2002
```

GUIP

Related concepts

[Date, time, and timestamp data types](#)

Although storing dates and times as numeric values is possible, using datetime data types is recommended. The datetime data types are DATE, TIME, and TIMESTAMP.

[Aggregate functions \(Db2 SQL\)](#)

Related reference

[Scalar functions \(Db2 SQL\)](#)

User-defined functions

User-defined functions are small programs that you can write to perform an operation. You can use a user-defined function wherever you can use a built-in function.

The CREATE FUNCTION statement is used to explicitly create a user-defined function.

Example

Assume that you define a distinct type called US_DOLLAR. You might want to allow instances of US_DOLLAR to be added. You can create a user-defined function that uses a built-in addition operation and takes instances of US_DOLLAR as input. This function, called a *sourced* function, requires no application coding. Alternatively, you might create a more complex user-defined function that can take a US_DOLLAR instance as input and then convert from U.S. dollars to another currency.

You name the function and specify its semantics so that the function satisfies your specific programming needs.

Related concepts

[Creation of user-defined functions](#)

You can create your own functions in Db2 to simplify your queries.

[Functions \(Db2 SQL\)](#)

Related tasks

[Creating a user-defined function \(Db2 Application programming and SQL\)](#)

CASE expressions

You can use a CASE expression to execute SQL expressions in several different ways depending on the value of a search condition.

One use of a CASE expression is to replace the values in a result table with more meaningful values.

Another use of a CASE expression is to prevent undesirable operations, such as division by zero, from being performed on column values.

GUPI

Example 1

Suppose that you want to display the employee number, name, and education level of all field representatives in the EMP table. Education levels are stored in the EDL column as small integers, but you want to replace the values in this column with more descriptive phrases. Use the following query:

```
SELECT EMPNO, FIRSTNAME, LASTNAME,
CASE
  WHEN EDL<=12 THEN 'HIGH SCHOOL OR LESS'
  WHEN EDL>12 AND EDL<=14 THEN 'JUNIOR COLLEGE'
  WHEN EDL>14 AND EDL<=17 THEN 'FOUR-YEAR COLLEGE'
  WHEN EDL>17 THEN 'GRADUATE SCHOOL'
  ELSE 'UNKNOWN'
END
AS EDUCATION
FROM EMP
WHERE JOB='FLD';
```

The result table looks like following example:

EMPNO	FIRSTNAME	LASTNAME	EDUCATION
000320	RAMMAL	MEHTA	FOUR-YEAR COLLEGE
000330	WING	LEE	JUNIOR COLLEGE
200340	ROY	ALONZO	FOUR-YEAR COLLEGE

The CASE expression replaces each small integer value of EDL with a description of the amount of each field representative's education. If the value of EDL is null, the CASE expression substitutes the word UNKNOWN.

Example 2

If you want to determine the ratio of employee commissions to their salaries, you can execute this query:

```
SELECT EMPNO, DEPT,
  COMM/SALARY AS "COMMISSION/SALARY",
FROM EMP;
```

This SELECT statement has a problem, however. If an employee has not earned any salary, a division-by-zero error occurs. By modifying the following SELECT statement with a CASE expression, you can avoid division by zero:

```
SELECT EMPNO, DEPT,
  (CASE WHEN SALARY=0 THEN NULL
   ELSE COMM/SALARY
  END) AS "COMMISSION/SALARY"
FROM EMP;
```

The CASE expression determines the ratio of commission to salary only if the salary is not zero. Otherwise, Db2 sets the ratio to a null value.

GUPI

Related reference

[CASE expressions \(Db2 SQL\)](#)

Ways to filter the number of returned rows

A variety of different comparison operators in the predicate of a WHERE clause let you filter the number of returned rows.

You can use a WHERE clause to select the rows that are of interest to you. For example, suppose you want to select only the rows that represent the employees who earn a salary greater than \$40,000. A WHERE clause specifies a *search condition*. A search condition is the criteria that Db2 uses to select rows. For any given row, the result of a search condition is true, false, or unknown. If the search condition evaluates to true, the row qualifies for additional processing. In other words, that row can become a row of the result table that the query returns. If the condition evaluates to false or unknown, the row does not qualify for additional processing.

A search condition consists of one or more *predicates* that are combined through the use of the logical operators AND, OR, and NOT. An individual predicate specifies a test that you want Db2 to apply to each row, for example, SALARY > 40000. When Db2 evaluates a predicate for a row, it evaluates to true, false, or unknown. Results are unknown only if a value (called an operand) of the predicate is null. If a particular employee's salary is not known (and is set to null), the result of the predicate SALARY > 40000 is unknown.

You can use a variety of different comparison operators in the predicate of a WHERE clause, as shown in the following table.

Table 10. Comparison operators used in conditions

Type of comparison	Specified with...	Example of predicate with comparison
Equal to null	IS NULL	COMM IS NULL
Equal to	=	DEPTNO = 'X01'
Not equal to	<>	DEPTNO <> 'X01'
Less than	<	AVG(SALARY) < 30000
Less than or equal to	<=	SALARY <= 50000
Greater than	>	SALARY > 25000
Greater than or equal to	>=	SALARY >= 50000
Similar to another value	LIKE	NAME LIKE ' ' or STATUS LIKE 'N_ '
At least one of two predicates	OR	HIREDATE < '2000-01-01' OR SALARY < 40000
Both of two predicates	AND	HIREDATE < '2000-01-01' AND SALARY < 40000
Between two values	BETWEEN	SALARY BETWEEN 20000 AND 40000
Equals a value in a set	IN (X, Y, Z)	DEPTNO IN ('B01', 'C01', 'D11')
Compares a value to another value	DISTINCT	value 1 IS DISTINCT from value 2

Table 10. Comparison operators used in conditions (continued)

Type of comparison	Specified with...	Example of predicate with comparison
--------------------	-------------------	--------------------------------------

Note: Another predicate, EXISTS, tests for the existence of certain rows. The result of the predicate is true if the result table that is returned by the subselect contains at least one row. Otherwise, the result is false.

The XMLEXISTS predicate can be used to restrict the set of rows that a query returns, based on the values in XML columns. The XMLEXISTS predicate specifies an XPath expression. If the XPath expression returns an empty sequence, the value of the XMLEXISTS predicate is false. Otherwise, XMLEXISTS returns true. Rows that correspond to an XMLEXISTS value of true are returned.

You can also search for rows that do not satisfy one of the predicates by using the NOT keyword before the specified predicate.

Related concepts

[Predicates \(Db2 SQL\)](#)

Related tasks

[Using predicates efficiently \(Db2 Performance\)](#)

Related reference

[Summary of predicate processing \(Db2 Performance\)](#)

[where-clause \(Db2 SQL\)](#)

Retrieving and excluding rows with null values

A *null value* indicates the absence of a column value in a row. A null value is not the same as zero or all blanks. You can retrieve or exclude rows that contain a null value in a specific row.

GUPI

Example 1

You can use a WHERE clause to retrieve rows that contain a null value in a specific column.

```
WHERE column-name IS NULL
```

Example 2

You can also use a predicate to exclude null values.

```
WHERE column-name IS NOT NULL
```

GUPI

You cannot use the equal sign to retrieve rows that contain a null value. (WHERE *column-name* = NULL is not allowed.)

Equalities and inequalities

You can use an equal sign (=), various inequality symbols, and the NOT keyword to specify search conditions in the WHERE clause.

Related tasks

[Using predicates efficiently \(Db2 Performance\)](#)

Related reference

[Basic predicate \(Db2 SQL\)](#)

[where-clause \(Db2 SQL\)](#)

How to test for equality

You can use an equal sign (=) to select rows for which a specified column contains a specified value.

GUI

To select only the rows where the department number is A00, use WHERE DEPT = 'A00' in your SELECT statement:

```
SELECT FIRSTNAME, LASTNAME
FROM EMP
WHERE DEPT = 'A00';
```

This query retrieves the first and last name of each employee in department A00.

GUI

How to test for inequalities

You can use inequality operators to specify search conditions in your SELECT statements.

GUI

You can use the following inequalities to specify search conditions:

```
<> < <= > >=
```

Examples

To select all employees that were hired before January 1, 2001, you can use this query:

```
SELECT HIREDATE, FIRSTNAME, LASTNAME
FROM EMP
WHERE HIREDATE < '2001-01-01';
```

This SELECT statement retrieves the hire date and name for each employee that was hired before 2001.

GUI

How to test for equality or inequality in a set of columns

You can use the equal operator or the not equal operator to test whether a set of columns is equal or not equal to a set of values.

Example

To select the rows in which the department number is A00 and the education level is 14, you can use this query:

```
SELECT FIRSTNAME, LASTNAME
FROM EMP
WHERE (DEPT, EDL) = ('A00', 14);
```

Example

To select the rows in which the department number is not A00, or the education level is not 14, you can use this query:

```
SELECT FIRSTNAME, LASTNAME
FROM EMP
WHERE (DEPT, EDL) <> ('A00', 14);
```

How to test for a false condition

You can use the NOT keyword to test for a false condition.

You can use the NOT keyword to select all rows for which the predicate is false (but not rows for which the predicate is unknown). The NOT keyword must precede the predicate.

GUI

Example

To select all managers whose compensation is not greater than \$40,000, use:

```
SELECT DEPT, EMPNO
FROM EMP
WHERE NOT (SALARY + COMM) > 40000 AND JOB = 'MGR'
ORDER BY DEPT;
```

The following table contrasts WHERE clauses that use a NOT keyword with comparison operators and WHERE clauses that use only comparison operators. The WHERE clauses are equivalent.

Table 11. Equivalent WHERE clauses

Using NOT	Equivalent clause without NOT
WHERE NOT DEPTNO = 'A00'	WHERE DEPTNO <> 'A00'
WHERE NOT DEPTNO < 'A00'	WHERE DEPTNO >= 'A00'
WHERE NOT DEPTNO > 'A00'	WHERE DEPTNO <= 'A00'
WHERE NOT DEPTNO <> 'A00'	WHERE DEPTNO = 'A00'
WHERE NOT DEPTNO <= 'A00'	WHERE DEPTNO > 'A00'
WHERE NOT DEPTNO >= 'A00'	WHERE DEPTNO < 'A00'

You cannot use the NOT keyword directly preceding equality and inequality comparison operators.

Example

The following WHERE clause results in an error:

```
Wrong:
WHERE DEPT NOT = 'A00'
```

Example

The following two clauses are equivalent:

```
Correct:
WHERE MGRNO NOT IN ('000010', '000020')
WHERE NOT MGRNO IN ('000010', '000020')
```

GUPI

Similarities of character data

You can use the LIKE predicate to specify a character string that is similar to the column value of rows that you want to select.

A LIKE pattern must match the character string in its entirety.

- Use a percent sign (%) to indicate any string of zero or more characters.
- Use an underscore (_) to indicate any single character.

You can also use NOT LIKE to specify a character string that is not similar to the column value of rows that you want to select.

Examples: How to select values similar to a string of unknown characters

The percent sign (%) means "any string or no string."

GUPI

Example

The following query selects data from each row for employees with the initials D B:

```
SELECT FIRSTNAME, LASTNAME, DEPT
FROM EMP
WHERE FIRSTNAME LIKE 'D%' AND LASTNAME LIKE 'B'
```

Example

The following query selects data from each row of the department table, where the department name contains "CENTER" anywhere in its name:

```
SELECT DEPTNO, DEPTNAME
FROM DEPT
WHERE DEPTNAME LIKE ' %CENTER %'
```

Example

Assume that the DEPTNO column is a three-character column of fixed length. You can use the following search condition to return rows with department numbers that begin with E and end with 1:

```
...WHERE DEPTNO LIKE 'E%1';
```

In this example, if E1 is a department number, its third character is a blank and does not match the search condition. If you define the DEPTNO column as a three-character column of varying length instead of fixed length, department E1 would match the search condition. Varying-length columns can have any number of characters, up to and including the maximum number that was specified when the column was created.

Example

The following query selects data from each row of the department table, where the department number starts with an E and contains a 1:

```
SELECT DEPTNO, DEPTNAME
FROM DEPT
WHERE DEPTNO LIKE 'E%1'
```

GUI

Example: How to select a value similar to a single unknown character

The underscore (_) means "any single character."

GUI

Consider the following query:

```
SELECT DEPTNO, DEPTNAME
FROM DEPT
WHERE DEPTNO LIKE 'E_1';
```

In this example, 'E_1' means E, followed by any character, followed by 1. (Be careful: '_' is an underscore character, not a hyphen.) 'E_1' selects only three-character department numbers that begin with E and end with 1; it does not select the department number 'E1'.

GUI

Related concepts

[String data types](#)

Db2 supports several types of string data: character strings, graphic strings, and binary strings.

Multiple conditions

You can use the AND and OR operators to combine predicates and search for data, based on multiple conditions.

Use the AND operator to specify that a search must satisfy both of the conditions. Use the OR operator to specify that the search must satisfy at least one of the conditions.

Examples

GUIP

Example

This query retrieves the employee number, hire date, and salary for each employee who was hired before 1998 and earns a salary of less than \$35,000 per year:

```
SELECT EMPNO, HIREDATE, SALARY
FROM EMP
WHERE HIREDATE < '1998-01-01' AND SALARY < 35000;
```

Example

This query retrieves the employee number, hire date, and salary for each employee who either was hired before 1998, or earns a salary less than \$35,000 per year or both :

```
SELECT EMPNO, HIREDATE, SALARY
FROM EMP
WHERE HIREDATE < '1998-01-01' OR SALARY < 35000;
```

GUIP

Examples: How to use parentheses with AND and OR

If you use more than two conditions with the AND or OR operators, you can use parentheses to specify the order in which you want Db2 to evaluate the search conditions. If you move the parentheses, the meaning of the WHERE clause can change significantly.

GUIP

Example

This query retrieves the row of each employee that satisfies at least one of the following conditions:

- The employee's hire date is before 1998 and salary is less than \$40,000.
- The employee's education level is less than 18.

```
SELECT EMPNO
FROM EMP
WHERE (HIREDATE < '1998-01-01' AND SALARY < 40000) OR (EDL < 18);
```

Example

This query retrieves the row of each employee that satisfies both of the following conditions:

- The employee's hire date is before 1998.
- The employee's salary is less than \$40,000 or the employee's education level is less than 18.

```
SELECT EMPNO
FROM EMP
WHERE HIREDATE < '1998-01-01' AND (SALARY < 40000 OR EDL < 18);
```

Example

This query retrieves the employee number of each employee that satisfies one of the following conditions:

- Hired before 1998 and salary is less than \$40,000.
- Hired after January 1, 1998, and salary is greater than \$40,000.

```
SELECT EMPNO
FROM EMP
WHERE (HIREDATE < '1998-01-01' AND SALARY < 40000)
OR (HIREDATE > '1998-01-01' AND SALARY > 40000);
```

GUIP

Examples: How to use NOT with AND and OR

When you use NOT with AND and OR, the placement of the parentheses is important.



Example

The following query retrieves the employee number, education level, and job title of each employee who satisfies both of the following conditions:

- The employee's salary is less than \$50,000.
- The employee's education level is less than 18.

```
SELECT EMPNO, EDL, JOB
FROM EMP
WHERE NOT (SALARY >= 50000) AND (EDL < 18);
```

In this query, the NOT operator affects only the first search condition (SALARY >= 50000).

Example

The following query retrieves the employee number, education level, and job title of each employee who satisfies at least one of the following conditions:

- The employee's salary is less than or equal to \$50,000.
- The employee's education level is less than or equal to 18.

```
SELECT EMPNO, EDL, JOB
FROM EMP
WHERE NOT (SALARY > 50000 AND EDL > 18);
```

To negate a set of predicates, enclose the entire set in parentheses and precede the set with the NOT keyword.



Ranges of values

You can use the BETWEEN predicate to select rows in which a column has a value within two limits.

You can specify the lower boundary of the BETWEEN predicate first, and then specify the upper boundary. The limits are *inclusive*.

Examples



Example 1

Suppose that you specify the following WHERE clause in which the value of the *column-name* column is an integer:

```
WHERE column-name BETWEEN 6 AND 8
```

Db2 selects all rows whose *column-name* value is 6, 7, or 8. If you specify a range from a larger number to a smaller number (for example, BETWEEN 8 AND 6), the predicate never evaluates to true.

Example 2

This query retrieves the department number and manager number of each department whose number is between C00 and D31:

```
SELECT DEPTNO, MGRNO
FROM DEPT
WHERE DEPTNO BETWEEN 'C00' AND 'D31';
```



You can also use NOT BETWEEN to select rows in which a column has a value that is outside the two limits.

Related reference

[where-clause \(Db2 SQL\)](#)

[BETWEEN predicate \(Db2 SQL\)](#)

Values in a list

You can use the IN predicate to select each row that has a column value that is equal to one of several listed values.

In the values list after the IN predicate, the order of the items is not important and does not affect the ordering of the result. Enclose the entire list of values in parentheses, and separate items by commas; the blanks are optional.

Examples

GUI

Example

The following query retrieves the department number and manager number for departments B01, C01, and D11:

```
SELECT DEPTNO, MGRNO
FROM DEPT
WHERE DEPTNO IN ('B01', 'C01', 'D11');
```

Using the IN predicate gives the same results as a much longer set of conditions that are separated by the OR keyword.

Example

You can alternatively code the WHERE clause in the SELECT statement in the previous example as:

```
WHERE DEPTNO = 'B01' OR DEPTNO = 'C01' OR DEPTNO = 'D11';
```

However, the IN predicate saves coding time and is easier to understand.

Example

The following query finds the projects that do not include employees in department C01 or E21:

```
SELECT PROJNO, PROJNAME, RESPEMP
FROM PROJ
WHERE DEPTNO NOT IN ('C01', 'E21');
```

GUI

Related concepts

[IN-list access \(ACCESSTYPE='N' or 'IN'\) \(Db2 Performance\)](#)

Related reference

[IN predicate \(Db2 SQL\)](#)

Ways to order rows

You can use the ORDER BY clause to retrieve rows in a specific order.

Using ORDER BY is the only way to guarantee that your rows are in the sequence in which you want them. This information demonstrates how to use the ORDER BY clause.

Related tasks

[Formatting the result table \(Db2 Application programming and SQL\)](#)

[Coding SQL statements to avoid unnecessary processing \(Db2 Performance\)](#)

Related reference

[order-by-clause \(Db2 SQL\)](#)

Sort key

You can specify the order of selected rows by using sort keys that you identify in the ORDER BY clause.

A sort key can be a column name, an integer that represents the number of a column in the result table, or an expression. You can identify more than one column.

You can list the rows in ascending or descending order. Null values are included last in an ascending sort and first in a descending sort.

Db2 sorts strings in the collating sequence that is associated with the encoding scheme of the table. Db2 sorts numbers algebraically and sorts datetime values chronologically.

Related tasks

[Ordering the result table rows \(Db2 Application programming and SQL\)](#)

Related reference

[order-by-clause \(Db2 SQL\)](#)

Ascending order

You can retrieve results in ascending order by specifying ASC in the ORDER BY clause of your SELECT statement.

GUI For example, the following query retrieves the employee numbers, last names, and hire dates of employees in department A00 in ascending order of hire dates:

```
SELECT EMPNO, LASTNAME, HIREDATE
FROM EMP
WHERE DEPT = 'A00'
ORDER BY HIREDATE ASC;
```

The result table looks like this:

EMPNO	LASTNAME	HIREDATE
000010	HAAS	1975-01-01
200010	HEMMINGER	1985-01-01
000120	CONNOR	1990-12-05

This SELECT statement shows the seniority of employees. ASC is the default sorting order.

GUI

Related reference

[order-by-clause \(Db2 SQL\)](#)

Descending order

You can retrieve results in descending order by specifying DESC in the ORDER BY clause.

GUI

For example, this query retrieves department numbers, last names, and employee numbers in descending order of department number:

```
SELECT DEPT, LASTNAME, EMPNO
FROM EMP
WHERE JOB = 'SLS'
ORDER BY DEPT DESC;
```

The result table looks like this:

DEPT	LASTNAME	EMPNO
====	=====	=====
C01	NICHOLLS	000140
A00	HEMMINGER	200010
A00	CONNOR	000120

GUI

Related reference

[order-by-clause \(Db2 SQL\)](#)

Sort keys with multiple columns

You can specify more than one column name in the ORDER BY clause to order rows by the values in more than one column.

When several rows have the same first ordering column value, those rows are in order of the second column that you identify in the ORDER BY clause, and then on the third ordering column, and so on.

Example

GUI

Consider this query:

```
SELECT JOB, EDL, LASTNAME
FROM EMP
WHERE DEPT = 'A00'
ORDER BY JOB, EDL;
```

The result table looks like the following example:

JOB	EDL	LASTNAME
====	===	=====
PRES	18	HAAS
SLS	14	CONNOR
SLS	18	HEMMINGER

GUI

Related reference

[order-by-clause \(Db2 SQL\)](#)

Sort keys with expressions

You can specify an expression with operators as the sort key for the result table of a SELECT statement.

When you specify an expression with operators as the sort key, the query to which ordering is applied must be a subselect.

Example

GUI The following query is a part of a subselect. The query retrieves the employee numbers, salaries, commissions, and total compensation (salary plus commission) for employees with a total compensation greater than 40000. Order the results by total compensation:

```
SELECT EMPNO, SALARY, COMM, SALARY+COMM AS "TOTAL COMP"
FROM EMP
WHERE SALARY+COMM > 40000
ORDER BY SALARY+COMM;
```

The result table looks like the following example:

EMPNO	SALARY	COMM	TOTAL COMP
=====	=====	=====	=====

000030	38250.00	3060.00	41310.00
000020	41250.00	3300.00	44550.00
200010	46500.00	4220.00	50720.00
000010	52750.00	4220.00	56970.00



Related reference

[order-by-clause \(Db2 SQL\)](#)

Ways to summarize group values

You can use the GROUP BY clause to summarize group values.

Use GROUP BY to group rows by the values of one or more columns. You can then apply aggregate functions to each group. You can use an expression in the GROUP BY clause to specify how to group the rows.

Except for the columns that are named in the GROUP BY clause, the SELECT statement must specify any other selected columns as an operand of one of the aggregate functions.

If a column that you specify in the GROUP BY clause contains null values, Db2 considers those null values to be equal, and all nulls form a single group.

Within the SELECT statement, the GROUP BY clause follows the FROM clause and any WHERE clause, and it precedes the HAVING and ORDER BY clauses.

You can also group the rows by the values of more than one column.

Examples

Example

This query lists, for each department, the lowest and highest education level within that department: The result table looks like this:

```
SELECT DEPT, MIN(EDL), MAX(EDL)
FROM EMP
GROUP BY DEPT;
```

DEPT		
=====	==	==
A00	14	18
B01	18	18
C01	18	20
D11	16	18
E21	14	16

Example

This query finds the average salary for employees with the same job in departments D11 and E21:

```
SELECT DEPT, JOB, AVG(SALARY) AS AVG_SALARY
FROM EMP
WHERE DEPT IN ('D11', 'E21')
GROUP BY DEPT, JOB;
```

The result table looks like this:

DEPT	JOB	AVG_SALARY
=====	===	=====
D11	DES	28790.00000000
D11	MGR	32250.00000000
E21	FLD	23053.33333333

In this example, Db2 groups the rows first by department number and next (within each department) by job before deriving the average salary value for each group.

Example

This query finds the average salary for all employees that were hired in the same year. You can use the following subselect to group the rows by the year of hire:

```
SELECT AVG(SALARY), YEAR(HIREDATE)
FROM EMP
GROUP BY YEAR(HIREDATE);
```

Related tasks

[Coding SQL statements to avoid unnecessary processing \(Db2 Performance\)](#)

Related reference

[group-by-clause \(Db2 SQL\)](#)

[order-by-clause \(Db2 SQL\)](#)

[select-statement \(Db2 SQL\)](#)

Ways to merge lists of values

There are several ways to use the UNION keyword for merging lists of values.

A *union* is an SQL operation that combines the results of two SELECT statements to form a single result table. When Db2 encounters the UNION keyword, it processes each SELECT statement to form an *interim result table*. Db2 then combines the interim result table of each statement. If you use UNION to combine two columns with the same name, the corresponding column of the result table inherits that name.

You can use the UNION keyword to obtain distinct rows in the result table of a union, or you can use UNION with the optional keyword ALL to obtain all rows, including duplicates.

Examples

Example 1: How to eliminate duplicates

Use UNION to eliminate duplicates when merging lists of values that are obtained from several tables. The following example combines values from the EMP table and the EMPPROJACT table.

List the employee numbers of all employees for which either of the following statements is true:

- The department number of the employee begins with 'D'.
- The employee is assigned to projects whose project numbers begin with 'MA'.

```
SELECT EMPNO FROM EMP
WHERE DEPT LIKE 'D%'
UNION
SELECT EMPNO FROM EMPPROJACT
WHERE PROJNO LIKE 'MA'
```

The result table looks like the following example:

```
EMPNO
=====
000010
000020
000060
000200
000220
```

The result is the union of two result tables, one formed from the EMP table, the other formed from the EMPPROJACT table. The result, a one-column table, is a list of employee numbers. The entries in the list are distinct.

Example: How to retain duplicates

If you want to keep duplicates in the result of a union, specify the optional keyword ALL after the UNION keyword. Replace the UNION keyword in the previous example with UNION ALL:

```
SELECT EMPNO FROM EMP
WHERE DEPT LIKE 'D%'
UNION ALL
```

```
SELECT EMPNO FROM EMPPROJECT
WHERE PROJNO LIKE 'MA'
```

The result table looks like the following example:

```
EMPNO
=====
000220
000200
000060
000010
000020
000010
```

Now, 000010 is included in the list more than once because this employee works in a department that begins with 'D' and also works on a project that begins with 'MA'.

Related reference

[fullselect \(Db2 SQL\)](#)

Ways to specify search conditions

You can use the HAVING clause in a variety of ways to specify search conditions.

Use HAVING to specify a search condition that each retrieved group must satisfy. The HAVING clause acts like a WHERE clause for groups, and it can contain the same kind of search conditions that you can specify in a WHERE clause. The search condition in the HAVING clause tests properties of each group rather than properties of individual rows in the group.

Examples

GUI

Example

Consider this query:

```
SELECT DEPT, AVG(SALARY) AS AVG_SALARY
FROM EMP
GROUP BY DEPT
HAVING COUNT(*) > 1
ORDER BY DEPT;
```

The result table looks like this:

```
DEPT      AVG_SALARY
=====
A00      42833.33333333
C01      31696.66666666
D11      29943.33333333
E21      23053.33333333
```

The HAVING COUNT(*) > 1 clause ensures that only departments with more than one member are displayed. (In this case, department B01 is not displayed because it consists of only one employee.)

Example

You can use the HAVING clause to retrieve the average salary and minimum education level of employees that were hired after 1990 and who report to departments in which the education level of all employees is greater than or equal to 14. Assuming that you want results only from departments A00 and D11, the following SQL statement tests the group property, MIN(EDL): **GUI**

```
SELECT DEPT, AVG(SALARY) AS AVG_SALARY,
MIN(EDL) AS MIN_EDL
FROM EMP
WHERE HIREDATE >= '1990-01-01' AND DEPT IN ('A00', 'D11')
GROUP BY DEPT
HAVING MIN(EDL) >= 14;
```

The result table looks like this:

DEPT	AVG_SALARY	MIN_EDL
=====	=====	=====
A00	29250.00000000	14
D11	29943.33333333	16

GUIP

When you specify both GROUP BY and HAVING, the HAVING clause must follow the GROUP BY clause in the syntax. A function in a HAVING clause can include multiple occurrences of the DISTINCT clause. You can also connect multiple predicates in a HAVING clause with AND and OR, and you can use NOT for any predicate of a search condition.

Related concepts

[Ways to summarize group values](#)

You can use the GROUP BY clause to summarize group values.

[Predicates \(Db2 SQL\)](#)

Related reference

[having-clause \(Db2 SQL\)](#)

[select-statement \(Db2 SQL\)](#)

[where-clause \(Db2 SQL\)](#)

Ways to join data from more than one table

When you want to see information from multiple tables, you can use a SELECT statement. SELECT statements can retrieve and join column values from two or more tables into a single row. The retrieval is based on a specified condition, typically of matching column values.

Join conditions

The main ingredient of a join is, typically, matching column values in rows of each table that participates in the join. The result of a join associates rows from one table with rows from another table. Depending on the type of join operation, some rows might be formed that contain column values in one table that do not match column values in another table.

A *joined-table* specifies an intermediate result table that is the result of either an inner join or an outer join. The table is derived by applying one of the join operators—INNER, FULL OUTER, LEFT OUTER, or RIGHT OUTER—to its operands. For more information, see [joined-table \(Db2 SQL\)](#).

Inner and outer joins

Db2 supports inner joins, outer joins, which include left outer joins, right outer joins, and full outer joins, and cross joins.

Inner join

An *inner join* result is the cross product of the tables, but it keeps only the rows where the join condition is true. The result of T1 INNER JOIN T2 consists of their paired rows. If a join operator is not specified, INNER is the default. The order in which a LEFT OUTER JOIN or RIGHT OUTER JOIN is performed can affect the result. For more information, see [Inner joins \(Db2 Application programming and SQL\)](#).

Outer join

An *outer join* result includes the rows that are produced by the inner join, plus the missing rows, depending on whether a left outer, full outer, right outer or full out join is used. For more information, see [Outer joins \(Db2 Application programming and SQL\)](#).

Left outer join

A *left outer join* result includes the rows from the left table that were missing from the inner join. The result of T1 LEFT OUTER JOIN T2 consists of their paired rows and, for each unpaired row

of T1, the concatenation of that row with the null row of T2. All columns derived from T2 allow null values. For more information, see [Left outer join \(Db2 Application programming and SQL\)](#).

Right outer join

A *right outer join* result includes the rows from the right table that were missing from the inner join. The result of T1 RIGHT OUTER JOIN T2 consists of their paired rows and, for each unpaired row of T2, the concatenation of that row with the null row of T1. All columns derived from T1 allow null values. For more information, see [Right outer join \(Db2 Application programming and SQL\)](#).

Full outer join

A *full outer join* result includes the rows from both tables that were missing from the inner join. The result of T1 FULL OUTER JOIN T2 consists of their paired rows and, for each unpaired row of T1, the concatenation of that row with the null row of T2, and for each unpaired row of T2, the concatenation of that row with the null row in T1. All columns of the result table allow null values. For more information, see [Full outer join \(Db2 Application programming and SQL\)](#).

Cross join

A *cross join* result includes the cross product of the tables, where each row of the left table is combined with every row of the right table. A cross join is also known as the *Cartesian product*. The result of T1 CROSS JOIN T2 consists of each row of T1 paired with each row of T2. A cross join can also be specified without the CROSS JOIN syntax, by listing the two tables in the FROM clause separated by commas without using a WHERE clause to supply join criteria.

Outer join examples

The following examples use two tables: the parts table (PARTS) and the products table (PRODUCTS), which consist of hardware supplies.

The following figure shows that each row in the PARTS table contains data for a single part: the part name, the part number, and the supplier of the part.

PARTS

PART	PROD#	SUPPLIER
WIRE	10	ACWF
OIL	160	WESTERN_CHEM
MAGNETS	10	BATEMAN
PLASTIC	30	PLASTK_CORP
BLADES	205	ACE_STEEL

Figure 11. Example PARTS table

The following figure shows that each row in the PRODUCTS table contains data for a single product: the product number, name, and price.

PRODUCTS

PROD#	PRODUCT	PRICE
505	SCREWDRIVER	3.70
30	RELAY	7.55
205	SAW	18.90
10	GENERATOR	45.75

Figure 12. Example PRODUCTS table

The following figure shows the ways to combine the PARTS and PRODUCTS tables by using outer join functions. The illustration is based on a subset of columns in each table.

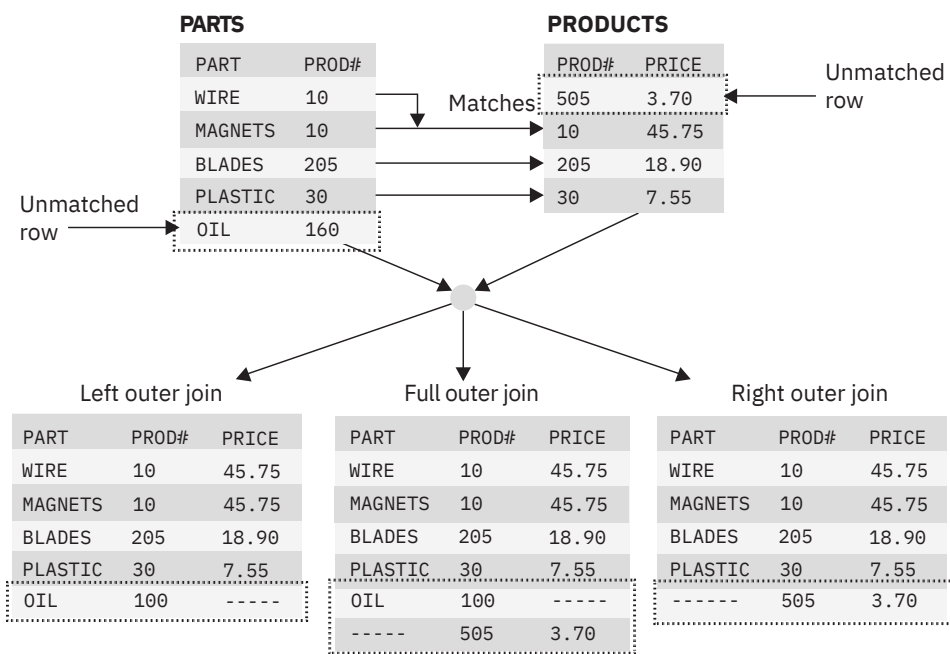


Figure 13. Outer joins of two tables

An inner join consists of rows that are formed from the PARTS and PRODUCTS tables, based on matching the equality of column values between the PROD# column in the PARTS table and the PROD# column in the PRODUCTS table. The inner join does not contain any rows that are formed from unmatched columns when the PROD# columns are not equal.

You can specify joins in the FROM clause of a query. Data from the rows that satisfy the search conditions are joined from all the tables to form the result table.

The result columns of a join have names if the outermost SELECT list refers to base columns. However, if you use a function (such as COALESCE) to build a column of the result, that column does not have a name unless you use the AS clause in the SELECT list.

Related concepts

[Investigating join operations \(Db2 Performance\)](#)

Related tasks

[Joining data from more than one table \(Db2 Application programming and SQL\)](#)

Related reference

[select-statement \(Db2 SQL\)](#)

[from-clause \(Db2 SQL\)](#)

[joined-table \(Db2 SQL\)](#)

Subqueries

You can use a subquery to narrow a search condition that is based on information in an interim table.

A *subquery* is a nested SQL statement, or *subselect*, that contains a SELECT statement within the WHERE or HAVING clause of another SQL statement. You can also code more complex subqueries, such as correlated subqueries and subqueries with quantified predicates.

You can use a subquery when you need to narrow your search condition that is based on information in an interim table. For example, you might want to find all employee numbers in one table that also exist for a given project in a second table.

Example

GUIP

Suppose that you want a list of the employee numbers, names, and commissions of all employees that work on a particular project, such as project number IF2000. The first part of the SELECT statement is easy to write:

```
SELECT EMPNO, LASTNAME, COMM
FROM EMP
WHERE EMPNO
:
```

GUIP

However, you cannot go further because the EMP table does not include project number data. You do not know which employees are working on project IF2000 without issuing another SELECT statement against the EMPPROJECT table.

You can use a subselect to solve this problem. The SELECT statement that surrounds the subquery is the outer SELECT.

Example

GUIP

This query expands the SELECT statement that started in the previous example to include a subquery:

```
SELECT EMPNO, LASTNAME, COMM
FROM EMP
WHERE EMPNO IN
  (SELECT EMPNO
   FROM EMPPROJECT
   WHERE PROJNO = 'IF2000');
```

To better understand what happens as a result from this SQL statement, imagine that Db2 goes through the following process:

1. Db2 evaluates the subquery to obtain a list of EMPNO values:

```
(SELECT EMPNO
 FROM EMPPROJECT
 WHERE PROJNO = 'IF2000');
```

The result is the following interim result table:

```
EMPNO
=====
000140
000140
000030
```

2. The interim result table then serves as a list in the search condition of the outer SELECT. Effectively, Db2 runs this SELECT statement:

```
SELECT EMPNO, LASTNAME, COMM
FROM EMP
WHERE EMPNO IN
  ('000140', '000030');
```

The result table looks like this:

EMPNO	LASTNAME	COMM
000140	NICHOLLS	2274.00
000030	KWAN	3060.00

GUIP

Related concepts

[Subqueries \(Db2 Application programming and SQL\)](#)

Related tasks

[Writing efficient subqueries \(Db2 Performance\)](#)

Related reference

[subselect \(Db2 SQL\)](#)

[Examples for subselect \(Db2 SQL\)](#)

Ways to access Db2 data that is not in a table

You can access Db2 data even if it is not in a table.

This method of data access can be accomplished in two ways.

GUPI

- Set the contents of a host variable to the value of an expression by using the SET host-variable assignment statement.

```
EXEC SQL SET :HVRANDVAL = RAND(:HVRAND);
```

- In addition, you can use the VALUES INTO statement to return the value of an expression in a host variable.

```
EXEC SQL VALUES RAND(:HVRAND)  
INTO :HVRANDVAL;
```

GUPI**Related concepts**

[Data access with host variables](#)

You can use host variables, host-variable arrays, and host structures in your application program to exchange data between the application and the DBMS.

Modifying Db2 data by using SQL statements

You can use SQL statements to add, modify, merge, and remove data in existing tables. You can use INSERT, UPDATE, MERGE, TRUNCATE, and DELETE statements to manipulate Db2 data. These data change statements are also sometimes categorized as Data Manipulation Language (DML) statements.

If you insert, update, merge, or delete data, you can retrieve the data immediately. If you open a cursor and then modify data, you see the modified data only in some circumstances.

Any modifications must maintain the integrity of table relationships. Db2 ensures that an insert, update, or delete operation does not violate any referential constraint or check constraint that is defined on a table.

Before modifying data in your tables, create duplicate tables for testing purposes so that the original table data remains intact. Assume that you created two new tables, NEWDEPT and NEWEMP, that duplicate the DEPT and EMP tables.

Related concepts

[Check constraints for column values](#)

You can use check constraints to ensure that only values from the domain for the column or attribute are allowed.

Related tasks

[Adding and modifying data in tables from application programs \(Db2 Application programming and SQL\)](#)

Related reference

[INSERT statement \(Db2 SQL\)](#)

[UPDATE statement \(Db2 SQL\)](#)

[MERGE statement \(Db2 SQL\)](#)
[TRUNCATE statement \(Db2 SQL\)](#)
[DELETE statement \(Db2 SQL\)](#)

Inserting Db2 data by using INSERT statements

You can use an INSERT statement to add new rows to a table or view.

The INSERT statement has the following three forms:

VALUES form

The *VALUES form* of the INSERT statement is used to insert a single row into the table or view using the values provided or referenced.

fullselect form

The *fullselect form* of the INSERT statement inserts one or more rows into the table or view using values from other tables, or views, or both.

FOR n ROWS form

The *FOR n ROWS form* of the INSERT statement inserts multiple rows into the table or view using values provided or referenced. Although not required, the values can come from host-variable arrays.

This form of INSERT is supported in SQL procedure applications. However, because host-variable arrays are not supported in SQL procedure applications, the support is limited to insertion of scalar values.

You can add new data to an existing table in other ways, too. You might need to add large amounts of data to an existing table. Some efficient options include copying a table into another table, writing an application program that enters data into a table, and using the Db2 LOAD utility to enter data.

Example INSERT statements

GUI

- Suppose that you want to add a new row to the NEWDEPT table. You can use the following INSERT statement.

```
INSERT INTO NEWDEPT (DEPTNO, DEPTNAME, MGRNO, ADMRDEPT)
VALUES ('E31', 'PUBLISHING', '000020', 'D11');
```

After inserting the new department row into the NEWDEPT table, you can use the following SELECT statement to see the modified table.

```
SELECT *
FROM NEWDEPT
WHERE DEPTNO LIKE 'E%'
ORDER BY DEPTNO;
```

The result table gives you the new department row that you inserted for department E31 and any existing departments with department numbers that begin with the letter E.

DEPTNO	DEPTNAME	MGRNO	ADMRDEPT
=====	=====	=====	=====
E21	SOFTWARE SUPPORT	-----	D11
E31	PUBLISHING	000020	D11

- The following statement inserts values into a single row of the DSN8C10.EMP sample table.

```
INSERT INTO DSN8C10.EMP
VALUES ('000205', 'MARY', 'T', 'SMITH', 'D11', '2866',
       '1981-08-10', 'ANALYST', 16, 'F', '1956-05-22',
       16345, 500, 2300);
```

- Assume that SMITH.TEMPEMPL is a created temporary table. The following statement populates the table with data from sample table DSN8C10.EMP.

```
INSERT INTO SMITH.TEMPEMPL
SELECT *
FROM DSN8C10.EMP;
```

- Assume that SESSION.TEMPEMPL is a declared temporary table. The following statement populates the table with data from department D11 in sample table DSN8C10.EMP.

```
INSERT INTO SESSION.TEMPEMPL
SELECT *
FROM DSN8C10.EMP
WHERE WORKDEPT='D11';
```

- The following statement inserts a row into sample table DSN8C10.EMP_PHOTO_RESUME. Set the value for column EMPNO to the value in host variable HV_ENUM. Let the value for column EMP_ROWID be generated because it was defined with a row ID data type and with clause GENERATED ALWAYS.

```
INSERT INTO DSN8C10.EMP_PHOTO_RESUME(EMPNO, EMP_ROWID)
VALUES (:HV_ENUM, DEFAULT);
```

You can only insert user-specified values into ROWID columns that are defined as GENERATED BY DEFAULT and not as GENERATED ALWAYS. Therefore, in the above example, if you were to try to insert a value into EMP_ROWID instead of specifying DEFAULT, the statement would fail unless you also specify OVERRIDING USER VALUE. For columns that are defined as GENERATED ALWAYS, the OVERRIDING USER VALUE clause causes Db2 to ignore any user-specified value and generate a value instead.

For example, assume that you want to copy the rows in DSN8C10.EMP_PHOTO_RESUME to another table that has a similar definition (both tables have a ROWID columns defined as GENERATED ALWAYS). For the following INSERT statement, the OVERRIDING USER VALUE clause causes Db2 to ignore the EMP_ROWID column values from DSN8C10.EMP_PHOTO_RESUME and generate values for the corresponding ROWID column in B.EMP_PHOTO_RESUME.

```
INSERT INTO B.EMP_PHOTO_RESUME
OVERRIDING USER VALUE
SELECT * FROM DSN8C10.EMP_PHOTO_RESUME;
```

- Assume that the T1 table has one column. The following statement inserts a variable (:hv) number of rows of data into the T1 table. The values to be inserted are provided in a host-variable array (:hva). In this example, :hva represents the host-variable array and :hvind represents the array of indicator variables.

```
EXEC SQL INSERT INTO T1 VALUES (:hva:hvind) FOR :hv ROWS ATOMIC;
```

- Assume that the T2 table has 2 columns, C1 is a SMALL INTEGER column, and C2 is an INTEGER column. The following statement inserts 10 rows of data into the T2 table. The values to be inserted are provided in host-variable arrays :hva1 (an array of INTEGERS) and :hva2 (an array of DECIMAL(15,0) values). The data values for :hva1 and :hva2 are represented in the following table:

Table 12. Data values for :hva1 and :hva2

Array entry	:hva1	:hva2
1	1	32768
2	-12	90000
3	79	2
4	32768	19
5	8	36
6	5	24

Table 12. Data values for :hva1 and :hva2 (continued)

Array entry	:hva1	:hva2
7	400	36
8	73	4000000000
9	-200	2000000000
10	35	88

```
EXEC SQL INSERT INTO T2 (C1, C2)
VALUES (:hva1:hvind1, :hva2:hvind2) FOR 10 ROWS
NOT ATOMIC CONTINUE ON SQLEXCEPTION;
```

After execution of the INSERT statement, the following information will be in the SQLCA:

```
SQLCODE = -253
SQLSTATE = 22529
SQLERRD3 = 8
```

Although an attempt was made to insert 10 rows, only 8 rows of data were inserted. Processing continued after the first failed insert because NOT ATOMIC CONTINUE ON SQLEXCEPTION was specified. You can use the GET DIAGNOSTICS statement to find further information, for example.

The result of this statement is num_rows = 8 and num_cond = 2 (2 conditions).

```
GET DIAGNOSTICS :num_rows = ROW_COUNT, :num_cond = NUMBER;
```

The result of this statement is sqlstate = 22003, sqlcode = -302, and row_num = 4.

```
GET DIAGNOSTICS CONDITION 2 :sqlstate = RETURNED_SQLSTATE,
                             :sqlcode = DB2_RETURNED_SQLCODE,
                             :row_num = DB2_ROW_NUMBER;
```

The result of this statement is sqlstate = 22003, sqlcode = -302, and row_num = 8.

```
GET DIAGNOSTICS CONDITION 1 :sqlstate = RETURNED_SQLSTATE,
                             :sqlcode = DB2_RETURNED_SQLCODE,
                             :row_num = DB2_ROW_NUMBER;
```

- Assume the above table T2 with two columns. C1 is a SMALL INTEGER column, and C2 is an INTEGER column. The following statement inserts 8 rows of data into the T2 table. The values to be inserted are provided in host-variable arrays :hva1 (an array of INTEGERS) and :hva2 (an array of DECIMAL(15,0) values.) The data values for :hva1 and :hva2 are represented in [Table 12 on page 88](#).

```
EXEC SQL INSERT INTO T2 (C1, C2)
VALUES (:hva1:hvind1, :hva2:hvind2) FOR 8 ROWS
NOT ATOMIC CONTINUE ON SQLEXCEPTION;
```

After execution of the INSERT statement, the following information will be in the SQLCA:

```
SQLCODE = -253
SQLSTATE = 22529
SQLERRD3 = 6
```

Although an attempt was made to insert 8 rows, only 6 rows of data were inserted. Processing continued after the first failed insert because NOT ATOMIC CONTINUE ON SQLEXCEPTION was specified. You can use the GET DIAGNOSTICS statement to find further information. For example, the result of the following statement is num_rows = 6 and num_cond = 2 (2 conditions).

```
GET DIAGNOSTICS :num_rows = ROW_COUNT, :num_cond = NUMBER;
```

The result of the following statement is sqlstate = 22003, sqlcode = -302, and row_num = 4.

```
GET DIAGNOSTICS CONDITION 2 :sqlstate = RETURNED_SQLSTATE,  
                             :sqlcode = DB2_RETURNED_SQLCODE,  
                             :row_num = DB2_ROW_NUMBER;
```

The result of the following statement is sqlstate = 22003, sqlcode = -302, and row_num = 8.

```
GET DIAGNOSTICS CONDITION 1 :sqlstate = RETURNED_SQLSTATE,  
                             :sqlcode = DB2_RETURNED_SQLCODE,  
                             :row_num = DB2_ROW_NUMBER;
```

- Assume that table T1 has two columns. Insert a variable number (:hvn) or rows into T1. The values to be inserted are in host-variable arrays :hva and :hvb. In this example, the INSERT statement is contained within the SELECT statement of cursor CS1. The SELECT statement makes use of two other input host variables (:hv1 and :hv2) in the WHERE clause. Either a static or dynamic INSERT statement can be used.

```
-- Static INSERT statement:  
DECLARE CS1 CURSOR WITH ROWSET POSITIONING FOR  
  SELECT *  
    FROM FINAL TABLE  
      (INSERT INTO T1 VALUES (:hva, :hvb) FOR :hvn ROWS)  
    WHERE C1 > :hv1 AND C2 < :hv2;  
OPEN CS1;  
-- Dynamic INSERT statement:  
PREPARE INSSTMT FROM  
  'SELECT *  
    FROM FINAL TABLE  
      (INSERT INTO T1 VALUES ( ? , ? ) FOR ? ROWS)  
    WHERE C1 > ? AND C2 < ?';  
DECLARE CS1 CURSOR WITH ROWSET POSITIONING FOR :INSSTMT;  
OPEN CS1 USING :hva, :hvb, :hvn, :hv1, :hv2; (or OPEN CS1 USING DESCRIPTOR ...)
```

If the host-variable arrays for the multiple-row INSERT statement were to be specified using a descriptor, that descriptor (SQLDA) would have to describe all input host variables in the statement, and the order of the entries in the SQLDA should be the same as the order of the order of the host variables, host-variable arrays, and values for the FOR n ROWS clauses in the statement. For example, given the statement above, the SQLVAR entries in the descriptor must be assigned in the following order: :hvn, :hva, :hvb, :hv1, hv2. Also, the SQLVAR entries for host-variable arrays must be tagged in the SQLDA as column arrays (by specifying a special value in part of the SQLNAME field for a host variable), and the SQLVAR entry for the number of rows value must be tagged in the SQLDA (by specifying another special value in part of the SQLNAME field for the host variable).

- Insert a row into table T1. The row contains the value 'xyz' for column COL1, and the cardinality of array INTA for column COL2.

```
CREATE TYPE INTARRAY AS INTEGER ARRAY [6];  
CREATE VARIABLE INTA AS INTARRAY;  
SET INTA = ARRAY [ 1, 2, 3, 4, 5 ];  
CREATE TABLE T1 (COL1 CHAR(7), COL2 INT);  
INSERT INTO T1 VALUES ('xyz', CARDINALITY(INTA));
```

- Insert the values from arrays CHARA and INTA into table T1. For a row of T1, a value of the CHARA array is used for column COL1, and the value of the INTA array with the same array index is used for column COL2.

```
CREATE TYPE INTARRAY AS INTEGER ARRAY[10];  
CREATE TYPE CHARARRAY AS CHAR(7) ARRAY[10];  
CREATE VARIABLE INTA AS INTARRAY;  
CREATE VARIABLE CHARA AS CHARARRAY;  
SET INTA = ARRAY[1, 2, 3, 4, 5];  
SET CHARA = ARRAY['a', 'b', 'c', 'd'];  
CREATE TABLE T1 (COL1 CHAR(7), COL2 INT);  
INSERT INTO T1  
  SELECT *  
    FROM UNNEST(CHARA, INTA) AS (COL1, COL2);
```

- Insert three rows of data into table T1. For each inserted row, assign the value of the tenth element in the INTA array variable to the COL1 column.

```
CREATE TYPE INTARRAY AS INTEGER ARRAY[10];
CREATE VARIABLE INTA AS INTARRAY;
CREATE VARIABLE VAR1 AS INTEGER;
CREATE VARIABLE VAR2 AS INTEGER;
SET INTA = ARRAY[10, 20, 30, 40, 50, 60, 70, 80, 90, 100];
CREATE TABLE T1 (COL1 INT, COL2 CHAR(10));
SET VAR1 = 10;
SET VAR2 = 3;
-- Perform a multiple row insert (specifying a FOR n ROWS clause).
-- The value to be inserted is specified by a reference to an array element.
INSERT INTO T1 (COL1) VALUES(INTA[VAR1]) FOR VAR2 ROWS;
```

The result of these operations is that a value of 100 is assigned to column COL1 for three rows.

GUIP

Related concepts

Programming for Db2 for z/OS

You use application programs that contain SQL statements to perform data definition or data manipulation operations on Db2 for z/OS objects.

Related tasks

[Loading data by issuing INSERT statements \(Db2 Administration Guide\)](#)

[Inserting data into tables \(Db2 Application programming and SQL\)](#)

[Inserting rows by using the INSERT statement \(Db2 Application programming and SQL\)](#)

Related reference

[INSERT statement \(Db2 SQL\)](#)

[where-clause \(Db2 SQL\)](#)

Updating Db2 data by using UPDATE statements

You can change the data in a table by using the UPDATE statement or the MERGE statement.

The UPDATE statement modifies zero or more rows of a table, depending on how many rows satisfy the search condition that you specify in the WHERE clause.

You can use an UPDATE or MERGE statement to specify the values that are to be updated in a single row. You can specify constants, host variables, expressions, DEFAULT, or NULL. Specify NULL to remove a value from a row's column (without removing the row).

Example UPDATE statements

GUIP

- Suppose that an employee gets a promotion. To update several items of the employee's data in the NEWEMP table that reflects the move, use this UPDATE statement:

```
UPDATE NEWEMP
SET JOB = 'MGR',
    DEPT = 'E21'
WHERE EMPNO = '100125';
```

- Change telephone number for employee number 000190 to 3565 in DSN8C10.EMP.

```
UPDATE DSN8C10.EMP
SET PHONENO='3565'
WHERE EMPNO='000190';
```

- Give each member of department D11 a 100-dollar raise.

```
UPDATE DSN8C10.EMP
SET SALARY = SALARY + 100
WHERE WORKDEPT = 'D11';
```

- Employee 000250 is going on a leave of absence. Set the employee's pay values (SALARY, BONUS, and COMMISSION) to null.

```
UPDATE DSN8C10.EMP
SET SALARY = NULL, BONUS = NULL, COMM = NULL
WHERE EMPNO='000250';
```

Alternatively, the statement could also be written as follows:

```
UPDATE DSN8C10.EMP
SET (SALARY, BONUS, COMM) = (NULL, NULL, NULL)
WHERE EMPNO='000250';
```

- Assume that a column named PROJSIZE has been added to DSN8C10.EMP. The column records the number of projects for which the employee's department has responsibility. For each employee in department E21, update PROJSIZE with the number of projects for which the department is responsible.

```
UPDATE DSN8C10.EMP
SET PROJSIZE = (SELECT COUNT(*)
                FROM DSN8C10.PROJ
                WHERE DEPTNO = 'E21')
WHERE WORKDEPT = 'E21';
```

- Double the salary of the employee represented by the row on which the cursor C1 is positioned.

```
EXEC SQL UPDATE DSN8C10.EMP
SET SALARY = 2 * SALARY
WHERE CURRENT OF C1;
```

- Assume that employee table EMP1 was created with the following statement:

```
CREATE TABLE EMP1
(EMP_ROWID ROWID GENERATED ALWAYS,
 EMPNO CHAR(6),
 NAME CHAR(30),
 SALARY DECIMAL(9,2),
 PICTURE BLOB(250K),
 RESUME CLOB(32K));
```

Assume that host variable *HV_EMP_ROWID* contains the value of the ROWID column for employee with employee number '350000'. Using that ROWID value to identify the employee and user-defined function UPDATE_RESUME, increase the employee's salary by \$1000 and update that employee's resume.

```
EXEC SQL UPDATE EMP1
SET SALARY = SALARY + 1000,
    RESUME = UPDATE_RESUME(:HV_RESUME)
WHERE EMP_ROWID = :HV_EMP_ROWID;
```

- In employee table X, give each employee whose salary is below average a salary increase of 10%.

```
EXEC SQL UPDATE EMP X
SET SALARY = 1.10 * SALARY
WHERE SALARY < (SELECT AVG(SALARY) FROM EMP Y
WHERE X.JOBCODE = Y.JOBCODE);
```

- Raise the salary of the employees in department 'E11' whose salary is below average to the average salary.

```
EXEC SQL UPDATE EMP T1
SET SALARY = (SELECT AVG(T2.SALARY) FROM EMP T2)
WHERE WORKDEPT = 'E11' AND
    SALARY < (SELECT AVG(T3.SALARY) FROM EMP T3);
```

- Give the employees in department 'E11' a bonus equal to 10% of their salary.

```
EXEC SQL
DECLARE C1 CURSOR FOR
```



```

SELECT BONUS
FROM DSN8710.EMP
WHERE WORKDEPT = 'E12'
FOR UPDATE OF BONUS;
EXEC SQL
UPDATE DSN8710.EMP
SET BONUS = ( SELECT .10 * SALARY FROM DSN8710.EMP Y
              WHERE EMPNO = Y.EMPNO )
WHERE CURRENT OF C1;

```

- Assuming that cursor CS1 is positioned on a rowset consisting of 10 rows in table T1, update all 10 rows in the rowset.

```
EXEC SQL UPDATE T1 SET C1 = 5 WHERE CURRENT OF CS1;
```

- Assuming that cursor CS1 is positioned on a rowset consisting of 10 rows in table T1, update the fourth row of the rowset.

```

short ind1, ind2;
int n, updt_value;

stmt = 'UPDATE T1 SET C1 = ? WHERE CURRENT OF CS1 FOR ROW ? OF ROWSET'

ind1 = 0;
ind2 = 0;
n = 4;
updt_value = 5;
...
strcpy(my_sqlda.sqldaid, "SQLDA");
my_sqlda.sqln = 2;
my_sqlda.sqld = 2;

my_sqlda.sqlvar[0].sqltype = 497;
my_sqlda.sqlvar[0].sqlllen = 4;
my_sqlda.sqlvar[0].sqldata = (int *) &updt_value;
my_sqlda.sqlvar[0].sqlind = (short *) &ind1;

my_sqlda.sqlvar[1].sqltype = 497;
my_sqlda.sqlvar[1].sqlllen = 4;
my_sqlda.sqlvar[1].sqldata = (int *) &n;
my_sqlda.sqlvar[1].sqlind = (short *) &ind2;

EXEC SQL PREPARE S1 FROM :stmt;
EXEC SQL EXECUTE S1 USING DESCRIPTOR :my_sqlda;

```

- Assume that table POLICY exists and that it is defined with a single inclusive-exclusive period, BUSINESS_TIME. The table contains a row where column BK has a value of 'P138', column CLIENT has a value of 'C882', column TYPE has a value of 'PPO', and the period has value ('2013-01-01', '2020-12-31'). Update the portion of the row beginning from '2014-01-01' to set the TYPE column to 'HMO':

```

UPDATE POLICY
FOR PORTION OF BUSINESS_TIME
FROM '2014-01-01' TO '9999-12-31'
SET TYPE='HMO'
WHERE BK='P138', CLIENT='C882';

```

After the UPDATE statement is processed, the table contains 2 rows in place of the original row. One row with period value ('2013-01-01', '2014-01-01') represents a value of 'PPO' for the TYPE column (the value before the update) and the other row with period value ('2014-01-01', '2020-12-31') represents a value of 'HMO' for the TYPE column (that began with the UPDATE statement).

- Suppose that the INTARRAY and CHARARRAY array types, the INTA, CHARA, and SI variables, and the T1 table are defined as follows:

```
CREATE TYPE INTARRAY AS INTEGER ARRAY [6];
CREATE TYPE CHARARRAY AS CHAR(20) ARRAY [7];
CREATE VARIABLE INTA AS INTARRAY;
CREATE VARIABLE CHARA AS CHARARRAY;
CREATE VARIABLE SI INT;
CREATE TABLE T1 (COL1 CHAR(7), COL2 INT);
```

Assign values to CHARA, INTA, and SI.

```
SET CHARA = ARRAY [ 'a', 'b', 'c' ];
SET INTA = ARRAY [ 1, 2, 3, 4, 5 ];
SET SI = 1;
```

Insert a row into table T1, and then update the row values using values from the CHARA and INTA arrays, which are indexed by the value of variable SI.

```
INSERT INTO T1 VALUES ('abc', 10);
UPDATE T1
SET COL1 = CHARA[SI],
    COL2 = INTA[SI];
```

In the table row, COL1 now contains 'a', and COL2 contains 1.

Set the value of column COL2 for all rows to the cardinality of array INTA.

```
UPDATE T1
SET COL2 = CARDINALITY(INTA);
```

In the table row, COL2 now contains 5.

- Assume that table POLICY exists and that it is defined with a single inclusive-inclusive period, BUSINESS_TIME. The table contains a row where column BK has a value of 'P138', column CLIENT has a value of 'C882', column TYPE has a value of 'PPO', and period has value ('2013-01-01', '2020-12-31'). Suppose that you issue the following UPDATE statement:

```
UPDATE POLICY
FOR PORTION OF BUSINESS_TIME
BETWEEN '2014-01-01' AND '9999-12-31'
SET TYPE='HMO'
WHERE BK='P138', CLIENT='C882';
```

After the UPDATE statement is processed, the table contains 2 rows in place of the original row. One row with period value ('2013-01-01', '2013-12-31') has a value of 'PPO' for the TYPE column (the value before the update) and the other row with period value ('2014-01-01', '2020-12-31') has a value of 'HMO' for the TYPE column.



Related tasks

[Updating table data \(Db2 Application programming and SQL\)](#)

Related reference

[UPDATE statement \(Db2 SQL\)](#)

[where-clause \(Db2 SQL\)](#)

Modifying Db2 data by using MERGE statements

The MERGE statement updates a target with specified input data.

The target of a MERGE statement can be a table or a view. Rows in the target that match the input data are updated as specified, and rows that do not exist in the target are inserted. You also can use a MERGE statement with host-variable arrays to insert and update data. The MERGE statement can also update underlying tables or views of a fullselect.

Example MERGE statements

GUI

- **FL 500** For activities whose description has been changed, update the description in the RECORDS table. For new activities, insert into the RECORDS table. The RECORDS and ACTIVITIES tables both have ACTIVITY as a primary key.

```
MERGE INTO RECORDS AR
  USING (SELECT ACTIVITY, DESCRIPTION FROM ACTIVITIES) AC
  ON (AR.ACTIVITY = AC.ACTIVITY)
  WHEN MATCHED THEN
    UPDATE SET
      DESCRIPTION = AC.DESCRPTION
  WHEN NOT MATCHED THEN
    INSERT
      (ACTIVITY, DESCRIPTION)
    VALUES (AC.ACTIVITY, AC.DESCRPTION);
```

- **FL 500** Using the SHIPMENT table, merge rows into the INVENTORY table: Increase the quantity by the part count in the SHIPMENT table for rows that match; else insert a row for the new part number into the inventory table.

```
MERGE INTO INVENTORY AS IN
  USING (SELECT PARTNO, DESCRIPTION, COUNT FROM SHIPMENT
  WHERE SHIPMENT.PARTNO IS NOT NULL) AS SH
  ON (IN.PARTNO = SH.PARTNO)
  WHEN MATCHED THEN
    UPDATE SET
      DESCRIPTION = SH.DESCRPTION,
      QUANTITY = IN.QUANTITY + SH.COUNT
  WHEN NOT MATCHED THEN
    INSERT
      (PARTNO, DESCRIPTION, QUANTITY)
    VALUES (SH.PARTNO, SH.DESCRPTION, SH.COUNT);
```

- **FL 500** Using the TRANSACTION table, merge rows into the ACCOUNT table: Update the balance from the set of transactions against an account ID, and insert new accounts from the consolidated transactions where they do not already exist.

```
MERGE INTO ACCOUNT AS A
  USING (SELECT ID, SUM(AMOUNT) SUM_AMOUNT FROM TRANSACTION
  GROUP BY ID) AS T
  ON A.ID = T.ID
  WHEN MATCHED THEN
    UPDATE SET
      BALANCE = A.BALANCE + T.SUM_AMOUNT
  WHEN NOT MATCHED THEN
    INSERT
      (ID, BALANCE)
    VALUES (T.ID, T.SUM_AMOUNT);
```

- **FL 500** Using the TRANSACTION_LOG table, merge rows into the EMPLOYEE_FILE table: Update the phone and office with the latest TRANSACTION_LOG row based on the transaction time, and insert the latest new EMPLOYEE_FILE row where the row does not already exist.

```
MERGE INTO EMPLOYEE_FILE AS E
  USING (SELECT EMPID, PHONE, OFFICE
  FROM (SELECT EMPID, PHONE, OFFICE,
  ROW_NUMBER() OVER (PARTITION BY EMPID
  ORDER BY TRANSACTION_TIME DESC) RN
  FROM TRANSACTION_LOG) AS NT
  WHERE RN = 1) AS T
  ON E.EMPID = T.EMPID
  WHEN MATCHED THEN
    UPDATE SET
      (PHONE, OFFICE) =
      (T.PHONE, T.OFFICE)
  WHEN NOT MATCHED THEN
    INSERT
      (EMPID, PHONE, OFFICE)
    VALUES (T.EMPID, T.PHONE, T.OFFICE);
```

- **FL 500** Update the list of activities organized by Group A in the RECORDS table. Delete all outdated activities and update the activities information (DESCRIPTION and DATE) in the RECORDS table if they have been changed. For new upcoming activities, insert into the RECORDS table. Signal an error if the date of the activity is not known. The date of the activities in the RECORDS table must be specified. Each group has an activities table. For example, ACTIVITIES_GROUPA contains all activities that group A organizes, and the RECORDS table contains all upcoming activities organized by different groups in a company. The RECORDS table has (GROUP, ACTIVITY) as the primary key, and DATE is not nullable. All activities tables have ACTIVITY as the primary key. The LAST_MODIFIED column in the RECORDS table is defined with CURRENT_TIMESTAMP as the default value.

```

MERGE INTO RECORDS AR
  USING (SELECT ACTIVITY, DESCRIPTION, DATE, LAST_MODIFIED
        FROM ACTIVITIES_GROUPA) AC
  ON (AR.ACTIVITY = AC.ACTIVITY) AND AR.GROUP = 'A'
  WHEN MATCHED AND AC.DATE IS NULL THEN
    SIGNAL SQLSTATE '70001'
    SET MESSAGE_TEXT =
      AC.ACTIVITY CONCAT ' CANNOT BE MODIFIED. REASON: DATE IS NOT KNOWN'
  WHEN MATCHED AND AC.DATE < CURRENT DATE THEN
    DELETE
  WHEN MATCHED AND AR.LAST_MODIFIED < AC.LAST_MODIFIED THEN
    UPDATE SET
      (DESCRIPTION, DATE, LAST_MODIFIED) = (AC.DESCRPTION, AC.DATE, DEFAULT)
  WHEN NOT MATCHED AND AC.DATE IS NULL THEN
    SIGNAL SQLSTATE '70002'
    SET MESSAGE_TEXT =
      AC.ACTIVITY CONCAT ' CANNOT BE INSERTED. REASON: DATE IS NOT KNOWN'
  WHEN NOT MATCHED AND AC.DATE >= CURRENT DATE THEN
    INSERT
      (GROUP, ACTIVITY, DESCRIPTION, DATE)
    VALUES ('A', AC.ACTIVITY, AC.DESCRPTION, AC.DATE)
  ELSE IGNORE;

```

- Update the descriptions for activities that exist in the RECORDS table. Otherwise, insert the activity and its description into the RECORDS table.

```

MERGE INTO RECORDS AR
  USING (VALUES (:hv_activity, :hv_description)
        FOR :hv_nrows ROWS)
  AS AC (ACTIVITY, DESCRIPTION)
  ON (AR.ACTIVITY = AC.ACTIVITY)
  WHEN MATCHED THEN UPDATE SET DESCRIPTION = AC.DESCRPTION
  WHEN NOT MATCHED THEN INSERT (ACTIVITY, DESCRIPTION)
    VALUES (AC.ACTIVITY, AC.DESCRPTION)
  NOT ATOMIC CONTINUE ON SQLEXCEPTION;

```

- Use the transaction data to merge rows into the account table. Update the balance from the transaction data against an account ID and insert new accounts from the transaction data where the accounts do not already exist.

```

MERGE INTO ACCOUNT AS A
  USING (VALUES (:hv_id, :hv_amount)
        FOR 3 ROWS)
  AS T (ID, AMOUNT)
  ON (A.ID = T.ID)
  WHEN MATCHED THEN UPDATE SET BALANCE = A.BALANCE + T.AMOUNT
  WHEN NOT MATCHED THEN INSERT (ID, BALANCE)
    VALUES (T.ID, T.AMOUNT)
  NOT ATOMIC CONTINUE ON SQLEXCEPTION;

```

- Update the list of activities that are organized by group A in the RECORDS table. Update the activities information (description and date when last modified) in the RECORDS table if the activities exist in the RECORDS table and are also organized by group A. Insert new activities into the RECORDS table.

```

-- hv_nrows = 3
-- hv_activity(1) = 'D'; hv_description(1) = 'Dance'; hv_date(1) = '03/01/07'
-- hv_activity(2) = 'S'; hv_description(2) = 'Singing'; hv_date(2) = '03/17/07'
-- hv_activity(3) = 'T'; hv_description(3) = 'Tai-chi'; hv_date(3) = '05/01/07'
-- hv_group = 'A';
-- note that hv_group is not an array. All 3 values contain the same values
MERGE INTO RECORDS AR
  USING (VALUES (:hv_activity, :hv_description, :hv_date, :hv_group)
        FOR :hv_nrows ROWS)

```

```

        AS AC (ACTIVITY, DESCRIPTION, DATE, GROUP)
    ON AR.ACTIVITY = AC.ACTIVITY AND AR.GROUP = AC.GROUP
    WHEN MATCHED
    THEN UPDATE SET (DESCRIPTION, DATE, LAST_MODIFIED)
                  = (AC.DESCRPTION, AC.DATE, CURRENT_TIMESTAMP)
    WHEN NOT MATCHED
    THEN INSERT (GROUP, ACTIVITY, DESCRIPTION, DATE, LAST_MODIFIED)
              VALUES (AC.GROUP, AC.ACTIVITY, AC.DESCRPTION, AC.DATE, CURRENT_TIMESTAMP)
    NOT ATOMIC CONTINUE ON SQLEXCEPTION;

```

- Use two arrays, CHARA and INTA, as input to a MERGE statement. Column COL2 is set to the cardinality of CHARA for matching rows, and COL2 is set to the cardinality of INTA for non-matching rows.

```

CREATE TYPE INTARRAY AS INTEGER ARRAY[6];
CREATE TYPE CHARARRAY AS CHAR(20) ARRAY[7];
CREATE VARIABLE INTA AS INTARRAY;
CREATE VARIABLE CHARA AS CHARARRAY;
CREATE VARIABLE SI INT;
SET CHARA = ARRAY['a', 'b', 'c'];
SET INTA = ARRAY [1, 2, 3, 4, 5];
CREATE TABLE T1 (COL1 CHAR(7), COL2 INT);
INSERT INTO T1 VALUES ('abc', 10);
MERGE INTO T1 AS A
  USING TABLE (VALUES ('rsk', 3 ) ) AS T (ID, AMOUNT)
  ON A.COL1 = T.ID
  WHEN MATCHED
    THEN UPDATE SET COL2 = CARDINALITY(CHARA)
  WHEN NOT MATCHED
    THEN INSERT (COL1, COL2 ) VALUES (T.ID, CARDINALITY(INTA));

```

•

GUIP

Related tasks

[Inserting data and updating data in a single operation \(Db2 Application programming and SQL\)](#)

Related reference

[MERGE statement \(Db2 SQL\)](#)

[where-clause \(Db2 SQL\)](#)

Deleting data by using DELETE statements

You can use the DELETE statement to remove entire rows from a table.

The DELETE statement removes zero or more rows of a table, depending on how many rows satisfy the search condition that you specify in the WHERE clause. If you omit a WHERE clause from a DELETE statement, Db2 removes all rows from the table or view you name. Therefore, use the DELETE statement carefully. The DELETE statement does not remove specific columns from the row.

GUIP

Example DELETE statements

GUIP

The following statement deletes each row in the NEWEMP table that has employee number 000060.

```

DELETE FROM NEWEMP
  WHERE EMPNO = '000060';

```

Assume that the statements in the following examples are embedded in PL/I programs.

- From the table DSN8C10.EMP, delete the row on which the cursor C1 is currently positioned.

```

EXEC SQL DELETE FROM DSN8C10.EMP WHERE CURRENT OF C1;

```

- From the table DSN8C10.EMP, delete all rows for departments E11 and D21.

```
EXEC SQL DELETE FROM DSN8C10.EMP
WHERE WORKDEPT = 'E11' OR WORKDEPT = 'D21';
```

- From employee table X, delete the employee who has the most absences.

```
EXEC SQL DELETE FROM EMP X
WHERE ABSENT = (SELECT MAX(ABSENT) FROM EMP Y
WHERE X.WORKDEPT = Y.WORKDEPT);
```

- Assuming that cursor CS1 is positioned on a rowset consisting of 10 rows of table T1, delete all 10 rows in the rowset.

```
EXEC SQL DELETE FROM T1 WHERE CURRENT OF CS1;
```

- Assuming cursor CS1 is positioned on a rowset consisting of 10 rows of table T1, delete the fourth row of the rowset.

```
EXEC SQL DELETE FROM T1 WHERE CURRENT OF CS1 FOR ROW 4 OF ROWSET;
```

- Delete rows in table T1 if the value for column COL2 matches the cardinality of array INTA. The array INTA is specified as an argument for the CARDINALITY function in the DELETE statement.

```
CREATE TYPE INTARRAY AS INTEGER ARRAY[6];
CREATE VARIABLE INTA AS INTARRAY;
SET INTA = ARRAY[1, 2, 3, 4, 5];
CREATE TABLE T1 (COL1 CHAR(7), COL2 INT);
INSERT INTO T1 VALUES('abc', 10);
DELETE FROM T1 WHERE COL2 = CARDINALITY(INTA);
```

- Delete only 3 rows from table T1 where the value of column C2 is greater than 10.

```
DELETE FROM T1
WHERE C2 > 10
FETCH FIRST 3 ROWS ONLY;
```

GUIP

Related tasks

[Deleting data from tables \(Db2 Application programming and SQL\)](#)

Related reference

[DELETE statement \(Db2 SQL\)](#)

[MERGE statement \(Db2 SQL\)](#)

[where-clause \(Db2 SQL\)](#)

Deleting data by using TRUNCATE statements

You can use the TRUNCATE statement to delete all rows for base tables or declared global temporary tables.

You can embed a TRUNCATE statement in an application program or issue it interactively. TRUNCATE statements are executable statements that you can prepare dynamically. To truncate a table, you must have the proper authorization or be the owner of the table. The TRUNCATE statement must not be confused with the TRUNCATE function.

Example TRUNCATE statements

GUIP

- Empty an unused inventory table regardless of any existing triggers and return its allocated space.

```
TRUNCATE TABLE INVENTORY
DROP STORAGE
IGNORE DELETE TRIGGERS;
```

- Empty an unused inventory table regardless of any existing delete triggers but preserve its allocated space for later reuse.

```
TRUNCATE TABLE INVENTORY
REUSE STORAGE
IGNORE DELETE TRIGGERS;
```

- Empty an unused inventory table permanently (a ROLLBACK statement cannot undo the truncate operation when the IMMEDIATE option is specified) regardless of any existing delete triggers and preserve its allocated space for immediate use.

```
TRUNCATE TABLE INVENTORY
REUSE STORAGE
IGNORE DELETE TRIGGERS
IMMEDIATE;
```



Related tasks

[Deleting data from tables \(Db2 Application programming and SQL\)](#)

Related reference

[TRUNCATE statement \(Db2 SQL\)](#)

[where-clause \(Db2 SQL\)](#)

Defining Db2 objects

You can use SQL statements to create, modify, and drop Db2 objects, by using various CREATE, ALTER and DROP statements.

Creation of tables

Designing tables that many applications use is a critical task. Table design can be difficult because you can represent the same information in many different ways. This information briefly describes how tables are created and altered, and how authorization is controlled.

You create tables by using the SQL CREATE TABLE statement. At some point after you create and start using your tables, you might need to make changes to them. The ALTER TABLE statement lets you add and change columns, add or drop a primary key or foreign key, add or drop table check constraints, or add and change partitions. Carefully consider design changes to avoid or reduce the disruption to your applications.

If you have DBADM (database administration) authority, you probably want to control the creation of Db2 databases and table spaces. These objects can have a big impact on the performance, storage, and security of the entire relational database. In some cases, you also want to retain the responsibility for creating tables. After designing the relational database, you can create the necessary tables for application programs. You can then pass the authorization for their use to the application developers, either directly or indirectly, by using views.

However, if you want to, you can grant the authority for creating tables to those who are responsible for implementing the application. For example, you probably want to authorize certain application programmers to create tables if they need temporary tables for testing purposes.

Some users in your organization might want to use Db2 with minimum assistance or control. You can define a separate storage group and database for these users and authorize them to create whatever data objects they need, such as tables.

Related concepts

[Authorization and security mechanisms for data access](#)

Authorization is an important part of controlling Db2. The security and authorization mechanisms that control access to Db2 data are both direct and indirect.

[Db2 objects and their relationships](#)

Logical data modeling and physical data modeling are two tasks that you need to perform to design a Db2 database.

Related tasks

[Creating tables from application programs \(Db2 Application programming and SQL\)](#)

Related reference

[CREATE TABLE statement \(Db2 SQL\)](#)

[ALTER TABLE statement \(Db2 SQL\)](#)

Related information

[Implementing Db2 tables \(Db2 Administration Guide\)](#)

Types of tables

In Db2, you store user data in tables. Db2 supports several types of tables, each of which has its own purpose and characteristics.

Db2 supports the following types of tables:

accelerator-only table

A table that stores rows only in the accelerator, not in Db2. The table and column definition of the accelerator-only table is contained in Db2 catalog tables. Any queries that reference the accelerator-only table, must be executed in the accelerator. If a query that references an accelerator-only table is not eligible for query acceleration, an error is issued. To change the contents of an accelerator-only table, the data change statement must be executed in the accelerator.

archive table

A table that stores rows that are deleted from another table.

archive-enabled table

A table that has an associated archive table. When rows are deleted from an archive-enabled table, Db2 can automatically insert those rows into an archive table.

auxiliary table

A table created with the SQL statement CREATE AUXILIARY TABLE and used to hold the data for a column that is defined in a base table.

base table

The most common type of table in Db2. You create a base table with the SQL CREATE TABLE statement. The Db2 catalog table, SYSIBM.SYSTABLES, stores the description of the base table. The table description and table data are persistent. All programs and users that refer to this type of table refer to the same description of the table and to the same instance of the table.

clone table

A table that is structurally identical to a base table. You create a clone table by using an ALTER TABLE statement for the base table that includes an ADD CLONE clause. The clone table is created in a different instance of the same table space as the base table, is structurally identical to the base table in every way, and has the same indexes, before triggers, and LOB objects. In the Db2 catalog, the SYSTABLESPACE table indicates that the table space has only one table in it, but SYSTABLESPACE.CLONE indicates that a clone table exists. Clone tables can be created only in a partition-by range or partition-by-growth table space that is managed by Db2. The base and clone table each have separate underlying VSAM data sets (identified by their data set instance numbers) that contain independent rows of data.

empty table

A table with zero rows.

history table

A table that is used to store historical versions of rows from the associated system-period temporal table.

materialized query table

A table, which you define with the SQL CREATE TABLE statement, that contains materialized data that is derived from one or more source tables. Materialized query tables are useful for complex queries that run on large amounts of data. Db2 can precompute all or part of such queries and use

the precomputed, or materialized, results to answer the queries more efficiently. Materialized query tables are commonly used in data warehousing and business intelligence applications.

Several Db2 catalog tables, including SYSIBM.SYSTABLES and SYSIBM.SYSVIEWS, store the description of the materialized query table and information about its dependency on a table, view, or function. The attributes that define a materialized query table tell Db2 whether the table is:

- System-maintained or user-maintained.
- Refreshable: All materialized tables can be updated with the REFRESH TABLE statement. Only user-maintained materialized query tables can also be updated with the LOAD utility and the UPDATE, INSERT, and DELETE SQL statements.
- Enabled for query optimization: You can enable or disable the use of a materialized query table in automatic query rewrite.

Materialized query tables can be used to improve the performance of dynamic SQL queries. If Db2 determines that a portion of a query could be resolved using a materialized query table, the query might be rewritten by Db2 to use the materialized query table. This decision is based in part on the settings of the CURRENT REFRESH AGE and the CURRENT MAINTAINED TABLE TYPES FOR OPTIMIZATION special registers.

result table

A table that contains a set of rows that Db2 selects or generates, directly or indirectly, from one or more base tables in response to an SQL statement. Unlike a base table or a temporary table, a result table is not an object that you define using a CREATE statement.

sample table

One of several tables shipped with the Db2 licensed program that contains sample data. Many examples in this information are based on sample tables.

temporal table

A table that records the period of time when a row is valid.

Db2 supports two types of periods, which are the system period (specified by a SYSTEM_TIME clause) and the application period (specified by a BUSINESS_TIME clause). The system period consists of a pair of columns with system-maintained values that indicates the period of time when a row is valid. The application period consists of a pair of columns with application-maintained values that indicates the period of time when a row is valid.

system-period temporal table

A system-period temporal table is a base table that is defined with system-period data versioning. You can modify an existing table to become a system-period temporal table by specifying the ADD PERIOD SYSTEM_TIME clause on the ALTER TABLE statement. After creating a history table that corresponds to the system-period temporal table, you can define system-period data versioning on the table by issuing the ALTER TABLE ADD VERSIONING statement with the USE HISTORY table clause.

application-period temporal table

An application-period temporal table is a base table that includes an application period (specified by a BUSINESS_TIME clause). You can modify an existing table to become an application-period temporal table by specifying the ADD PERIOD BUSINESS_TIME clause on the ALTER TABLE statement.

bitemporal table

A bitemporal table is a table that is both a system-period temporal table and an application-period temporal table. You can use a bitemporal table to keep application period information and system-based historical information. Therefore, you have a lot of flexibility in how you query data, based on periods of time.

temporary table

A table that is defined by the SQL statement CREATE GLOBAL TEMPORARY TABLE or DECLARE GLOBAL TEMPORARY TABLE to hold data temporarily. Temporary tables are especially useful when you need to sort or query intermediate result tables that contain many rows, but you want to store only a small subset of those rows permanently.

created global temporary table

A table that you define with the SQL CREATE GLOBAL TEMPORARY TABLE statement. The Db2 catalog table, SYSIBM.SYSTABLES, stores the description of the created temporary table. The description of the table is persistent and shareable. However, each individual application process that refers to a created temporary table has its own distinct instance of the table. That is, if application process A and application process B both use a created temporary table named TEMPTAB:

- Each application process uses the same table description.
- Neither application process has access to or knowledge of the rows in the other application instance of TEMPTAB.

declared global temporary table

A table that you define with the SQL DECLARE GLOBAL TEMPORARY TABLE statement. The Db2 catalog does not store a description of the declared temporary table. Therefore, the description and the instance of the table are not persistent. Multiple application processes can refer to the same declared temporary table by name, but they do not actually share the same description or instance of the table. For example, assume that application process A defines a declared temporary table named TEMP1 with 15 columns. Application process B defines a declared temporary table named TEMP1 with five columns. Each application process uses its own description of TEMP1; neither application process has access to or knowledge of rows in the other application instance of TEMP1.

XML table

A special table that holds only XML data. When you create a table with an XML column, Db2 implicitly creates an XML table space and an XML table to store the XML data.

These different types of tables differ in other ways that this topic does not describe.

Related concepts

Creation of materialized query tables

Materialized query tables improve the performance of complex queries that operate on large amounts of data.

Db2 catalog

Db2 maintains a set of tables that contain information about the data that Db2 controls. These tables are collectively known as the *Db2 catalog*. The Db2 catalog consists of tables of data about everything defined to the Db2 system, including table spaces, indexes, tables, copies of table spaces and indexes, and storage groups.

Archive-enabled tables and archive tables

If you have a table that contains a significant amount of historical data that is not often referenced, consider creating archive tables. An *archive table* is a table that stores older rows from another table.

Related tasks

Creating large objects

Defining large objects to Db2 is different than defining other types of data and objects.

Related reference

Db2 sample tables

Much of the Db2 information refers to or relies on the Db2 sample tables. As a group, the tables include information that describes employees, departments, projects, and activities, and they make up a sample application that exemplifies many of the features of Db2.

Archive-enabled tables and archive tables

If you have a table that contains a significant amount of historical data that is not often referenced, consider creating archive tables. An *archive table* is a table that stores older rows from another table.

The original table is called an *archive-enabled table*. Db2 can automatically store rows that are deleted from an archive-enabled table in an associated archive table. When you query an archive-enabled table, you can specify whether you want those queries to include data from the archive table.

Archive tables have the following advantages:

- Db2 can manage historical data for you. You do not have to manually move data to a separate table.
- Because rows that are infrequently accessed are stored in a separate table, you can potentially improve the performance of queries against the archive-enabled table.
- You can modify queries to include or exclude archive table data without having to change the SQL statement and prepare the application again. Instead, you can control the scope of the query with a global variable.
- You can store archive tables on a lower-cost device to reduce operating costs.

To create an archive table, follow the instructions in [Creating an archive table \(Db2 Administration Guide\)](#). When you create an archive table, use the SYSIBMADM.MOVE_TO_ARCHIVE built-in global variable to specify whether Db2 is to automatically archive any rows that are deleted from the archive-enabled table.

When you query an archive-enabled table, you can specify whether you want the query to consider rows in the archive table. You do not have to modify the SQL. Instead, you can control the scope of the query by using the SYSIBMADM.GET_ARCHIVE built-in global variable and the ARCHIVESENSITIVE bind option. To retrieve data from an archive table, set SYSIBMADM.GET_ARCHIVE to Y and bind the plan or package with ARCHIVESENSITIVE(YES).

The ARCHIVESENSITIVE bind option has no effect on the SYSIBMADM.MOVE_TO_ARCHIVE value.

You can later remove the relationship between the archive-enabled table and the archive table. To remove this relationship, issue the ALTER TABLE statement for the archive-enabled table and specify the DISABLE ARCHIVE clause. Both tables still exist, but the relationship is removed.

Related reference

[GET_ARCHIVE built-in global variable \(Db2 SQL\)](#)

[MOVE_TO_ARCHIVE built-in global variable \(Db2 SQL\)](#)

[ARCHIVESENSITIVE bind option \(Db2 Commands\)](#)

Creation of base tables

You use the CREATE TABLE statement to create a base table that you have designed.

When you create a table, Db2 records a definition of the table in the Db2 catalog. Creating a table does not store the application data. You can put data into the table by using several methods, such as the LOAD utility or the INSERT statement.

GUI

Example

The following CREATE TABLE statement creates the EMP table, which is in a database named MYDB and in a table space named MYTS:

```
CREATE TABLE EMP
  (EMPNO    CHAR(6)      NOT NULL,
   FIRSTNME VARCHAR(12)  NOT NULL,
   LASTNAME VARCHAR(15)  NOT NULL,
   DEPT     CHAR(3)      ,
   HIREDATE DATE          ,
   JOB      CHAR(8)      ,
   EDL      SMALLINT     ,
   SALARY   DECIMAL(9,2)  ,
   COMM     DECIMAL(9,2)  ,
   PRIMARY KEY (EMPNO))
IN MYDB.MYTS;
```

The preceding CREATE TABLE statement shows the definition of multiple columns.

GUI

Related concepts

[Db2 table columns](#)

A column definition has two basic components, the column name and the data type.

Related reference

[CREATE TABLE statement \(Db2 SQL\)](#)

Creation of temporary tables

Temporary tables can help you identify a small subset of rows from an intermediate result table that you want to store permanently. The two types of temporary tables are created temporary tables and declared temporary tables.

You can use temporary tables to sort large volumes of data and to query that data. Then, when you have identified the smaller number of rows that you want to store permanently, you can store them in a base table. The two types of temporary tables in Db2 are the created temporary table and the declared temporary table. The following topics describe how to define each type.

Created temporary table

Sometimes you need a permanent, shareable description of a table but need to store data only for the life of an application process. In this case, you can define and use a created temporary table. Db2 does not log operations that it performs on created temporary tables; therefore, SQL statements that use them can execute more efficiently. Each application process has its own instance of the created temporary table.

Example: Created temporary table

For example, the following statement defines a created temporary table, TEMPPROD:

```
CREATE GLOBAL TEMPORARY TABLE TEMPPROD
(SERIALNO CHAR(8) NOT NULL,
DESCRIPTION VARCHAR(60) NOT NULL,
MFGCOSTAMT DECIMAL(8,2) ,
MFGDEPTNO CHAR(3) ,
MARKUPPCT SMALLINT ,
SALESDEPTNO CHAR(3) ,
CURDATE DATE NOT NULL);
```

Declared temporary table

Sometimes you need to store data for the life of an application process, but you do not need a permanent, shareable description of the table. In this case, you can define and use a declared temporary table.

Unlike other Db2 DECLARE statements, DECLARE GLOBAL TEMPORARY TABLE is an executable statement that you can embed in an application program or issue interactively. You can also dynamically prepare the statement.

When a program in an application process issues a DECLARE GLOBAL TEMPORARY TABLE statement, Db2 creates an empty instance of the table. You can populate the declared temporary table by using INSERT statements, modify the table by using searched or positioned UPDATE or DELETE statements, and query the table by using SELECT statements. You can also create indexes on the declared temporary table. The definition of the declared temporary table exists as long as the application process runs.

Example: Declared temporary table



For example, the following statement defines a declared temporary table, TEMP_EMP. (This example assumes that you have already created the WORKFILE database and corresponding table space for the temporary table.)

```
DECLARE GLOBAL TEMPORARY TABLE SESSION.TEMP_EMP
(EMPNO CHAR(6) NOT NULL,
SALARY DECIMAL(9, 2) ,
COMM DECIMAL(9, 2));
```

GUPI

If specified explicitly, the qualifier for the name of a declared temporary table, must be SESSION. If the qualifier is not specified, it is implicitly defined to be SESSION.

At the end of an application process that uses a declared temporary table, Db2 deletes the rows of the table and implicitly drops the description of the table.

Related concepts

[Temporary tables \(Db2 Application programming and SQL\)](#)

Related reference

[CREATE GLOBAL TEMPORARY TABLE statement \(Db2 SQL\)](#)

[DECLARE GLOBAL TEMPORARY TABLE statement \(Db2 SQL\)](#)

Creation of materialized query tables

Materialized query tables improve the performance of complex queries that operate on large amounts of data.

Using a materialized query table, Db2 pre-computes the results of data that is derived from one or more tables. When you submit a query, Db2 can use the results that are stored in a materialized query table rather than compute the results from the underlying source tables on which the materialized query table is defined. If the rewritten query is less costly, Db2 chooses to optimize the query by using the rewritten query, a process called *automatic query rewrite*.

To take advantage of automatic query rewrite, you must define, populate, and periodically refresh the materialized query table. You use the CREATE TABLE statement to create a table as a materialized query table.

Example

GUPI

The following CREATE TABLE statement defines a materialized query table named TRANSCNT. TRANSCNT summarizes the number of transactions in table TRANS by account, location, and year:

```
CREATE TABLE TRANSCNT (ACCTID, LOCID, YEAR, CNT) AS
  (SELECT ACCOUNTID, LOCATIONID, YEAR, COUNT(*)
   FROM TRANS
   GROUP BY ACCOUNTID, LOCATIONID, YEAR )
DATA INITIALLY DEFERRED
REFRESH DEFERRED
MAINTAINED BY SYSTEM
ENABLE QUERY OPTIMIZATION;
```

The fullselect, together with the DATA INITIALLY DEFERRED clause and the REFRESH DEFERRED clause, defines the table as a materialized query table.

GUPI

Related tasks

[Dropping and re-creating a table space to change its attributes \(Db2 Administration Guide\)](#)

Creation of a table with table-controlled partitioning

Table-controlled partitioning does not require an index for partitioning and is defined by PARTITION clauses on the CREATE TABLE statement.

When you define a partitioning index on a table in a partitioned table space, you specify the partitioning key and the limit key values in the PARTITION clause of the CREATE INDEX statement. This type of partitioning is known as *index-controlled partitioning*. Because the index is created separately from the associated table, you cannot insert data into the table until the partitioning index is created.

Db2 also supports a method called *table-controlled partitioning* for defining table partitions. You can use table-controlled partitioning instead of index-controlled partitioning.

With table-controlled partitioning, you identify column values that delimit partition boundaries with the `PARTITION BY` clause and the `PARTITION ENDING AT` clause of the `CREATE TABLE` statement. When you use this type of partitioning, an index is not required for partitioning.

GUIP

Assume that you need to create a large transaction table that includes the date of the transaction in a column named `POSTED`. You want to keep the transactions for each month in a separate partition. To create the table, use the following statement:

```
CREATE TABLE TRANS
  (ACCTID ...,
   STATE ...,
   POSTED ...,
   ... ,...)
PARTITION BY (POSTED)
(PARTITION 1 ENDING AT ('01/31/2003'),
 PARTITION 2 ENDING AT ('02/28/2003'),
 ...
 PARTITION 13 ENDING AT ('01/31/2004'));
```

GUIP

Related concepts

[Indexes on partitioned tables \(Db2 Administration Guide\)](#)

Creation of temporal tables

A *temporal table* is a table that records the period of time when a row is valid.

You can create application-period temporal tables and system-period temporal tables. An *application-period temporal table* includes an application period, which is a period in which you maintain the beginning and ending values for a row. A *system-period temporal table* has a system period, and you can define system-period data versioning on the table to manage historical and current table data. A *bitemporal table* is a table that is both a system-period temporal table and an application-period temporal table.

System-period data versioning specifies that old rows are archived into another table. The table that contains the current active rows of a table is called the system-period temporal table. The table that contains the archived rows is called the history table. When you define a base table to use system-period data versioning or when you define system-period data versioning on an existing table, you must create a history table. You must specify a name for the history table and create a table space to hold that table.

When you update or delete a row in a system-period temporal table, Db2 inserts the previous version of the row into the history table. The historical versions of rows are written to the history table to record committed versions of the data in the associated system-period temporal table. Intermediate or uncommitted versions of rows in a system-period temporal table are not normally recorded in the history table. If a row in a system-period temporal table is updated multiple times within a single unit of work, and then a commit occurs, only one new historical version of that row is recorded in the history table. If a row is inserted into a system-period temporal table, and the insert is rolled back, nothing is recorded in the history table for the insert that was never committed. You can query a system-period temporal table with timestamp criteria to retrieve previous data values. You can specify the timestamp criteria in the query or by using special registers.

You can use system-period data versioning instead of developing your own programs for maintaining multiple versions of data within a database. With Db2, system-period data versioning is a more efficient method for maintaining versioned data.

You can use system-period temporal tables to track auditing information about when data changes occurred. If you want to track more information, such as who changed the data and the SQL operation that changed the data, you can include non-deterministic generated expression columns in the system-period temporal table.

Related concepts

[Temporal tables and data versioning \(Db2 Administration Guide\)](#)

Related tasks[Querying temporal tables \(Db2 Administration Guide\)](#)

Creation of table spaces

Often you do not need to create a table space before you create a table. You only need to create a table space explicitly when you define a declared temporary table or if you manage all of your own data sets.

Advantages of universal table spaces

Universal table spaces (UTS) combine the benefits of data partitions and segmented organization. Each UTS always contains only a single table.

Universal table spaces offer the following advantages, when compared to the deprecated non-UTS types:

- A choice of partitioning methods:
 - Partitions based on ranges of data values, with [Partition-by-range table spaces \(Db2 Administration Guide\)](#) (PBR)
 - Partitions based on data growth and automatically managed by Db2, with [Partition-by-growth table spaces \(Db2 Administration Guide\)](#) (PBG)
- Improved space management for varying-length rows because a segmented space-map page has more information about free space than a partitioned space-map page
- Improved mass delete performance because mass delete in a segmented table space organization tends to be faster than in non-segmented table space organizations
- Table scans that are localized to segments
- Immediate reuse of all or most of the segments of a table after the table is dropped or mass deleted

Comparison of table space types

The following table compares the characteristics of the various table space types that Db2 for z/OS supports.

Table 13. Comparison of table space types in Db2 for z/OS

Type	Segmented?	Partitioned?	Remarks
Partition-by-range (PBR) table space	Yes	Yes, based on data value ranges	The following attribute keywords distinguish a PBR table space: • SEGSIZE (non-zero) • Numparts • PAGENUM RELATIVE (RPN) PBR table spaces can use absolute or relative page numbering (RPN). However, PBR table spaces with absolute page numbers are deprecated. Use RPN in most cases.

Table 13. Comparison of table space types in Db2 for z/OS (continued)

Type	Segmented?	Partitioned?	Remarks
Partition-by-growth (PBG) table space or "classic partitioned"	Yes	Yes, based on data growth	The following attribute keywords distinguish a PBG table space: • SEGSIZE (non-zero) • MAXPARTITIONS
LOB table space	No	No	For auxiliary tables that contain the data for LOB columns.
XML table space	Yes	Yes, based on the partitioning method of the base table.	UTS table spaces that are created implicitly for XML data.
Partitioned (non-UTS) table space	No	Yes, based on data value ranges. "3" on page 108	This type is deprecated. "1" on page 108
Segmented (non-UTS) table space	Yes	No	This type is deprecated. "1" on page 108
Simple table space	No	No	This type is deprecated. "2" on page 108

Notes:

1. [FL 504](#) Non-UTS table spaces for base tables are deprecated. CREATE TABLESPACE statements that run at application compatibility level V12R1M504 or higher always create a partition-by-growth or partition-by-range table space, and CREATE TABLE statements that specify a non-UTS table space (including existing multi-table segmented table spaces) return an error. However, you can use a lower application compatibility level to create table spaces of the deprecated types if needed, such as for recovery situations. For instructions, see [Creating non-UTS table spaces \(deprecated\)](#) (Db2 Administration Guide).
2. Db2 12 does not support creating simple table spaces. Existing simple table spaces remain supported, but they are likely to be unsupported in the future.
3. Partitioned (non-UTS) table spaces can use either index-controlled or table-controlled partitions. A table space that still uses index-controlled partitions must be converted to use table-controlled partitions before it can be converted to UTS. For more information, see [Converting table spaces to use table-controlled partitioning](#) (Db2 Administration Guide).

Implicit creation of table spaces

Db2 creates a table space implicitly if you issue a CREATE TABLE statement without specifying an existing table space name. Db2 derives the table space name from the name of your table. The table space is created in the either the specified database or, if the database is not specified, in an implicitly created database. For more information, see [Implicitly defined table spaces](#) (Db2 Administration Guide).

Explicit creation of table spaces

If you must create table spaces explicitly, you can issue CREATE TABLESPACE statements.

When you create or alter a table space, the clauses that you specify in a CREATE TABLESPACE or ALTER TABLESPACE statement determine the type of table space. For more information, see [Creating table spaces explicitly](#) (Db2 Administration Guide).

Related concepts

Db2 table spaces

A Db2 *table space* is a set of volumes on disks that hold the data sets in which tables are actually stored. Every table is stored in table space.

Related tasks

[Creating partition-by-range table spaces \(Db2 Administration Guide\)](#)

[Creating partition-by-growth table spaces \(Db2 Administration Guide\)](#)

Related reference

[CREATE TABLESPACE statement \(Db2 SQL\)](#)

Assignment of table spaces and index spaces to physical storage

You can store table spaces and index spaces in user-managed storage, SMS-managed storage, or in Db2-managed storage groups. (A *storage group* is a set of disk volumes.)

If you do not use SMS, you need to name the Db2 storage groups when you create table spaces or index spaces. Db2 allocates space for these objects from the named storage group. You can assign different partitions of the same table space to different storage groups.

Recommendation: Use products in the IBM Storage Management Subsystem (SMS) family, such as Data Facility SMS (DFSMS), to manage some or all of your data sets. Organizations that use SMS to manage Db2 data sets can define storage groups with the VOLUMES(*) clause. You can also assign management class, data class, and storage class attributes. As a result, SMS assigns a volume to the table spaces and index spaces in that storage group.

The following figure shows how storage groups work together with the various Db2 data structures.

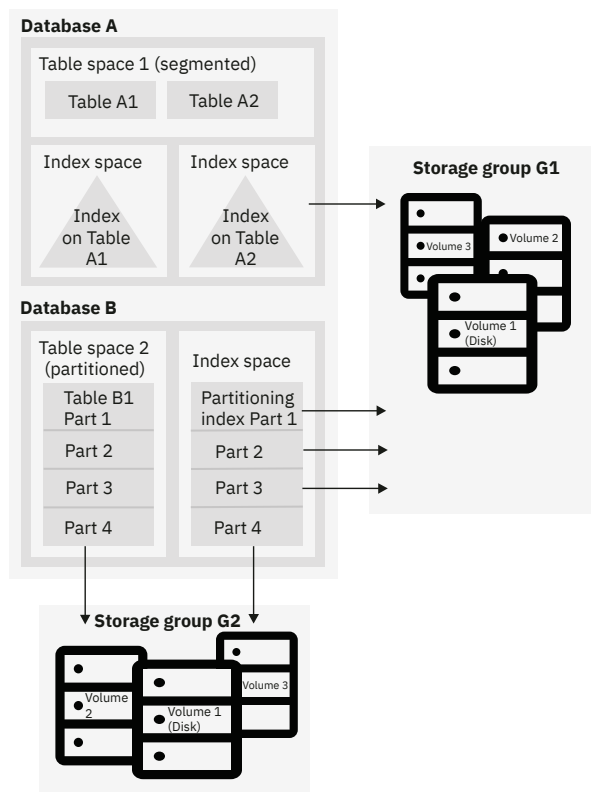


Figure 14. Hierarchy of Db2 structures

To create a Db2 storage group, use the SQL statement `CREATE STOGROUP`. Use the `VOLUMES(*)` clause to specify the SMS management class (MGMTCLAS), SMS data class (DATACLAS), and SMS storage class (STORCLAS) for the Db2 storage group.

After you define a storage group, Db2 stores information about it in the Db2 catalog. The catalog table SYSIBM.SYSTOGROUP has a row for each storage group, and SYSIBM.SYSVOLUMES has a row for each volume in the group.

The process of installing Db2 includes the definition of a default storage group, SYSDEFLT. If you have authorization, you can define tables, indexes, table spaces, and databases. Db2 uses SYSDEFLT to allocate the necessary auxiliary storage. Db2 stores information about SYSDEFLT and all other storage groups in the catalog tables SYSIBM.SYSTOGROUP and SYSIBM.SYSVOLUMES.

Recommendation: Use storage groups whenever you can, either explicitly or implicitly (by using the default storage group). In some cases, organizations need to maintain closer control over the physical storage of tables and indexes. These organizations choose to manage their own user-defined data sets rather than using storage groups. Because this process is complex, this information does not describe the details.

Example

GUI

Consider the following CREATE STOGROUP statement:

```
CREATE STOGROUP MYSTOGRP
  VOLUMES (*)
  VCAT ALIASICF;
```

This statement creates storage group MYSTOGRP. The asterisk (*) on the VOLUMES clause indicates that SMS is to manage your storage group. The VCAT *catalog-name* clause identifies ALIASICF as the name or alias of the catalog of the integrated catalog facility that the storage group is to use. The catalog of the integrated catalog facility stores entries for all data sets that Db2 creates on behalf of a storage group.

The data sets are VSAM linear data sets cataloged in the integrated catalog facility catalog that *catalog-name* identifies. For more information about *catalog-name* values, see [Naming conventions in SQL \(Db2 SQL\)](#).

GUI

IBM Storage Management Subsystem

Db2 for z/OS includes the Storage Management Subsystem (SMS) capabilities. A key product in the SMS family is the Data Facility Storage Management Subsystem (DFSMS). DFSMS can automatically manage all the data sets that Db2 uses and requires. If you use DFSMS to manage your data sets, the result is a reduced workload for Db2 database administrators and storage administrators.

You can experience the following benefits by using DFSMS:

- Simplified data set allocation
- Improved allocation control
- Improved performance management
- Automated disk space management
- Improved management of data availability
- Simplified data movement

Db2 database administrators can use DFSMS to achieve all their objectives for data set placement and design. To successfully use DFSMS, Db2 database administrators and storage administrators need to work together to ensure that the needs of both groups are satisfied.

Related concepts

[Db2 storage groups](#)

Db2 *storage groups* are a set of volumes on disks that hold the data sets in which tables and indexes are stored.

[Implementing Db2 indexes \(Db2 Administration Guide\)](#)

Related tasks

[Choosing data page sizes \(Db2 Performance\)](#)

Related reference

[CREATE STOGROUP statement \(Db2 SQL\)](#)

Related information

[Implementing Db2 table spaces \(Db2 Administration Guide\)](#)

Creation of indexes

Indexes provide efficient access to table data, but can require additional processing when you modify data in a table.

You can create indexes to enforce uniqueness. You can also choose to use indexes because of access requirements.

Using indexes involves a trade-off. A greater number of indexes can simultaneously improve the performance of a certain transaction and require additional processing for inserting, updating, and deleting index keys.

After you create an index, Db2 maintains the index, but you can perform necessary maintenance, such as reorganizing it or recovering it, as necessary.

Related concepts

[Indexes on table columns \(Db2 Administration Guide\)](#)

[Implementing Db2 indexes \(Db2 Administration Guide\)](#)

[Index access \(ACCESSTYPE is 'I', 'IN', 'I1', 'N', 'NR', 'MX', or 'DX'\) \(Db2 Performance\)](#)

Related tasks

[Designing indexes for performance \(Db2 Performance\)](#)

Types of indexes

In Db2 for z/OS, you can create a number of different types of indexes. Carefully consider which type or types best suit your data and applications.

All of the index types are listed in the following tables. These index types are not necessarily mutually exclusive. For example, a unique index can also be a clustering index. Restrictions are noted.

The following table lists the types of indexes that you can create on any table.

Table 14. Index types that are applicable to any table type

Type	Description	SQL syntax	More information
Unique index	An index that ensures that the value in a particular column or set of columns is unique.	<code>CREATE INDEX... UNIQUE...</code>	Unique indexes (Db2 Administration Guide) Nonunique indexes (Db2 Administration Guide)

Table 14. Index types that are applicable to any table type (continued)

Type	Description	SQL syntax	More information
Primary index	A unique index on the primary key of the table. A <i>primary key</i> is column or set of columns that uniquely identifies one row of a table. You define a primary key when you create or alter a table; specify PRIMARY KEY in the CREATE TABLE statement or ALTER TABLE statement. Primary keys are optional. If you define a primary key on a table, you must define a primary index on that key. Otherwise, if the table does not have a primary key, it cannot have a primary index. Each table can have only one primary index. However, the table can have additional unique indexes.	No keywords are required in the CREATE INDEX statement. An index is a primary index if the index key that is specified in the CREATE INDEX statement matches the primary key of the table.	Defining a parent key and unique index (Db2 Application programming and SQL)
Secondary index	An index that is not a primary index. In the context of a partitioned table, a secondary index can also mean an index that is not a partitioning index. See Table 15 on page 112.	No keywords are required in the CREATE INDEX statement. A secondary index is any index that is not a primary index or partitioning index.	None
Clustering index	An index that ensures a logical grouping. When data is inserted into the table, the clustering index attempts to maintain the clustering sequence within the partition. Each table can have only one clustering index.	CREATE INDEX... CLUSTER... or ALTER INDEX... CLUSTER...	Clustering indexes (Db2 Administration Guide)
Expression-based index	An index that is based on a general expression. Use expression-based indexes when you want an efficient evaluation of queries that involve a column-expression.	In the CREATE INDEX or ALTER INDEX statement, the index key is defined as an expression rather than a column or set of columns.	Expression-based indexes (Db2 Administration Guide)

The following table lists the types of indexes that you can create on partitioned tables. These indexes apply to partition-by-range table spaces. They do not apply to partition-by-growth table spaces.

Table 15. Index types that are applicable to only partitioned tables

Type	Description	SQL syntax	More information
Partitioned index	An index that is physically partitioned. A partitioned index consists of multiple data sets. Each data set corresponds to a table partition.	CREATE INDEX... PARTITIONED...	Indexes on partitioned tables (Db2 Administration Guide)

Table 15. Index types that are applicable to only partitioned tables (continued)

Type	Description	SQL syntax	More information
Partitioning index (PI)	An index that corresponds to the columns that partition the table. These columns are called the <i>partitioning key</i> and are specified in the PARTITION BY clause of the CREATE TABLE statement. All partitioning indexes must also be partitioned. Partitioning indexes are not required.	No keywords are required in the CREATE INDEX statement. An index is a partitioning index if the index key matches the partitioning key. To confirm that an index is partitioning index, check the SYSIBM.SYSINDEXES catalog table. The INDEXTYPE column for that index contains a P if the index is a partitioning index.	Indexes on partitioned tables (Db2 Administration Guide)
Secondary index	Depending on the context, a secondary index can mean one of the following two things: • An index that is not a partitioning index. • An index that is not a primary index.	No keywords are required in the CREATE INDEX statement. A secondary index is any index that is not a primary index or partitioning index.	Indexes on partitioned tables (Db2 Administration Guide)
Data partitioned secondary index (DPSI)	A partitioned index that is not a partitioning index. These indexes are also called partitioned secondary indexes (PSIs).	<pre>CREATE INDEX... PARTITIONED...</pre> Also, the specified index key must not match the partitioning key.	Indexes on partitioned tables (Db2 Administration Guide)
Nonpartitioned secondary index (NPSI)	An index that is not partitioned or partitioning. These indexes are also called nonpartitioned indexes (NPIs).	The CREATE INDEX statement does not include the PARTITIONED keyword. Also, the index key does not match the partitioning key.	Indexes on partitioned tables (Db2 Administration Guide)
Multi-piece index	A nonpartitioned index that has multiple data sets. The data sets do not correspond to data partitions. Use a multi-piece index to spread a large index across multiple data sets and thus reduce the physical I/O contention on the index.	<pre>CREATE INDEX... PIECESIZE ...</pre> or <pre>ALTER INDEX... PIECESIZE ...</pre>	None

The following table lists the XML index type.

Table 16. Index types that are applicable to only tables with XML columns

Type	Description	SQL syntax	More information
XML index	An index that uses a particular XML pattern expression to index paths and values in XML documents that are stored in a single XML column.	CREATE INDEX... GENERATE KEY USING XMLPATTERN	XML index attributes (Db2 Administration Guide)
		or CREATE INDEX... GENERATE KEYS USING XMLPATTERN	

Additionally, when you create any of these types of indexes, you can define whether they have the following characteristics:

Table 17. General index characteristics

Characteristic	Description	SQL syntax	More information
Padded	Any varying-length string columns in the index are padded with the default pad character to their maximum length.	CREATE INDEX... PADED ...	Indexes that are padded or not padded (Db2 Administration Guide)
		or ALTER INDEX... PADED ...	
Compressed	The data is compressed to reduce the size of the index on disk.	CREATE INDEX... COMPRESS YES...	Compression of indexes (Db2 Administration Guide)
		or ALTER INDEX... COMPRESS YES...	

Related concepts

[Index keys \(Db2 Administration Guide\)](#)

[Implementing Db2 indexes \(Db2 Administration Guide\)](#)

Related reference

[ALTER INDEX statement \(Db2 SQL\)](#)

[CREATE INDEX statement \(Db2 SQL\)](#)

How indexes can help to avoid sorts

Db2 can use indexes to avoid sorts when processing queries with the ORDER BY clause.

When a query contains an ORDER BY clause, Db2 looks for indexes that satisfy the order in the query. For Db2 to be able to use an index to access ordered data, you must define an index on the same columns as specified in the ORDER BY clause.

Forward index scan

For Db2 to use a forward index scan, the ordering must be exactly the same as in the ORDER BY clause.

Backward index scan

For Db2 to use a backward index scan, the ordering must be exactly the opposite of what is requested in the ORDER BY clause.

In addition to forward and backward scans, you have the option to create indexes with a pseudo-random order. This ordering option is useful when ascending insertions or hotspots cause contention within the indexes. Indexes created with the RANDOM option do not support range scans. They do support equality lookups.

Examples

GUIP

Example 1

For example, if you define an index by specifying DATE DESC, TIME ASC as the column names and order, Db2 can use this same index for both of the following ORDER BY clauses:

- Forward scan for ORDER BY DATE DESC, TIME ASC
- Backward scan for ORDER BY DATE ASC, TIME DESC

You do not need to create two indexes for the two ORDER BY clauses. Db2 can use the same index for both forward index scan and backward index scan.

Example 2

Suppose that the query includes a WHERE clause with a predicate of the form COL=*constant*. For example:

```
...  
WHERE CODE = 'A'  
ORDER BY CODE, DATE DESC, TIME ASC
```

Db2 can use any of the following index keys to satisfy the ordering:

- CODE, DATE DESC, TIME ASC
- CODE, DATE ASC, TIME DESC
- DATE DESC, TIME ASC
- DATE ASC, TIME DESC

Db2 can ignore the CODE column in the ORDER BY clause and the index because the value of the CODE column in the result table of the query has no effect on the order of the data. If the CODE column is included, it can be in any position in the ORDER BY clause and in the index.

GUIP

Related reference

[order-by-clause \(Db2 SQL\)](#)

Index keys

The usefulness of an index depends on the design of its key, which you define at the time that you create the index.

An *index key* is a column, an ordered collection of columns, or an expression on which you define an index. Db2 uses an index key to determine the order of index entries. Good candidates for index keys are columns or expressions that you use frequently in operations that select, join, group, and order data.

All index keys do not need to be unique. For example, an index on the SALARY column of the sample EMP table allows duplicates because several employees can earn the same salary.

A *composite key* is an index key that is built on 2 or more columns. An index key can contain up to 64 columns.

GUIP

For example, the following SQL statement creates a unique index on the EMPPROJECT table. A composite key is defined on two columns, PROJNO and STDTE.

```
CREATE UNIQUE INDEX XPROJAC1
ON EMPPROJECT
  (PROJNO ASC,
   STDATE ASC)
:
```

This composite key is useful when you need to find project information by start date. Consider a SELECT statement that has the following WHERE clause:

```
WHERE PROJNO='MA2100' AND STDATE='2004-01-01'
```

This SELECT statement can execute more efficiently than if separate indexes are defined on PROJNO and on STDATE.

GUIP

In general, try to create an index that is selective, because the more selective an index is, the more efficient it is. An efficient index contains multiple columns, is ordered in the same sequence as the SQL statement, and is used often in SQL statements. To create an efficient index, consider the following recommendations when you create an index and define the index keys:

- Define as few indexes as possible on a column that is updated frequently because every change to the column data must be reflected in each index.
- Consider using a composite key, which might be more useful than a key on a single column when the comparison is for equality. A single multicolumn index is more efficient when the comparison is for equality and the initial columns are available. However, for more general comparisons, such as *A > value* AND *B > value*, multiple indexes might be more efficient.

Related concepts

[Query and application performance analysis](#)

Performance analysis for queries and applications begins by answering some basic questions.

General index attributes

You typically determine which type of index you need to define after you define a table space. An index can have many different attributes.

Index attributes fall into two broad categories: general attributes that apply to indexes on all tables and specific attributes that apply to indexes on partitioned tables only. The following table summarizes these categories.

Table 18. Index attributes

Table or table space type	Index attribute
Any	• Unique or nonunique • Clustering or nonclustering • Padded or not padded • Exclude nulls
Partitioned	• Partitioning • Secondary

This topic explains the types of indexes that apply to all tables. Indexes that apply to partitioned tables only are covered separately.

Related concepts

[Indexes on partitioned tables \(Db2 Administration Guide\)](#)

Unique indexes

Db2 uses unique indexes to ensure that no identical key values are stored in a table.

When you create a table that contains a primary key or a unique constraint, you must create a unique index for the primary key and for each unique constraint. Db2 marks the table definition as incomplete until the explicit creation of the required enforcing indexes, which can be created implicitly depending on whether the table space was created implicitly, the schema processor, or the CURRENT RULES special register. If the required indexes are created implicitly, the table definition is not marked as incomplete.

Restrict access with unique indexes

You can also use indexes to meet access requirements.

Example

A good candidate for a unique index is the EMPNO column of the EMP table. The following figure shows a small set of rows from the EMP table and illustrates the unique index on EMPNO.

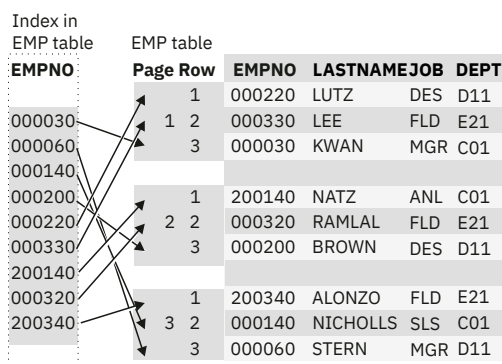


Figure 15. A unique index on the EMPNO column

Db2 uses this index to prevent the insertion of a row to the EMP table if its EMPNO value matches that of an existing row. The preceding figure illustrates the relationship between each EMPNO value in the index and the corresponding page number and row. Db2 uses the index to locate the row for employee 000030, for example, in row 3 of page 1.

If you do not want duplicate values in the key column, create a unique index by using the UNIQUE clause of the CREATE INDEX statement.

Example

GUI The DEPT table does not allow duplicate department IDs. Creating a unique index, as the following example shows, prevents duplicate values.

```
CREATE UNIQUE INDEX MYINDEX
ON DEPT (DEPTNO);
```

The index name is MYINDEX, and the indexed column is DEPTNO.

If a table has a primary key (as the DEPT table has), its entries must be unique. Db2 enforces this uniqueness by defining a unique index on the primary key columns, with the index columns in the same order as the primary key columns.

GUI

Before you create a unique index on a table that already contains data, ensure that no pair of rows has the same key value. If Db2 finds a duplicate value in a set of key columns for a unique index, Db2 issues an error message and does not create the index.

If an index key allows nulls for some of its column values, you can use the WHERE NOT NULL clause to ensure that the non-null values of the index key are unique.

Unique indexes are an important part of implementing referential constraints among the tables in your Db2 database. You cannot define a foreign key unless the corresponding primary key already exists and has a unique index defined on it.

When not to use a unique index

In some cases you might not want to use a unique index. You can improve the performance of data access when the values of the columns in the index are not necessarily unique by creating a default index.

When you create a default index, Db2 allows you to enter duplicate values in a key column.

For example, assume that more than one employee is named David Brown. Consider an index that is defined on the FIRSTNAME and LASTNAME columns of the EMP table.

GUIP

```
CREATE INDEX EMPNAME ON EMP (FIRSTNAME, LASTNAME);
```

GUIP

This is an example of an index that can contain duplicate entries.

Tip: Do not create this type of index on very small tables because scans of the tables are more efficient than using indexes.

INCLUDE columns

Unique indexes can include additional columns that are not part of a unique constraint. Those columns are called INCLUDE columns. When you specify INCLUDE columns in a unique index, queries can use the unique index for index-only access. Including these columns can eliminate the need to maintain extra indexes that are used solely to enable index-only access.

Related reference

[CREATE INDEX statement \(Db2 SQL\)](#)

Nonunique indexes

You can use nonunique indexes to improve the performance of data access when the values of the columns in the index are not necessarily unique.

Recommendation: Do not create nonunique indexes on very small tables, because scans of the tables are more efficient than using indexes.

To create nonunique indexes, use the SQL CREATE INDEX statement. For nonunique indexes, Db2 allows users and programs to enter duplicate values in a key column.

Example

GUIP Assume that more than one employee is named David Brown. Consider an index that is defined on the FIRSTNAME and LASTNAME columns of the EMP table.

```
CREATE INDEX EMPNAME  
ON EMP (FIRSTNAME, LASTNAME);
```

This index is an example of a nonunique index that can contain duplicate entries.

GUIP

Related tasks

[Designing indexes for performance \(Db2 Performance\)](#)

Related reference

[CREATE INDEX statement \(Db2 SQL\)](#)

Clustering indexes

A *clustering index* determines how rows are physically ordered (clustered) in a table space. Clustering indexes provide significant performance advantages in some operations, particularly those that involve many records. Examples of operations that benefit from clustering indexes include grouping operations, ordering operations, and comparisons other than equal.

Any index, except for an expression-based index or an XML index, can be a clustering index. You can define only one clustering index on a table.

You can define a clustering index on a partitioned table space or on a segmented table space. On a partitioned table space, a clustering index can be a partitioning index or a secondary index. If a clustering index on a partitioned table is not a partitioning index, the rows are ordered in cluster sequence within each data partition instead of spanning partitions. (Prior to Version 8 of Db2 UDB for z/OS, the partitioning index was required to be the clustering index.)

Restriction: An expression based index or an XML index cannot be a clustering index.

When a table has a clustering index, an INSERT statement causes Db2 to insert the records as nearly as possible in the order of their index values. The first index that you define on the table serves implicitly as the clustering index unless you explicitly specify CLUSTER when you create or alter another index. For example, if you first define a unique index on the EMPNO column of the EMP table, Db2 inserts rows into the EMP table in the order of the employee identification number unless you explicitly define another index to be the clustering index.

Although a table can have several indexes, only one index can be a clustering index. If you do not define a clustering index for a table, Db2 recognizes the first index that is created on the table as the implicit clustering index when it orders data rows.

Tip:

- Always define a clustering index. Otherwise, Db2 might not choose the key that you would prefer for the index.
- Define the sequence of a clustering index to support high-volume processing of data.

You use the CLUSTER clause of the CREATE INDEX or ALTER INDEX statement to define a clustering index.

Example

GUPI Assume that you often need to gather employee information by department. In the EMP table, you can create a clustering index on the DEPTNO column.

```
CREATE INDEX DEPT_IX
  ON EMP
  (DEPTNO ASC)
  CLUSTER;
```

As a result, all rows for the same department are probably close together. Db2 can generally access all the rows for that department in a single read. (Using a clustering index does not guarantee that all rows for the same department are stored on the same page. The actual storage of rows depends on the size of the rows, the number of rows, and the amount of available free space. Likewise, some pages may contain rows for more than one department.)

GUPI

The following figure shows a clustering index on the DEPT column of the EMP table; only a subset of the rows is shown.

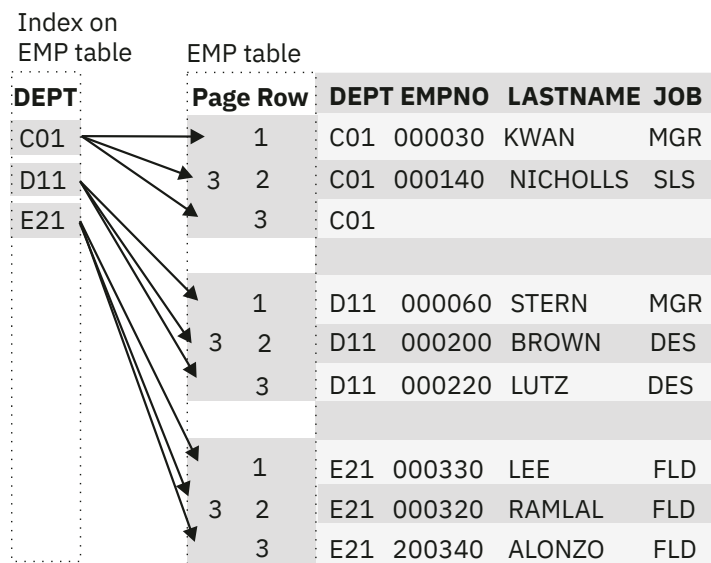


Figure 16. A clustering index on the EMP table

Suppose that you subsequently create a clustering index on the same table. In this case, Db2 identifies it as the clustering index but does not rearrange the data that is already in the table. The organization of the data remains as it was with the original nonclustering index that you created. However, when the REORG utility reorganizes the table space, Db2 clusters the data according to the sequence of the new clustering index. Therefore, if you know that you want a clustering index, you should define the clustering index before you load the table. If that is not possible, you must define the index and then reorganize the table. If you create or drop and re-create a clustering index after loading the table, those changes take effect after a subsequent reorganization.

Related reference

[Employee table \(DSN8C10.EMP\)](#)

The sample employee table identifies all employees by an employee number and lists basic personnel information.

[CREATE INDEX statement \(Db2 SQL\)](#)

Indexes that exclude NULL keys

You can exclude NULL keys from an index to reduce the size of an index and improve the performance of an index.

Some table values are never used in queries and are unnecessary in an index. NULL key columns add to index size and can reduce the performance of index scans. If you exclude NULL key columns from an index, Db2 only creates index entries for key columns that are not null. You can specify that an index excludes null keys when you create an index with the CREATE INDEX statement.

A NULL key in an index is not the same as a null foreign key.

Related reference

[CREATE INDEX statement \(Db2 SQL\)](#)

Indexes that are padded or not padded

The NOT PADDED and PADDED options of the CREATE INDEX and ALTER INDEX statements specify how varying-length string columns are stored in an index.

GUI

You can choose not to pad varying-length string columns in the index to their maximum length (the default), or you can choose to pad them.

If you specify the NOT PADDED clause on a CREATE INDEX statement, any varying-length columns in the index key are not padded to their maximum length. If an existing index key includes varying-length columns, you can consider altering the index to use the NOT PADDED clause. However, using the NOT PADDED clause on the ALTER INDEX statement to change the padding places the index in the REBUILD-pending (RBDP) state. You should rebuild the index to remove the RBDP state.

GUIP

The PADIX subsystem parameter controls whether varying length columns are padded by default. For more information, see [PAD INDEXES BY DEFAULT field \(PADIX subsystem parameter\) \(Db2 Installation and Migration\)](#).

Padded indexes prevent index-only access to data in the following situations:

- Varying-length data is returned
- A VARCHAR column has a LIKE predicate
- An index on expression is the left-hand side of a LIKE predicate, and the index does not contain varying length data.

Tip: If your application typically accesses varying-length columns, use the NOT PADDED clause to implement index-only access.

Using the NOT PADDED clause has the following advantages:

- Db2 can use index-only access for the varying-length columns within the index key, which enhances performance.
- Db2 stores only actual data, which reduces the storage requirements for the index key.

However, using the NOT PADDED clause might also have the following disadvantages:

- Index key comparisons are slower because Db2 must compare each pair of corresponding varying-length columns individually instead of comparing the entire key when the columns are padded to their maximum length.
- Db2 stores an additional 2-byte length field for each varying-length column. Therefore, if the length of the padding (to the maximum length) is less than or equal to 2 bytes, the storage requirements could actually be greater for varying-length columns that are not padded.

Related concepts

[Index access \(ACCESSTYPE is 'I', 'IN', 'I1', 'N', 'NR', 'MX', or 'DX'\) \(Db2 Performance\)](#)

Related reference

[CREATE INDEX statement \(Db2 SQL\)](#)

Expression-based indexes

By using the expression-based index capability of Db2, you can create an index that is based on a general expression. You can enhance query performance if Db2 chooses the expression-based index.

Use expression-based indexes when you want an efficient evaluation of queries that involve a column-expression. In contrast to simple indexes, where index keys consist of a concatenation of one or more table columns that you specify, the index key values are not the same as values in the table columns. The values have been transformed by the expressions that you specify.

You can create the index by using the CREATE INDEX statement, and specifying an expression, rather than a column name. If an index is created with the UNIQUE option, the uniqueness is enforced against the values that are stored in the index, not against the original column values.

Db2 does not use expression-based indexes for queries that use sensitive static scrollable cursors.

Related concepts

[Expressions \(Db2 SQL\)](#)

[Index keys \(Db2 Administration Guide\)](#)

Related reference

[CREATE INDEX statement \(Db2 SQL\)](#)

Compression of indexes

You can reduce the amount of space that an index occupies on disk by compressing the index.

The COMPRESS YES/NO clause of the ALTER INDEX and CREATE INDEX statements allows you to compress the data in an index and reduce the size of the index on disk. However, index compression is heavily data-dependent, and some indexes might contain data that does not yield significant space savings. Compressed indexes might also use more real and virtual storage than non-compressed indexes. The amount of additional real and virtual storage that is required depends on the compression ratio that is used for the compressed keys, the amount of free space, and the amount of space that is used by the key map.

You can choose 8 KB, 16 KB, and 32 KB buffer pool page sizes for the index. Use the DSN1COMP utility on existing indexes to estimate the appropriate page size for new indexes. Choosing a 32 KB buffer pool instead of a 16 KB or an 8 KB buffer pool accommodates a potentially higher compression ratio, but this choice also increases the potential to use more storage. Estimates for index space savings from the DSN1COMP utility, either on the true index data or some similar index data, are not exact.

If I/O is needed to read an index, the CPU degradation for an index scan is probably relatively small, but the CPU degradation for random access is likely to be very significant.

CPU degradation for deletes and updates is significant even if no read I/O is necessary.

Related reference

[ALTER INDEX statement \(Db2 SQL\)](#)

[CREATE INDEX statement \(Db2 SQL\)](#)

XML index attributes

You can create an index on an XML column for efficient evaluation of XQuery expressions to improve performance for queries on XML documents.

In simple relational indexes, index keys are composed of one or more table columns that you specified. However, an *XML index* uses a particular XML pattern expression to index paths and values in XML documents that are stored in a single XML column.

In an XML index, only the attribute nodes, text nodes, or element nodes that match the XML pattern expression are indexed. An XML index only indexes the nodes that match the specific XML pattern and not the document itself. Two more key fields are added to the index to form the composite index key. The extra key fields, which identify the XML document and the node position within the document, are stored in the catalog. These fields are not involved in uniqueness checking for unique indexes.

Use the CREATE INDEX statement with the XMLPATTERN keyword to create an XML index. You must also specify the XML path to be indexed. An index key is formed by concatenating the values that are extracted from the nodes in the XML document that satisfy the specified XML path with the document and node ID.

You specify a data type for every XML index. XML indexes support the following data types:

- VARCHAR
- DECFLOAT
- TIMESTAMP(12)
- DATE

You can use the following clauses to control whether Db2 inserts values into a table that are not compatible with the index data type:

- IGNORE INVALID VALUES
- REJECT INVALID VALUES

When you index an XML column with XMLPATTERN, only the parts of the document that satisfy the XML pattern expression are indexed. Multiple parts of the document might satisfy the XML pattern that you specified in the XMLPATTERN. Therefore, more than one index key entry might be generated and inserted into the index for the insertion of a single document.

Only one XML index specification is allowed per CREATE INDEX statement. However, you can create an XML index with multiple keys, or create multiple XML indexes on an XML column.

Restriction: Partitioned XML indexes are not supported

Examples

Example 1

GUIP Assume that you must search for a specific employee's surname (name/last) on the employee elements. You can use the following CREATE INDEX statement to create an index on the ' / department/emp/name/last ' XML pattern expression:

```
CREATE INDEX EMPINDEX ON DEPARTMENT (DEPTDOCS)
  GENERATE KEYS USING XMLPATTERN '/department/emp/name/last'
  AS SQL VARCHAR(20)
```

After the EMPINDEX index is created successfully, several entries are populated in the catalog tables.

GUIP

Example 2

You can create two XML indexes with the same pattern expression by using different data types for each. You can use the different indexes to choose how you want to interpret the result of the expression as multiple data types. For example, the value '12345' has a character representation but it can also be interpreted as the number 12,345. For example, assume that you want to index the path ' / department/emp/@id ' as both a character string and a number. You must create two indexes, one for the VARCHAR data type and one for the DECFLOAT data type. The values in the document are cast to the specified data type for each index.

Related concepts

[Storage structure for XML data \(Db2 Programming for XML\)](#)

[Processing XML data with Db2 pureXML](#)

You can use pureXML with your client applications to manage XML data in Db2 tables. You can store well-formed XML documents in their hierarchical form and retrieve all or portions of those documents.

[XML data indexing \(Db2 Programming for XML\)](#)

[Pattern expressions \(Db2 Programming for XML\)](#)

[Best practices for XML performance in Db2 \(Db2 Performance\)](#)

Related reference

[XMLEXISTS predicate \(Db2 SQL\)](#)

[CREATE INDEX statement \(Db2 SQL\)](#)

Indexes on partitioned tables

The following types of indexes apply to only partitioned tables: partitioned indexes, partitioning indexes (PIs), data-partitioned secondary indexes (DPSIs), and nonpartitioned secondary indexes (NPIs or NPSIs).

Partitioned index

A *partitioned index* is an index that is physically partitioned. Any index on a partitioned table, except for an XML index, can be physically partitioned.

To create a partitioned index, specify PARTITIONED in the CREATE INDEX statement.

A partitioned index consists of multiple data sets. Each data set corresponds to a table partition. The following figure illustrates the difference between a partitioned index and a nonpartitioned index.

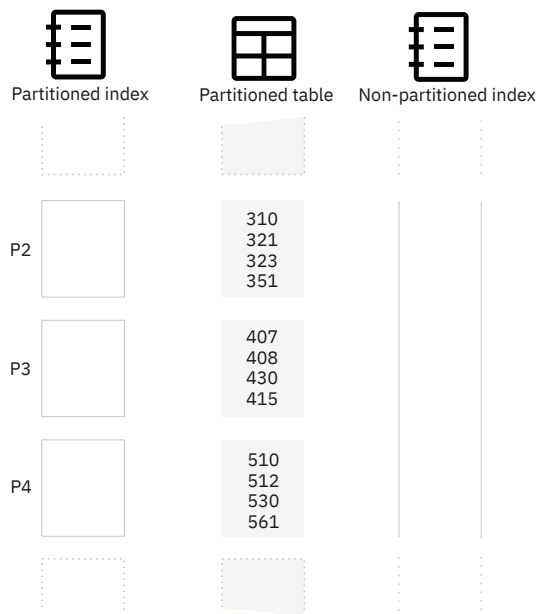


Figure 17. Comparison of partitioned and nonpartitioned index

Partitioning index

A *partitioning index* is an index on the column or columns that partition the table. Partitioning indexes are generally not required because Db2 uses *table-controlled partitioning*, where the partitioning scheme (the partitioning key and limit key values) are already defined in the table definition.

The CREATE INDEX statement does not have a specific SQL keyword that designates an index as a partitioning index. Instead, an index is a partitioning index if the index key that is specified in the CREATE INDEX statement matches the partitioning key. The *partitioning key* is the column or columns that are specified in the PARTITION BY clause of the CREATE TABLE statement. Those columns partition the table. An index key matches the partitioning key if it has the same leftmost columns and collating sequence (ASC/DESC) as the columns in the partitioning key.

A partitioning key is different from the limit key values. A partitioning key defines the columns on which the table is partitioned. The limit key values define which values belong in each partition. Specifically, a *limit key value* is the value of the partitioning key that defines the partition boundary. It is the highest value of the partitioning key for an ascending index, or the lowest value for a descending index. Limit key values are specified in the PARTITION... ENDING AT clause of a CREATE TABLE statement or ALTER TABLE statement. The specified ranges partition the table space and the corresponding partitioning index space.

Remember: Partitioning is different from clustering. Whereas, *partitioning* guarantees that rows are grouped into certain partitions based on value ranges defined partition limit key, *clustering* controls how rows are physically ordered in a partition or table space. Clustering is controlled by a clustering index and can apply to any type of table space. For more information, see [Clustering indexes \(Db2 Administration Guide\)](#).

Tables created in earlier Db2 releases might still use *index-controlled partitioning*, where the partitioning scheme was not defined as part of the table definition. In this case, a partitioning index is required to specify the partitioning scheme. (The partitioning key and the limit key values were specified in the PART VALUES clause of the CREATE INDEX statement.)

Deprecated function: Db2 12 can still process range-partitioned tables and indexes that use index-controlled partitioning. However, such tables and indexes are deprecated. For best results, convert them to use table-controlled partitioning (and a PBR table space) as soon as possible. For more information, see [Converting table spaces to use table-controlled partitioning \(Db2 Administration Guide\)](#).

Example partitioning index

GUIP For example, assume that you created an AREA_CODES table that contains area codes by state by issuing the following CREATE TABLE statement.

```
CREATE TABLE AREA_CODES
  (AREACODE_NO INTEGER NOT NULL,
   STATE CHAR (2) NOT NULL)

  PARTITION BY RANGE (AREACODE_NO)
  (PARTITION 1 ENDING AT (299),
   PARTITION 2 ENDING AT (399),
   PARTITION 3 ENDING AT (499),
   PARTITION 4 ENDING AT (599),
   PARTITION 5 ENDING AT (699),
   PARTITION 6 ENDING AT (799),
   PARTITION 7 ENDING AT (899),
   PARTITION 8 ENDING AT (MAXVALUE));
```

Optionally, you can issue the following CREATE INDEX statement to create a partitioning index on the example AREA_CODES table. This index is not required because the partitioning scheme of the AREA_CODES table is defined in its CREATE TABLE statement, and AREA_CODES uses table -controlled partitioning.

```
CREATE INDEX AREACODE_IX1 ON AREA_CODES (AREACODE_NO)
  CLUSTER PARTITIONED;
```

Tip: If you use a partitioning index for clustering, the data rows can be physically ordered across the entire table space.

GUIP

The following figure illustrates the partitioning index on the AREA_CODES table.

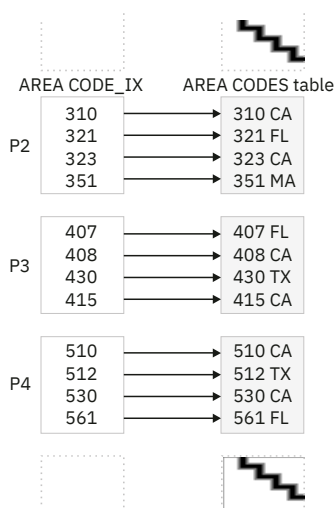


Figure 18. Partitioning index on the AREA_CODES table

Related information

[Partitioning data in Db2 tables \(Db2 Administration Guide\)](#)

[“Creation of a table with table-controlled partitioning” on page 105](#)

[Changing the boundary between partitions \(Db2 Administration Guide\)](#)

[Clustering indexes \(Db2 Administration Guide\)](#)

[Converting table spaces to use table-controlled partitioning \(Db2 Administration Guide\)](#)

Nonpartitioning index (secondary index)

An index that is not a partitioning index is a *nonpartitioning index* or *secondary index*. You can create a secondary index on a table to enforce a unique constraint, to cluster data, or to provide access paths to data for queries.

The usefulness of an index depends on the columns in its key and the cardinality of the key. Columns that you frequently select, join, group, or order are good candidates for keys. In addition, the number of distinct values in an index key for a large table must be sufficient for Db2 to use the index to retrieve data. Otherwise, Db2 might choose to do a table space scan.

You can create two types of secondary indexes: those that are partitioned (called data-partitioned secondary indexes) and those that are nonpartitioned (called nonpartitioned secondary indexes).

Data-partitioned secondary index (DPSI)

A *data-partitioned secondary index* (DPSI) is a nonpartitioning index that is physically partitioned according to the partitioning scheme of the underlying data.

A DPSI has as many partitions as the number of partitions in the table space. Each DPSI partition contains keys for the rows of the corresponding table space partition only. For example, if the table space has three partitions, the keys in DPSI partition 1 reference only the rows in table space partition 1; the keys in DPSI partition 2 reference only the rows in table space partition 2, and so on.

Restrictions:

- You can create a DPSI only on a table in a partitioned table space.
- You cannot create a DPSI for a partition-by-growth table space.
- An XML index cannot be a DPSI.

To define a DPSI, use the **PARTITIONED** keyword in the **CREATE INDEX** statement and specify an index key that does not match the partitioning key columns. If the leftmost columns of the index that you specify with the **PARTITIONED** keyword match the partitioning key, Db2 creates the index as a DPSI only if the collating sequence of the matching columns is different.

The use of DPSIs promotes partition independence and therefore provides the following performance advantages, among others:

- Eliminates contention between parallel LOAD utility jobs with the **PART** option that target different partitions of a table space
- Facilitates partition-level operations such as adding a partition or rotating a partition to be the last partition
- Improves the recovery time of secondary indexes on partitioned table spaces

However, the use of DPSIs does not always improve the performance of queries. For example, for queries with predicates that reference only the columns in the key of the DPSI, Db2 must probe each partition of the index for values that satisfy the predicate.

DPSIs provide performance advantages for queries that meet all of the following criteria:

- The query has predicates on the DPSI columns.
- The query contains additional predicates on the partitioning columns of the table that limit the query to a subset of the partitions in the table.

Nonpartitioned secondary index (NPI or NPSI)

A *nonpartitioned secondary index* (NPI or NPSI) is any index that is not defined as a partitioning index and is not physically partitioned. An NPI index has one index space that contains keys for the rows of all partitions of the table space.

You can create an NPI on a table in a partitioned table space. These indexes do not apply to nonpartitioned table spaces.

Although an NPI is not physically partitioned, it does have logical partitions. A *logical partition* is the subset of index entries that correspond to a particular table space partition.

NPIs provide performance advantages for queries that meet the following criteria:

- The query does not contain predicates on the partitioning columns of the table that limit the query to a small subset of the partitions in the table.
- The query qualifications match the index columns.
- The SELECT list columns are included in the index (for index-only access).

Examples of DPSI and NPI advantages

GUIP

To understand the advantages of using DPSIs and NPIs, consider the following example indexes on the AREA_CODES table:

A data partitioned secondary index (DPSI) on the STATE column

Assuming that the AREA_CODES table not partitioned on the STATE column, the following CREATE INDEX statement creates a DPSI on the AREA_CODES table.

```
CREATE INDEX DPSIIX2 ON AREA_CODES (STATE) PARTITIONED;
```

The following example query can make efficient use of the example DPSI. The number of key values that need to be searched is limited to just the key values of the qualifying partitions, which are only those with partitioning key values that are less than or equal to 300.

```
SELECT STATE FROM AREA_CODES
WHERE AREACODE_NO <= 300 AND STATE = 'CA';
```

A nonpartitioned index (NPI) on the STATE column

Assuming that the AREA_CODES table not partitioned on the STATE column, the following CREATE INDEX statement creates an NPI on the AREA_CODES table.

```
CREATE INDEX NPSIIX3 ON AREA_CODES (STATE);
```

The following query can make efficient use of the example NPI. The number of key values that need to be searched is limited to scanning the index key values that are greater than 'CA'.

```
SELECT STATE FROM AREA_CODES
WHERE STATE > 'CA';
```

The following figure illustrates the structure of the example indexes.

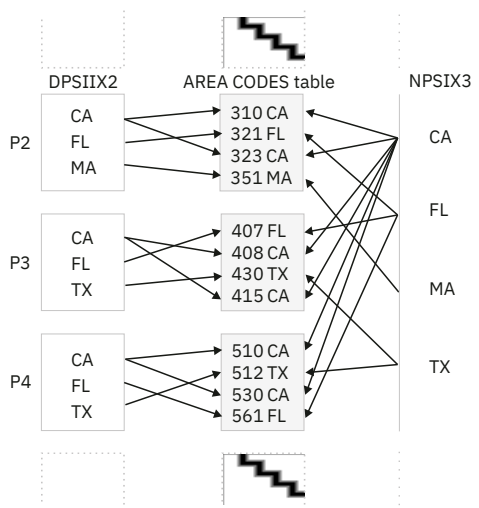


Figure 19. DPSI and NPI on the AREA_CODES table

GUIP

DPSIs provide advantages over NPIs for utility processing. For example, utilities such as COPY, REBUILD INDEX, and RECOVER INDEX can operate on physical partitions rather than logical partitions because the keys for a data partition reside in a single DPSI partition. This method can provide greater availability.

Related concepts

[Page range screening \(PAGE_RANGE='Y'\) \(Db2 Performance\)](#)

[Efficient queries for tables with data-partitioned secondary indexes \(Db2 Performance\)](#)

[Table space scan access \(ACCESSTYPE='R' and PREFETCH='S'\) \(Db2 Performance\)](#)

Related tasks

[Designing indexes for performance \(Db2 Performance\)](#)

Creation of views

When you design your database, you might need to give users access to only certain pieces of data. You can give users controlled access by designing and using views.

Use the CREATE VIEW statement to define and name a view. Unless you specifically list different column names after the view name, the column names of the view are the same as the column names of the underlying table. When you create different column names for your view, remember the naming conventions that you established when designing the relational database.

A SELECT statement describes the information in the view. The SELECT statement can name other views and tables, and it can use the WHERE, GROUP BY, and HAVING clauses. It cannot use the ORDER BY clause or name a host variable.

Related concepts

[Db2 views](#)

A *view* is an alternative way of representing data that exists in one or more tables. A view can include all or some of the columns from one or more base tables.

[Customized data views](#)

A view offers an alternative way of describing data that exists in one or more tables.

A view on a single table

You can create views on individual tables when you need to limit access to particular columns.

Examples

GUI

Example

Assume that you want to create a view on the DEPT table. Of the four columns in the table, the view needs only three: DEPTNO, DEPTNAME, and MGRNO. The order of the columns that you specify in the SELECT clause is the order in which they appear in the view:

```
CREATE VIEW MYVIEW AS
  SELECT DEPTNO,DEPTNAME,MGRNO
  FROM DEPT;
```

Example

In the preceding example, no column list follows the view name, MYVIEW. Therefore, the columns of the view have the same names as those of the DEPT table on which it is based. You can execute the following SELECT statement to see the view contents:

```
SELECT * FROM MYVIEW;
```

The result table looks like this:

DEPTNO	DEPTNAME	MGRNO
=====	=====	=====
A00	CHAIRMANS OFFICE	000010
B01	PLANNING	000020

C01	INFORMATION CENTER	000030
D11	MANUFACTURING SYSTEMS	000060
E21	SOFTWARE SUPPORT	-----

GUI

Related reference

[CREATE VIEW statement \(Db2 SQL\)](#)

A view that combines information from several tables

You can create a view that contains a union of more than one table. A union of more than one table is called a *join*.

Db2 provides two types of joins—an outer join and an inner join. An *outer join* includes rows in which the values in the join columns don't match, and rows in which the values match. An *inner join* includes only rows in which matching values in the join columns are returned.

Examples

GUI

Example

The following example is an inner join of columns from the DEPT and EMP tables. The WHERE clause limits the view to just those columns in which the MGRNO in the DEPT table matches the EMPNO in the EMP table:

```
CREATE VIEW MYVIEW AS
  SELECT DEPTNO, MGRNO, LASTNAME, ADMRDEPT
  FROM DEPT, EMP
  WHERE EMP.EMPNO = DEPT.MGRNO;
```

The result of executing this CREATE VIEW statement is an inner join view of two tables, which is shown below:

DEPTNO	MGRNO	LASTNAME	ADMRDEPT
=====	=====	=====	=====
A00	000010	HAAS	A00
B01	000020	THOMPSON	A00
C01	000030	KWAN	A00
D11	000060	STERN	D11

Example

Suppose that you want to create the view in the preceding example, but you want to include only those departments that report to department A00. Suppose also that you prefer to use a different set of column names. Use the following CREATE VIEW statement:

```
CREATE VIEW MYVIEWA00
  (DEPARTMENT, MANAGER, EMPLOYEE_NAME, REPORT_TO_NAME)
AS
  SELECT DEPTNO, MGRNO, LASTNAME, ADMRDEPT
  FROM EMP, DEPT
  WHERE EMP.EMPNO = DEPT.MGRNO
  AND ADMRDEPT = 'A00';
```

You can execute the following SELECT statement to see the view contents:

```
SELECT * FROM MYVIEWA00;
```

When you execute this SELECT statement, the result is a view of a subset of the same data, but with different column names, as follows:

DEPARTMENT	MANAGER	EMPLOYEE_NAME	REPORT_TO_NAME
=====	=====	=====	=====
A00	000010	HAAS	A00
B01	000020	THOMPSON	A00
C01	000030	KWAN	A00

GUPI

Related concepts

[Ways to merge lists of values](#)

There are several ways to use the UNION keyword for merging lists of values.

Inserts and updates of data through views

If you define a view on a single table, you can refer to the name of a view in insert, update, or delete operations. If the view is complex or involves multiple tables, you must define an INSTEAD OF trigger before that view can be referenced in an INSERT, UPDATE, MERGE, or DELETE statement. This information explains how the simple case is dealt with, where Db2 makes an insert or update to the base table.

To ensure that the insert or update conforms to the view definition, specify the WITH CHECK OPTION clause. The following example illustrates some undesirable results of omitting that check.

GUPI

Examples

GUPI

Example

Suppose that you define a view, V1, as follows:

```
CREATE VIEW V1 AS
  SELECT * FROM EMP
  WHERE DEPT LIKE 'D%'
```

A user with the SELECT privilege on view V1 can see the information from the EMP table for employees in departments whose IDs begin with D. The EMP table has only one department (D11) with an ID that satisfies the condition.

Assume that a user has the INSERT privilege on view V1. A user with both SELECT and INSERT privileges can insert a row for department E01, perhaps erroneously, but cannot select the row that was just inserted.

Example

The following example shows an alternative way to define view V1. You can avoid the situation in which a value that does not match the view definition is inserted into the base table. To do this, instead define view V1 to include the WITH CHECK OPTION clause:

```
CREATE VIEW V1 AS SELECT * FROM EMP
  WHERE DEPT LIKE 'D%' WITH CHECK OPTION;
```

With the new definition, any insert or update to view V1 must satisfy the predicate that is contained in the WHERE clause: DEPT LIKE 'D%'. The check can be valuable, but it also carries a processing cost; each potential insert or update must be checked against the view definition. Therefore, you must weigh the advantage of protecting data integrity against the disadvantage of the performance degradation.

Related tasks

[Inserting, updating, and deleting data in views by using INSTEAD OF triggers \(Db2 Application programming and SQL\)](#)

[Changing data by using views that reference temporal tables \(Db2 Administration Guide\)](#)

Related reference

[CREATE VIEW statement \(Db2 SQL\)](#)

Creating large objects

Defining large objects to Db2 is different than defining other types of data and objects.

Before you begin

Db2 sometimes implicitly creates the LOB table space, auxiliary table, and index on the auxiliary table for each LOB column in a table or partition. For more information, see [LOB table space implicit creation \(Db2 Administration Guide\)](#).

If Db2 does not implicitly create the LOB table spaces, auxiliary tables, and indexes on the auxiliary tables, you must create these objects by issuing CREATE TABLESPACE, CREATE AUXILIARY TABLE, and CREATE INDEX statements.

About this task

Large object and *LOB* refer to Db2 objects that you can use to store large amounts of data. A LOB is a varying-length character string that can contain up to 2 GB - 1 byte of data. Db2 supports the following LOB data types:

Binary large object (BLOB)

Use a BLOB to store binary data such as pictures, voice, and mixed media.

Character large object (CLOB)

Use a CLOB to store SBCS or mixed character data, such as documents.

Double-byte character large object (DBCLOB)

Use a DBCLOB to store data that consists of only DBCS data.

For more information about LOB data types, see [Large objects \(LOBs\) \(Db2 SQL\)](#).

Procedure

To define LOB columns and move large-object data into Db2, complete the following step:

1. Optional: Define at most one ROWID column when you create or alter the table, even if the table is going to have multiple LOB columns. If you do not create a ROWID column before you define a LOB column, Db2 implicitly creates a ROWID column with the IMPLICITLY HIDDEN attribute and appends it as the last column of the table.

If you add a ROWID column after you add a LOB column, the table has two ROWID columns: the implicitly-created column and the explicitly-created column. In this case, Db2 ensures that the values of the two ROWID columns are always identical.

2. Define one or more columns of the appropriate LOB type column by issuing a CREATE TABLE statement or one or more ALTER TABLE statements.
3. Create table spaces and auxiliary tables for the LOB data, unless Db2 creates them implicitly for you. For more information, see [LOB table space implicit creation \(Db2 Administration Guide\)](#).

You must create one LOB table space for each table partition and one auxiliary table for each LOB column. For example, if your base table has three partitions, you must create three LOB table spaces and three auxiliary tables for each LOB column. Use the following statements to create these objects: [CREATE LOB TABLESPACE \(Db2 SQL\)](#) and [CREATE AUXILIARY TABLE statement \(Db2 SQL\)](#).

The privilege set must include the following privileges:

- The USE privilege on the buffer pool and the storage group that is used by the LOB objects
 - If the base table space is explicitly created, CREATETS is also required on the database that contains the table (DSNDB04 if the database is implicitly created)
4. Create one index for each auxiliary table by using the CREATE INDEX statement. Each auxiliary table must have exactly one index in which each index entry refers to a LOB.
 5. Put the LOB data into Db2.

If the total length of a LOB column and the base table row is less than 32 KB, you can use the LOAD utility to put the data in Db2. You can also use SQL to put LOB data into Db2 that is less than 32KB. Even though the data resides in the auxiliary table, the LOAD utility statement or SQL statement that changes the data specifies the base table. Using INSERT or MERGE statements can be difficult because your application needs enough storage to hold the entire value that goes into the LOB column.

Example

GUIP

Assume that you must define a LOB table space and an auxiliary table to hold employee resumes. You must also define an index on the auxiliary table. You must define the LOB table space in the same database as the associated base table. Assume that EMP_PHOTO_RESUME is a base table. This base table has a LOB column named EMP_RESUME. You can use statements like this to define the LOB table space, the auxiliary table space, and the index:

```
CREATE LOB TABLESPACE RESUMETS
  IN MYDB
  LOG NO;
COMMIT;
CREATE AUXILIARY TABLE EMP_RESUME_TAB
  IN MYDB.RESUMETS
  STORES EMP_PHOTO_RESUME
  COLUMN EMP_RESUME;
CREATE UNIQUE INDEX XEMP_RESUME
  ON EMP_RESUME_TAB;
COMMIT;
```

You can use the LOG clause to specify whether changes to a LOB column in the table space are to be logged. The LOG NO clause in the preceding CREATE LOB TABLESPACE statement indicates that changes to the RESUMETS table space are not to be logged.

GUIP

Related concepts

[Large objects \(LOBs\) \(Db2 SQL\)](#)

Related tasks

[Storing LOB data in Db2 tables \(Db2 Application programming and SQL\)](#)

Related reference

[CREATE LOB TABLESPACE \(Db2 SQL\)](#)

[CREATE AUXILIARY TABLE statement \(Db2 SQL\)](#)

[ALTER TABLE statement \(Db2 SQL\)](#)

[CREATE TABLE statement \(Db2 SQL\)](#)

Creation of databases

When you define a Db2 database, you name an eventual collection of tables, associated indexes, and the table spaces in which they are to reside.

When you decide whether to define a new database for a new set of objects or use an existing database, consider the following facts:

- You can start and stop an entire database as a unit. You can display the status of all objects in the database by using a single command that names only the database. Therefore, place a set of related tables into the same database. (The same database holds all indexes on those tables.)
- If you want to improve concurrency and memory use, keep the number of tables in a single database relatively small (maximum of 20 tables). For example, with fewer tables, Db2 performs a reorganization in a shorter length of time.
- Having separate databases allows data definitions to run concurrently and also uses less space for control blocks.

To create a database, use the CREATE DATABASE statement. A name for a database is an unqualified identifier of up to eight characters. A Db2 database name must not be the same as the name of any other Db2 database.

Implicit creation of databases

If you do not specify the IN clause in a CREATE TABLE statement, Db2 implicitly creates a database.

The name of the database is DSNxxxxx, where xxxxx is the next five-digit number from a sequence, for example:

DSN00001, DSN00002, DSN00003, ...

However, if you do not specify the IN clause in a CREATE TABLESPACE statement, the table space is created in database DSNDB04.

Example

 The following example shows a valid database name:

Object
Name
Database
MYDB

This CREATE DATABASE statement creates the database MYDB:

```
CREATE DATABASE MYDB
  STOGROUP MYSTGRP
  BUFFERPOOL BP8K4
  INDEXBP BP4;
```

The STOGROUP, BUFFERPOOL, and INDEXBP clauses that this example shows establish default values. You can override these values on the definitions of the table space or index space. 

Related reference

[CREATE DATABASE statement \(Db2 SQL\)](#)

Creation of relationships with referential constraints

Referential integrity is a condition in which all intended references from data in one table column to data in another table column are valid. By using referential constraints, you can define relationships between entities that you define in Db2.

Organizations that choose to enforce referential constraints have at least one thing in common. They need to ensure that values in one column of a table are valid with respect to other data values in the database.

For example:

- A manufacturing company wants to ensure that each part in a PARTS table identifies a product number that equals a valid product number in the PRODUCTS table.
- A company wants to ensure that each value of DEPT in the EMP table equals a valid DEPTNO value in the DEPT table.

If the DBMS did not support referential integrity, programmers would need to write and maintain application code that validates the relationship between the columns. Some programs might not enforce business rules, even though it is recommended.

This programming task can be complex because of the need to make sure that only valid values are inserted or updated in the columns. When the DBMS supports referential integrity, as Db2 does, programmers avoid some complex programming tasks and can be more productive in their other work.

Related concepts

[Application of business rules to relationships](#)

Whether a given relationship is one-to-one, one-to-many, many-to-one, or many-to-many, your relationships need to make good business sense.

Related tasks

[Creating tables for data integrity \(Db2 Application programming and SQL\)](#)

[Altering a table for referential integrity \(Db2 Administration Guide\)](#)

[Using referential integrity for data consistency \(Managing Security\)](#)

How Db2 enforces referential constraints

This information describes what Db2 does to maintain referential integrity.

You define referential constraints between a foreign key and its parent key. Before you start to define the referential relationships and constraints, you should understand what Db2 does to maintain referential integrity. You should understand the rules that Db2 follows when users attempt to modify information in columns that are involved in referential constraints.

To maintain referential integrity, Db2 enforces referential constraints in response to any of the following events:

- An insert to a dependent table
- An update to a parent table or dependent table
- A delete from a parent table
- Running the CHECK DATA utility or the LOAD utility on a dependent table with the ENFORCE CONSTRAINTS option

When you define the constraints, you have the following choices:

CASCADE

Db2 propagates the action to the dependents of the parent table.

NO ACTION

An error occurs, and Db2 takes no action.

RESTRICT

An error occurs, and Db2 takes no action.

SET NULL

Db2 places a null value in each nullable column of the foreign key that is in each dependent of the parent table.

Db2 does not enforce referential constraints in a predefined order. However, the order in which Db2 enforces constraints can affect the result of the operation. Therefore, you should be aware of the restrictions on the definition of delete rules and on the use of certain statements. The restrictions relate to the following SQL statements: CREATE TABLE, ALTER TABLE, INSERT, UPDATE, MERGE, and DELETE.

You can use the NOT ENFORCED option of the referential constraint definition in a CREATE TABLE or ALTER TABLE statement to define an informational referential constraint. You should use this type of referential constraint only when an application process verifies the data in a referential integrity relationship.

Insert rules

The insert rules for referential integrity apply to parent and dependent tables.

The following insert rules for referential integrity apply to parent and dependent tables:

Parent table rules

You can insert a row at any time into a parent table without taking any action in the dependent table. For example, you can create a new department in the DEPT table without making any change to the EMP table. If you are inserting rows into a parent table that is involved in a referential constraint, the following restrictions apply:

- A unique index must exist on the parent key.
- You cannot enter duplicate values for the parent key.

- You cannot insert a null value for any column of the parent key.

Dependent table rules

You cannot insert a row into a dependent table unless a row in the parent table has a parent key value that equals the foreign key value that you want to insert. You can insert a foreign key with a null value into a dependent table (if the referential constraint allows this), but no logical connection exists if you do so. If you insert rows into a dependent table, the following restrictions apply:

- Each nonnull value that you insert into a foreign key column must be equal to some value in the parent key.
- If any field in the foreign key is null, the entire foreign key is null.
- If you drop the index that enforces the parent key of the parent table, you cannot insert rows into either the parent table or the dependent table.

Example

For example, assume your company doesn't want to have a row in the PARTS table unless the PROD# column value in that row matches a valid PROD# in the PRODUCTS table. The PRODUCTS table has a primary key on PROD#. The PARTS table has a foreign key on PROD#. The constraint definition specifies a RESTRICT constraint. Every inserted row of the PARTS table must have a PROD# that matches a PROD# in the PRODUCTS table.

Update rules

The update rules for referential integrity apply to parent and dependent tables.

The following update rules for referential integrity apply to parent and dependent tables:

Parent table rules

You cannot change a parent key column of a row that has a dependent row. If you do, the dependent row no longer satisfies the referential constraint, so Db2 prohibits the operation.

Dependent table rules

You cannot change the value of a foreign key column in a dependent table unless the new value exists in the parent key of the parent table.

Example

When an employee transfers from one department to another, the department number for that employee must change. The new value must be the number of an existing department, or it must be null. You should not be able to assign an employee to a department that does not exist. However, in the event of a company reorganization, employees might temporarily not report to a valid department. In this case, a null value is a possibility.

If an update to a table with a referential constraint fails, Db2 rolls back all changes that were made during the update.

Delete rules

Delete rules, which are applied to parent and dependent tables, are an important part of Db2 referential integrity.

The following delete rules for referential integrity apply to parent and dependent tables:

For parent tables

For any particular relationship, Db2 enforces delete rules that are based on the choices that you specify when you define the referential constraint.

For dependent tables

At any time, you can delete rows from a dependent table without acting on the parent table.

To delete a row from a table that has a parent key and dependent tables, you must obey the delete rules for that table. To succeed, the DELETE must satisfy all delete rules of all affected relationships. The DELETE fails if it violates any referential constraint.

Examples

Example 1

Consider the parent table in the department-employee relationship. Suppose that you delete the row for department C01 from the DEPT table. That deletion affects the information in the EMP table about Sally Kwan, Heather Nicholls, and Kim Natz, who work in department C01.

Example 2

Consider the dependent in the department-employee relationship. Assume that an employee retires and that a program deletes the row for that employee from the EMP table. The DEPT table is not affected.

Constructing a referential structure

When you build a referential structure, you need to create a set of tables and indexes in the correct order.

During logical design, you express one-to-one relationships and one-to-many relationships as if the relationships are bi-directional. For example:

- An employee has a resume, and a resume belongs to an employee (one-to-one relationship).
- A department has many employees, and each employee reports to a department (one-to-many relationship).

During physical design, you restate the relationship so that it is unidirectional; one entity becomes an implied parent of the other. In this case, the employee is the parent of the resume, and the department is the parent of the assigned employees.

During logical design, you express many-to-many relationships as if the relationships are both bidirectional and multivalued. During physical design, database designers resolve many-to-many relationships by using an associative table. The relationship between employees and projects is a good example of how referential integrity is built. This is a many-to-many relationship because employees work on more than one project, and a project can have more than one employee assigned.

Example

To resolve the many-to-many relationship between employees (in the EMP table) and projects (in the PROJ table), designers create a new associative table, EMP_PROJ, during physical design. EMP and PROJ are both parent tables to the child table, EMP_PROJ.

When you establish referential constraints, you must create parent tables with at least one unique key and corresponding indexes before you can define any corresponding foreign keys on dependent tables.

Related concepts

[Database design with denormalization](#)

The rules of normalization do not consider performance. In some cases, you need to consider denormalization to improve performance.

[Entities for different types of relationships](#)

In a relational database, separate entities must be defined for different types of relationships.

Related tasks

[Using referential integrity for data consistency \(Managing Security\)](#)

Creating tables in a referential structure

In a referential structure, you can create table spaces in any order. Using a model for the structure can be helpful.

You can create table spaces in any order. However, you need to create the table spaces before you perform the following steps. (This procedure uses the DEPT and EMP tables.)

Examples



1. Create the DEPT table and define its primary key on the DEPTNO column. The PRIMARY KEY clause of the CREATE TABLE statement defines the primary key.

```
CREATE TABLE DEPT
:
PRIMARY KEY (DEPTNO);
```

2. Create the EMP table and define its primary key as EMPNO and its foreign key as DEPT. The FOREIGN KEY clause of the CREATE TABLE statement defines the foreign key.

```
CREATE TABLE EMP
:
PRIMARY KEY (EMPNO)
FOREIGN KEY (DEPT)
REFERENCES DEPT (DEPTNO)
ON DELETE SET NULL;
```

3. Alter the DEPT table to add the definition of its foreign key, MGRNO.

```
ALTER TABLE DEPT
FOREIGN KEY (MGRNO)
REFERENCES EMP (EMPNO)
ON DELETE RESTRICT;
```



Related tasks

[Using referential integrity for data consistency \(Managing Security\)](#)

Creation of exception tables

Before you load tables that are involved in a referential constraint or check constraint, you need to create exception tables. An *exception table* contains the rows that the CHECK DATA utility identified because they violate referential constraints or check constraints.

Related reference

[Exception tables for the CHECK DATA utility \(Db2 Utilities\)](#)

Creation of triggers

You can use triggers to define and enforce business rules that involve different states of the data. Triggers automatically execute a set of SQL statements whenever a specified event occurs. These statements validate and edit database changes, read and modify the database, and invoke functions that perform various operations.

You define triggers by using the CREATE TRIGGER statement. The SELECT privilege is required on the table or view for which the trigger is defined.

For example, assume that the majority of your organization's salary increases are less than or equal to 10 percent. Assume also that you need to receive notification of any attempts to increase a value in the salary column by more than that amount. To enforce this requirement, Db2 compares the value of a salary before a salary increase to the value that would exist after a salary increase. You can use a trigger in this case. Whenever a program updates the salary column, Db2 activates the trigger. In the triggered action, you can specify that Db2 is to perform the following actions:

- Update the value in the salary column with a valid value, rather than preventing the update altogether.
- Notify an administrator of the attempt to make an invalid update.

As a result of using a trigger, the notified administrator can decide whether to override the original salary increase and allow a larger-than-normal salary increase.

Recommendation: For rules that involve only one condition of the data, consider using referential constraints and check constraints rather than triggers.

Triggers also move the application logic that is required to enforce business rules into the database, which can result in faster application development and easier maintenance. In the previous example, which limits salary increases, the logic is in the database, rather than in an application. Db2 checks the validity

of the changes that any application makes to the salary column. In addition, if the logic ever changes (for example, to allow 12 percent increases), you don't need to change the application programs.

Db2 supports two types of triggers, basic and advanced. For more information about the different trigger types, see [“Triggers” on page 37](#).

Related reference

[CREATE TRIGGER statement \(advanced trigger\) \(Db2 SQL\)](#)

[CREATE TRIGGER statement \(basic trigger\) \(Db2 SQL\)](#)

Creation of user-defined functions

You can create your own functions in Db2 to simplify your queries.

There are three primary types of user-defined functions.

Sourced functions

Functions that are based on existing functions.

External functions

Functions that are developed by users.

SQL functions

Functions that are defined to the database by use of SQL statements only.

External user-defined functions can return a single value or a table of values.

- External functions that return a single value are called *user-defined scalar functions*.
- External functions that return a table are called *user-defined table functions*.

User-defined functions, like built-in functions or operators, support the manipulation of distinct types.

Examples

GUPI

The following two examples demonstrate how to define and use both a user-defined function and a distinct type.

Example 1

Suppose that you define a table called EUROEMP. One column of this table, EUROSAL, has a distinct type of EURO, which is based on DECIMAL(9,2). You cannot use the built-in AVG function to find the average value of EUROSAL because AVG operates on built-in data types only. You can, however, define an AVG function that is sourced on the built-in AVG function and accepts arguments of type EURO:

```
CREATE FUNCTION AVG(EURO)
  RETURNS EURO
  SOURCE SYSIBM.AVG(DECIMAL);
```

Example 2

You can then use this function to find the average value of the EUROSAL column:

```
SELECT AVG(EUROSAL) FROM EUROEMP;
```

The next two examples demonstrate how to define and use an external user-defined function.

Example 3

Suppose that you define and write a function, called REVERSE, to reverse the characters in a string. The definition looks like the following example:

```
CREATE FUNCTION REVERSE(VARCHAR(100))
  RETURNS VARCHAR(100)
  EXTERNAL NAME 'REVERSE'
  PARAMETER STYLE SQL
  LANGUAGE C;
```

Example 4

You can then use the REVERSE function in an SQL statement wherever you would use any built-in function that accepts a character argument, as shown in the following example:

```
SELECT REVERSE (:CHARSTR)
FROM SYSDUMMY1;
```

Although you cannot write user-defined aggregate functions, you can define sourced user-defined aggregate functions that are based on built-in aggregate functions. This capability is useful in cases where you want to refer to an existing user-defined function by another name or where you want to pass a distinct type.

The next two examples demonstrate how to define and use a user-defined table function.

Example 5

You can define and write a user-defined table function that users can invoke in the FROM clause of a SELECT statement. For example, suppose that you define and write a function called BOOKS. This function returns a table of information about books on a specified subject. The definition looks like the following example:

```
CREATE FUNCTION BOOKS      (VARCHAR(40))
  RETURNS TABLE (TITLE_NAME  VARCHAR(25),
                  AUTHOR_NAME VARCHAR(25),
                  PUBLISHER_NAME VARCHAR(25),
                  ISBNNO     VARCHAR(20),
                  PRICE_AMT   DECIMAL(5,2),
                  CHAP1_TXT    CLOB(50K))
LANGUAGE COBOL
PARAMETER STYLE SQL
EXTERNAL NAME BOOKS;
```

Example 6

You can then include the BOOKS function in the FROM clause of a SELECT statement to retrieve the book information, as shown in the following example:

```
SELECT B.TITLE_NAME, B.AUTHOR_NAME, B.PUBLISHER_NAME, B.ISBNNO
FROM TABLE(BOOKS('Computers')) AS B
WHERE B.TITLE_NAME LIKE '%COBOL%';
```



Related concepts

[User-defined functions](#)

User-defined functions are small programs that you can write to perform an operation. You can use a user-defined function wherever you can use a built-in function.

[Sample user-defined functions \(Db2 SQL\)](#)

Related tasks

[Creating a user-defined function \(Db2 Application programming and SQL\)](#)

[Creating user-defined functions \(Db2 Administration Guide\)](#)

[Controlling user-defined functions \(Db2 Administration Guide\)](#)

Character conversion

A *string* is a sequence of bytes that can represent characters. Within a string, all the characters are represented by a common encoding representation. In some cases, it might be necessary to convert these characters to a different encoding representation. The process of conversion is known as *character conversion*.

Important: For best results, try to avoid character conversions whenever possible, because conversions can potentially slow performance and sometimes cause data loss. The best way to avoid conversions is to use the same CCSID for all of your data. For more information, see [Possible consequences of character conversion \(Db2 Internationalization Guide\)](#).

Character conversion, when required, is automatic, and when successful, it is transparent to the application.

In client/server environments, character conversion can occur when an SQL statement is executed remotely. Consider, for example, the following two cases. In either case, the data could have a different representation at the sending and receiving systems.

- The values of data sent from the requester to the current server
- The values of data sent from the current server to the requester

Conversion can also occur during string operations on the same system, as in the following examples:

- An overriding CCSID is specified.

For example, an SQL statement with a descriptor, which requires an SQLDA. In the SQLDA, the CCSID is in the SQLNAME field for languages other than REXX, and in the SQLCCSID field for REXX. (For more information, see [SQL descriptor area \(SQLDA\) \(Db2 SQL\)](#)). A DECLARE VARIABLE statement can also be issued to associate a CCSID with the host variables into which data is retrieved from a table.

- The value of the ENCODING bind option or the APPLICATION ENCODING SCHEMA option of the CREATE PROCEDURE or ALTER PROCEDURE statement for a native SQL procedure (static SQL statements) or the CURRENT APPLICATION ENCODING SCHEME special register (for dynamic SQL) is different than encoding scheme of the data being retrieved.
- A mixed character string is assigned to an SBCS column or host variable.
- An SQL statement refers to data that is defined with different CCSIDs.

The text of an SQL statement is also subject to character conversion because it is a character string.

The following list defines some of the terms used for character conversion.

ASCII

Acronym for American Standard Code for Information Interchange, an encoding scheme used to represent characters. The term ASCII is used throughout this information to refer to IBM-PC Data or ISO 8-bit data.

character set

A defined set of characters, a character being the smallest component of written language that has semantic value. For example, the following character set appears in several code pages:

- 26 nonaccented letters A–Z
- 26 nonaccented letters a–z
- digits 0–9
- . , : ; ? () ' " / - _ & + % * = < >

code page

A set of assignments of characters to code points. For example, in EBCDIC, "A" is assigned code point X'C1', and "B" is assigned code point X'C2'. In Unicode UTF-8, "A" is assigned code point X'41', and "B" is assigned code point X'42'. Within a code page, each code point has only one specific meaning.

code point

A unique bit pattern that represents a character. It is a numerical index or position in an encoding table used for encoding characters.

coded character set

A set of unambiguous rules that establishes a character set and the one-to-one relationships between the characters of the set and their coded representations. It is the assignment of each character in a character set to a unique numeric code value.

coded character set identifier (CCSID)

A two-byte, unsigned binary integer that uniquely identifies an encoding scheme and one or more pairs of character sets and code pages.

EBCDIC

Acronym for Extended Binary-Coded Decimal Interchange Code, an encoding scheme used to represent character data, a group of coded character sets that consist of 8 bit coded characters.

EBCDIC coded character sets use the first 64 code positions (X'00' to X'3F') for control codes. The range X'41' to X'FE' is used for single-byte characters. For double-byte characters, the first byte is in the range X'41' to X'FE' and the second byte is also in the range X'41' to X'FE', while X'4040' represents a double-byte space.

encoding scheme

A set of rules used to represent character data. All string data stored in a table must use the same encoding scheme and all tables within a table space must use the same encoding scheme, except for global temporary tables, declared temporary tables, and work file table spaces. Db2 supports these encoding schemes:

- ASCII
- EBCDIC
- Unicode

substitution character

A unique character that is substituted during character conversion for any characters in the source encoding representation that do not have a match in the target encoding representation.

Unicode

A universal encoding scheme for written characters and text that enables the exchange of data internationally. It provides a character set standard that can be used all over the world. It provides the ability to encode all characters used for the written languages of the world and treats alphabetic characters, ideographic characters, and symbols equivalently because it specifies a numeric value and a name for each of its characters. It includes punctuation marks, mathematical symbols, technical symbols, geometric shapes, and dingbats. Db2 supports these two encoding forms:

- UTF-8: Unicode Transformation Format, a 8 bit encoding form designed for ease of use with existing ASCII-based systems. UTF-8 can encode any of the Unicode characters. A UTF-8 character is 1,2,3, or 4 bytes in length. A UTF-8 data string can contain any combination of SBCS and MBCS data, including supplementary characters. The CCSID value for data in UTF-8 format is 1208. UTF-8 has multiple code points for spaces, including the '20'X single-byte space that Db2 uses for padding UTF-8 data.
- UTF-16: Unicode Transformation Format, a 16 bit encoding form designed to provide code values for over a million characters and a superset of UCS-2. UTF-16 can encode any of the Unicode characters. In UTF-16 encoding, characters are 2 bytes in length, except for supplementary characters, which take two 2 byte string units per character. The CCSID value for data in UTF-16 format is 1200. UTF-16 has multiple code points for spaces, including the '0020'X single-byte space that Db2 uses for padding UTF-16 data.

Character data (CHAR, VARCHAR, and CLOB) is encoded in Unicode UTF-8. Character strings are also used for mixed data (that is a mixture of single-byte characters and multi-byte characters) and for data that is not associated with any character set (called bit data). Graphic data (GRAPHIC, VARGRAPHIC, and DBCLOB) is encoded in Unicode UTF-16. For a comparison of some UTF-8 and UTF-16 code points for some sample characters, see “Character sets and code pages” on page 142. This table shows how a UTF-8 character can be 1 to 4 bytes in length, a non-supplementary UTF-16 character is 2 bytes in length, and how a supplementary character in either UTF-8 or UTF-16 takes two 2 byte code points.

Character conversion can affect the results of several SQL operations. In this information, the effects are described in the following topics:

- [String assignments \(Db2 SQL\)](#)
- [Conversion rules for comparisons \(Db2 SQL\)](#)
- [Character conversion in set operations and concatenations \(Db2 SQL\)](#)

Related concepts

[International data and character conversion in Db2 for z/OS \(Db2 Internationalization Guide\)](#)

Related reference

[Character conversion terminology \(Db2 Internationalization Guide\)](#)

Character sets and code pages

Even with the same encoding scheme, different CCSIDs exist, and the same code point can represent a different character in different CCSIDs. Furthermore, a byte in a character string does not necessarily represent a character from a single-byte character set (SBCS).

The following figure shows how a typical character set might map to different code points in two different code pages.

Code page: pp1 (ASCII)

	0	1	2	3	4	5		E	F
0				0	@	P		Â	
1				1	A	Q		À	α
2			†	2	B	R		Å	β
3				3	C	S		Á	γ
4				4	D	T		Ä	σ
5			%	5	E	U		Ä	ε
E			.	>	N			¼	ö
F			/	*	O			®	

Code point: 2F

Character set ss1
(in code page pp2)

Code page: pp2(EBDIC)

	0	1		A	B	C	D	E	F
0					#				0
1					\$	A	J		1
2				s	%	B	K	S	2
3				t	¬	C	L	T	3
4				u	*	D	M	U	4
5				v	(	E	N	V	5
E					!	:	Â	}	
F				À	□	;	Á	{	

Character set ss1
(in code page pp1)

Figure 20. Code page mappings for character set ss1 in ASCII and EBCDIC

For Unicode, there is only one CCSID for UTF-8 and only one CCSID for UTF-16. The following figure shows how the first 127 single code points for UTF-8 are the same as ASCII with a CCSID of 367. For example, in both UTF-8 and ASCII CCSID 367, an A is X'41' and a 1 is X'31'.

First 127 code points for UTF-8 code page

	0	1	2	3	4	5	6	7
0			(sp)	0	@	P	`	p
1			!	1	A	Q	a	q
2			"	2	B	R	b	r
3			#	3	C	S	c	s
4			\$	4	D	T	d	t
5			%	5	E	U	e	u
6			&	6	F	V	f	v
7			'	7	G	W	g	w
8			(	8	H	X	h	x
9			)	9	I	Y	i	y
A			*	:	J	Z	j	z
B			+	;	K	[	k	{
C			,	<	L	\	l	
D			-	=	M	]	m	}
E			.	>	N	^	n	~
F			/	?	O	_	o	

code point:2f

Figure 21. Code point mapping for the first 127 code points for UTF-8 single-byte characters (CCSID 1208)

The following figure shows a comparison of how some UTF-16 and UTF-8 code points map to some sample characters. The character for the eighth note musical symbol takes two 2 byte code points because it is a supplementary character.

Character glyph	UTF-8 code point	UTF-16 code point
M	4D	004D
Ä	C384	00C4
事	E4BA8B	4E8B
♪	F09D85A0	D834DD60

Figure 22. A comparison of how some UTF-8 and UTF-16 code points map to some sample characters

Related concepts

[International data and character conversion in Db2 for z/OS \(Db2 Internationalization Guide\)](#)

Related information

[Unicode Consortium](#)

Coded character sets and CCSIDS

The IBM *character data representation architecture (CDRA)* deals with the differences in string representation and encoding. The *Coded Character Set Identifier (CCSID)* is a key element of this architecture. A CCSID is a 2 byte (unsigned) binary number that uniquely identifies an encoding scheme and one or more pairs of character sets and code pages.

A CCSID is an attribute of strings, just as length is an attribute of strings. All values of the same string column have the same CCSID.

Character conversion is described in terms of CCSIDs of the source and target. With Db2 for z/OS, two methods are used to identify valid source and target combinations and to perform the conversion from one coded character set to another:

- Db2 catalog table SYSIBM.SYSSTRINGS

Each row in the catalog table describes a conversion from one coded character set to another.

- z/OS support for Unicode.

In some cases, no conversion is necessary even though the strings involved have different CCSIDs.

Different types of conversions might be supported by each database manager. Round-trip conversions attempt to preserve characters in one CCSID that are not defined in the target CCSID so that if the data is subsequently converted back to the original CCSID, the same original characters result. Enforced subset match conversions do not attempt to preserve such characters. Which type of conversion is used for a specific source and target CCSID is product-specific.

Related concepts

[International data and character conversion in Db2 for z/OS \(Db2 Internationalization Guide\)](#)

[z/OS Unicode Services User's Guide and Reference](#)

Related tasks

[Working with international data \(Db2 Installation and Migration\)](#)

Related reference

[SYSSTRINGS catalog table \(Db2 SQL\)](#)

Encoding scheme and CCSID rules for strings

An encoding scheme and a CCSID are attributes of strings, just as length is an attribute of strings. All values of the same string column have the same encoding scheme and CCSID.

Every string has an encoding scheme and a CCSID that identifies the manner in which the characters in the string are encoded. Strings can be encoded in ASCII, EBCDIC, or Unicode. The CCSID that is associated with a string value depends on the SQL statement in which the data is referenced and the type of expression.

Conditions for type 1 and type 2 rules

The encoding scheme and CCSID rules for SQL statements are divided into type 1 and type 2 rules.

The **type 1 rules** apply when the SQL statement operates with a single set of CCSIDs (SBCS, mixed, and graphic).

The **type 2 rules** apply in any of the following situations:

- The SQL statement contains any of the following items, which cause it to operate with a multiple CCSIDs:
 - References to columns from multiple tables or views that are defined with CCSIDs from more than one set of CCSIDs (SBCS, mixed, and graphic)
 - References to an EBCDIC table that contains a Unicode column
 - Graphic hexadecimal (GX) or hexadecimal Unicode (UX) string constants
 - References to the XMLCLOB built-in function
 - Cast specifications with a CCSID clause
 - User-defined table functions
- The SQL statement is any of the following statements:
 - CALL
 - SET *assignment-statement*
 - SET *special-register*
 - VALUES statement
 - VALUES INTO
- Any of the following built-in functions are referenced:
 - ASCII_CHR

- ASCII_STR (or ASCIISTR)
 - CHR
 - DECRYPT_BIT
 - DECRYPT_CHAR
 - DECRYPT_DATAKEY_BIT
 - DECRYPT_DATAKEY_CLOB
 - DECRYPT_DATAKEY_DBCLOB
 - DECRYPT_DATAKEY_VARCHAR
 - DECRYPT_DATAKEY_VARGRAPHIC
 - DECRYPT_DB
 - EBCDIC_CHR
 - EBCDIC_STR
 - GETVARIABLE
 - JSON_VAL
 - NORMALIZE_STRING
 - UNICODE_STR (or UNISTR)
 - XMLSERIALIZE
- The SQL statement includes a collection-derived table (UNNEST).

Rules for statements that use SYSDUMMYx tables

For those SQL statements and tools that use a SYSDUMMYx table, use the SYSDUMMYx table that has the same encoding scheme as the other objects in your SQL statement, to avoid conversions. A SYSDUMMYx table is available in each encoding scheme.

The last character of the table name identifies the SYSDUMMYx table that is associated with each encoding scheme as follows:

- SYSIBM.SYSDUMMY1 uses the EBCDIC encoding scheme.
- SYSIBM.SYSDUMMYE uses the EBCDIC encoding scheme.
- SYSIBM.SYSDUMMYA uses the ASCII encoding scheme.
- SYSIBM.SYSDUMMYU uses the UNICODE encoding scheme.

For example, suppose that your SQL statement references a SYSDUMMYx table that is in a different encoding scheme from other objects in the statement. Db2 treats this statement as one that references objects with different CCSIDs (type 2 rules), and conversion is likely to occur. To avoid this situation, reference the SYSDUMMYx table that has the same encoding scheme as the other objects in your SQL statement.

For more information, see [“SYSDUMMYx tables” on page 351](#).

String data type encoding scheme and CCSID rules

The following table describes the rules for determining the CCSID of derived string data when the source data has a string type.

Table 19. Rules for determining the CCSID that is associated with string data derived from string data

Source of the string data	Type 1 rules	Type 2 rules
String constant	If the statement references a table or view, the encoding scheme of that table or view determines the encoding scheme for the string constant. The CCSID is the appropriate character string CCSID of the encoding scheme.	The CCSID is the appropriate character string CCSID of the application encoding scheme.“1” on page 148
Datetime constant	If the statement references a table or view, the encoding scheme of that table or view determines the encoding scheme for the string constant. Otherwise, the default EBCDIC encoding scheme is used for the string constant. The CCSID is the appropriate character string CCSID of the encoding scheme.	The CCSID is the appropriate character string CCSID of the application encoding scheme.“1” on page 148
Hexadecimal string constant (X'...')	If the statement references a table or view, the encoding scheme of that table or view determines the encoding scheme for the string constant. Otherwise, the default EBCDIC encoding scheme is used for the string constant. The CCSID is the appropriate graphic string CCSID of the encoding scheme.	The CCSID is the appropriate character string CCSID of the application encoding scheme.“1” on page 148
Graphic string constant (G'...')	If the statement references a table or view, the encoding scheme of that table or view determines the encoding scheme for the graphic string constant. Otherwise, the default EBCDIC encoding scheme is used for the graphic string constant. The CCSID is the graphic string CCSID of the encoding scheme.	The CCSID is the graphic string CCSID of the application encoding scheme.“1” on page 148
Graphic hexadecimal constant (GX'...')	Not applicable.	The CCSID is the graphic string CCSID of the application encoding scheme, which must be ASCII or EBCDIC.
Hexadecimal Unicode string constant (UX'....')	Not applicable.	The CCSID is 1200 (UTF-16).

Table 19. Rules for determining the CCSID that is associated with string data derived from string data (continued)

Source of the string data	Type 1 rules	Type 2 rules
Special register	If the statement references a table or view, the encoding scheme of that table or view determines the encoding scheme for the special register. Otherwise, the default EBCDIC encoding scheme is used for the special register. The CCSID is the appropriate character string CCSID of the encoding scheme.	The CCSID is the appropriate CCSID of the application encoding scheme.^{"1" on page 148}
Column of a table	The CCSID is the CCSID that is associated with the column of the table.	The CCSID is the CCSID that is associated with the column of the table.
Column of a view	The CCSID is the CCSID of the column of the result table of the fullselect of the view definition.	The CCSID is the CCSID of the column of the result table of the fullselect of the view definition.
Expression	The CCSID is the CCSID of the result of the expression.	The CCSID is the CCSID of the result of the expression.
Result of a built-in function	If the description of the function, in Built-in functions and functions that are supplied with Db2 (Db2 SQL), indicates what the CCSID of the result is, the CCSID is that CCSID. Otherwise, if the description of the function refers to this table for the CCSID, the CCSID is the appropriate CCSID of the CCSID set that is used by the statement for the data type of the result.	If the description of the function, in Built-in functions and functions that are supplied with Db2 (Db2 SQL), indicates what the CCSID of the result is, the CCSID is that CCSID. Otherwise, if the description of the function refers to this table for the CCSID, the CCSID is the appropriate CCSID of the application encoding scheme for the data type of the result.^{"1" on page 148}
Parameter of a user-defined routine	The CCSID is the CCSID that was determined when the function or procedure was created.	The CCSID is the CCSID that was determined when the function or procedure was created.
The expression in the RETURN statement of a CREATE statement for a user-defined SQL scalar function	If the expression in the RETURN statement is string data, the encoding scheme is the same as for the parameters of the function. The CCSID is determined from the encoding scheme and the attributes of the data.	The CCSID is determined from the CCSID of the result of the expression specified in the RETURN statement.

Table 19. Rules for determining the CCSID that is associated with string data derived from string data (continued)

Source of the string data	Type 1 rules	Type 2 rules
String host variable	If the statement references a table or view, the encoding scheme of that table or view determines the encoding scheme for the host variable. Graphic variables are an exception if the table or view is EBCDIC or ASCII and the value of the MIXED DATA field on the DSNTIPF panel is NO. In this case, the Unicode encoding scheme is used for the graphic host variable. Otherwise, the default EBCDIC encoding scheme is used for the host variable. The CCSID is the appropriate CCSID of the data type of the host variable.	At package prepare time, the CCSID is the appropriate CCSID of the data type of the host variable for the application encoding scheme. Graphic variables are an exception if the application or encoding scheme is EBCDIC or ASCII and the value of the MIXED DATA field on the DSNTIPF panel is NO. In this case, the Unicode encoding scheme is used for the graphic host variable. At run time, the CCSID specified in the declare variable statement, or as an override in the SQLDA. Otherwise, the CCSID is the appropriate CCSID of the application encoding scheme for the data type of the host variable.

Notes:

1. If the context is within a check constraint or a package for a basic trigger, the CCSID is the appropriate CCSID for Unicode, instead of the application encoding scheme. If the context is within a package for an advanced trigger, the CCSID is determined from the implicitly or explicitly specified APPLICATION ENCODING SCHEME option; otherwise the CCSID is the appropriate CCSID for Unicode instead of the application encoding scheme.

Numeric data type encoding scheme and CCSID rules

The following table shows the rules for determining the CCSID of derived string data when the source data has a numeric type.

Table 20. Rules for determining the CCSID that is associated with string data derived from numeric data

Source of the numeric data	Type 1 rules	Type 2 rules
Numeric data	If the statement references a table or view, the encoding scheme of that table or view determines the encoding scheme for the string data that is derived from a numeric value. Otherwise, the default EBCDIC encoding scheme is used for the string data derived from a numeric value. The CCSID is the appropriate character or graphic string CCSID of the application encoding scheme.	The CCSID is the appropriate character or graphic string CCSID of the application encoding scheme. “1” on page 149

Table 20. Rules for determining the CCSID that is associated with string data derived from numeric data (continued)

Source of the numeric data	Type 1 rules	Type 2 rules
Notes: 1. If the context is within a check constraint or a package for a basic trigger, the CCSID is the appropriate CCSID for Unicode, instead of the application encoding scheme. If the context is within a package for an advanced trigger, the CCSID is determined from the implicitly or explicitly specified APPLICATION ENCODING SCHEME option; otherwise the CCSID is the appropriate CCSID for Unicode instead of the application encoding scheme.		

Type 1 rules example

Assume that the default encoding scheme for the installation is EBCDIC and that the installation does not support mixed and graphic data. The following statement conforms to the type 1 rules in Table 19 on page 146. Therefore, the X'40' is interpreted as EBCDIC SBCS data because the statement references a table that is in EBCDIC. The CCSID for X'40' is the default EBCDIC SBCS CCSID for the installation.

```
SELECT * FROM EBCDIC_TABLE WHERE COL1 = X'40';
```

the result of the query includes each row that has a value in column COL1 that is equal to a single EBCDIC blank.

Type 2 rules example

The following statement references data from two different tables that use different encoding schemes. This statement does not conform to the rules for Type 1 statements in Table 19 on page 146. Therefore, the rules for type 2 statements are used. The CCSID for X'40' is dependent on the current application encoding scheme. Assuming that the current application encoding scheme is EBCDIC, X'40' represents a single EBCDIC blank.

```
SELECT * FROM EBCDIC_TABLE, UNICODE_TABLE WHERE COL1 = X'40';
```

As with Example 1, the result of the query includes each row that has a value in column COL1 that is equal to a single EBCDIC blank. If the current application encoding scheme were ASCII or Unicode, X'40' would represent something different and the results of the query would be different.

Related concepts

[Application encoding scheme \(Db2 Internationalization Guide\)](#)

[Code pages and CCSIDs \(Db2 Internationalization Guide\)](#)

[Encoding schemes \(Db2 Internationalization Guide\)](#)

Related tasks

[Specifying a CCSID for your application \(Db2 Internationalization Guide\)](#)

Expanding conversions

An *expanding conversion* occurs when the length of the converted string is greater than that of the source string.

For example, an expanding conversion occurs when an ASCII mixed data string that contains DBCS characters is converted to EBCDIC mixed data. To prevent the loss of data on expanding conversions, use a varying-length string variable with a maximum length that is sufficient to contain the expansion.

Expanding conversions also can occur when string data is converted to or from Unicode. It can also occur between UTF-8 and UTF-16, depending on the data being converted. UTF-8 uses 1, 2, 3, or 4 bytes per character. UTF-16 uses 2 bytes per character, except for supplementary characters, which use two 2 byte

code points for each character. If UTF-8 were being converted to UTF-16, a 1 byte character would be expanded to 2 bytes.

Contracting conversions

A *contracting conversion* occurs when the length of the converted string is smaller than that of the source string.

For example, a contracting conversion occurs when an EBCDIC mixed data string that contains DBCS characters is converted to ASCII mixed data due to the removal of shift codes.

Contracting conversions also can occur when string data is converted to or from Unicode data. It can also occur between UTF-8 and UTF-16, depending on the data being converted.

Distributed relational databases

The database managers in a distributed relational database communicate and cooperate with each other in a way that allows a Db2 application program to use SQL to access data at any of the interconnected computer systems.

A *distributed relational database* consists of a set of tables and other objects that are spread across different, but interconnected, computer systems. Each computer system has a relational database manager, such as Db2, that manages the tables in its environment. The database managers communicate and cooperate with each other in a way that allows a Db2 application program to use SQL to access data at any of the computer systems. The Db2 subsystem where the application plan is bound is known as the *local Db2 subsystem*. Any database server other than the local Db2 subsystem is considered a *remote database server*, and access to its data is a distributed operation.

Distributed relational databases are built on formal requester-server protocols and functions. An *application requester* component supports the application end of a connection. It transforms an application's database request into communication protocols that are suitable for use in the distributed database network. These requests are received and processed by an *application server* component at the database server end of the connection. Working together, the application requester and application server handle the communication and location considerations so that the application is isolated from these considerations and can operate as if it were accessing a local database.

For more information on Distributed Relational Database Architecture (DRDA) communication protocols, see [DRDA Volume 1: Distributed Relational Database Architecture \(Open Group Library - Data Management Standards\)](#).

Related concepts

Distributed data access

Distributed computing environments typically involve requests from users at one DBMS client that are processed by a DBMS server. The server DBMS is typically remote to the client. Certain programming techniques and performance implications apply to distributed computing environments.

Related tasks

[Improving performance for applications that access distributed data \(Db2 Performance\)](#)

Connections

A *connection* is an association between an application process and a local or remote database server. Connections are managed by applications.

An application process must be connected to the application server facility of a database manager before SQL statements that reference tables or views can be executed. An application can use the CONNECT statement to establish a connection to a database server and make that database server the current server of the application process.

Commit processing

When Db2 for z/OS acts as a requester, it negotiates with the database server during the connection process to determine how to perform commits. If the remote server does not support two-phase commit protocol, Db2 downgrades to perform one-phase commits. Otherwise, Db2 always performs two-phase commits, which allow applications to update one or more databases in a single unit of work and are more reliable than one-phase commits. Two-phase commit is a two-step process:

1. First, all database managers involved in the same unit of work are pooled to determine whether they are ready to commit.
2. Then, if all database managers respond positively, they are directed to execute commit processing. If all database managers do not respond positively, they are directed to execute backout processing.

Db2 can also provide coordination for transactions that include both two-phase commit resources and one-phase commit resources. If an application has multiple connections to several different database servers, and if any of the connections are one-phase commit connections, then only one database that is involved in the transaction can be updated. The connections to all the other databases that are involved in the transaction are read-only.

To execute a static SQL statement that references tables or views, the bound form of the statement is taken from a package that the database manager previously created through a bind operation or when a version of a native SQL procedure was defined.

Supported SQL statements and clauses

For the most part, an application can use the statements and clauses that are supported by the database manager of the current server, even though that application might be running via the application requester of a database manager that does not support some of those statements and clauses. Restrictions to this

general rule for Db2 for z/OS are documented in  [SQL Reference for Cross-Platform Development - Version 6](#).

Distributed unit of work

The *distributed unit of work facility* provides for the remote preparation and execution of SQL statements.

An application process at computer system A can connect to a database server at computer system B and, within one or more units of work, execute any number of static or dynamic SQL statements that reference objects at B. All objects referenced in a single SQL statement must be managed by the same database server. Any number of database servers can participate in the same unit of work, and any number of connections can exist between an application process and a database server. A commit or rollback operation that does not specify a savepoint ends the unit of work.

Connection management

How connections are managed depends on what states the SQL connection and the application process are in.

At any time:

- An SQL connection is in one of the following states:
 - Current and held
 - Current and release-pending
 - Dormant and held
 - Dormant and release-pending
- An application process is in the connected or unconnected state, and has a set of zero or more SQL connections. Each SQL connection is uniquely identified by the name of the database server at the other end of the connection. Only one SQL connection is active (current) at a time.

Initial state of an application process: An application process is initially in the connected state, and it has exactly one SQL connection. The server of that connection is the local Db2 subsystem.

Initial state of an SQL connection: An SQL connection is initially in the current and held state.

The following figure shows the state transitions:

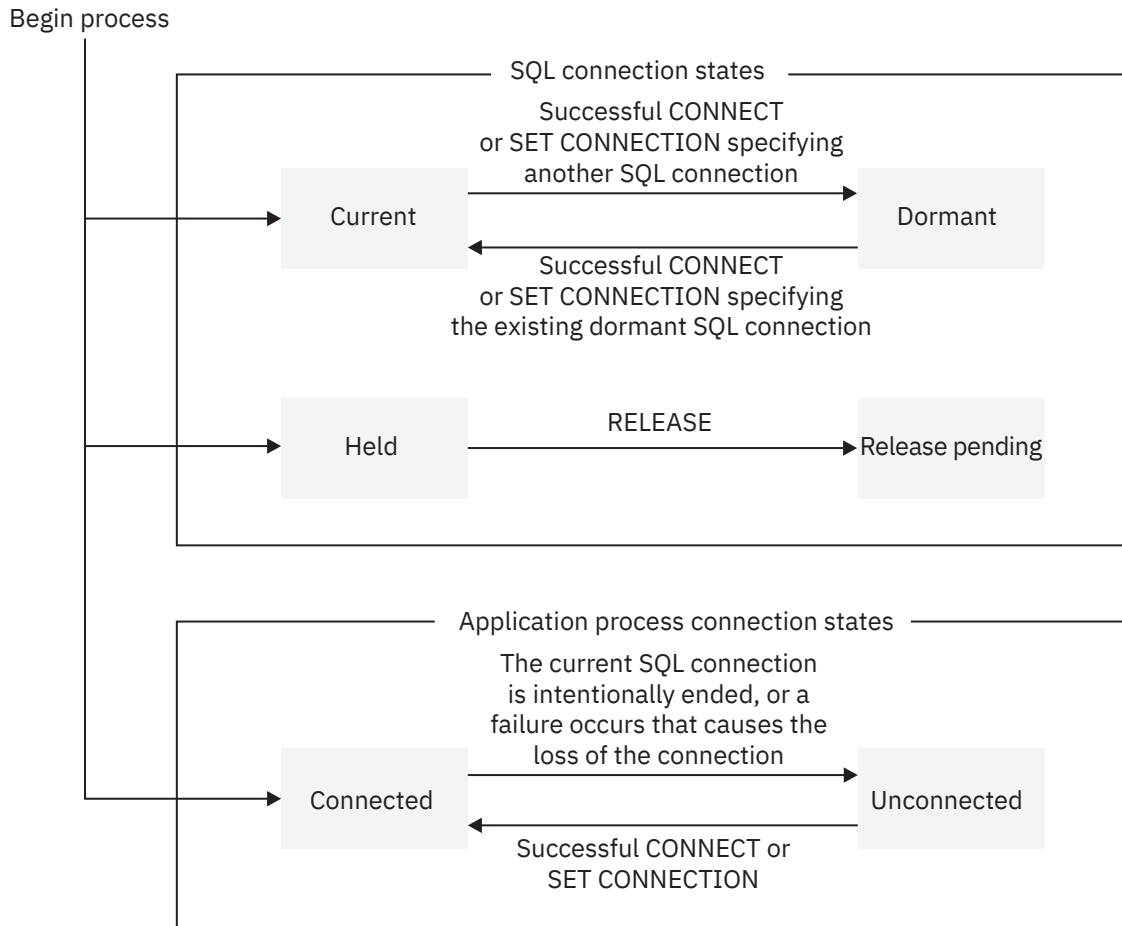


Figure 23. State transitions for an SQL connection and an application process connection in a distributed unit of work

SQL connection states

If an application process successfully executes a CONNECT statement, the SQL connection states of the connections change.

If an application process successfully executes a CONNECT statement:

- The current connection is placed in the dormant and held state.
- The new connection is placed in the current and held state.
- The location name is added to the set of existing connections.

If the location name is already in the set of existing connections, an error is returned.

An SQL connection in the dormant state is placed in the current state using:

- The SET CONNECTION statement, or
- The CONNECT statement, if the SQLRULES(DB2) bind option is in effect.

When an SQL connection is placed in the current state, the previously-current SQL connection, if any, is placed in the dormant state. No more than one SQL connection in the set of existing connections of an application process can be current at any time. Changing the state of an SQL connection from current to dormant or from dormant to current has no effect on its held or release-pending state.

An SQL connection is placed in the release-pending state by the RELEASE statement. When an application process executes a commit operation, every release-pending connection of the process is ended. Changing the state of an SQL connection from held to release-pending has no effect on its current or dormant state. Thus, an SQL connection in the release-pending state can still be used until the next commit operation. No way exists to change the state of a connection from release-pending to held.

Application process connection states

In a distributed unit of work, an application process can be in a connected or unconnected state. Depending on the state, the application process can execute only certain SQL statements successfully.

A connection to a different database server can be established by the explicit or implicit execution of a CONNECT statement. The following rules apply:

- An application process cannot have more than one SQL connection to the same database server at the same time.
- When an application process executes a SET CONNECTION statement, the specified location name must be in the set of existing connections of the application process.
- When an application process executes a CONNECT statement and the SQLRULES(STD) bind option is in effect, the specified location must not be in the set of existing connections of the application process.

If an application process has a current SQL connection, the application process is in a *connected* state. The CURRENT SERVER special register contains the name of the database server of the current SQL connection. The application process can execute SQL statements that refer to objects managed by that server. If the server is a Db2 subsystem, the application process can also execute certain SQL statements that refer to objects managed by a Db2 subsystem with which that server can establish a connection.

An application process in an unconnected state enters a connected state when it successfully executes a CONNECT or SET CONNECTION statement.

If an application process does not have a current SQL connection, the application process is in an *unconnected* state. The CURRENT SERVER special register contains blanks. The only SQL statements that can be executed successfully are CONNECT, RELEASE, COMMIT, ROLLBACK, and any of the following local SET statements.

- SET CONNECTION
- SET CURRENT APPLICATION ENCODING SCHEME
- SET CURRENT PACKAGE PATH
- SET CURRENT PACKAGESET
- SET *host-variable* = CURRENT APPLICATION ENCODING SCHEME
- SET *host-variable* = CURRENT PACKAGESET
- SET *host-variable* = CURRENT SERVER

Because the application process is in an unconnected state, a COMMIT or ROLLBACK statement is processed by the local Db2 subsystem.

An application process in a connected state enters an unconnected state when its current SQL connection is intentionally ended, or the execution of an SQL statement is unsuccessful because of a failure that causes a rollback operation at the current server and loss of the SQL connection. SQL connections are intentionally ended when an application process successfully executes a commit operation and either of the following are true:

- The SQL connection is in the release-pending state.
- The SQL connection is not in the release-pending state, but it is a remote connection and either of the following are true:
 - The DISCONNECT(AUTOMATIC) bind option is in effect
 - The DISCONNECT(CONDITIONAL) bind option is in effect and an open WITH HOLD cursor is not associated with the connection

An implicit CONNECT to a default Db2 subsystem is executed when an application process executes an SQL statement other than COMMIT, CONNECT TO, CONNECT RESET, SET CONNECTION, RELEASE, or ROLLBACK, and if all of the following conditions apply:

- The CURRENTSERVER bind option was specified when creating the application plan of the application process and the identified server is not the local Db2.
- An explicit CONNECT statement has not already been successfully or unsuccessfully executed by the application process.
- An implicit connection has not already been successfully or unsuccessfully executed by the application process. An implicit connection occurs as the result of execution of an SQL statement that contains a three-part name in a package that is bound with the DBPROTOCOL(DRDA) option.

If the implicit CONNECT fails, the application process is placed in an *unconnected* state.

When a connection is ended, all resources that were acquired by the application process through the connection and all resources that were used to create and maintain the connection are returned to the connection pool. For example, if application process *P* placed the connection to application server *X* in the release-pending state, all cursors of *P* at *X* are closed and returned to the connection pool when the connection is ended during the next commit operation.

When a connection is ended as a result of a communications failure, the application process is placed in an unconnected state.

All connections of an application process are ended when the process ends.

Remote unit of work

The *remote unit of work facility* also provides for the remote preparation and execution of SQL statements, but in a much more restricted fashion than the distributed unit of work facility.

An application process at computer system A can connect to a database server at computer system B and, within one or more units of work, execute any number of static or dynamic SQL statements that reference objects at B. All objects referenced in a single SQL statement must be managed by the same database server, and all SQL statements in the same unit of work must be executed by the same database server. However, unlike a distributed unit of work, an application process can have only one connection at a time. The process cannot connect to a new server until it executes a commit or rollback operation on the current server to end that unit of work. This restricts the situations in which a CONNECT statement can be executed.

Connection management

How connections are managed depends on what states the SQL connection and the application process are in.

An application process is in one of four states at any time:

- Connectable and connected
- Unconnectable and connected
- Connectable and unconnected
- Unconnectable and unconnected

Initial state of an application process: An application process is initially in the connectable and connected state. The database server to which the application process is connected is determined by a product-specific option that might involve an implicit CONNECT operation. An implicit connect operation cannot occur if an implicit or explicit connect operation has already successfully or unsuccessfully occurred. Thus, an application process cannot be implicitly connected to a database server more than once. Other rules for implicit connect operations are product-specific.

[Figure 24 on page 155](#) shows the state transitions:

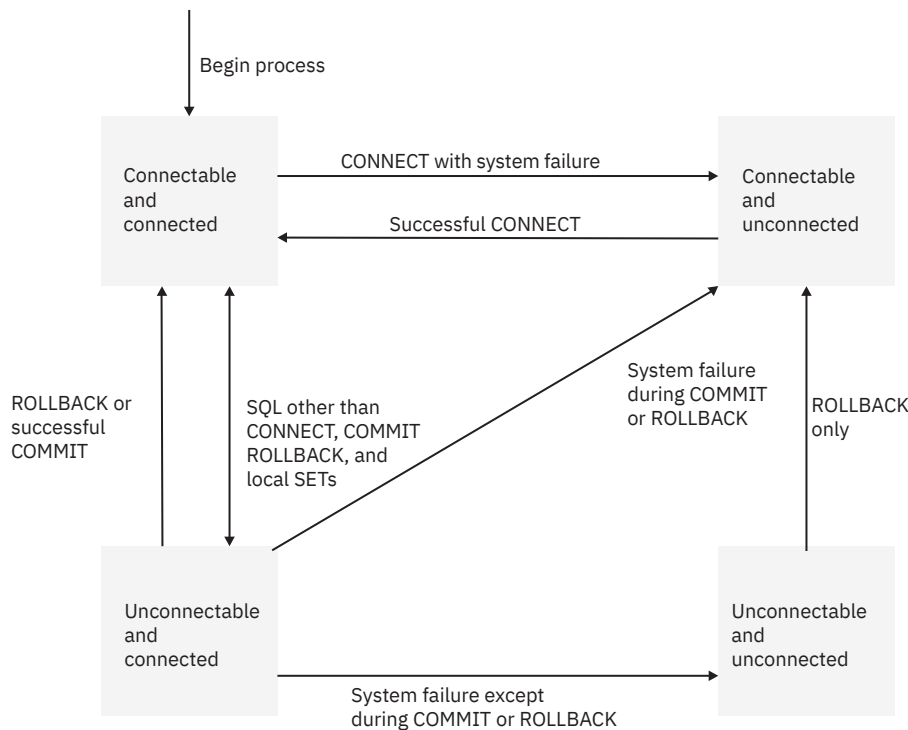


Figure 24. State transitions for an application process connection in a remote unit of work

In the following descriptions of application process connections, CONNECT can mean:

- CONNECT TO
- CONNECT RESET
- CONNECT *authorization*

It cannot mean CONNECT with no operand, which is used to return information about the current server.

Consecutive CONNECT statements can be executed successfully because CONNECT does not remove an application process from the connectable state. A CONNECT statement does not initiate a new unit of work; a unit of work is initiated by the first SQL statement that accesses data. CONNECT cannot execute successfully when it is preceded by any SQL statement other than CONNECT, COMMIT, RELEASE, ROLLBACK, or SET CONNECTION. To avoid an error, execute a commit or rollback operation before a CONNECT statement is executed.

Connectable and connected state: In the connectable and connected state, an application process is connected to a database server, and CONNECT statements that target the current server can be executed. An application process re-enters this state when either of the following is true:

- The process completes a rollback or a successful commit from an unconnectable and connected state.
- The process successfully executes a CONNECT statement from a connectable and unconnected state.

Unconnectable and connected state: In the unconnectable and connected state, an application process is connected to a database server, and only a CONNECT statement with no operands can be executed. An application process enters this state from a connectable and connected state when it executes any SQL statement other than CONNECT, COMMIT, or ROLLBACK.

Connectable and unconnected state: In the connectable and unconnected state, an application process is not connected to a database server. The only SQL statement that can be executed is CONNECT. An application process enters this state if any of the following is true:

- The process does not successfully execute a CONNECT statement from a connectable and connected state.
- The process executes a COMMIT statement when the SQL connection is in a release-pending state.
- A system failure occurs during a COMMIT or ROLLBACK from an unconnectable and connected state.

- The process executes a ROLLBACK statement from an unconnectable and unconnected state.

Other product-specific reasons can also cause an application process to enter the connectable and unconnected state.

Unconnectable and unconnected state: In the unconnectable and unconnected state, an application process is not connected to a database server and CONNECT statements cannot be executed. The only SQL statement that can be executed is ROLLBACK. An application process enters this state from an unconnectable and connected state as a result of a system failure, except during a COMMIT or ROLLBACK, at the server.

Distributed data

Distributed data is data that resides on a DBMS other than your local system.

Your *local* DBMS is the one on which you bind your package. All other DBMSs are *remote*.

Many businesses need to manage data from a wide variety of sources and locations. A distributed environment provides the flexibility that is required to allocate resources for data that is located at different sites or database management systems (DBMSs) in a computer network.

Related concepts

Distributed data access

Distributed computing environments typically involve requests from users at one DBMS client that are processed by a DBMS server. The server DBMS is typically remote to the client. Certain programming techniques and performance implications apply to distributed computing environments.

Remote servers

A remote server can be physically remote, or it can be part of the same operating system under which your local DBMS runs.

When you request services from a remote DBMS, the remote DBMS is a *server*, and your local system is a *requester* or *client*. Conceptually, a server is like a food server who takes food orders, delivers food, and provides other services to customers. The customer is like the requester, or client. The purpose of the server is to provide services to its clients.

A remote server can be truly remote in the physical sense (thousands of miles away), or a remote server can be part of the same operating system under which your local DBMS runs. This information generally assumes that your local DBMS is an instance of Db2 for z/OS. A remote server can be another instance of Db2 for z/OS or an instance of one of many other products.

The following figure shows the client/server environment.

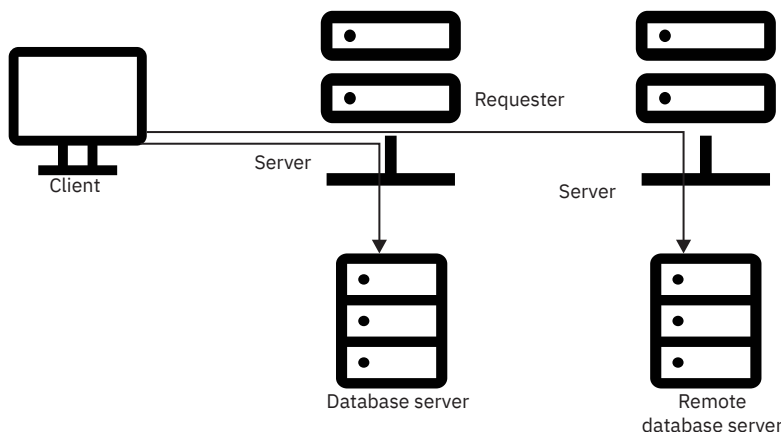


Figure 25. Client/server processing environment

Connectivity in distributed environments

Connectivity in the client/server environment enables communication between applications and database systems on disparate operating systems.

Connectivity in the client/server environment requires an architecture that can handle the stringent performance requirements of a transaction-based system and the flexibility of a decision-support system by using ODBC or JDBC.

Using standard communication protocols, Db2 can bind and rebind packages at other servers and run the statements in those packages. Communication protocols are rules for managing the flow of data across a computer network just as traffic lights and traffic rules manage the flow of car traffic. These protocols are invisible to Db2 applications.

In a distributed environment, applications can connect to multiple databases on different servers and can complete transactions, including commit and rollback operations, at the same time. This type of connectivity is known as a *distributed unit of work*.

Distributed data access

Distributed computing environments typically involve requests from users at one DBMS client that are processed by a DBMS server. The server DBMS is typically remote to the client. Certain programming techniques and performance implications apply to distributed computing environments.

The Db2 distributed environment supports both a two-tier and a multitier architecture.

A DBMS, whether local or remote, is known to your Db2 subsystem by its location name. Remote systems use the location name, or an alias location name, to access a Db2 subsystem. You can define a maximum of eight location names for a Db2 subsystem.

The location name of the Db2 subsystem is defined in the BSDS during Db2 installation. The communications database (CDB) records the location name and the network address of a remote DBMS. The CDB is a set of tables in the Db2 catalog.

The primary method that Db2 uses for accessing data at a remote server is based on Distributed Relational Database Architecture (DRDA).

If your application performs updates to two or more DBMSs, a transaction manager coordinates the two-phase commit process and guarantees that units of work are consistently committed or rolled back. If Db2 requests updates to two or more DBMSs, Db2 acts as the transaction manager. The distributed commit protocols that are used on the network connection dictate whether both DBMSs can perform updates or whether updates are restricted to a single DBMS.

Examples

The following examples show statements that you can use to access distributed data.



Example 1

To access data at a remote server, write statements like the following example:

```
EXEC SQL
  CONNECT TO CHICAGO;
SELECT * FROM IDEMP01.EMP
  WHERE EMPNO = '000030';
```

You can also accomplish the same task by writing the query like the following example:

```
SELECT * FROM CHICAGO.IDEMP01.EMP
  WHERE EMPNO = '000030';
```

Before you can execute either query at location CHICAGO, you must bind a package at the CHICAGO server.

Example 2

You can call a stored procedure to access data at a remote server. Your program executes these statements:

```
EXEC SQL
  CONNECT TO ATLANTA;
EXEC SQL
  CALL procedure_name (parameter_list);
```

The parameter list is a list of host variables that is passed to the stored procedure and into which it returns the results of its execution. The stored procedure must exist at location ATLANTA.

GUPI

Related concepts

Distributed data

Distributed data is data that resides on a DBMS other than your local system.

Data distribution and Web access

The ability to distribute data and provide Web access to that data is vital to service providers and their customers.

Effects of distributed data on program preparation

In a distributed data environment, several items affect precompile and bind options that are used for DRDA access and package resolution.

Web application environment

Web-based applications run on a web application server and access data on an enterprise information system, such as a Db2 database server. The components of web-based applications are spread across multiple tiers, or layers.

Related tasks

[Improving performance for applications that access distributed data \(Db2 Performance\)](#)

Ways to implement distributed data in programs

You can connect to a remote server in different ways. You can code an application that uses DRDA to access data at a remote location by using either CONNECT statements or three-part names and aliases.

Using either method, you must bind the DBRMs for the SQL statements that are to execute at the server to packages that reside at the server.

- At the local Db2 subsystem, use the BIND PLAN command to build an application plan.
- At the remote location, use the BIND PACKAGE command to build an application package that uses the local application plan.

Related concepts

[The DRDA database protocol \(Db2 Installation and Migration\)](#)

Explicit CONNECT statements

Using CONNECT statements provides application portability across all Db2 clients and requires the application to manage connections.

With the CONNECT statement, an application program explicitly connects to each server. You must bind the DBRMs for the SQL statements that are to execute at the server to packages that reside at that server.

The application connects to each server based on the location name in the CONNECT statement. You can explicitly specify a location name, or you can specify a location name in a host variable. Issuing the CONNECT statement changes the special register CURRENT SERVER to show the location of the new server.

Example

GUPI

Assume that an application includes a program loop that reads a location name, connects to the location, and executes an INSERT statement. The application inserts a new location name into a host variable, LOCATION_NAME, and executes the following statements:

```
EXEC SQL
    CONNECT TO :LOCATION_NAME;
EXEC SQL
    INSERT INTO IDP101.PROJ VALUES (:PROJNO, :PROJNAME, :DEPTNO,
    :RESPEMP, :MAJPROJ);
```

The host variables match the declaration for the PROJ table.

GUIP

Db2 guarantees the consistency of data across a distributed transaction. To keep the data consistent at all locations, the application commits the work only after the program loop executes for all locations. Either every location commits the INSERT, or, if a failure prevents any location from inserting, all other locations roll back the INSERT.

Related concepts

[The DRDA database protocol \(Db2 Installation and Migration\)](#)

Three-part names and aliases in distributed data applications

Using three-part object names and aliases provides the application with location transparency; objects can move to a new location without requiring changes to the application. Instead, the DBMS manages the underlying connections.

You can use three-part names to access data at a remote location, including tables and views. Using a three-part name, or an alias, an application program implicitly connects to each server. With these access methods, the database server controls where the statement executes.

A three-part name consists of:

- A LOCATION name that uniquely identifies the remote server that you want to access
- An AUTHORIZATION ID that identifies the owner of the object (the table or view) at the location that you want to access
- An OBJECT name that identifies the object at the location that you want to access

Example

GUIP

This example shows how an application uses a three-part name in INSERT, PREPARE, and EXECUTE statements. Assume that the application obtains a location name, 'SAN_JOSE'. Next, it creates the following character string:

```
INSERT INTO SAN_JOSE.IDP101.PROJ VALUES (?, ?, ?, ?, ?)
```

The application assigns the character string to the variable INSERTX, and then executes these statements:

```
EXEC SQL
    PREPARE STMT1 FROM :INSERTX;
EXEC SQL
    EXECUTE STMT1 USING :PROJNO, :PROJNAME, :DEPTNO, :RESPEMP, :MAJPROJ;
```

The host variables match the declaration for the PROJ table.

GUIP

Recommendation: If you plan to port your application from a z/OS server to another server, you should not use three-part names. For example, a client application might connect to a z/OS server and then issue a three part-name for an object that resides on a Linux server. Db2 for z/OS is the only server that automatically forwards SQL requests that reference objects that do not reside on the connected server.

A convenient alternative approach is to use aliases when creating character strings that become prepared statements, instead of using full three-part names.

Deciding between three-part names and aliases

Three-part names and aliases each have different advantages.

You can always use three-part names to reference data at another remote server. The advantage of three-part names is that they allow application code to run at different Db2 locations without the additional overhead of maintaining aliases. However, if the table locations change, you must also change the affected applications.

The advantage of aliases is that they allow you to move data around without needing to modify application code or interactive queries. However, if you move a table or view, you must drop the aliases that refer to those tables or views. Then, you can re-create the aliases with the new location names.

Related concepts

[Aliases](#)

An *alias* is a substitute for the three-part name of a table or view.

[The DRDA database protocol \(Db2 Installation and Migration\)](#)

Ways that other tasks are affected by distributed data

When you operate in a distributed environment, you need to consider how the environment affects planning and programming activities.

Effects of distributed data on planning

When you work in a distributed environment, you need to consider how authorization works and the cost of running SQL statements.

The appropriate authorization ID must have authorization at a remote server to connect to and to use resources at that server.

You can use the *resource limit facility* at the server to set resource limits for distributed SQL statements. Using this facility, a server can govern how much of its resources a given package can consume by using DRDA access.

Related concepts

[Distributed data](#)

Distributed data is data that resides on a DBMS other than your local system.

[Remote servers and distributed data \(Db2 Application programming and SQL\)](#)

[Coding methods for distributed data \(Db2 Application programming and SQL\)](#)

Related tasks

[Granting privileges for accessing distributed data \(Managing Security\)](#)

[Improving performance for applications that access distributed data \(Db2 Performance\)](#)

[Limiting resources for statements from remote locations \(Db2 Performance\)](#)

Related information

[Managing distributed access \(Managing Security\)](#)

Effects of distributed data on programming

There are several effects that you must be aware of when you write programs for a distributed environment.

Keep in mind the following considerations when you write programs that are used in a distributed environment:

- Stored procedures

If you use DRDA access, your program can call stored procedures. Stored procedures behave like subroutines that can contain SQL statements and other operations.

- Three-part names and multiple servers

Assume that a statement runs at a remote server (server 1). That statement uses a three-part name or an alias that resolves to a three-part name. The statement includes a location name of a different server (server 2). To ensure that access to the second remote server is by DRDA access, bind the package that contains the three-part name at the second server.

- SQL differences at servers other than Db2 for z/OS

With explicit connections, a program that uses DRDA access can use SQL statements that a remote server supports, even if the local server does not support them. A program that uses three-part object names cannot execute non-z/OS SQL.

Effects of distributed data on program preparation

In a distributed data environment, several items affect precompile and bind options that are used for DRDA access and package resolution.

The following table lists the z/OS precompiler options that are relevant to preparing a package that is to be run using DRDA access.

Table 21. Db2 precompiler options for DRDA access

z/OS precompiler options	Usage
CONNECT	Use CONNECT(2) to allow your application program to make updates at more than one DBMS.
SQL	Use SQL(ALL) for binding a package to a non-z/OS server; otherwise, use SQL(Db2).

Usually, binding a package to run at a remote location is like binding a package to run at your local Db2 subsystem. Binding a plan to run the package is like binding any other plan. The following table gives you guidelines for which z/OS bind options to choose when binding a package and planning to run using DRDA access.

Table 22. z/OS bind options for DRDA access

z/OS bind options	Usage
DEFER(PREPARE)	For dynamic SQL, use DEFER(PREPARE) to send PREPARE and EXECUTE statements together over the network to improve performance.
SQLERROR	Use SQLERROR(CONTINUE) to create a package even if the bind process finds SQL errors.
SQLRULES	Use SQLRULES(Db2) for more flexibility in coding your applications, particularly for LOB data, and to improve performance.

JDBC, SQLJ, and ODBC use different methods for binding packages that involve less preparation for accessing a z/OS server.

The CURRENT PACKAGE PATH special register provides a benefit for applications that use DRDA from a z/OS requester. The package collection ID is resolved at the server. Applications on the server can take advantage of the list of collections, and network traffic is minimal.

Example



Assume that five packages exist and that you want to invoke the first package at the server. The package names are SCHEMA1.PKG1, SCHEMA2.PKG2, SCHEMA3.PKG3, SCHEMA4.PKG4, and SCHEMA5.PKG5.

Rather than issuing a SET CURRENT PACKAGESET statement to invoke each package, you can use a single SET statement if the server supports the CURRENT PACKAGE PATH special register:

```
SET CURRENT PACKAGE PATH = SCHEMA1, SCHEMA2, SCHEMA3, SCHEMA4, SCHEMA5;
```



Related concepts

[Preparation process for an application program](#)

How you prepare an application program to run depends on the type of application. The program preparation steps for applications vary based on the type of programming language that is used.

How updates are coordinated across distributed systems

Various products are available to work with Db2 to coordinate updates across a distributed transaction. Db2 coordinates updates at servers that support different types of connections.

Related concepts

[The DRDA database protocol \(Db2 Installation and Migration\)](#)

Db2 transaction manager support

Db2 supports a wide range of transaction manager products to coordinate updates across a distributed transaction. A distributed transaction typically involves multiple recoverable resources, such as Db2 tables, IBM MQ messages, and IMS databases.

Application environments that use Db2 Connect to access Db2 remotely can use the following transaction manager products:

- WebSphere Application Server
- CICS
- IBM TXSeries® (CICS and Encina)
- IBM MQ
- Microsoft Transaction Server (MTS)
- Java applications that support Java Transaction API (JTA) and Enterprise JavaBeans (EJBs)
- BEA (Tuxedo and WebLogic)
- Other transaction manager products that support standard XA protocols

The XA interface is a bidirectional interface between a transaction manager and resource managers that provides coordinated updates across a distributed transaction. The Open Group defined XA protocols based on the specification [Distributed TP: The XA Specification](#).

Application environments that access Db2 locally can use the following transaction manager products:

- WebSphere Application Server
- CICS transaction server
- IMS

For application environments that do not use a transaction manager, Db2 coordinates updates across a distributed transaction by using DRDA-protected connections.

Related concepts

[The DRDA database protocol \(Db2 Installation and Migration\)](#)

Servers that support two-phase commit

Updates in a two-phase commit situation are coordinated if they must all commit or all roll back in the same unit of work.

Example

You can update an IMS database and a Db2 table in the same unit of work. Suppose that a system or communication failure occurs between committing the work on IMS and on Db2. In that case, the two programs restore the two systems to a consistent point when activity resumes.

Db2 coordinates commits even when a connection is using one-phase commit in a distributed transaction. In this case, however, only one location can perform an update.

Related concepts

Coordinated updates for consistency between servers

In a distributed system, a transaction might occur at more than one server. To ensure data consistency, each subsystem that participates in a single transaction must coordinate update operations. Transactions must be either committed or backed out.

[The DRDA database protocol \(Db2 Installation and Migration\)](#)

Servers that do not support two-phase commit

In a distributed transaction, Db2 can coordinate a mixture of two-phase and one-phase connections.

You cannot have coordinated updates with a DBMS that does not implement two-phase commit. You can, however, achieve the effect of coordinated updates when you access a server that does not implement two-phase commit; such a server is called a *restricted system*.

Db2 prevents you from updating both a restricted system and any other system in the same unit of work. In this context, *update* includes the statements INSERT, DELETE, UPDATE, CREATE, ALTER, DROP, GRANT, REVOKE, and RENAME.

You can achieve the effect of coordinated updates with a restricted system. You must first update one system and commit that work, and then update the second system and commit its work. However, suppose that a failure occurs after the first update is committed and before the second update is committed. No automatic provision is available to bring the two systems back to a consistent point. Your program must handle that task.

When you access a mixture of systems, some of which might be restricted, you can take the following actions to ensure data integrity:

- Read from any of the systems at any time.
- Update any one system many times in one unit of work.
- Update many systems, including CICS or IMS, in one unit of work if no system is a restricted system. If the first system you update is not restricted, any attempt to update a restricted system within a unit of work results in an error.
- Update one restricted system in a unit of work. You can do this action only if you do not try to update any other system in the same unit of work. If the first system that you update is restricted, any attempt to update another system within that unit of work results in an error.

Related concepts

[The DRDA database protocol \(Db2 Installation and Migration\)](#)

Ways to reduce network traffic

The key to improving performance in a network computing environment is to minimize network traffic.

Stored procedures are an excellent method for sending many SQL statements in a single network message and, as a result, running many SQL statements at the Db2 server. This topic introduces you to other ways to improve performance when accessing remote servers.

Related tasks

[Improving performance for applications that access distributed data \(Db2 Performance\)](#)

Improvements in query efficiency

Queries almost always execute faster on a local server than they do when the same query is sent to a remote server. To increase efficiency when accessing remote servers, try to write queries that send few messages over the network.

For example:

- Reduce the number of columns and rows in the result table that is returned to your application. Keep your SELECT lists as short as possible. Creative use of the clauses WHERE, GROUP BY, and HAVING can eliminate unwanted data at the remote server.
- Use FOR READ ONLY. For example, retrieving thousands of rows as a continuous stream is reasonable. Sending a separate message for each one can be much slower.
- When possible, do not bind application plans and packages with ISOLATION(RR). If your application does not need to refer again to rows it reads once, another isolation level might reduce lock contention and message overhead during COMMIT processing.
- Minimize the use of parameter markers.

When your program uses DRDA access, Db2 can streamline the processing of dynamic queries that do not have parameter markers. However, parameter markers are needed for effective dynamic statement caching.

When a Db2 requester encounters a PREPARE statement for such a query, it anticipates that the application is going to open a cursor. The requester therefore sends the server a single message that contains a combined request for PREPARE, DESCRIBE, and OPEN. A Db2 server that receives this message sequence returns a single reply message sequence that includes the output from the PREPARE, DESCRIBE, and OPEN operations. As a result, the number of network messages that are sent and received for these operations is reduced from two to one.

Related tasks

[Improving performance for applications that access distributed data \(Db2 Performance\)](#)

[Choosing an ISOLATION option \(Db2 Performance\)](#)

Related reference

[group-by-clause \(Db2 SQL\)](#)

[having-clause \(Db2 SQL\)](#)

[read-only-clause \(Db2 SQL\)](#)

[where-clause \(Db2 SQL\)](#)

[PREPARE statement \(Db2 SQL\)](#)

Reduction in the volume of messages

Db2 capabilities that combine multiple rows of data during fetch and insert operations can significantly reduce the number of messages that are sent across the network. Those capabilities include block fetch and rowset fetches and inserts.

Related tasks

[Improving performance for applications that access distributed data \(Db2 Performance\)](#)

Block fetch

You can use block fetch to retrieve a set of rows and transmit them all in one message over the network.

Db2 uses a *block fetch* to group the rows that an SQL query retrieves into as large a "block" of rows as can fit in a message buffer, and then transmits the block over the network. By sending multiple rows in a block, Db2 avoids sending a message for every row.

A block fetch is used only with cursors that do not update data.

Db2 can use two different types of block fetch:

Limited block fetch

An operation that optimizes data transfer by minimizing the number of messages that are transmitted from the requester whenever a remote fetch operation is performed.

Continuous block fetch

An operation that optimizes data transfer by minimizing the number of messages that are transmitted from the requester to retrieve the entire result set. In addition, overlapped processing is performed at the requester and the server.

To use block fetch, Db2 must determine that the cursor is not used for update or delete. You can indicate in your program by adding the clause `FOR READ ONLY` or `FOR FETCH ONLY` to the query. If you do not specify `FOR READ ONLY` or `FOR FETCH ONLY`, the way in which Db2 uses the cursor determines whether it uses block fetch. For scrollable cursors, the sensitivity of the cursor and the bind options affect whether Db2 can use block fetch.

Related concepts

[Limited block fetch \(Db2 Performance\)](#)

[Continuous block fetch \(Db2 Performance\)](#)

Related tasks

[Improving performance for applications that access distributed data \(Db2 Performance\)](#)

[Enabling block fetch for distributed applications \(Db2 Performance\)](#)

Related reference

[read-only-clause \(Db2 SQL\)](#)

[DECLARE CURSOR statement \(Db2 SQL\)](#)

Rowset fetches and inserts

For rowset-positioned cursors, when the cursor is opened for rowset processing, the answer set is returned in a single query block. The query block contains exactly the number of rows that are specified for the rowset.

Because a rowset is returned in a single query block, the size of a rowset is limited to 10 MB. This rowset size minimizes the impact to the network when retrieving a large rowset with a single fetch operation.

Rowset-positioned cursors also enable a form of multiple-row inserts. The `FOR n ROWS` form of the `INSERT` statement inserts multiple rows into a table or view by using values that are provided in a host-variable array. With this form of `INSERT`, all insert data is sent in a single network message, instead of separate `INSERT` statements being sent for inserted row. For more information about this form of `INSERT` statement, see [Inserting multiple rows of data from host-variable arrays \(Db2 Application programming and SQL\)](#).

Rowset fetches and accelerated queries

Accelerated queries are also limited to a rowset size of 10 MB, to minimize network impact and Db2 storage consumption.

The 10 MB limit applies regardless of whether the cursor is declared `WITH ROWSET POSITIONING`. For example, a JDBC application might need to reduce the number of network exchanges that are required to fetch an accelerated query's results. To do that, the application needs to use the `Statement.setFetchSize` method and the `ResultSet.setFetchSize` method to specify the default fetch size for the accelerated query.

When you specify a fetch size, ensure that the fetch size does not exceed the rowset size limit of 10 MB for a single query block.

Related concepts

Row retrieval with a cursor

Db2 has a mechanism called a *cursor*. Using a cursor is like keeping your finger on a particular line of text on a printed page.

Related tasks

[Improving performance for applications that access distributed data \(Db2 Performance\)](#)

Related reference

[DECLARE CURSOR statement \(Db2 SQL\)](#)

Optimization for large and small result sets

Several options on the SELECT statement let you limit the number of rows that are returned to a client program.

Enabling a Db2 client to request that multiple query blocks on each transmission can reduce network activity and improve performance significantly for applications that use DRDA access to download large amounts of data.

You can specify a large value of *n* in the OPTIMIZE FOR *n* ROWS clause of a SELECT statement to increase the number of DRDA query blocks that a Db2 server returns in each network transmission for a nonscrollable cursor.

If *n* is greater than the number of rows that fit in a single DRDA query block, the OPTIMIZE FOR *n* ROWS clause lets the DRDA client request multiple blocks of query data on each network transmission instead of requesting another block when the first block is full. This use of the OPTIMIZE FOR *n* ROWS clause is intended for applications that open a cursor and download large amounts of data. The OPTIMIZE FOR *n* ROWS clause does not affect scrollable cursors.

When a client does not need all the rows from a potentially large result set, preventing the Db2 server from returning all the rows for a query can reduce network activity and improve performance significantly for DRDA applications. You can use either the OPTIMIZE FOR *n* ROWS clause or the FETCH FIRST *n* ROWS ONLY clause of a SELECT statement to limit the number of rows that are returned to a client program.

Related tasks

[Improving performance for applications that access distributed data \(Db2 Performance\)](#)

[Optimizing retrieval for a small set of rows \(Db2 Application programming and SQL\)](#)

[Fetching a limited number of rows \(Db2 Performance\)](#)

Related reference

[optimize-clause \(Db2 SQL\)](#)

[fetch-clause \(Db2 SQL\)](#)

Support for high availability

Because Db2 provides support for high availability, frequently starting or stopping Db2 is not necessary.

You can run Db2 for several weeks without stopping and starting the subsystem. Some customers have managed to keep Db2 running continuously for several years. One key to achieving high availability is to use data sharing. Data sharing allows access to data even when one Db2 subsystem in a group is stopped. Another key to high availability is the ability to get the Db2 subsystem back up and running quickly after an unplanned outage.

- Some restart processing can occur concurrently with new work. Also, you can choose to postpone some processing.
- During a restart, Db2 applies data changes from the log. This technique ensures that data changes are not lost, even if some data was not written at the time of the failure. Some of the process of applying log changes can run in parallel
- You can register Db2 to the Automatic Restart Manager of z/OS. This facility automatically restarts Db2 if it goes down as a result of a failure.

Related concepts

[Backup, recovery, and restart](#)

Although high availability of data is a goal for all Db2 subsystems, unplanned outages are difficult to avoid entirely. A good backup, recovery, and restart strategy, however, can reduce the elapsed time of an unplanned outage.

Db2 data sharing

In Db2 *data sharing*, applications that reside in multiple Db2 subsystems in a Parallel Sysplex can read from and write to the same Db2 for z/OS data concurrently, with integrity, performance, scalability, and dynamic workload balancing.

Db2 subsystems that share data must belong to a Db2 data sharing group, which runs on a IBM zSystems Parallel Sysplex cluster. A *data sharing group* is a collection of one or more Db2 subsystems that access shared Db2 data.

A Parallel Sysplex is a cluster of z/OS systems that communicate and cooperate with each other. The Parallel Sysplex is a highly sophisticated cluster architecture. It consists of two key pieces of technology:

Coupling facility

Provides specialized hardware, specialized high-speed links and adapters, and a shared, nonvolatile electronic storage for fast intersystem data sharing protocols.

Sysplex Timer

Provides a common time source across all the systems in the cluster, thereby delivering an efficient way to provide log-record sequencing and event ordering across the different systems.

The coupling facility and the Sysplex Timer are exclusive to the IBM zSystems environment. They provide strong performance and scalability in a multisystem clustered DBMS environment with shared disks.

Each Db2 subsystem that belongs to a particular data sharing group is a *member* of that group. All members of a data sharing group use the same shared Db2 catalog.

You can use some capabilities that are described in this information regardless of whether you share data. The term *data sharing environment* refers to a situation in which a data sharing group is defined with at least one member. In a non-data-sharing environment, no group is defined.

Related concepts

[Introduction to Db2 data sharing \(Db2 Data Sharing Planning and Administration\)](#)

[Planning for Db2 data sharing \(Db2 Installation and Migration\)](#)

Related tasks

[Installing, migrating, and enabling Db2 data sharing \(Db2 Installation and Migration\)](#)

Advantages of Db2 data sharing

You can use data sharing to enhance the capabilities of Db2.

Db2 data sharing improves Db2 availability, enables scalable growth, and provides more flexible ways to configure your environment. You don't need to change SQL in your applications to use data sharing, although you might need to do some tuning for optimal performance.

Related concepts

[Advantages of Db2 data sharing \(Db2 Data Sharing Planning and Administration\)](#)

Improved availability of data

Db2 data sharing helps you meet your service objective by improving availability during both planned and unplanned outages.

As the following figure illustrates, if one subsystem fails, users can access their Db2 data from another subsystem. Transaction managers are informed that Db2 is down and can switch new user work to another Db2 subsystem in the group. For unplanned outages, the z/OS automatic restart manager can automate restart and recovery.

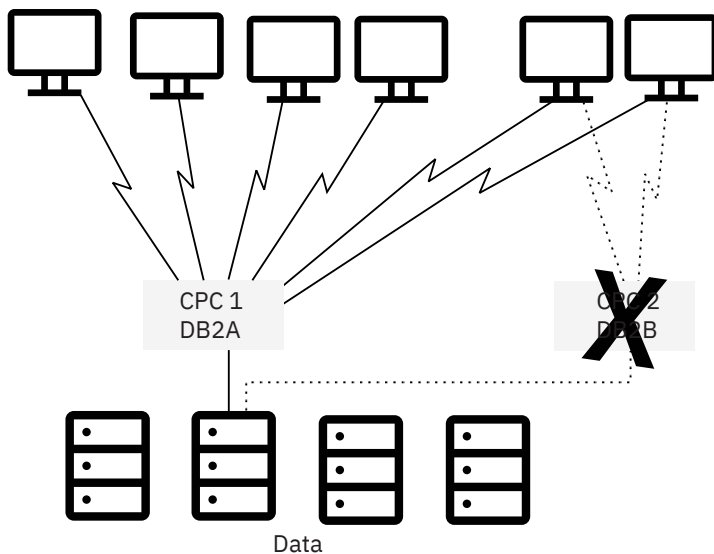


Figure 26. How data sharing improves availability during outages

Although the increased availability of Db2 has some performance cost, the overhead for interprocessor communication and caching changed data is minimized. Db2 provides efficient locking and caching mechanisms and uses coupling facility hardware. A *coupling facility* is a special logical partition that runs the coupling facility control program. It provides high-speed caching, list processing, and locking functions in a Sysplex. The Db2 structures in the coupling facility benefit from high availability. The coupling facility uses automatic structure rebuild and duplexing of the structures that are used for caching data.

Related concepts

[Advantages of Db2 data sharing \(Db2 Data Sharing Planning and Administration\)](#)

Scalable growth

As you move more data processing into a Db2 environment, your processing needs can exceed the capacity of a single system. Data sharing might relieve that constraint.

Db2 for z/OS is optimized to use your existing hardware more efficiently and manage a larger workload. Db2 now has more 64 bit storage, and supports more concurrent threads than previous versions. These improvements greatly increase the capabilities of a single system, which reduces the cost of new hardware, software, and maintenance. However, there are several ways to scale the capabilities of your system to meet your business needs.

Without data sharing

Without Db2 data sharing, you have the following options:

- Copy the data, or split the data into separate Db2 subsystems.

This approach requires that you maintain separate copies of the data. No communication takes place among Db2 subsystems, and the Db2 catalog is not shared.

- Install another Db2 subsystem, and rewrite applications to access the original data as distributed data.

This approach might relieve the workload on the original Db2 subsystem, but it requires changes to your applications and has performance overhead of its own. Nevertheless, for Db2 subsystems that are separated by great distance or for a Db2 subsystem that needs to share data with a system that outside the data sharing group, the distributed data facility is still your only option.

- Install a larger processor and move data and applications to that machine.

This option can be expensive. In addition, this approach requires your system to come down while you move to the new, larger machine.

With data sharing

With Db2 data sharing, you can take advantage of the following benefits:

Incremental growth

The Parallel Sysplex cluster can grow incrementally. You can add a new Db2 subsystem onto another central processor complex and access the same data through the new Db2 subsystem. You no longer need to manage copies or distribute data. All Db2 subsystems in the data sharing group have concurrent read-write access, and all Db2 subsystems use a single Db2 catalog.

Workload balancing

Db2 data sharing provides flexibility for growth and workload balancing. With the partitioned data approach to parallelism (sometimes called the *shared-nothing* architecture), a one-to-one relationship exists between a particular DBMS and a segment of data. By contrast, data in a Db2 data sharing environment does not need to be redistributed when you add a new subsystem or when the workload becomes unbalanced. The new Db2 member has the same direct access to the data as all other existing members of the data sharing group.

Db2 works closely with the z/OS Workload Manager (WLM) to ensure that incoming work is optimally balanced across the systems in the cluster. WLM manages workloads that share system resources and have different priorities and resource-use characteristics.

For example, assume that large queries with a low priority are running on the same system as online transactions with a higher priority. WLM can ensure that the queries do not monopolize resources and do not prevent the online transactions from achieving acceptable response times. WLM works in both a single-system and a multisystem (data sharing) environment.

Capacity when you need it

A data sharing configuration can handle your peak loads. You can start data sharing members to handle peak loads, such as end-of-quarter processing, and then stop them when the peak passes.

You can take advantage of all these benefits, whether your workloads are for online transaction processing (OLTP), or a mixture of OLTP, batch, and queries.

Higher transaction rates

Data sharing gives you opportunities to put more work through the system. As the following figure illustrates, you can run the same application on more than one Db2 subsystem to achieve transaction rates that are higher than are possible on a single subsystem.

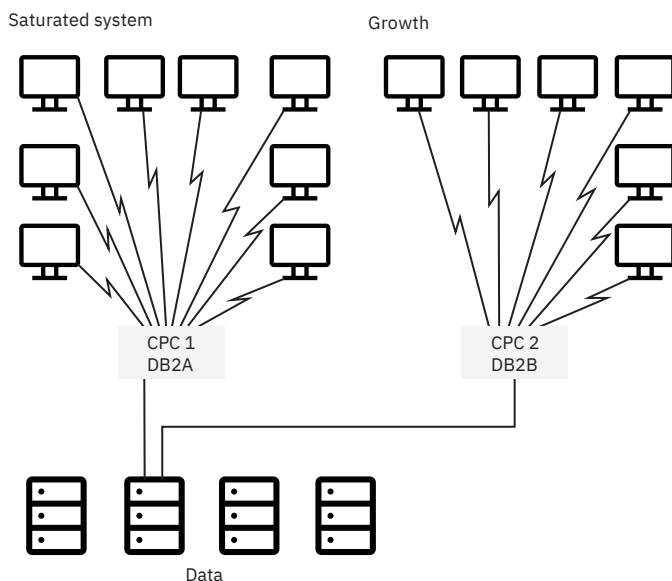


Figure 27. How data sharing enables growth

Chapter 4. Db2 performance management

Managing the performance of a Db2 subsystem involves understanding a wide range of system components. You need to understand the performance of those components, how to monitor the components, and how to identify problem areas.

System resources, database design, and query performance are among the many performance issues to consider, and each of these factors influences the others. For example, a well-designed query does not run efficiently if system resources are not available when it needs to run.

To manage Db2 performance, you need to establish performance objectives and determine whether objects, resources, and processes are meeting your performance expectations. Tips and guidelines help you tune your Db2 subsystem to improve performance. Several tools are available to make performance analysis easier for you.

Related information

[Db2 12 for z/OS Performance Topics \(IBM Redbooks\)](#)

[Db2 12 for z/OS Optimizer](#)

[Db2 11 for z/OS Performance Topics \(IBM Redbooks\)](#)

[Subsystem and Transaction Monitoring and Tuning with Db2 11 for z/OS \(IBM Redbooks\)](#)

[DB2 10 for z/OS Performance Topics \(IBM Redbooks\)](#)

Initial steps for performance management

The first step in managing Db2 performance is understanding performance issues. You need to know how to recognize different types of performance problems and to know what tools are available to help you solve them.

Performance objectives

Establishing performance objectives can help you make good choices as you work with Db2. Although performance objectives vary for every business, how your site defines good Db2 performance depends on data processing needs and priorities.

In all cases, performance objectives must be realistic, understandable, and measurable. Typical objectives include values for:

- Acceptable response time (a duration within which some percentage of all applications have completed)
- Average throughput (the total number of transactions or queries that complete within a given time)
- System availability, including mean time to failure and the durations of downtimes

Objectives such as these define the workload for the system and determine the requirements for resources, which include processor speed, amount of storage, additional software, and so on.

For example, an objective might be that 90% of all response times on a local network during a prime shift are under 2 seconds. Another objective might be that the average response time does not exceed 6 seconds, even during peak periods. (For remote networks, response times are substantially higher.)

Often, though, available resources limit the maximum acceptable workload, which requires that you revise the objectives.

Related tasks

[Setting performance objectives and defining your workloads \(Db2 Performance\)](#)

Application design for performance

Designing the database and applications to be as efficient as possible is an important first step to good system and application performance. As you code applications, consider performance objectives in your application design.

Tip: Query tuning capabilities that can help you with this task, such as *visual explain* and *statistics advisor*, are available in [IBM Db2 Administration Foundation for z/OS](#) and [IBM Db2 for z/OS Developer Extension](#).

Some factors that affect the performance of applications include how the program uses host variables and what bind options you choose. In turn, those factors affect how long Db2 takes to determine an access path for the SQL statements in the application.

Later in this information you can read about locking and concurrency, including recommendations for database and application design that improve performance.

After you run an application, you need to decide if it meets your performance objectives. You might need to test and debug the application to improve its performance.

Related concepts

[Performance information for SQL application programming](#)

Efficient applications are an important first step to good system and application performance. As you code applications that access data in Db2, consider performance objectives in your application design.

[Performance objectives](#)

Establishing performance objectives can help you make good choices as you work with Db2. Although performance objectives vary for every business, how your site defines good Db2 performance depends on data processing needs and priorities.

[Setting reasonable performance objectives \(Db2 Performance\)](#)

[Investigating SQL performance by using EXPLAIN \(Db2 Performance\)](#)

[Interpreting data access by using EXPLAIN \(Db2 Performance\)](#)

Related tasks

[Programming applications for performance \(Db2 Performance\)](#)

[Improving concurrency \(Db2 Performance\)](#)

[Programming for concurrency \(Db2 Performance\)](#)

Origin of performance problems

After running an application, if you determine that it does not meet your performance objectives, you need to determine the origin of the problem. This information describes how you identify performance problems and what tools can help you.

To identify a performance problem, you begin by looking at the overall system before you decide that you have a problem in Db2. In general, look closely to see why application processes are progressing slowly or why a given resource is being heavily used.

Within Db2, the performance problem is usually either poor response time or an unexpected and unexplained high use of resources. Check factors such as total processor usage, disk activity, and memory usage.

First, get a picture of task activity, from classes 1, 2, and 3 of the accounting trace. Db2 provides a *trace facility* that lets you monitor and collect detailed information about Db2, including performance and statistical information. Then, focus on specific activities, such as specific application processes or a specific time interval. You might see problems such as these:

- Slow response time. You can collect detailed information about a single slow task, a problem that can occur for several reasons. For example, users might be trying to do too much work with certain applications, and the system simply cannot do all the work that they want done.

- Real storage constraints. Applications progress more slowly than expected because of paging interrupts. The constraints result in delays between successive requests that are recorded in the Db2 trace.

If you identify a performance problem in Db2, you can look at specific reports. Reports give you information about:

- Whether applications are able to read from buffer pools rather than from disk
- Whether and how long applications must wait to write to disk or wait for a lock
- Whether applications are using more than the usual amount of resources

Db2 also provides several tools that help you analyze performance.

Related concepts

The role of buffer pools in caching data

Buffer pools are a key element of Db2 performance, and help you to avoid delays when retrieving data.

Ways to improve performance for multiple users

Locking ensures the integrity and accuracy of data. However, locking is sometimes too restrictive and can cause slow performance and poor concurrency. Understanding how locking works can help you make good decisions for performance. Locking is critical to performance, and also provides recommendations for promoting concurrency.

Tools for performance analysis

You can use workstation and browser-based tools to simplify performance analysis for Db2 for z/OS.

Db2 trace output (Db2 Performance)

Db2 trace (Db2 Performance)

Tools for performance analysis

You can use workstation and browser-based tools to simplify performance analysis for Db2 for z/OS.

For example, the following tools simplify performance analysis for Db2 for z/OS:

- “IBM Data Server Manager” on page 324
- “IBM Data Studio” on page 325
- Tivoli OMEGAMON XE for Db2 Performance Expert on z/OS
- Buffer Pool Analyzer overview (Tivoli OMEGAMON XE for Db2 Performance Expert on z/OS)
- Db2 SQL Performance Analyzer for z/OS
- https://www.ibm.com/support/knowledgecenter/SSAURY_3.3.0/topics/cqmhome.html

You can also use the Db2 EXPLAIN facility to analyze query access paths. For more information, see Investigating SQL performance by using EXPLAIN (Db2 Performance).

IBM OMEGAMON for Db2 Performance Expert on z/OS

IBM OMEGAMON for Db2 Performance Expert on z/OS integrates performance monitoring, reporting, buffer pool analysis, and a performance warehouse function into one tool. It provides a single-system overview that monitors all subsystems and instances across many different platforms in a consistent way.

IBM OMEGAMON for Db2 Performance Expert on z/OS includes the function of Db2 Performance Monitor on z/OS. It provides the following features:

- Combined information from EXPLAIN and from the Db2 catalog.
- Displays of access paths, indexes, tables, table spaces, plans, packages, DBRMs, host variable definitions, ordering, table access sequences, join sequences, and lock types.
- An immediate "snapshot" view of Db2 for z/OS activities that the online monitor provides. The monitor allows for exception processing while the system is operational.

IBM OMEGAMON for Db2 Performance Expert on z/OS offerings support Db2 for z/OS on IBM zSystems, and Db2 for Linux, UNIX, and Windows, HP-UX, Sun Solaris, and IBM AIX®).

Related concepts

[The role of buffer pools in caching data](#)

Buffer pools are a key element of Db2 performance, and help you to avoid delays when retrieving data.

Related reference

[Db2 Tools for z/OS](#)

Related information

[Db2 Tools for z/OS Product Documentation](#)

Ways to move data efficiently through the system

As data progresses through a Db2 subsystem, it moves from disk to memory and to the user or to applications. You need to tune the system resources and objects such as buffer pools, table spaces, and indexes, that contain data to keep the flow of data efficient.

The role of buffer pools in caching data

Buffer pools are a key element of Db2 performance, and help you to avoid delays when retrieving data.

Db2 can retrieve a page from a buffer pool faster than it can from disk. When data is already in a buffer, an application program avoids the delay of waiting for Db2 to retrieve the data from disk.

You can specify that Db2 uses up to 50 buffer pools that contain 4 KB pages. You can also specify up to 10 buffer pools each that contain 8 KB, 16 KB, and 32 KB pages.

The following figure shows buffer pools with 4 KB and 8 KB pages. The number of pages that a buffer pool contains depends on the size of the buffer pool.

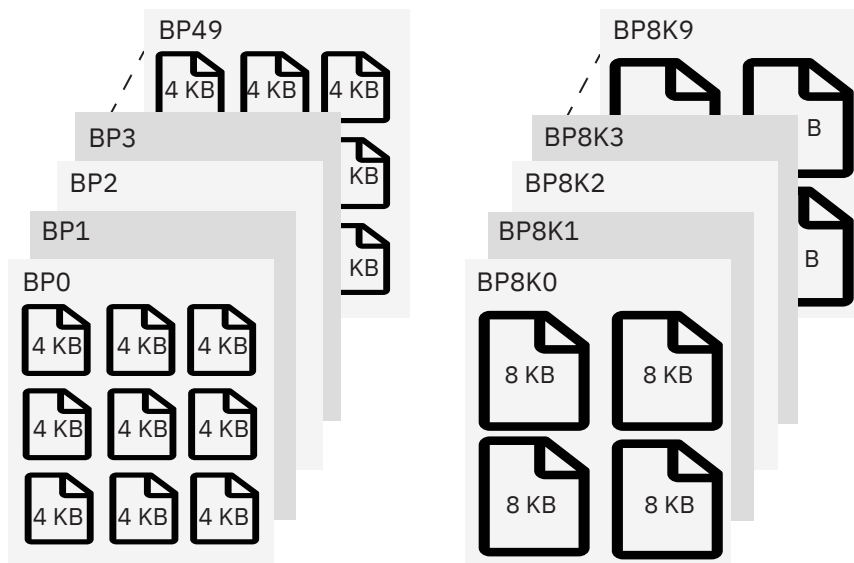


Figure 44. Buffer pools with 4 KB and 8 KB pages

At any time, pages in a virtual buffer pool can be in-use, updated, or available:

- *In-use pages* are currently being read or updated. The data that they contain is available for use by other applications.
- *Updated pages* contain changed data that is not yet written to disk.
- *Available pages* are ready for use. An incoming page of new data can overwrite available pages.

To avoid disk I/O, you can use updated and available pages that contain data.

When data in the buffer changes, that data must eventually be written back to disk. Because Db2 does not need to write the data to disk right away, the data can remain in the buffer pool for other uses. The data remains in the buffer until Db2 decides to use the space for another page. Until that time, applications can read or change the data without a disk I/O operation.

The key factor that affects the performance of buffer pools is their size. The method that Db2 uses to access buffer pools also affects performance.

Buffer pool size

The size of buffer pools is critical to the performance characteristics of an application or a group of applications that access data in those buffer pools.

By tuning your buffer pools, you can improve the response time and throughput for your applications, and provide optimum use of resources. For example, applications that process online transactions are more likely to need large buffer pools because they often must reaccess the same data. In those cases, storing large amounts of data in a buffer pool enables applications to access data more efficiently.

By making buffer pools as large as possible, you can achieve the following benefits:

- Fewer I/O operations result, which means faster access to your data.
- I/O contention is reduced for the most frequently used tables and indexes.
- Sort speed is increased because of the reduction in I/O contention for work files.

You can use the ALTER BUFFERPOOL command to change the size and other characteristics of a buffer pool at any time while Db2 is running. Use the DISPLAY BUFFERPOOL and ALTER BUFFERPOOL commands to gather buffer pool information and change buffer pool sizes.

Db2 Buffer Pool Analyzer for z/OS helps database administrators manage buffer pools more efficiently by providing information about current buffer pool behavior and by using simulation to anticipate future behavior. Using this tool, you can take advantage of these features:

- Collection of data about virtual buffer pool activity
- Comprehensive reporting of the buffer pool activity
- Simulated buffer pool usage
- Reports and simulation results
- Expert analysis that is available through an easy-to-use wizard

Db2 Buffer Pool Analyzer capabilities are included in IBM OMEGAMON for Db2 Performance Expert on z/OS Db2 Performance Expert.

Efficient page access

Db2 determines when to use a method called *sequential prefetch* to read data pages faster. With sequential prefetch, Db2 determines in advance that a set of data pages is about to be used. Db2 then reads the set of pages into a buffer with a single I/O operation. The prefetch method is always used for table space scans and is sometimes used for index scans. Prefetching is processed concurrently with other application I/O operations.

In addition to a predetermined sequential prefetch, Db2 also supports dynamic prefetch. A dynamic prefetch is a more robust and flexible method that is based on sequential detection.

Related concepts

[How z/OS uses physical and virtual storage \(z/OS basic skills\)](#)

[What is paging? \(z/OS Basic Skills\)](#)

[Buffer pools](#)

Buffer pools are areas of virtual storage that temporarily store pages of table spaces or indexes.

[Read operations and prefetch I/O \(Db2 Performance\)](#)

[Tools for performance analysis](#)

You can use workstation and browser-based tools to simplify performance analysis for Db2 for z/OS.

Related tasks

[Tuning database buffer pools \(Db2 Performance\)](#)

Related reference

[-ALTER BUFFERPOOL command \(Db2\) \(Db2 Commands\)](#)

[-DISPLAY BUFFERPOOL command \(Db2\) \(Db2 Commands\)](#)

The effect of data compression on performance

In many cases, compressing the data in a table space significantly reduces the amount of disk space that is needed to store data. Compressing data can also help improve buffer pool performance. For example, you can store more data in a buffer pool, and Db2 can scan large amounts of data more easily.

With compressed data, performance improvements depend on the SQL workload and the amount of compression. You might see some of the following benefits:

- Higher buffer pool hit ratios. The hit ratio measures how often a page is accessed without requiring an I/O operation.
- Fewer operations in which Db2 accesses a data page.

The compression ratio that you achieve depends on the characteristics of your data and the type of compression that is being used. Compression can work well for large table spaces. With small table spaces, the process of compressing data can negate the space savings that compression provides.

Consider these factors when deciding whether to compress data using fixed-length compression:

- Db2 compresses data one row at a time. If Db2 determines that compressing the row yields no savings, the row is not compressed. The closer that the average row length is to the actual page size, the less efficient compression can be.
- Compressing data costs processing time. Although decompressing data costs less than compressing data, the overall cost depends on the patterns in your data.

If the compression ratio is less than 10%, compression is not beneficial and, therefore, is not recommended.

You can use the DSN1COMP utility to determine the probable effectiveness of compressing your data if Db2 is using fixed-length compression.

If Db2 is using fixed-length compression, you use the COMPRESS clause of the CREATE TABLESPACE and ALTER TABLESPACE statements to compress data in a table space, data in a partition of a partitioned table space, or data in indexes.

FL 504If Db2 is using Huffman compression, see [Compressing your data \(Db2 Performance\)](#) for information about compressing data.

Related concepts

[Deciding whether to compress data \(Db2 Performance\)](#)

[Support for compressing LOB data \(Db2 for z/OS What's New?\)](#)

Related tasks

[Compressing your data \(Db2 Performance\)](#)

[Compressing indexes \(Db2 Performance\)](#)

Related reference

[DSN1COMP \(Db2 Utilities\)](#)

How data organization can affect performance

To achieve optimal performance for table spaces and indexes, you need to keep data organized efficiently. The use of space and the organization of data in a table space and the associated indexes sometimes affects performance.

Related tasks

[Maintaining data organization \(Db2 Performance\)](#)

Use of free space in data and index storage

An important factor that affects how well your table spaces and indexes perform is the amount of available *free space*. Free space refers to the amount of space that Db2 leaves free in a table space or index when data is loaded or reorganized.

Freeing pages or portions of pages can improve performance, especially for applications that perform high-volume inserts or that update varying-length columns. When you specify a sufficient amount of free space, you trade the amount of used disk space for the performance of certain SQL statements. For example, inserting new rows into free space is faster than splitting index pages.

You use the `FREEPAGE` and `PCTFREE` clauses of the `CREATE` and `ALTER TABLESPACE` and `INDEX` statements to set free space values.

Related tasks

[Reserving free spaces for indexes \(Db2 Performance\)](#)

[Reserving free space in table spaces \(Db2 Performance\)](#)

Related reference

[CREATE TABLESPACE statement \(Db2 SQL\)](#)

[ALTER TABLESPACE statement \(Db2 SQL\)](#)

[CREATE INDEX statement \(Db2 SQL\)](#)

[ALTER INDEX statement \(Db2 SQL\)](#)

Guidelines for data reorganization

You must consider several factors before you reorganize your data.

You must run the REORG utility only when you determine that data needs to be reorganized. If application performance is not degraded, you might not need to reorganize data. Even when some statistics indicate that data is becoming unorganized, a REORG utility job is not always required, unless the lack of organization exceeds a specified threshold.

In the following situations, data reorganization is advisable:

Data is in REORG-pending status

When table spaces or partitions are in *REORG-pending* (*REORP*) status, you cannot select, insert, update, or delete data. You must reorganize table spaces or partitions when REORG-pending status imposes this restriction.

You can use the **DISPLAY DATABASE RESTRICT** command to identify the table spaces and partitions that need to be reorganized.

Data is in advisory REORG-pending status

After you change table or index definitions, consider reorganizing data to improve performance. After you change data types or column lengths by using `ALTER TABLE` statements, Db2 places the table space that contains the modified data in advisory REORG-pending (*AREO**) status. The table space is in *AREO** status because the existing data is not immediately converted to its new definition. Reorganizing the table space prevents possible performance degradation.

Recommendation: When data is in REORG-pending or AREO* status, use the REORG utility with the SCOPE PENDING option to automatically reorganize partitions. With this option, you do not need to first identify which partitions need to be reorganized or to customize the REORG control statement.

Data is skewed

When you use partitioned table spaces, you might sometimes find that data is out of balance, or skewed. When data is skewed, performance can be negatively affected because of contention for I/O and other resources. You might also have a situation in which some partitions are approaching their maximum size, and other partitions have excess space.

You can correct the skewed data by redistributing the data across partitions. For more information, see [Redistributing data in partitioned table spaces \(Db2 Administration Guide\)](#).

Data is unorganized or fragmented

When data becomes unorganized or fragmented, you need to consider reorganizing your table spaces and index spaces.

You need to consider the following situations to evaluate when data reorganization is necessary:

Unused space

In simple table spaces, dropped tables use space that is not reclaimed until you reorganize the table space. Consider running REORG if the percentage of space that is occupied by rows of dropped tables is greater than 10%. The PERCDROP value in the SYSIBM.SYSTABLEPART catalog table identifies this percentage.

Page gaps

Indexes can have multiple levels of pages. An index page that contains pairs of keys and identifiers and that points directly to data is called a *leaf page*.

Deleting index keys can result in page gaps within leaf pages. Gaps can also occur when Db2 inserts an index key that does not fit onto a full page. Sometimes Db2 detects sequential inserts and splits the index pages asymmetrically to improve space usage and reduce split processing. You can improve performance even more by choosing the appropriate page size for index pages. If page gaps occur, consider running the REORG utility.

The LEAFNEAR and LEAFFAR columns of SYSIBM.SYSINDEXPART store information about the organization of physical leaf pages by indicating the number of pages that are not in an optimal position.

I/O activity

You can determine when I/O activity on a table space might be increasing. A large number (relative to previous values that you received) for the NEARINDREF or the FARINDREF option indicates an increase in I/O activity. Consider a reorganization when the sum of NEARINDREF and FARINDREF values exceeds 10%.

The NEARINDREF and FARINDREF values in the SYSIBM.SYSTABLEPART and SYSIBM.SYSTABLEPART_HIST catalog tables identify the number of reallocated rows.

Recommendation: When increased I/O activity occurs, use a non-zero value for the PCTFREE clause of the table space definition. The PCTFREE clause specifies what percentage of each page in a table space or index is left free when data is loaded or reorganized. PCTFREE is a better choice than FREEPAGE.

Clustering

You can determine if clustering is becoming degraded. Clustering becomes degraded when the rows of a table are not stored in the same order as the entries of its clustering index. A large value for the FAROFFPOSF option might indicate poor clustering. Reorganizing the table space can improve performance. Although less critical, a large value for the NEAROFFPOSF option can also indicate that reorganization might improve performance. The FAROFFPOSF and NEAROFFPOSF values in the

SYSIBM.SYSINDEXPART and SYSIBM.SYSINDEXPART_HIST catalog tables identify the number of rows that are far from and near to optimal position.

REORG thresholds

You can use the RUNSTATS, REORG, REBUILD INDEX, and LOAD utilities to collect statistics that describe the fragmentation of table spaces and indexes. These statistics can help you decide when to run the REORG utility to improve performance or reclaim space.

You can set up your REORG job in accordance with threshold limits that you set for relevant statistics from the catalog. The OFFPOSLIMIT and INDREFLIMIT options specify when to run REORG on table spaces. When a REORG job runs with these options, it queries the catalog for relevant statistics. The REORG job does not occur unless one of the thresholds that you specify is exceeded. You can also specify the REPORTONLY option to produce a report that tells you whether a REORG job is recommended.

Related tasks

[Maintaining data organization \(Db2 Performance\)](#)

Related reference

[REORG-pending status \(Db2 Utilities\)](#)

[REORG TABLESPACE \(Db2 Utilities\)](#)

[REORG INDEX \(Db2 Utilities\)](#)

[RUNSTATS \(Db2 Utilities\)](#)

[LOAD \(Db2 Utilities\)](#)

[SYSTABLEPART catalog table \(Db2 SQL\)](#)

[SYSINDEXPART catalog table \(Db2 SQL\)](#)

Ways to improve performance for multiple users

Locking ensures the integrity and accuracy of data. However, locking is sometimes too restrictive and can cause slow performance and poor concurrency. Understanding how locking works can help you make good decisions for performance. Locking is critical to performance, and also provides recommendations for promoting concurrency.

Db2 uses locks on user data. The main reason for using locks is to ensure the integrity, or accuracy, of the data. Without locks, one user might be retrieving a specific data item while another user might be changing that data. The result is that the first user retrieves inaccurate data. In the Db2 for z/OS environment, which includes vast amounts of data and large numbers of users and transactions, the prospect of inaccurate data is unacceptable. Therefore, Db2 for z/OS provides comprehensive locking to ensure data integrity.

Despite the importance of data integrity, locking can sometimes be too restrictive. If an application process locks too much data, other users, utilities, and application processes must wait for the locked data. This situation results in poor concurrency. *Concurrency* is the ability of more than one application process to access the same data at essentially the same time. Db2 for z/OS handles the trade-off between concurrency and data integrity to maximize concurrency without sacrificing the integrity of the data.

Related concepts

[Lock contention, suspension, deadlock, and timeout \(Db2 Performance\)](#)

Related tasks

[Improving concurrency \(Db2 Performance\)](#)

Related reference

[Concurrency and locking controls in Db2 for z/OS \(Db2 Performance\)](#)

Improved performance through the use of locks

Db2 uses locks on various data objects.

Locks can be placed on rows, pages, tables, table space segments, table space partitions, entire table spaces, and databases. When an application acquires a lock, the application "holds" or "owns" the lock.

Db2 uses of the lock modes to determine whether one lock is *compatible* with another. Some lock modes do not exclude all other users. For example, assume that application process A holds a lock on a table space that process B also wants to access. Db2 requests, on behalf of process B, a lock of some particular mode. If the mode of the lock for process A permits the lock requested by process B, the modes of the two locks are said to be compatible. However, if the two locks are not compatible, process B cannot proceed. It must wait until process A releases its lock, and until all other existing incompatible locks are released.

The following lock modes provide different degrees of protection:

S lock (share)

The lock owner and any concurrent processes can read, but not change, the locked page or row. Concurrent processes can acquire S or U locks on the page or row or might read data without acquiring a page or row lock. Only one U lock can be acquired concurrently with one or more S locks.

U lock (update)

The lock owner can read, but not change, the locked page or row. Concurrent processes can acquire S locks or might read data without acquiring a page or row lock, but no concurrent process can acquire a U lock.

U locks reduce the chance of deadlocks when the lock owner is reading a page or row to determine whether to change it. The owner can start with the U lock and then promote the lock to an X lock to change the page or row.

X lock (exclusive)

The lock owner can read or change the locked page or row. A concurrent process cannot acquire S, U, or X locks on the page or row. However, concurrent processes, such as those processes bound with the CURRENTDATA(NO) or ISOLATION(UR) bind options or running with YES specified for the EVALUNC subsystem parameter, can read the data without acquiring a page or row lock.

The following table shows the compatibility of any two modes for page and row locks. No question of compatibility arises between page and row locks, because a partition or table space cannot use both page and row locks.

Table 26. Compatibility matrix of page lock and row lock modes			
Lock mode	Share (S-lock)	Update (U-lock)	Exclusive (X-lock)
Share (S-lock)	Yes	Yes	No
Update (U-lock)	Yes	No	No
Exclusive (X-lock)	No	No	No

The share, update, and exclusive locks apply to row or page locks. These facts apply only to application processes that acquire an intent lock on the table space and the table, if the table is in a segmented table space.

When a page or row is locked, the table, partition, or table space that contains it is also locked. This *intent lock* indicates the plan that the application process has for accessing the data. In that case, the table, partition, or table space lock has one of the *intent modes*: either IS for *intent share*, IX for *intent exclusive*, or SIX for *share with intent exclusive*.

Compatibility for table space locks is slightly more complex than for page and row locks.

Locks are important for maintaining concurrency in the Db2 environment. However, locks might cause several types of *contention* situations that degrade Db2 performance, including suspension, timeout, and deadlock.

Suspension

An application encounters *suspension* when it requests a lock that is already held by another application process and cannot be shared. The suspended process temporarily stops running. A suspended process resumes when all processes that hold the conflicting lock release them or the requesting process experiences a timeout or deadlock and the process resumes and handles an error condition.

Timeout

An application process encounters a *timeout* when it terminates because of a suspension that exceeds a preset interval. Db2 terminates the process, issues messages, and returns error codes. Commit and rollback operations do not timeout. The STOP DATABASE command, however, can time out, in which case Db2 sends messages to the console. When this happens Db2 retries the STOP DATABASE command as many as 15 times.

Deadlock

A *deadlock* occurs when two or more application processes each hold locks on resources that the others need and without which they cannot proceed. After a preset time interval, Db2 can roll back the current unit of work for one of the processes or request a process to terminate. In determining which process to roll back or terminate, Db2 assesses many characteristics of the processes that are involved in the deadlock and chooses the one that, if terminated, will cause the least impact relative to the other processes. By choosing a process to roll back or terminate, Db2 frees the locks and allows the remaining processes to continue.

Although some locking problems can occur, you can avoid system and application locking problems.

The following scenarios illustrate the importance of locks.

Scenario: Avoidance of the loss of updated data

Two users, Kathy and Frank, are both trying to access the same Db2 table. Here is what happens:

1. Kathy reads the data value, 100, into a host variable.
2. Frank reads the same column value into a host variable.
3. Kathy adds 10 to the host variable value and saves the new value, 110, in the Db2 table column.
4. Frank adds 20 to the host variable value and saves the new value, 120, in the Db2 table column.

This scenario does not use locking. It shows that the updated value in the column depends on which user commits the data first. If Kathy commits first, the updated column value is 120, and Kathy's update is lost. If Frank commits first, the updated column value is 110, and Frank's update is lost.

The scenario changes if it includes locking. When you read the process below, assume the use of an updatable cursor. Here is what happens:

1. Kathy reads column value 100 into a host variable with the intention of updating the value. Db2 then grants an update lock to Kathy.
2. Frank wants to read the same column value into a host variable with the intention of updating the value. According to the compatibility matrix in the table above, Db2 does not grant Frank an update lock (U-lock) on the Db2 object that contains column value 100. Therefore, Frank must wait to read the column value until Kathy releases the lock.
3. Kathy adds 10 to the host variable value and wants to save the new value, 110, in the Db2 table column. At this point, Db2 changes the U-lock to an exclusive lock (X-lock) on the Db2 object that contains the column value.
4. Kathy commits the change. Db2 then releases the X-lock on the Db2 object that contains the column value. Next, Db2 grants the U-lock to Frank on the same object (unless Frank timed out while waiting for access). The host variable that Frank specified now contains the updated value of 110.
5. Frank adds 20 to the host variable value and wants to save the new value, 130, in the table column. Db2 changes the U-lock to an X-lock on the Db2 object that contains the column value.
6. Frank commits the change. Db2 then releases the X-lock on the Db2 object that contains the column value.

If this scenario did not include updatable cursors, Db2 would grant a share lock (S-lock) to Kathy instead of a U-lock in step 1. Db2 would also grant an S-lock to Frank in step 2. When both Kathy and Frank try to update the column value, they would encounter a deadlock. When a deadlock occurs, Db2 decides whether to roll back Kathy's work or Frank's work. A rollback occurs when Db2 reverses a change that an individual application process tried to make. If Db2 rolls back Kathy's changes, Kathy releases the locks, and Frank can then complete the process. Conversely, if Db2 rolls back Frank's changes, Frank releases the locks, and Kathy can complete the process.

Application programs can minimize the risk of deadlock situations by using the FOR UPDATE OF clause in the DECLARE CURSOR statement. The program does not actually acquire the U-lock until any other U-locks or X-locks on the data object are released.

Scenario: Avoidance of read access to uncommitted data

Two users, Kathy and Frank, are both trying to access the same Db2 table.

1. Kathy updates the value of 100 to 0 in the Db2 table column.
2. Frank reads the updated value of 0 and makes program decisions based on that value.
3. Kathy cancels the process and changes the value of 0 back to 100 for the Db2 table column.

This scenario does not include locks. It shows that Frank made an incorrect program decision. As a result, the business data in the database might be inaccurate.

When this scenario includes locking, this is what happens:

1. Kathy attempts to update the value of 100 to 0 in the table column. Db2 grants an X-lock to Kathy on the Db2 object that contains the column value that requires an update.
2. Frank tries to read the updated column value so that he can make program decisions based on that value. Db2 does not allow Frank to read the updated column value of 0. Frank tries to acquire an S-lock on the Db2 object that currently has the X-lock. Frank must wait until Kathy commits or rolls back the work.
3. Kathy cancels the process and changes the value of 0 back to the original value of 100 for the Db2 table column. Db2 makes the actual change to the data and releases the X-lock for Kathy. Db2 then grants the S-lock to Frank on the Db2 object that contains the column value. Frank then reads the value of 100.

When the scenario includes locks, Frank reads the correct data and can therefore make the correct program decision. As a result, the business data in the database is accurate.

Scenario: Avoidance of updates between multiple reads within a unit of work

In this scenario, Kathy wants to read the same data twice. No other program or user can change the data between the two reads.

GUIP

Assume that Kathy uses the following SQL statement:

```
SELECT * FROM EMP
WHERE SALARY >
      (SELECT AVG(SALARY) FROM EMP);
```

GUIP

This SQL statement reads the EMP table twice:

1. It calculates the average of the values in the SALARY column of all rows in the table.
2. It finds all rows in the EMP table that have a value in the SALARY column that exceeds the average value.

If Kathy does not lock the data between the two read processes, another user can update the EMP table between the two read processes. This update can lead to an incorrect result for Kathy.

Kathy could use Db2 locks to ensure that no changes to the table occur in between the two read processes. Kathy can choose from these options:

- Using the package or plan isolation level of repeatable read (RR) or using the WITH RR clause in the SQL SELECT statement.
- **GUIP**
- Locking the table in share or exclusive mode, using one of these statements:
 - LOCK TABLE EMP IN SHARE MODE
 - LOCK TABLE EMP IN EXCLUSIVE MODE

GUIP

Related concepts

[Concurrency and locks \(Db2 Performance\)](#)

[Lock modes and compatibility of locks \(Db2 Performance\)](#)

[Lock contention, suspension, deadlock, and timeout \(Db2 Performance\)](#)

Related tasks

[Programming for concurrency \(Db2 Performance\)](#)

[Designing databases for concurrency \(Db2 Performance\)](#)

Improved performance through concurrency control

Concurrency control relies on the *isolation level* and *current data* options of applications to determine how much to isolate different applications that access the same data from each other. Too much isolation might result in contention, whereas too little isolation might result in the return of non-current data to applications.

The *ISOLATION* option of an application specifies the degree to which operations are isolated from the possible effects of other operations that act concurrently. The ISOLATION options specify how soon Db2 can release S and U locks on rows or pages. Regardless of the isolation level that you specify, outstanding claims on Db2 objects can inhibit the execution of Db2 utilities or commands.

The *CURRENTDATA* option of an application specifies whether data currency is required for read-only and ambiguous cursors when the ISOLATION(CS) option is used. This option enables a trade-off between the improved ability of multiple applications to access the same data concurrently and the risk that non-current data might be returned to the application.

The basic recommendation is to bind most applications with the ISOLATION(CS) and CURRENTDATA(NO) options. ISOLATION(CS) typically enables Db2 to release acquired locks as soon as possible. The CURRENTDATA(NO) typically enables Db2 to acquire the fewest number of locks, for better *lock avoidance*.

Db2 provides the following isolation levels:

Cursor stability (CS)

The ISOLATION (CS) or *cursor stability* option allows maximum concurrency with data integrity. Under the ISOLATION (CS) option, a transaction holds locks only on its uncommitted changes and on the current row of each of its cursors.

Uncommitted read (UR)

The ISOLATION (UR) or *uncommitted read* option allows an application to read while acquiring few locks, at the risk of reading uncommitted data. UR isolation applies only to the following read-only operations: SELECT, SELECT INTO, or FETCH from a read-only result table.

Read stability (RS)

The ISOLATION (RS) or *read stability* option enables an application to read the same pages or rows more than once and prevents updates or deletes to qualifying rows by other processes. However, other applications can insert or update rows that did not satisfy the search condition of the original application.

Repeatable read (RR)

The ISOLATION (RR) or *repeatable read* option allows the application to read the same pages or rows more than once without allowing any update, insert, or delete operations by other processes. All accessed rows or pages are locked, even if they do not satisfy the predicate. Under the ISOLATION (RR) option, the data that an application references cannot be updated by any other applications before the application reaches a commit point.

Db2 uses and depends on locks because of the requirement for data integrity. However, locks are sometimes the cause of problems with contention, such as deadlocks, timeouts, and suspensions. To minimize these problems and promote concurrency, database designers and application designers can take various actions.

Related tasks

[Improving concurrency \(Db2 Performance\)](#)

[Choosing an ISOLATION option \(Db2 Performance\)](#)

[Designing databases for concurrency \(Db2 Performance\)](#)

[Specifying the size of locks for a table space \(Db2 Performance\)](#)

[Programming for concurrency \(Db2 Performance\)](#)

Related reference

[BIND and REBIND options for packages, plans, and services \(Db2 Commands\)](#)

Concurrency recommendations for database designers

Database designers can take certain general actions to promote concurrency without compromising data integrity.

Procedure

To promote concurrency without compromising data integrity, use any of the following approaches:

- Keep like things together in the database.
- Keep unlike things apart from each other in the database.
- Use LOCKSIZE ANY or PAGE as a design default. Consider LOCKSIZE ROW only when applications encounter significant lock contention, including deadlock and timeout.
- Examine small tables, looking for opportunities to improve concurrency by reorganizing data or changing the locking approach.
- Partition secondary indexes to promote partition independence and reduce lock contention.
- Minimize update activity that moves rows across partitions.
- Store fewer rows of data in each data page.

Related tasks

[Designing databases for concurrency \(Db2 Performance\)](#)

[Improving concurrency \(Db2 Performance\)](#)

Concurrency recommendations for application designers

Application designers can take certain general actions to promote concurrency without compromising data integrity.

Procedure

To promote concurrency without compromising data integrity, use any of the following approaches:

- Program applications to access data in the same order.
- Commit work as soon as doing so is practical, to avoid unnecessary lock contention, even in read-only applications.

- Include logic in your application program to retry after a deadlock or timeout to attempt recovery from the contention situation without assistance.
- Bind most applications with the ISOLATION(CS) and CURRENTDATA(NO) options.
These options enable Db2 to release locks early and avoid taking locks in many cases.
- Use global transactions, which enables Db2 and other transaction managers to participate in a single transaction and thereby share the same locks and access the same data.

Related tasks

[Programming for concurrency \(Db2 Performance\)](#)

[Improving concurrency \(Db2 Performance\)](#)

Ways to improve query performance

Access paths have a significant impact on Db2 performance. Db2 chooses access paths, but you can use tools to understand how access paths affect performance in certain situations.

An *access path* is the path that Db2 uses to locate data that is specified in SQL statements. An access path can be indexed or sequential.

Two important factors in the performance of an SQL statement are the amount of time that Db2 uses to determine the access path at run time and the efficiency of the access path. Db2 determines the access path for a statement either when you bind the plan or package that contains the SQL statement or when the SQL statement executes.

The time at which Db2 determines the access path depends on whether the statement is executed statically or dynamically and whether the statement contains input host variables.

The access path that Db2 chooses determines how long the SQL statement takes to run. For example, to execute an SQL query that joins two tables, Db2 has several options. Db2 might make any of the following choices to process those joins:

- Scan the PARTS table for every row that matches a row in the PRODUCTS table.
- Scan the PRODUCTS table for every row that matches a row in the PARTS table.
- Sort both tables in PROD# order; then merge the ordered tables to process the join.

Choosing the best access path for an SQL statement depends on a number of factors. Those factors include the content of any tables that the SQL statement queries and the indexes on those tables.

Db2 also uses extensive statistical information about the database and resource use to make the best access choices.

In addition, the physical organization of data in storage affects how efficiently Db2 can process a query.

Related tasks

[Writing efficient SQL queries \(Db2 Performance\)](#)

[Maintaining data organization and statistics \(Db2 Performance\)](#)

[Programming applications for performance \(Db2 Performance\)](#)

Tools for improving query performance

Performance analysis and query tuning tools can help you improve the performance of your SQL statements and workloads.

IBM Data Studio or IBM Data Server Manager

IBM Data Studio and IBM Data Server Manager are sets of tools that help you to perform a wide range of database management activities, including query performance tuning.

Db2 Query Workload Tuner for z/OS

Db2 Query Workload Tuner for z/OS helps database administrators and SQL developers optimize the performance of SQL statements in applications that query Db2 for z/OS subsystems.

IBM OMEGAMON for Db2 Performance Expert on z/OS

IBM OMEGAMON for Db2 Performance Expert on z/OS is a performance analysis, monitoring, and tuning tool for Db2 for z/OS environments.

EXPLAIN statement and EXPLAIN tables

In Db2 for z/OS, EXPLAIN statements produces the following information:

- A plan, package, or SQL statement when it is bound. The output appears in a table that you create, called a *plan table*.
- The estimated cost of executing a SELECT, INSERT, UPDATE, or DELETE statement. The output appears in a table that you create, called a *statement table*.
- User-defined functions that are referred to in an SQL statement, including the specific function name and schema. The output appears in a table that you create, called a *function table*.

Db2 SQL Performance Analyzer for z/OS

Db2 SQL Performance Analyzer for z/OS provides you with an extensive analysis of SQL queries without executing them. This analysis aids you in tuning your queries to achieve maximum performance. Db2 SQL Performance Analyzer helps you reduce the escalating costs of database queries by estimating their cost before execution.

Using Db2 SQL Performance Analyzer for z/OS helps you:

- Estimate how long queries are likely to take
- Prevent queries from running too long
- Analyze new access paths
- Code queries efficiently using hints and tips that the tool provides

Related concepts

[Investigating SQL performance by using EXPLAIN \(Db2 Performance\)](#)

[Interpreting data access by using EXPLAIN \(Db2 Performance\)](#)

[IBM Data Server Manager](#)

IBM Data Server Manager is an integrated database management tools platform for monitoring performance, tracking configuration changes, and database administration in Db2 for z/OS subsystems. IBM Data Server Manager is being replaced by Db2 Administration Foundation and Db2 Developer Extension.

[IBM Data Studio](#)

IBM Data Studio is a set of tools for managing enterprise data, databases, and data-driven applications. IBM Data Studio is being replaced by Db2 Administration Foundation and Db2 Developer Extension.

Related tasks

[Investigating access path problems \(Db2 Performance\)](#)

Related reference

[Facilities and tools for Db2 performance monitoring \(Db2 Performance\)](#)

[EXPLAIN statement \(Db2 SQL\)](#)

[EXPLAIN tables \(Db2 Performance\)](#)

Query and application performance analysis

Performance analysis for queries and applications begins by answering some basic questions.

To improve query performance and application performance, you need to answer some basic questions to determine how well your queries and applications perform.

Tip: Query tuning capabilities that can help you with this task, such as *visual explain* and *statistics advisor*, are available in [IBM Db2 Administration Foundation for z/OS](#) and [IBM Db2 for z/OS Developer Extension](#).

Are the catalog statistics up-to-date?

Keeping object statistics current is an important activity. Db2 needs those statistics to choose an optimal access path to data.

Db2 utilities can collect statistics about Db2 objects. The RUNSTATS utility is designed for collecting statistics, but you can also collect inline statistics when you run certain other utilities, such as the LOAD, REORG INDEX, REORG TABLESPACE, and REBUILD INDEX utilities. These statistics are stored in the Db2 catalog. Db2 uses this information during the bind process to choose an access path. If you do not collect statistics and then rebind your packages or plans, Db2 does not have the information that it needs to choose the most efficient access path. Lack of statistical information can result in unnecessary I/O operations and excessive processor consumption.

Recommendation: Collect statistics at least once for each table and its associated indexes. How often you rerun the utility depends on how current you need the catalog data to be. If data characteristics of a table vary significantly over time, you must keep the catalog current with those changes. Collecting statistics is most beneficial for the following objects:

- Table spaces that contain frequently accessed tables
- Tables that are involved in sort operations
- Tables with many rows
- Tables that are queried by SELECT statements that include many search arguments
- Tables with indexes

For more information, see

[Investigating access path problems \(Db2 Performance\)](#)
[Maintaining Db2 database statistics \(Db2 Performance\)](#)
[RUNSTATS \(Db2 Utilities\)](#)

Does the estimated filter factor match the filtering at run time?

Db2 uses estimated filter factors to choose efficient access paths. When the estimated and actual filtering differs, Db2 might choose an access path that performs poorly because the cost estimates are inaccurate. When the estimated and actual factors differ, you might collect more statistics or reoptimize statements at run time to improve access path selection.

For more information, see

[Predicate filter factors \(Db2 Performance\)](#)
[Investigating access path problems \(Db2 Performance\)](#)
[Reoptimizing SQL statements at run time \(Db2 Performance\)](#)
[Overriding predicate selectivities at the statement level \(Db2 Performance\)](#)

Is the query coded as simply and efficiently as possible?

Ensure that the SQL query is coded as simply and efficiently as possible. Make sure that:

- Unused columns are not selected.
- No unneeded ORDER BY or GROUP BY clauses are in the query.

- No unneeded predicates are in the query.

For more information, see

[Writing efficient SQL queries \(Db2 Performance\)](#)

[Coding SQL statements to avoid unnecessary processing \(Db2 Performance\)](#)

[Using predicates efficiently \(Db2 Performance\)](#)

Are you using materialized query tables?

Define materialized query tables to improve the performance of dynamic queries that operate on large amounts of data and that involve multiple joins. Db2 generates the results of all or parts of the queries in advance and stores the results in materialized query tables. Db2 determines when using precomputed results is likely to optimize the performance of dynamic queries.

Note:

For more information, see

[Using materialized query tables to improve SQL performance \(Db2 Performance\)](#)

Is access through an index?

An index provides efficient access to data. Db2 uses different types of index scans, each of which affects performance differently. Sometimes Db2 can avoid a sort by using an index.

If a query is satisfied by using only the index, Db2 uses a method called *index-only access*.

- For a SELECT operation, if all the columns that are needed for the query can be found in the index, Db2 does not need to access the table.
- For an UPDATE or DELETE operation, an index-only scan can be performed to search for qualifying rows to update or delete. After the qualifying rows are identified, Db2 must retrieve those rows from the table space before they are updated or deleted.

Other types of index scans that Db2 might use are matching or nonmatching index scans.

- In a matching index scan, the query uses predicates that match the index columns. Predicates provide filtering; Db2 needs to access only specific index and data pages.
- In a nonmatching index scan, Db2 reads all index keys and their rows of data. This type of scan is less likely to provide an efficient access path than a matching index scan.

In addition to providing selective access to data, indexes can also order data, and sometimes they eliminate the need for sorting. You can avoid some sorts if index keys are in the order that is needed by ORDER BY, GROUP BY, a join operation, or DISTINCT in an aggregate function. When you want to prevent a sort, consider creating an index on the columns that are necessary to provide that ordering.

For more information, see

[Index access \(ACCESSTYPE is 'I', 'IN', 'I1', 'N', 'NR', 'MX', or 'DX'\) \(Db2 Performance\)](#)

[Indexes on table columns \(Db2 Administration Guide\)](#)

Is a table space scan used?

When index access is not possible, Db2 uses a table space scan. Db2 typically uses the sequential prefetch method to scan table spaces.

GUIP

For example, assume that table T has no index on column C1. Db2 uses a table space scan in the following example:

```
SELECT * FROM T WHERE C1 = VALUE;
```


In this case, every row in table T must be examined to determine if the value of C1 matches the given value.

GUIP

A table space scan on a partitioned table space can be more efficient than a scan on a nonpartitioned table space. Db2 can take advantage of the partitions by limiting the scan of data in a partitioned table space to one or more partitions.

For more information, see:

[Table space scan access \(ACCESSTYPE='R' and PREFETCH='S'\) \(Db2 Performance\)](#)
[Sequential prefetch \(PREFETCH='S'\) \(Db2 Performance\)](#)

Are sorts performed?

Minimizing the need for Db2 to use sorts to process a query can result in better performance. In general, try to create indexes that match the predicates in your queries before trying to avoid sorts in your queries.

For more information, see:

[Sorts of data \(Db2 Performance\)](#)
[Sort access \(Db2 Performance\)](#)

Is data accessed or processed in parallel?

Parallel processing applies to read-only queries. Db2 can use parallel I/O and CPU operations to improve performance. For example, Db2 can use multiple parallel operations to access data from a table or index. The response time for data-intensive or processor-intensive queries can be reduced.

For more information, see [Programming for parallel processing \(Db2 Performance\)](#).

Are host variables used?

When you specify the bind option REOPT(VARS), Db2 determines the access paths at both bind time and run time for statements that contain one or more host variables, parameter markers, or special registers. At run time, Db2 uses the values in those variables to determine access paths.

Db2 spends extra time determining the access path for statements at run time. But if Db2 finds a better access path using the variable values, you might see an overall performance improvement.

For static SQL applications with host variables, if you specify REOPT(VARS), Db2 determines the access path at bind time and again at run time, using the values of input variables.

For static SQL applications with no host variables, Db2 determines the access path when you bind the plan or package. This situation yields the best performance because the access path is already determined when the program runs.

For applications that contain dynamic SQL statements with host variables, using REOPT(VARS) is the recommended approach for binding.

For more information, see [Reoptimizing SQL statements at run time \(Db2 Performance\)](#).

Are dynamic SQL statements used?

For dynamic SQL statements, Db2 determines the access path at run time, when the statement is prepared.

When an application performs a commit operation, it must issue another PREPARE statement if that SQL statement is to be executed again. For a SELECT statement, the ability to declare a cursor WITH HOLD provides some relief but requires that the cursor be open at the commit point. Using the WITH HOLD option also causes some locks to be held for any objects that the prepared statement depends on. Also, the WITH HOLD option offers no relief for SQL statements that are not SELECT statements.

You can use the dynamic statement cache to decrease the number of times that those dynamic statements must be prepared. Using the dynamic statement cache is useful when you execute the same SQL statement often.

Db2 can save prepared dynamic statements in a cache. The cache is a Db2-wide cache that all application processes can use to store and retrieve prepared dynamic statements. After an SQL statement is prepared and is automatically stored in the cache, subsequent prepare requests for that same SQL statement can use the statement in the cache to avoid the costly preparation process. Different threads, plans, or packages can share cached statements.

The SELECT, UPDATE, INSERT, and DELETE statements are eligible for caching.

Note:

For more information, see [Held and non-held cursors \(Db2 Application programming and SQL\)](#).

Related concepts

[Interpreting data access by using EXPLAIN \(Db2 Performance\)](#)

Related tasks

[Programming applications for performance \(Db2 Performance\)](#)

[Investigating access path problems \(Db2 Performance\)](#)

Related reference

[EXPLAIN statement \(Db2 SQL\)](#)

Using EXPLAIN to understand the access path

You can use the EXPLAIN statement to determine the access paths for the SELECT parts of your statements.

Tip: Query tuning capabilities that can help you with this task, such as *visual explain* and *statistics advisor*, are available in [IBM Db2 Administration Foundation for z/OS](#) and [IBM Db2 for z/OS Developer Extension](#).

This information describes what EXPLAIN provides and how you can obtain information from EXPLAIN. The information in the plan table can help you when you need to perform the following tasks:

- Determine the access path that Db2 chooses for a query
- Design databases, indexes, and application programs
- Determine when to rebind an application

For each access to a single table, EXPLAIN indicates whether Db2 uses index access or a table space scan. For indexes, EXPLAIN indicates how many indexes and index columns are used and what I/O methods are used to read the pages. For joins of tables, EXPLAIN indicates the join method and type, the order in which Db2 joins the tables, and the occasions when and reasons why it sorts any rows.

The following steps summarize how to obtain information from EXPLAIN:

1. Create the plan table.

Before you can use EXPLAIN, you must create a plan table to hold the results of EXPLAIN.

2. Populate the plan table.

You can populate the plan table by executing the SQL statement EXPLAIN. You can also populate a plan table when you bind or rebind a plan or package by specifying the option EXPLAIN(YES). EXPLAIN obtains information about the access paths for all explainable SQL statements in a package or in the DBRMs of a plan.

3. Select information from the plan table.

Several processes can insert rows into the same plan table. To understand access paths, you must retrieve the rows for a particular query in an appropriate order.

EXPLAIN helps you answer questions about query performance; the answers give you the information that you need to make performance improvements. EXPLAIN indicates whether Db2 used an index to access data, whether sorts were performed, whether parallel processing was used, and so on.

As you gain experience working with Db2, you can use the plan table to give optimization hints to Db2 that influence access path selection.

Related concepts

[Investigating SQL performance by using EXPLAIN \(Db2 Performance\)](#)

[Interpreting data access by using EXPLAIN \(Db2 Performance\)](#)

Related tasks

[Investigating access path problems \(Db2 Performance\)](#)

Related reference

[EXPLAIN tables \(Db2 Performance\)](#)

[EXPLAIN statement \(Db2 SQL\)](#)

[EXPLAIN bind option \(Db2 Commands\)](#)

[IBM Data Studio product overview \(IBM Data Studio\)](#)

Related information

[IBM Db2 Data Management Console Documentation](#)

Questions for investigating data access

You can focus your investigation of access paths that Db2 uses to process SQL statements by using the PLAN_TABLE data to answer certain questions.

Tip: You can generate diagrams of the access paths used for your SQL statements by using the *visual explain* capability in tools such as [IBM Db2 Administration Foundation for z/OS](#) and [IBM Db2 for z/OS Developer Extension](#).

PSPI

You can use the following questions to guide your initial analysis of the access paths when you analyze PLAN_TABLE data:

How are indexes used to access the data?

The ACESSTYPE and MATCHOLS values contain information about the use of indexes in an access path.

1. Is an index used? For information about interpreting index access, see [Index access \(ACESSTYPE is 'I', 'IN', 'I1', 'N', 'NR', 'MX', or 'DX'\) \(Db2 Performance\)](#).
2. How many indexes are used? For information about interpreting access through multiple indexes, see [Multiple index access \(ACESSTYPE='M', 'MX', 'MI', 'MU', 'DX', 'DI', or 'DU'\) \(Db2 Performance\)](#).
3. How many index columns are used in matching? For more information about finding the number of matching index columns, see [Matching index scan \(MATCHCOLS>0\) \(Db2 Performance\)](#).
4. Is the query satisfied by the index alone? For more information about analyzing index-only access, see [Index-only access \(INDEXONLY='Y'\) \(Db2 Performance\)](#).
5. How many index screening columns are used? For more information about analyzing index screening, see [Index screening \(Db2 Performance\)](#).

Is direct row access used?

Direct row access can only be used only when the table contains a column of the ROWID data type. For information about direct row access, see [Direct row access \(PRIMARY_ACESSTYPE='D'\) \(Db2 Performance\)](#) and “ROWID data type” on page 27.

What possibly costly operations are used?

1. Is a view or nested table expression materialized? For more information about analyzing materialization, see [View and nested table expression access \(Db2 Performance\)](#).

2. Was a scan limited to certain partitions? For information about analyzing the use of page-range screening, see [Prefetch access paths \(PREFETCH='D', 'S', 'L', or 'U'\) \(Db2 Performance\)](#).
3. What prefetch type is expected? For information about analyzing the use of prefetch, see [Prefetch access paths \(PREFETCH='D', 'S', 'L', or 'U'\) \(Db2 Performance\)](#).
4. Is data accessed or processed in parallel? For information about analyzing the use of parallelism in the access path, see [Parallel processing access \(PARALLELISM_MODE='C'\) \(Db2 Performance\)](#).
5. Is data sorted? For information about analyzing the use of sort operations, see [Sort access \(Db2 Performance\)](#).
6. Is a subquery transformed to a join? For information about analyzing subquery access, see [Subquery access \(Db2 Performance\)](#).
7. When are aggregate functions evaluated? For information about analyzing when Db2 evaluates aggregate functions, see [Aggregate function access \(COLUMN_FN_EVAL\) \(Db2 Performance\)](#).
8. Is a complex trigger WHEN clause used? For information about identifying the use of triggers in a WHEN clause, see [Complex trigger WHEN clause access \(QBLOCKTYPE='TRIGGR'\) \(Db2 Performance\)](#).

What are the object dependencies for the access path?

Access paths have dependencies on the objects that are identified in the TNAME and ACCESSNAME columns of the PLAN_TABLE. However, access paths might also depend on objects that Db2 does not actually use when it processes the selected access paths. Such dependencies are not shown in EXPLAIN output, but they are recorded in the [SYSPACKDEP catalog table \(Db2 SQL\)](#) or the [SYSDYNQRYDEP catalog table \(Db2 SQL\)](#).

PSPI

Related concepts

[Interpreting data access by using EXPLAIN \(Db2 Performance\)](#)

Related reference

[PLAN_TABLE \(Db2 Performance\)](#)

[EXPLAIN statement \(Db2 SQL\)](#)

Hash access paths (deprecated)

Db2 can use hash access paths to directly access a single row in a table and avoid scanning the table or index. Hash-organized table spaces are deprecated and will be unsupported in the future.

Deprecated function:

FL 504 Hash-organized tables are deprecated. Beginning in Db2 12, packages that are bound with APPLCOMPAT(V12R1M504) or higher cannot create hash-organized tables or alter existing tables to use hash-organization. Existing hash-organized tables remain supported, but they will be unsupported in the future.

Db2 can generate a hash access path from the equal predicate of an SQL statement that fetches only a single row of a table. This access method is faster and more efficient than scanning the table or the index.

Hash access can reduce the query access time and the CPU load for queries that require access to a single row. However, tables that are enabled for hash access might require as much as twice the amount of disk space as tables that are not organized for hash access.

Related concepts

[Hash access \(ACCESSTYPE='H', 'HN', or 'MH'\) \(deprecated\) \(Db2 Performance\)](#)

[Db2 hash spaces \(deprecated\)](#)

A *hash space* is a defined disk space that organizes table data for hash access. Hash-organized table spaces are deprecated and will be unsupported in the future.

Related tasks

[Monitoring hash access \(deprecated\) \(Db2 Performance\)](#)

[Managing space and page size for hash-organized tables \(deprecated\) \(Db2 Performance\)](#)

Related information

[DB2 10 for z/OS Performance Topics \(IBM Redbooks\)](#)

[DB2 10 for z/OS Technical Overview \(IBM Redbooks\)](#)

Chapter 5. Programming for Db2 for z/OS

You use application programs that contain SQL statements to perform data definition or data manipulation operations on Db2 for z/OS objects.

Where can I find application programming samples?

See [Db2 programming samples \(Db2 Programming samples\)](#).

What application development tools can I use?

See [“Tools and IDEs for developing Db2 applications” on page 247](#).

What do I need to do in an SQL application?

In an application, you need to complete these tasks:

1. Declare variables for passing data to and from the Db2 database system.
2. Execute SQL statements.
3. Check for and handle SQL errors

What programming languages can I use to write SQL applications?

You can write application programs for Db2 for z/OS data servers in any of the following programming languages:

- [APL2®](#)
 - [Assembler](#)
 - [C and C++](#)
 - [C# “2” on page 243 \(through .NET\)](#)
 - [COBOL](#)
 - [Fortran](#)
 - [High-level Assembler \(or HLASM\)](#)
 - [Java “1” on page 243](#)
 - [Node.js “3” on page 243](#)
 - [Perl “2” on page 243](#)
 - [PHP “2” on page 243](#)
 - [PL/I](#)
 - [Python “4” on page 243](#)
 - [REXX](#)
 - [Ruby on Rails “2” on page 243](#)
 - [SQL procedural language](#)
 - [Visual Basic “2” on page 243](#)
1. By connecting to Db2 for z/OS using [IBM Data Server Driver for JDBC and SQLJ](#).
 2. By remote connections to Db2 for z/OS using [Db2 for Linux, UNIX, and Windows client drivers](#).
 3. By connecting to Db2 for z/OS using the [IBM_DB Node.js driver](#).
 4. By connecting to Db2 for z/OS using the [IBM_DB Python driver](#).

How do I pass data between SQL applications and a Db2 table?

You use host variables and indicator variables to pass data between SQL applications and Db2 tables. *Host variables* are application variables that contain the data that is exchanged with the Db2 table. *Indicator variables* are application variables whose values denote whether there is data in the host variables.

What are static and dynamic SQL applications?

In a static SQL application, the SQL statements are written directly in the application. When the application runs, the user can change only the variable values. In a dynamic SQL application, the user provides the SQL statements when the application runs. In general, dynamic SQL applications are more flexible than static SQL applications. However, dynamic applications are harder to write, and often run more slowly because the application has more processing to do at run time.

The following table lists some sample programs that are shipped with Db2, which demonstrate how to write static and dynamic SQL applications. You can find the source code in data set *prefix.SDSNSAMP*.

Program name	Language	Type of SQL
DSN8BC3	COBOL	Static
DSN8BD3	C	Static
DSN8BP3	PL/I	Static
DSNTEP2	PL/I	Dynamic

How do I prepare an SQL application for execution?

Preparing most SQL applications involves the following tasks:

1. For each source file in the application:
 - Modify a copy of the source file to convert the SQL statements into host language calls to an attachment facility.
 - Create a database request module (DBRM).

The preferred way to accomplish this step is to direct the compiler to call the Db2 coprocessor. However, you can include a separate step that runs the Db2 precompiler before the step that runs the compiler.
2. Compile the modified source files to produce z/OS object code.
3. Link edit the object code for all the object modules to produce a z/OS load module.
4. Bind the DBRM for each source file into a Db2 package, and binding the Db2 packages into a Db2 plan.

Note:

For C or C++ programs that use ODBC, Java programs that use JDBC or SQLJ, and REXX programs, the steps for preparing an SQL application for execution are different. The following table lists those steps.

Type of application	Program preparation steps
ODBC	Compile, prelink, link edit
JDBC	Compile
SQLJ	Translate, compile, customize, bind
pureQuery	Use IBM Optim pureQuery Runtime
REXX	No program preparation

Where do I find more information?

The following tables provide more information about application programming for each of the commonly used languages.

Table 27. Resources for writing Db2 applications in C and C++

Subject	See the following topics
Declaring variables	Declaring host variables and indicator variables in C and C++ (Db2 Application programming and SQL) , Equivalent SQL and C data types (Db2 Application programming and SQL)
Executing SQL statements	C and C++ applications that issue SQL statements (Db2 Application programming and SQL)
Checking for and handling SQL errors	Defining the SQL communications area, SQLSTATE, and SQLCODE in C and C++ (Db2 Application programming and SQL) , Checking the execution of SQL statements (Db2 Application programming and SQL)
Preparing an application to run	Preparing an application to run on Db2 for z/OS (Db2 Application programming and SQL)

Table 28. Resources for writing ODBC applications

Subject	See the following topics
Declaring variables	Data types and data conversion (Db2 Programming for ODBC)
Executing SQL statements	Transaction processing in Db2 ODBC (Db2 Programming for ODBC)
Checking for and handling SQL errors	Diagnostics (Db2 Programming for ODBC)
Preparing an application to run	Overview of preparing and executing a Db2 ODBC application (Db2 Programming for ODBC)

Table 29. Resources for writing Db2 applications in COBOL

Subject	See the following topics
Declaring variables	Declaring host variables and indicator variables in COBOL (Db2 Application programming and SQL) , Equivalent SQL and COBOL data types (Db2 Application programming and SQL)
Executing SQL statements	COBOL applications that issue SQL statements (Db2 Application programming and SQL)
Checking for and handling SQL errors	Defining the SQL communications area, SQLSTATE, and SQLCODE in COBOL (Db2 Application programming and SQL) , Checking the execution of SQL statements (Db2 Application programming and SQL)
Preparing an application to run	Preparing an application to run on Db2 for z/OS (Db2 Application programming and SQL)

Table 30. Resources for writing JDBC applications

Subject	See the following topics
Starting point	JDBC application programming (Db2 Application Programming for Java)
Declaring variables	Variables in JDBC applications (Db2 Application Programming for Java) , Data types that map to database data types in Java applications (Db2 Application Programming for Java)
Executing SQL statements	JDBC interfaces for executing SQL (Db2 Application Programming for Java)
Checking for and handling SQL errors	Exceptions and warnings under the IBM Data Server Driver for JDBC and SQLJ (Db2 Application Programming for Java)
Preparing an application to run	Program preparation for JDBC programs (Db2 Application Programming for Java)

Table 31. Resources for writing SQLJ applications

Subject	See the following topics
Starting point	SQLJ application programming (Db2 Application Programming for Java)
Declaring variables	Variables in SQLJ applications (Db2 Application Programming for Java) , Data types that map to database data types in Java applications (Db2 Application Programming for Java)
Executing SQL statements	SQL statement execution in SQLJ applications (Db2 Application Programming for Java)
Checking for and handling SQL errors	Handling SQL errors and warnings in SQLJ applications (Db2 Application Programming for Java)
Preparing an application to run	Program preparation for SQLJ programs (Db2 Application Programming for Java)

Table 32. Resources for programming with JSON

Subject	See the following topics
Starting point	JSON application development for IBM data servers (Programming with JSON)
JSON installation requirements	Installation requirements for the JSON Java API (Programming with JSON)
Working with JSON documents	Working with JSON documents with the Java API (Programming with JSON)

Table 33. Resources for writing Db2 applications in PL/I

Subject	See the following topics
Declaring variables	Declaring host variables and indicator variables in PL/I (Db2 Application programming and SQL) , Equivalent SQL and PL/I data types (Db2 Application programming and SQL)
Executing SQL statements	PL/I applications that issue SQL statements (Db2 Application programming and SQL)
Checking for and handling SQL errors	Defining the SQL communications area, SQLSTATE, and SQLCODE in PL/I (Db2 Application programming and SQL) , Checking the execution of SQL statements (Db2 Application programming and SQL)
Preparing an application to run	Preparing an application to run on Db2 for z/OS (Db2 Application programming and SQL)

Table 34. Resources for writing Db2 applications in REXX

Subject	See the following topics
Declaring variables	Passing the data type of an input data type to Db2 for REXX programs (Db2 Application programming and SQL) , Equivalent SQL and REXX data types (Db2 Application programming and SQL)
Executing SQL statements	REXX applications that issue SQL statements (Db2 Application programming and SQL)
Checking for and handling SQL errors	Defining the SQL communications area, SQLSTATE, and SQLCODE in REXX (Db2 Application programming and SQL) , Checking the execution of SQL statements (Db2 Application programming and SQL)
Preparing an application to run	No program preparation is necessary.

Other learning resources

For information on training in Db2 application programming, see [Db2 for z/OS training paths](#).

Tools and IDEs for developing Db2 applications

Application developers can use various application development tools and integrated development environments (IDEs) to create database applications, and stored procedures that access and use data in Db2 for z/OS.

The following application development tools and IDEs support the development of Db2 for z/OS database applications and routines.

IBM Db2 for z/OS Developer Extension for Microsoft Visual Studio Code

[IBM Db2 for z/OS Developer Extension for Visual Studio Code \(Db2 Developer Extension\)](#) is a free downloadable extension that provides SQL language support to a modern and widely adopted IDE. Db2 Developer Extension delivers capabilities that help developers to write syntactically correct Db2 for z/OS SQL quickly. For more information, see [Introduction to Db2 for z/OS Developer Extension](#).

Coupled with [IBM Z Open Editor for Visual Studio Code](#), Db2 Developer Extension delivers a comprehensive set of tools for developing applications that run on the IBM zSystems platform. Both IBM Z Open Editor and Db2 Developer Extension are also supported by the [Eclipse Theia IDE](#) platform.

IBM Data Studio

IBM Data Studio is a suite of Eclipse-based tools that are for development database administrators and application developers.

Important: The following tools are replacing **IBM Data Studio** and **IBM Data Server Manager** for use with Db2 for z/OS:

Db2 Administration Foundation provides Db2 for z/OS database administrators who have minimal z/OS or Db2 experience with a modern, browser-based interface for working with Db2 subsystems and their catalogs.

Db2 Developer Extension is a free downloadable extension for Microsoft Visual Studio Code that provides SQL language support to a modern and widely adopted IDE. Db2 Developer Extension introduces capabilities that help application developers to quickly write syntactically correct SQL for Db2 for z/OS.

You can use IBM Data Studio for the following tasks:

- Developing pureQuery applications in a Java project
- Creating, testing, debugging, and deploying routines, such as stored procedures and user-defined functions
- Creating, editing, and running SQL queries
- Connecting to data sources and browsing data objects and their properties
- Creating and altering data objects

IBM Developer for z/OS

IBM Developer for z/OS can improve efficiency and helps with IBM zSystems application development, web development, and integrated mixed workload or composite development. By using IBM Developer for z/OS, you can accelerate the development of your web applications, traditional COBOL and PL/I applications, web services, and XML-based interfaces.

IBM Developer for z/OS provides a common workbench and an integrated set of tools that support end-to-end, model-based application development, run time testing, and rapid deployment of on-Demand applications. With the interactive, workstation-based environment, you can quickly access your z/OS data .

For more information about connecting to Db2 for z/OS and developing SQL applications with IBM Developer for z/OS, see [Developing with Db2 for z/OS](#).

Rational Application Developer for WebSphere Software

IBM Rational® software provides a full range of tools to meet your analysis, design, and construction needs, whether you are an application developer, application architect, systems engineer, or database designer. IBM Rational Application Developer for WebSphere Software helps developers to quickly design, develop, analyze, test, profile, and deploy high-quality web, Service-oriented Architecture (SOA), Java, J2EE, and portal applications.

By using Rational Application Developer, you can increase productivity, minimize your learning curve, and shorten development and test cycles so that you can deploy applications quickly.

WebSphere Application Server Developer Tools for Eclipse

WebSphere Application Server Developer Tools for Eclipse is a fully integrated Java development environment. Using WebSphere Application Server Developer Tools for Eclipse, you can build, compile,

and test J2EE (Java 2 Enterprise Edition) applications for enterprise On Demand business applications with:

- JSP (JavaServer Pages) files
- EJB (Enterprise JavaBeans) components
- 100% Pure Java applets and servlets

For more information see, [IBM WebSphere Application Server Developer Tools for Eclipse](#).

ODBC based development tools

A wide variety of tools are available for performing tasks such as querying a database. These tools include ODBC-based tools such as Lotus Approach, Microsoft Access, Microsoft Visual Basic, Microsoft Excel, and many others. The ODBC-based tools provide a simpler alternative to developing applications than using a high-level programming language. QMF for Workstation provides access to Db2 data for these tools. With all of these tools, you can specify Db2 for z/OS as the database to access.

Related concepts

[Use of development tools to create a stored procedure](#)

Workstation-based development tools can help you create, install, and test stored procedures for Db2 for z/OS.

Programming languages and methods for developing application programs that access Db2 for z/OS

You can use a wide variety of programming languages and techniques to develop application programs for Db2 for z/OS. In addition, several methods are available for communicating with Db2.

You can write application programs for Db2 for z/OS data servers in any of the following programming languages:

- [APL2](#)
- [Assembler](#)
- [C and C++](#)
- [C# “2” on page 249](#) (through [.NET](#))
- [COBOL](#)
- [Fortran](#)
- [High-level Assembler \(or HLASM\)](#)
- [Java “1” on page 249](#)
- [Node.js “3” on page 249](#)
- [Perl “2” on page 249](#)
- [PHP “2” on page 249](#)
- [PL/I](#)
- [Python “4” on page 249](#)
- [REXX](#)
- [Ruby on Rails “2” on page 249](#)
- [SQL procedural language](#)
- [Visual Basic “2” on page 249](#)

1. By connecting to Db2 for z/OS using [IBM Data Server Driver for JDBC and SQLJ](#).
2. By remote connections to Db2 for z/OS using [Db2 for Linux, UNIX, and Windows client drivers](#).
3. By connecting to Db2 for z/OS using the [IBM_DB Node.js driver](#).
4. By connecting to Db2 for z/OS using the [IBM_DB Python driver](#).

You can use any of the following programming methods:

Static SQL

The source form of a static SQL statement is embedded within an application program that is written in a traditional programming language. (Traditional programming languages include C, C++, COBOL, Fortran, PL/I, and Assembler.) Static SQL is a good choice when you know what statements an application needs to execute before the application runs.

Dynamic SQL

Unlike static SQL, dynamic statements are constructed and prepared at run time. Dynamic SQL is a good choice when you do not know the format of an SQL statement when you write a program. It is also a good choice when the program needs to generate part or all of an SQL statement based on input from its users.

ODBC

ODBC is an application programming interface (API) that C and C++ application programs can use to access relational databases. ODBC is well suited to the client/server environment.

pureQuery

pureQuery is a high-performance data access platform for Java applications that makes it easier to develop, optimize, secure, and manage data access.

SQLJ and JDBC

Like ODBC and C++, the SQLJ and JDBC Java interfaces let you write portable application programs that are independent of any one database product.

- SQLJ application support lets you write static SQL applications in the Java programming language. With SQLJ, you can embed SQL statements in your Java applications.
- JDBC application support lets you write dynamic SQL applications in the Java programming language. JDBC is similar to ODBC, but it is designed specifically for use with Java.

Related concepts

[Use of an application program as a stored procedure](#)

A stored procedure is a compiled program that can execute SQL statements.

[Dynamic SQL applications](#)

With dynamic SQL, Db2 prepares and executes the SQL statements within a program while the program is running. Dynamic SQL is a good choice when you do not know the format of an SQL statement before you write or run a program.

[Differences between static and dynamic SQL \(Db2 Application programming and SQL\)](#)

[Use of Java to execute static and dynamic SQL](#)

Db2 for z/OS supports SQLJ and JDBC. In general, Java applications use SQLJ for static SQL, JDBC for dynamic SQL, and PureQuery for both static and dynamic SQL.

[Use of ODBC to execute dynamic SQL](#)

Open Database Connectivity (ODBC) lets you access data through ODBC function calls in your application. The ODBC interface eliminates the need for precompiling and binding your application and increases the portability of your application.

[Introduction to Db2 ODBC \(Db2 Programming for ODBC\)](#)

[Static SQL applications](#)

For most Db2 users, static SQL provides a straightforward, efficient path to Db2 data.

Related tasks

[Planning for and designing Db2 applications \(Db2 Application programming and SQL\)](#)

Performance information for SQL application programming

Efficient applications are an important first step to good system and application performance. As you code applications that access data in Db2, consider performance objectives in your application design.

The following topics can help you understand how application programmers can consider performance as they write applications that access data in Db2 for z/OS.

Concurrency and programming

The goal is to program and prepare applications in a way that:

- Protects the integrity of the data that is being read or updated from being changed by other applications.
- Minimizes the length of time that other access to the data is prevented.

For more information about data concurrency in Db2 and recommendations for improving concurrency in your application programs, see the following topics:

- [Programming for concurrency \(Db2 Performance\)](#)
- [Designing databases for concurrency \(Db2 Performance\)](#)
- [Concurrency and locks \(Db2 Performance\)](#)
- [Improving concurrency \(Db2 Performance\)](#)
- [Improving concurrency in data sharing environments \(Db2 Data Sharing Planning and Administration\)](#)

Writing efficient queries

The predicates, subqueries, and other structures in SQL statements affect the access paths that Db2 uses to access the data.

For information about how to write SQL statements that access data efficiently, see the following topics:

- [“Ways to improve query performance” on page 233](#)
- [Writing efficient SQL queries \(Db2 Performance\)](#)

Analyzing access paths

By analyzing the access path that Db2 uses to access the data for an SQL statement, you can discover potential problems. You can use this information to modify your statement to perform better.

Tip: Query tuning capabilities that can help you with this task, such as *visual explain* and *statistics advisor*, are available in [IBM Db2 Administration Foundation for z/OS](#) and [IBM Db2 for z/OS Developer Extension](#).

For information about how you can use EXPLAIN tables to analyze the access paths for your SQL statements, see the following topics:

- [Investigating access path problems \(Db2 Performance\)](#)
- [00C200A4 \(Db2 Codes\)](#)
- [Investigating SQL performance by using EXPLAIN \(Db2 Performance\)](#)
- [Interpreting data access by using EXPLAIN \(Db2 Performance\)](#)
- [EXPLAIN tables \(Db2 Performance\)](#)
- [EXPLAIN statement \(Db2 SQL\)](#)

Distributed data access performance

The goal is to reduce the amount of network traffic that is required to access the distributed data, and to manage the use of system resources such as distributed database access threads and connections.

For information about improving the performance of applications that access distributed data, see the following topics:

- [“Ways to reduce network traffic” on page 163](#)
- [Managing Db2 threads \(Db2 Performance\)](#)
- [Improving performance for applications that access distributed data \(Db2 Performance\)](#)
- [Improving performance for SQL statements in distributed applications \(Db2 Performance\)](#)

Stored procedures performance

For information about stored procedures and Db2 performance, see the following topics:

- [Implementing Db2 stored procedures \(Stored procedures provided by Db2\)](#)
- [Improving the performance of stored procedures and user-defined functions \(Db2 Performance\)](#)

Related concepts

[SQL: The language of Db2 \(Db2 SQL\)](#)

[Programming for Db2 for z/OS](#)

You use application programs that contain SQL statements to perform data definition or data manipulation operations on Db2 for z/OS objects.

Related tasks

[Programming applications for performance \(Db2 Performance\)](#)

[Planning for and designing Db2 applications \(Db2 Application programming and SQL\)](#)

Preparation process for an application program

How you prepare an application program to run depends on the type of application. The program preparation steps for applications vary based on the type of programming language that is used.

Db2 applications require different methods of program preparation, depending on the type of the application.

Applications that contain embedded static or dynamic SQL statements

Db2 applications embed SQL statements in traditional language programs. To use these programs, you must follow the typical preparation steps (compile, link-edit, and run) plus SQL statement processing and bind steps.

Applications in interpreted languages, such as REXX and APL2

REXX procedures use dynamic SQL. You do any program preparation for Db2 REXX procedures before you run them.

Applications that contain ODBC calls

These applications pass dynamic SQL statements as arguments. You do not process or bind ODBC applications. ODBC applications use a standard set of functions to execute SQL statements and related services at run time.

Java applications, which can contain JDBC calls or embedded SQL statements

Preparing a Java program that contains only JDBC methods is the same as preparing any other Java program. You can prepare the program using the `javac` command. JDBC applications do not require the SQL processing or bind steps.

You can use IBM pureQuery can create and prepare Java applications. IBM pureQuery enables SQL access to databases or Java objects that are in memory and facilitates SQL best practices.

Preparing an SQLJ program requires a precompile step and a bind step.

The following program preparations steps are required by host programming languages:

1. *Process SQL statements* in the program. This step is required to prevent compiler errors because most compilers do not recognize SQL statements. You can use the host language Db2 coprocessor for C, C++, COBOL, and PL/I to process SQL statements as you compile your program. The Db2 coprocessor perform scans the program and returns modified source code at compile time. For more information, see [Processing SQL statements by using the Db2 coprocessor \(Db2 Application programming and SQL\)](#).

Another option is to use the Db2 precompiler before you compile or assemble C, COBOL, Fortran, PL/I, and Assembler language programs. The Db2 precompiler scans the program and returns modified source code, which you can then compile and link-edit. For more information see [Processing SQL statements by using the Db2 precompiler \(Db2 Application programming and SQL\)](#).

Tip: The Db2 coprocessor is the recommended method for processing SQL statements in application programs. Compared to the Db2 precompiler, the Db2 coprocessor has fewer restrictions on SQL

programs, and more fully supports the latest SQL and programming language enhancements. See [Processing SQL statements by using the Db2 coprocessor \(Db2 Application programming and SQL\)](#).

The main output from the Db2 coprocessor or Db2 precompiler is a *database request module (DBRM)*. A DBRM is a data set that contains SQL statements and host variable information that is extracted from the source program during program preparation. The purpose of a DBRM is to communicate your SQL requests to Db2 during the bind process. For more information, see [Output from the Db2 coprocessor \(Db2 Application programming and SQL\)](#).

2. *Bind* the application program. Before your Db2 application can run, you must bind the DBRM to a package. You might decide to put certain SQL statements together in the same program to precompile them into the same DBRM and then bind them into a single package. When the program runs, Db2 uses a timestamp to verify that the program matches the correct plan or package.

A *collection* is a group of associated packages. By binding packages into package collections you can add packages to an existing application plan without needing to bind the entire plan again. If you include a collection name in the package list when you bind a plan, any package that is in the collection becomes available to the plan. The collection can even be empty when you first bind the plan. Later, you can add packages to the collection and drop or replace existing packages without binding the plan again.

The CURRENT PACKAGE PATH special register specifies a value that identifies a list of collections that Db2 uses when resolving references to packages that you use to run SQL statements.

For more information about the bind process, see [Binding application packages and plans \(Db2 Application programming and SQL\)](#).

3. *Compile* and *link-edit* the program. To enable your application to interface with the Db2 subsystem, you must use a link-edit procedure to build an executable load module that satisfies the requirements of your environment (such as CICS, IMS, TSO, or batch). The *load module* is a program unit that is loaded into main storage for execution.
4. *Run* the program: After you complete the preceding steps, you can run your Db2 application. You can use the following methods to prepare an application to run:
 - Use Db2 Interactive (DB2I) panels, which lead you step by step from preparing the program to running the program.
 - Submit an application in the TSO foreground or in batch in the TSO background.
 - Start the program preparation command list (CLIST) in TSO foreground or batch.
 - Use the DSN command processor.
 - Use JCL procedures that you include in your data sets (such as SYS1.PROCLIB) at Db2 installation time.

You can also precompile and prepare an application program by using a Db2-supplied procedure. Db2 has a unique procedure for each supported language.

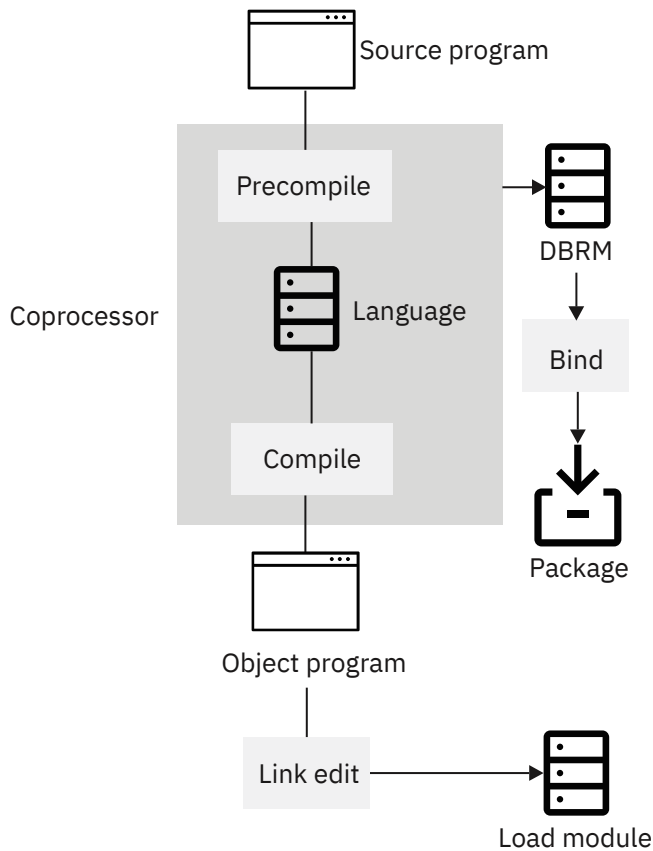


Figure 45. Overview of the program preparation process for applications that contain embedded SQL. The Db2 coprocessor can combine the precompile and compile steps for certain languages.

Db2 Bind Manager tool

The Db2 Bind Manager tool helps application programmers:

- Predict whether a bind of a DBRM results in a changed access path
- Run access path checks on a batch of DBRMs
- Eliminate unnecessary bind steps between application programs and the database
- Compare DBRMs to subsystems and load modules

Db2 Path Checker tool

The Db2 Path Checker helps you increase the stability of your Db2 environments and avoid painful and costly disruptions. The Db2 Path Checker can help you discover and correct unwanted and unexpected access path changes before you are notified about them.

Related tasks

[Preparing an application to run on Db2 for z/OS \(Db2 Application programming and SQL\)](#)

Application programming default values

Each Db2 for z/OS subsystem has a set of *application programming default values*. These values are the default settings for SQL applications that run on that subsystem. For example, the ENSHEME application programming default value specifies the default encoding scheme for new objects. Some users colloquially refer to these values as *DECP values*.

System administrators generally set the application programming default values when they install or migrate Db2. After installation or migration, a system administrator should change these values only if

necessary and according to the update advice that is documented for each application programming value. Changing an application programming default value requires Db2 to be stopped and restarted.

For a list of application programming defaults, see [Directory of subsystem parameters, panel fields, and application default values \(Db2 Installation and Migration\)](#).

Application programming default values are contained in the DSNHDECP load module that is generated by installation job DSNTIJUZ. In a data sharing environment, each member is not required to have a separate application defaults load module. The DSNHDECP module is also required by the Db2 precompiler and coprocessor for C, C++, COBOL, and PL/I.

One way to check the application programming default values is to call stored procedure SYSPROC.ADMIN_INFO_SYSPARM by running sample job DSNTJ6Z. For certain application programming defaults, you can also check the value by invoking the [GETVARIABLE](#) function.

Related concepts

[Subsystem parameters](#)

Subsystem parameters are settings that apply to a Db2 for z/OS subsystem. For example, the AUTH subsystem parameter controls whether Db2 checks authorizations. Many users colloquially refer to subsystem parameters as *zparms*.

[Subsystem parameters and application defaults in a data sharing environment \(Db2 Installation and Migration\)](#)

Related tasks

[Generating tailored Db2 12 installation, migration, or function level activation jobs \(Db2 Installation and Migration\)](#)

Related reference

[Directory of subsystem parameters, panel fields, and application default values \(Db2 Installation and Migration\)](#)

[DSNTIPF: Application programming defaults panel 1 \(Db2 Installation and Migration\)](#)

Static SQL applications

For most Db2 users, static SQL provides a straightforward, efficient path to Db2 data.

The source form of a static SQL statement is embedded within an application program that is written in a traditional programming language such as C. The statement is prepared before the program is executed, and the operational form of the statement persists beyond the execution of the program. You can use static SQL when you know before run time what SQL statements your application needs to run.

When you use static SQL, you cannot change the form of SQL statements unless you make changes to the program. However, you can increase the flexibility of those statements by using host variables. Using static SQL and host variables is more secure than using dynamic SQL.

Example

GUPI

For example, assume that you are coding static SQL in a COBOL program. The following UPDATE statement can update the salary of any employee. When you write your program, you know that salaries must be updated, but you do not know until run time whose salaries should be updated, and by how much.

```
01 IOAREA.
   02 EMPID          PIC X(06).
   02 NEW-SALARY     PIC S9(7)V9(2) COMP-3.
: (Other declarations)
READ CARDIN RECORD INTO IOAREA
  AT END MOVE 'N' TO INPUT-SWITCH.
: (Other COBOL statements)
EXEC SQL
  UPDATE EMP
  SET SALARY = :NEW-SALARY
```

```
WHERE EMPNO = :EMPID  
END-EXEC.
```

GUIP

The UPDATE statement does not change, nor does its basic structure, but the input can change the results of the UPDATE statement.

Basic SQL coding concepts apply to traditional programming languages: C, C++, COBOL, Fortran, PL/I, and Assembler.

Suppose that you are writing an application program to access data in a Db2 database. When your program executes an SQL statement, the program needs to communicate with Db2. When Db2 finishes processing an SQL statement, Db2 sends back a return code, called the SQL *return code*. Your program should test the return code to examine the results of the operation.

Unique instructions and details apply to each language.

Related concepts

[Submitting SQL statements to Db2](#)

You can use several different methods to submit SQL statements to Db2 interactively or through application programs.

[Static SQL statements](#)

The source form of a *static SQL statement* is embedded within an application program written in a host language such as COBOL. The statement is prepared before the program is executed and the operational form of the statement persists beyond the execution of the program.

Declaration of table and view definitions

Declaring table or view definitions is optional, but they offer several advantages. You can declare a table or view by including an SQL DECLARE statement in your program.

Before your program issues SQL statements that retrieve, update, delete, or insert data, you must declare the tables and views that your program accesses. Declaring tables or views is not required; however, declaring them offers advantages such as documenting application programs and providing the precompiler with information that is used to check your embedded SQL statements.

Example

GUIP

The DECLARE TABLE statement (written in COBOL) for the DEPT table looks like the following example:

```
EXEC SQL  
  DECLARE DEPT TABLE  
    (DEPTNO CHAR(3) NOT NULL,  
     DEPTNAME VARCHAR(36) NOT NULL,  
     MGRNO CHAR(6) ,  
     ADMRDEPT CHAR(3) NOT NULL )  
END-EXEC.
```

GUIP

For each traditional language, you delimit an SQL statement in your program between EXEC SQL and a statement terminator. In the preceding example, the EXEC SQL and END-EXEC delimit the SQL statement in a COBOL program.

As an alternative to coding the DECLARE statement yourself, you can use the Db2 subcomponent DCLGEN, the declarations generator.

Related reference

[DECLARE STATEMENT statement \(Db2 SQL\)](#)

Data access with host variables

You can use host variables, host-variable arrays, and host structures in your application program to exchange data between the application and the DBMS.

A *host variable* is a data item that you declare in a program for use within an SQL statement. You can:

- Retrieve data into the host variable for your application program's use.
- Place data into the host variable to insert into a table or to change the contents of a row.
- Use the data in the host variable when evaluating a WHERE or HAVING clause.
- Assign the value in the host variable to a special register. A *special register* is a storage area that Db2 defines for a process to hold information that SQL statements can reference.

For example, the CURRENT SQLID special register contains the SQL authorization ID of a process, which is set in an SQL statement. Db2 replaces the register name with the value of the authorization ID when the SQL statement runs.

- Use the host variable to indicate a null value

How you code a host variable varies according to the programming language that you use. Some languages require a separate declaration section for SQL variables. In this case, you can code the BEGIN and END DECLARE SECTION statements in an application program wherever variable declarations can appear according to the rules of the host language. A host variable declaration section starts with the BEGIN DECLARE SECTION statement and ends with the END DECLARE SECTION statement. The host variable must be preceded with a *:hostvar*

The INTO clause of the SELECT statement names one or more host variables to contain the returned column values. For host variables and host-variable arrays, the named variables correspond one-to-one with the list of column names in the SELECT list.

Example

The example that follows uses a host variable to retrieve a single row of data.

GUIP Suppose that you want to retrieve the EMPNO, LASTNAME, and DEPT column values from a single row in the EMP table. You can define a host variable in your program to hold each column. The host variable consists of the local variable name, preceded by a colon. You then can name the data areas with an INTO clause, as shown:

```
EXEC SQL
  SELECT EMPNO, LASTNAME, DEPT
  INTO :CBLEMPNO, :CBLNAME, :CBLDEPT
  FROM EMP
  WHERE EMPNO = :EMPID
END-EXEC.
```

GUIP

You must declare the host variables CBLEMPNO, CBLNAME, and CBLDEPT in the data declaration portion of the program. The data types of the host variables must be compatible with the SQL data types of the columns EMPNO, LASTNAME, and DEPT of the EMP table.

Suppose that you don't know how many rows Db2 will return, or you expect more than one row to return. In either case, you must use an alternative to the SELECT ... INTO statement. Using a Db2 cursor, an application can process a set of rows and retrieve rows from the result table.

Related concepts

Row retrieval with a cursor

Db2 has a mechanism called a *cursor*. Using a cursor is like keeping your finger on a particular line of text on a printed page.

[Host variables \(Db2 Application programming and SQL\)](#)

[Host variables \(Db2 SQL\)](#)

Data access with host-variable arrays

A *host-variable array* is a data array that is declared in a host language for use within an SQL statement. You can retrieve data into host-variable arrays for use by your application program. You can also place data into host-variable arrays to insert rows into a table.

You can specify host-variable arrays in C, C++, COBOL, or PL/I. Each host-variable array contains values for a column, and each element of the array corresponds to a value for a column. You must declare the array in the host program before you use it.

Host-variable arrays can be referenced only as a simple reference in the following contexts. In syntax diagrams, *host-variable-array* designates a reference to a host-variable array.

- In a FETCH statement for a multiple-row fetch. See [FETCH statement \(Db2 SQL\)](#).
- In the FOR *n* ROWS form of the INSERT statement with a host-variable array for the source data. See [INSERT statement \(Db2 SQL\)](#).
- In a MERGE statement with multiple rows of source data. See [MERGE statement \(Db2 SQL\)](#).
- In an EXECUTE statement to provide a value for a parameter marker in a dynamic FOR *n* ROWS form of the INSERT statement or a MERGE statement. See [EXECUTE statement \(Db2 SQL\)](#).

If an application references a host-variable array in other contexts that are not supported, Db2 returns the SQLCODE -312 error.

Host-variable arrays are defined by statements of the host language, as explained in the following topics:

- [Host-variable arrays in C and C++ \(Db2 Application programming and SQL\)](#)
- [Host-variable arrays in COBOL \(Db2 Application programming and SQL\)](#)
- [Host-variable arrays in PL/I \(Db2 Application programming and SQL\)](#)

Tip: Host-variable arrays are not supported for assembler, FORTRAN, or REXX programs. However, you can use SQL descriptor areas (SQLDA) to achieve similar results in any host language. For more information see [Defining SQL descriptor areas \(SQLDA\) \(Db2 Application programming and SQL\)](#).

Example

GUPI

The following statement uses the main host-variable array, COL1, and the corresponding indicator array, COL1IND. Assume that COL1 has 10 elements. The first element in the array corresponds to the first value, and so on. COL1IND must have at least 10 entries.

```
EXEC SQL
  SQL FETCH FIRST ROWSET FROM C1 FOR 5 ROWS
    INTO :COL1 :COL1IND
END-EXEC.
```

GUPI

Related concepts

[Host-variable arrays \(Db2 SQL\)](#)

[Host variables \(Db2 Application programming and SQL\)](#)

Data access with host structures

A *host structure* is a group of host variables that an SQL statement can refer to by using a single name. When the host language environment allows it, you can use host language statements to define the host structures.

GUPI

Assume that your COBOL program includes the following SQL statement:

```
EXEC SQL
  SELECT EMPNO, FIRSTNME, LASTNAME, DEPT
  INTO :EMPNO, :FIRSTNME, :LASTNAME, :WORKDEPT
  FROM VEMP
  WHERE EMPNO = :EMPID
END-EXEC.
```

Now assume that you want to avoid listing the host variables in the preceding example.

You can substitute the name of a structure, such as :PEMP, that contains :EMPNO, :FIRSTNME, :LASTNAME, and :DEPT:

```
EXEC SQL
  SELECT EMPNO, FIRSTNME, LASTNAME, WORKDEPT
  INTO :PEMP
  FROM VEMP
  WHERE EMPNO = :EMPID
END-EXEC.
```

GUPI

You can declare a host structure in your program. You can also use DCLGEN to generate a COBOL record description, PL/I structure declaration, or C structure declaration that corresponds to the columns of a table.

Related concepts

[Host variables \(Db2 Application programming and SQL\)](#)

Row retrieval with a cursor

Db2 has a mechanism called a *cursor*. Using a cursor is like keeping your finger on a particular line of text on a printed page.

In Db2, an application program uses a cursor to point to one or more rows in a set of rows that are retrieved from a table. You can also use a cursor to retrieve rows from a result set that is returned by a stored procedure. Your application program can use a cursor to retrieve rows from a table.

You can retrieve and process a set of rows that satisfy the search condition of an SQL statement. When you use a program to select the rows, the program processes one or more rows at a time.

The SELECT statement must be within a DECLARE CURSOR statement and cannot include an INTO clause. The DECLARE CURSOR statement defines and names the cursor, identifying the set of rows to retrieve with the SELECT statement of the cursor. This set of rows is referred to as the result table.

After the DECLARE CURSOR statement executes, you process the result table of a cursor as follows:

1. Open the cursor before you retrieve any rows.

To tell Db2 that you are ready to process the first row of the result table, have your program issue the OPEN statement. Db2 then uses the SELECT statement within the DECLARE CURSOR statement to identify a set of rows. If you use host variables in that SELECT statement, Db2 uses the *current value* of the variables to select the rows.

2. Use a FETCH statement to retrieve one or more rows.

The simplest form of the FETCH statement retrieves a single row of the result table by using a *row-positioned* cursor. At any point in time, a row-positioned cursor retrieves at most a single row from the result table into host variables. You can use a FETCH statement to retrieve more than one row of the result table by using a cursor that is enabled to process rowsets. A *rowset* is a set of rows that is retrieved through a multiple-row fetch.

When your program issues a row-positioned FETCH statement, Db2 uses the cursor to point to a row in the result table, making it the current row. Db2 then moves the current row contents into the program host variables that you specified in the INTO clause of the FETCH statement. The FETCH statement

moves the cursor. You can use host-variable arrays and return multiple rows of data with a single FETCH statement.

3. Close the cursor when the end-of-data condition occurs.

If you finish processing the rows of the result table and you want to use the cursor again, issue a CLOSE statement to close the cursor.

Recommendation: Explicitly close the cursor when you finish using it.

Your program can have several cursors. Each cursor has the following requirements:

- DECLARE CURSOR statement to define the cursor
- OPEN and CLOSE statements to open and close the cursor
- FETCH statement to retrieve rows from the result table of the cursor

You must declare host variables before you refer to them in a DECLARE CURSOR statement. To define and identify a set of rows that are to be accessed with a cursor, issue a DECLARE CURSOR statement. The DECLARE CURSOR statement names a cursor and specifies a SELECT statement. The SELECT statement defines the criteria for the rows that belong in the result table.

You can use cursors to fetch, update, or delete one or more rows of a table, but you cannot use them to insert a row into a table.

Examples

Suppose that your program examines data about people in department D11 and keeps the data in the EMP table. The following examples show the SQL statements that you must include in a COBOL program to define and use a cursor. In these examples, the program uses the cursor to process a set of rows from the EMP table.

GUPI

Example: define the cursor

The following statement defines a cursor named THISEMP:

```
EXEC SQL
  DECLARE THISEMP CURSOR FOR
    SELECT EMPNO, LASTNAME,
           DEPT, JOB
    FROM EMP
    WHERE DEPT = 'D11'
    FOR UPDATE OF JOB
END-EXEC.
```

Example: open the cursor

The following statement opens the cursor:

```
EXEC SQL
  OPEN THISEMP
END-EXEC.
```

Example: use the cursor to retrieve a row

The following statement uses the cursor, THISEMP, to retrieve a row:

```
EXEC SQL
  FETCH THISEMP
  INTO :EMP-NUM, :NAME2,
       :DEPT, :JOB-NAME
END-EXEC.
```

Example: Update the current row using the cursor

The following statement uses the cursor, THISEMP, to update the JOB value for specific employees in department D11:

```
EXEC SQL
  UPDATE EMP
  SET JOB = :NEW-JOB
```

```
WHERE CURRENT OF THISEMP  
END-EXEC.
```

Example close the cursor

The following statement closes the cursor:

```
EXEC SQL  
CLOSE THISEMP  
END-EXEC.
```

More about cursors

GUPI

If the cursor is not scrollable, each fetch positions the cursor at the next sequential row, or set of rows. A *scrollable cursor* can scroll forward and backward, and can be repositioned at the beginning, at the end, or at a relative offset point. Applications can use a powerful set of SQL statements to fetch data by using a cursor in random order. Scrollable cursors are especially useful for screen-based applications. You can specify that the data in the result table is to remain static. For example, an accounting application can require that data is to remain constant, whereas an airline reservation system application must display the latest flight availability information.

You can also define options on the DECLARE CURSOR statement that specify how sensitive a scrollable cursor is to changes in the underlying data when inserts, updates, or deletes occur.

- A *sensitive cursor* is sensitive to changes that are made to the database after the result table is generated. For example, when an application executes positioned UPDATE and DELETE statements with the cursor, those changes are visible in the result table.
- An *insensitive cursor* is not sensitive to inserts, updates, or deletes that are made to the underlying rows of a result table after the result table is generated. For example, the order of the rows and the values for each row of the result table do not change after the application opens the cursor.

To indicate that a cursor is scrollable, you declare it with the SCROLL keyword.

GUPI For example, the following example shows a declaration for an insensitive scrollable cursor:

```
EXEC SQL DECLARE C1 INSENSITIVE SCROLL CURSOR FOR  
SELECT DEPTNO, DEPTNAME, MGRNO  
FROM DEPT  
ORDER BY DEPTNO  
END-EXEC.
```

To use this cursor to fetch the fifth row of the result table, you can use a FETCH statement like the following example:

```
EXEC SQL FETCH ABSOLUTE +5 C1 INTO :HVDEPTNO, :DEPTNAME, :MGRNO;
```

GUPI

Db2 for z/OS provides another type of cursor called a *dynamic scrollable cursor*. With a dynamic scrollable cursor, applications can scroll directly on a base table while accessing the most current data.

Related reference

[DECLARE CURSOR statement \(Db2 SQL\)](#)

[FETCH statement \(Db2 SQL\)](#)

Ways to check the execution of SQL statements

Db2 offers several ways to check the execution of SQL statements in an program.

A program that includes SQL statements can have an area that is set apart for communication with Db2—an *SQL communication area (SQLCA)*. When Db2 processes an SQL statement in your program, it places return codes in the SQLSTATE and SQLCODE host variables or in corresponding fields of the SQLCA. The return codes indicate whether the statement executed successfully or failed.

Recommendation: Because the SQLCA is a valuable problem-diagnosis tool, include the necessary instructions to display some of the information that is in the SQLCA in your application programs.

You can use a GET DIAGNOSTICS statement or a WHENEVER statement in your program to supplement checking SQLCA fields after each SQL statement runs.

- The GET DIAGNOSTICS statement returns diagnostic information about the last SQL statement that was executed. You can request specific types of diagnostic information or all available diagnostic information about a statement. For example, the GET DIAGNOSTICS statement returns the number of rows that are affected by a data insert, update, or delete.
- The WHENEVER statement allows you to specify what to do if a general condition is true. Db2 checks the SQLCA and continues processing your program. If an error, exception, or warning results when an SQL statement is executed, Db2 branches to another area in your program. The program can then examine the SQLSTATE or SQLCODE to react specifically to the error or exception.

Related reference

[GET DIAGNOSTICS statement \(Db2 SQL\)](#)

[WHENEVER statement \(Db2 SQL\)](#)

Dynamic SQL applications

With dynamic SQL, Db2 prepares and executes the SQL statements within a program while the program is running. Dynamic SQL is a good choice when you do not know the format of an SQL statement before you write or run a program.

An application that uses dynamic SQL generates an SQL statement in the form of a character string or accepts an SQL statement as input. Depending on the needs of the application, you might be able to simplify the programming. Try to plan the application so that it does not use SELECT statements, or so that it uses only those statements that return a known number of values of known data types. In general, more complex dynamic programs are those in which you do not know in advance about the SQL statements that the application issues.

A typical dynamic SQL application takes the following steps:

1. Translates the input data into an SQL statement.
2. Prepares the SQL statement to execute and acquires a description of the result table (if any).
3. Obtains, for SELECT statements, enough main storage to contain retrieved data.
4. Executes the statement or fetches the rows of data.
5. Processes the returned information.
6. Handles SQL return codes.

GUI This example shows a portion of a C program that dynamically issues SQL statements to Db2. Assume that you are writing a program to keep an inventory of books. The table that you need to update depends on input to your program. This example shows how you can build an SQL statement and then call Db2 to execute it.

```
/* *****  
/* Determine which table to update, then build SQL          */  
/* statement dynamically into 'stmt' variable.             */  
/* *****  
strcpy(stmt, "UPDATE ");  
  
EXEC SQL SELECT TYPE INTO :book_type FROM BOOK_TYPES WHERE  
TITLE=:bktitle;  
  
IF (book_type=='FICTION') strcpy(table_name, "FICTION_BOOKS");  
ELSE strcpy(table_name, "NON_FICTION_BOOKS");  
  
strcat(stmt, table_name);  
strcat(stmt,  
" SET INVENTORY = INVENTORY-1 WHERE TITLE = :bktitle");  
/* *****  
/* PREPARE and EXECUTE the statement                      */  
/* *****
```

```
EXEC SQL PREPARE OBJSTMT FROM :stmt;  
EXEC SQL EXECUTE OBJSTMT;
```

GUIP

Related concepts

[Submitting SQL statements to Db2](#)

You can use several different methods to submit SQL statements to Db2 interactively or through application programs.

[Differences between static and dynamic SQL \(Db2 Application programming and SQL\)](#)

[Dynamic SQL plan stability \(Db2 Performance\)](#)

[Use of Java to execute static and dynamic SQL](#)

Db2 for z/OS supports SQLJ and JDBC. In general, Java applications use SQLJ for static SQL, JDBC for dynamic SQL, and PureQuery for both static and dynamic SQL.

[How authorization IDs control data access](#)

One of the ways that Db2 controls access to data is through the use of identifiers. A set of one or more Db2 identifiers, called *authorization IDs*, represents every process that connects to or signs on to Db2.

[Differences between Db2 ODBC and embedded SQL \(Db2 Programming for ODBC\)](#)

Related tasks

[Including dynamic SQL in your program \(Db2 Application programming and SQL\)](#)

[Improving dynamic SQL performance \(Db2 Performance\)](#)

Use of ODBC to execute dynamic SQL

Open Database Connectivity (ODBC) lets you access data through ODBC function calls in your application. The ODBC interface eliminates the need for precompiling and binding your application and increases the portability of your application.

The ODBC interface is specifically designed for C and C++ applications to access relational databases. Applications that use the ODBC interface might be executed on a variety of data sources without being compiled against each of the databases. ODBC ideally suits the client/server environment in which the target data source might be unknown when the application is built.

You execute SQL statements by passing them to Db2 through an ODBC function call. The function calls allow an application to connect to the data source, issue SQL statements, and receive returned data and status information.

You can prepare an SQL statement by calling the ODBC SQLPrepare() function. You then execute the statement by calling the ODBC SQLExecute() function. In both cases, the application does not contain an embedded PREPARE or EXECUTE statement. You can execute the statement, without preparation, by passing the statement to the ODBC SQLExecDirect() function.

Another advantage of ODBC access is that it can help hide the differences between system catalogs of different database servers. Unlike embedded SQL, Db2 ODBC provides a consistent interface for applications to query and retrieve system catalog information across the Db2 Database family of database management systems. This capability reduces the need to write catalog queries that are specific to each database server. Db2 ODBC can return result tables to those programs.

Examples

GUIP

This example shows a portion of an ODBC program for keeping an inventory of books.

```
/* *****  
/* Determine which table to update  
/* *****  
rc = SQLBindParameter( hStmt,  
                        1,  
                        SQL_PARAM_INPUT,  
                        SQL_C_CHAR,  
                        SQL_CHAR,
```

```

        50,
        0,
        bktitle,
        sizeof(bktitle),
        &bktitle_len);
if( rc != SQL_SUCCESS ) goto dberror;

rc = SQLExecDirect( hStmt,
                    "SELECT TYPE FROM BOOK_TYPES WHERE TITLE=?"
                    SQL_NTS );
if( rc != SQL_SUCCESS ) goto dberror;

rc = SQLBindCol( hStmt,
                 1,
                 SQL_C_CHAR,
                 book_type,
                 sizeof(book_type),
                 &book_type_len);
if( rc != SQL_SUCCESS ) goto dberror;

rc = SQLFetch( hStmt );
if( rc != SQL_SUCCESS ) goto dberror;

rc = SQLCloseCursor( hStmt );
if( rc != SQL_SUCCESS ) goto dberror;
/***** Update table *****/
strcpy( (char *)update_sqlstmt, (char *)"UPDATE ");
if( strcmp( (char *)book_type, (char *)"FICTION") == 0)
{
    strcat( (char *)update_sqlstmt, (char *)"FICTION_BOOKS" );
}
else
{
    strcpy( (char *)update_sqlstmt, (char *)"NON_FICTION_BOOKS" );
}
strcat( (char *)update_sqlstmt,
        (char *)" SET INVENTORY = INVENTORY-1 WHERE TITLE = ?");

rc = SQLPrepare( hStmt, update_sqlstmt, SQL_NTS );
if( rc != SQL_SUCCESS ) goto dberror;

rc = SQLExecute( hStmt );
if( rc != SQL_SUCCESS ) goto dberror;

rc = SQLEndTran( SQL_HANDLE_DBC, hDbc, SQL_COMMIT );
if( rc != SQL_SUCCESS ) goto dberror;

```



Related concepts

Dynamic SQL applications

With dynamic SQL, Db2 prepares and executes the SQL statements within a program while the program is running. Dynamic SQL is a good choice when you do not know the format of an SQL statement before you write or run a program.

Related tasks

[Enabling dynamic SQL statement caching for ODBC function calls \(Db2 Programming for ODBC\)](#)

Use of Java to execute static and dynamic SQL

Db2 for z/OS supports SQLJ and JDBC. In general, Java applications use SQLJ for static SQL, JDBC for dynamic SQL, and PureQuery for both static and dynamic SQL.

By using the Java programming language you gain the following key advantages:

- You can write an application on any Java-enabled platform and run it on any platform to which the Java Development Kit (JDK) is ported.
- You can develop an application once and run it anywhere, which offers the following potential benefits:
 - Reduced development costs
 - Reduced maintenance costs

- Reduced systems managements costs
- Flexibility in supporting diverse hardware and software configurations

The following table shows some of the major differences between SQLJ and JDBC.

Table 35. Comparison of SQLJ and JDBC

SQLJ characteristics	JDBC characteristics
SQLJ follows the static SQL model and offers performance advantages over JDBC.	JDBC follows the dynamic SQL model.
SQLJ source programs are smaller than equivalent JDBC programs because SQLJ automatically generates certain code that developers must include in JDBC programs.	JDBC source programs are larger than equivalent SQLJ programs because certain code that the developer must include in JDBC programs is generated automatically by SQLJ.
SQLJ checks data-types during the program preparation process and enforces strong typing between table columns and Java host expressions.	JDBC passes values to and from SQL tables without checking data types at compile time.
In SQLJ programs, you can embed Java host expressions in SQL statements.	JDBC requires a separate statement for each bind variable and specifies the binding by position number.
SQLJ provides the advantages of static SQL authorization checking. With SQLJ, the authorization ID under which SQL statements run is the plan or package owner. Db2 checks the table privileges at bind time.	Because JDBC uses dynamic SQL, the authorization ID under which SQL statements run is not known until run time, so no authorization checking of table privileges can occur until run time.

SQLJ support

Db2 for z/OS includes SQLJ, which provides support for embedding static SQL statements in Java applications and servlets. *Servlets* are application programs that are written in Java and that run on a web server.

Because SQLJ coexists with JDBC, an application program can create a JDBC connection and then use that connection to run dynamic SQL statements through JDBC and embedded static SQL statements through SQLJ.

A group of companies that includes Oracle, Hewlett Packard, and IBM, initially developed SQLJ to complement the dynamic SQL JDBC model with a static SQL model.

The SQLJ coding to update the salary of any employee is as follows:

GUIP

```
#sql [myConnCtxt] { UPDATE EMP
                        SET SALARY = :newSalary
                        WHERE EMPNO = :empID };
```

GUIP

By using SQLJ you gain the following advantages:

- Portable applications across platforms and database management systems.
- Strong typing, with compile and bind-time checking to ensure that applications are well designed for the database.
- Superior performance, manageability, and authorization checking of static SQL.
- Improved programmer productivity and easier maintenance. In comparison to a JDBC application, the resulting program is typically shorter and easier to understand.
- Familiarity for programmers who use embedded SQL in other traditional programming languages.

Related concepts

Db2 database design

After building a logical design and physical design of your relational database and collecting the processing requirements, you can move to the implementation stage. In general, implementing your physical design involves defining the various objects and enforcing the constraints on the data relationships.

JDBC support

Db2 for z/OS supports applications that use Sun Microsystems JDBC interfaces to access Db2 data by using dynamic SQL. Support for JDBC enables organizations to write Java applications that access local Db2 data, and access remote relational data on a server that supports DRDA.

Sun Microsystems developed the JDBC specifications. The JDBC specifications define a set of APIs (based on ODBC) that allow Java applications to access relational data. The APIs provide a generic interface for writing applications that run on multiple platforms and can access any SQL database. The APIs are defined within 16 classes, and they support basic SQL functions for connecting to a database, running SQL statements, and processing results. Together, these interfaces and classes represent the JDBC capabilities by which a Java application can access relational data.

GUIP

This example shows a portion of a JDBC program for that keeps an inventory of books.

```
/* *****  
/* Determine which table to update, then build SQL      */  
/* statement dynamically.                                */  
/* *****  
String tableName = null;  
Statement stmt = con.createStatement();  
  
ResultSet rs = stmt.executeQuery("SELECT TYPE FROM " +  
" BOOK_TYPES WHERE " +  
" TITLE = \"" + bkTitle + "\"");  
if (rs.next())  
{  
    if (rs.getString(1).equalsIgnoreCase("FICTION"))  
        tableName = "FICTION_BOOKS";  
    else  
        tableName = "NON_FICTION_BOOKS";  
/* *****  
/* PREPARE and EXECUTE the statement                    */  
/* *****  
stmt.executeUpdate("UPDATE " + tableName + " SET INVENTORY = INVENTORY-1 " +  
"WHERE TITLE = \"" + bkTitle + "\"");  
}  
rs.close();  
stmt.close();
```

GUIP

Db2 for z/OS support for JDBC offers a number of advantages for accessing Db2 data:

- JDBC combines the benefit of running your applications in a z/OS environment with the portability and ease of writing Java applications.
- The JDBC interface offers the ability to change between drivers and access various databases without recoding your Java program.
- JDBC applications do not require precompiles or binds.
- JDBC provides a consistent interface for applications to query and retrieve system catalog information across the Db2 Database family of database management systems. This capability reduces the need to write catalog queries that are specific to each database server.

Related concepts

Submitting SQL statements to Db2

You can use several different methods to submit SQL statements to Db2 interactively or through application programs.

Dynamic SQL applications

With dynamic SQL, Db2 prepares and executes the SQL statements within a program while the program is running. Dynamic SQL is a good choice when you do not know the format of an SQL statement before you write or run a program.

Use of ODBC to execute dynamic SQL

Open Database Connectivity (ODBC) lets you access data through ODBC function calls in your application. The ODBC interface eliminates the need for precompiling and binding your application and increases the portability of your application.

Related tasks

Including dynamic SQL in your program (Db2 Application programming and SQL)

Use of an application program as a stored procedure

A stored procedure is a compiled program that can execute SQL statements.

Stored procedures are stored at the Db2 local or remote server where they run. A typical stored procedure contains two or more SQL statements and some manipulative or logical processing in a program. A client application program uses the SQL CALL statement to invoke the stored procedure.

Consider using stored procedures for a client/server application that does at least one of the following things:

- Executes multiple remote SQL statements.

Remote SQL statements can result in several send and receive operations across the network, which increases processor costs and elapsed times.

Stored procedures can encapsulate many SQL statements into a single message to the Db2 server. The network traffic of stored procedures is a single send and receive operation for a series of SQL statements.

Locks on Db2 tables are not held across network transmissions, which reduces contention for resources at the server.

- Accesses tables from a dynamic SQL environment in which table privileges for the application that is running are undesirable.

Stored procedures allow static SQL authorization from a dynamic environment.

- Accesses host variables for which you want to check security and integrity.

Stored procedures remove SQL applications from the workstation, preventing workstation users from manipulating the contents of sensitive SQL statements and host variables.

- Creates a result set of rows to return to the client application.

Related concepts

Stored procedures (Db2 Application programming and SQL)

Related tasks

Implementing Db2 stored procedures (Db2 Administration Guide)

Languages used to create stored procedures

Stored procedures can be written in a variety of programming languages from object-oriented programming languages to traditional programming languages.

You can write stored procedures in the following programming languages:

Java

If you have more experience writing applications in an object-oriented programming environment, you might want to create stored procedures by using Java

SQL procedural language

If your application consists entirely of SQL statements, some simple control flow logic, and no complex application logic, you might choose to create your stored procedures by using the SQL procedural language.

REXX

You can create stored procedures by using REXX programs that can contain dynamic SQL. DBAs and programmers generally use REXX for administrative tasks.

Traditional programming languages: C, C++, COBOL, PL/I, and Assembler

All traditional language programs must be designed to run using Language Environment®. COBOL and C++ stored procedures can contain object-oriented extensions.

The program that calls the stored procedure can be in any language that supports the SQL CALL statement. ODBC and JDBC applications can use an escape clause to pass a stored procedure call to Db2.

Related concepts

Use of Java to execute static and dynamic SQL

Db2 for z/OS supports SQLJ and JDBC. In general, Java applications use SQLJ for static SQL, JDBC for dynamic SQL, and PureQuery for both static and dynamic SQL.

Use of the SQL procedural language to create a stored procedure

With SQL procedural language, you can write stored procedures that consist entirely of SQL statements.

Stored procedure processing

There are several steps to stored procedure processing.

The following figure illustrates processing without stored procedures.

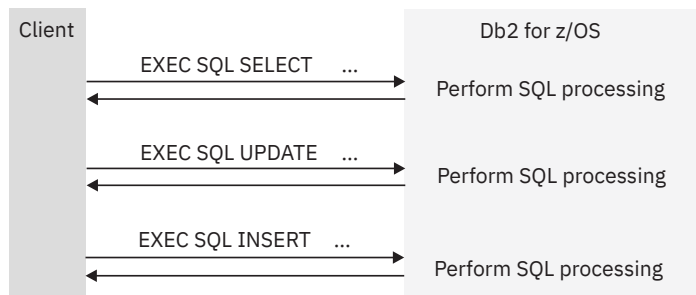


Figure 46. Processing without stored procedures

The following figure illustrates processing with stored procedures.

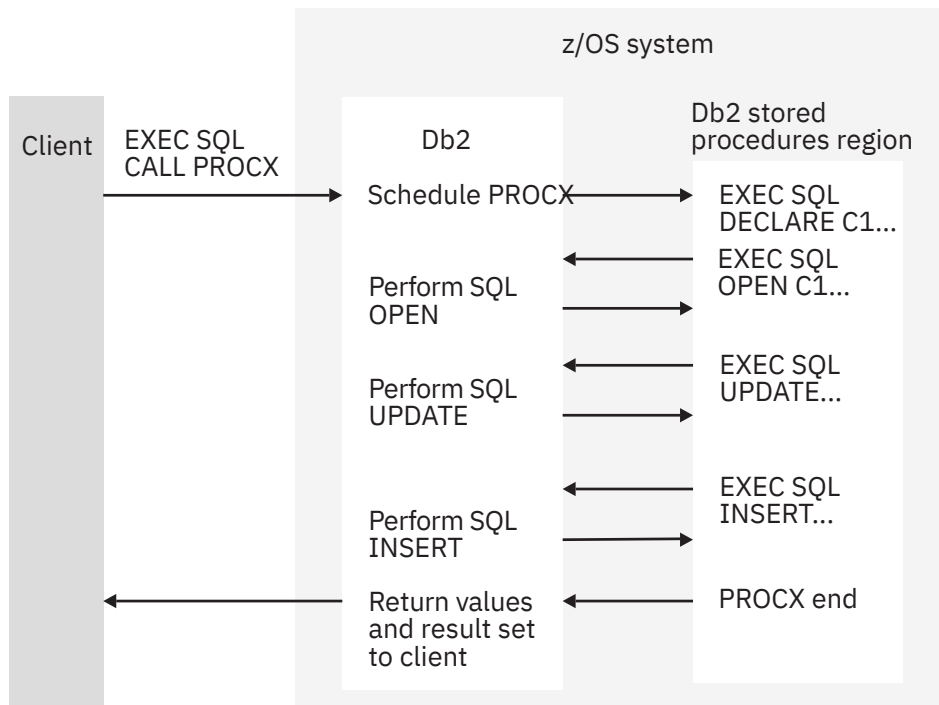


Figure notes:

- The workstation application uses the SQL CONNECT statement to create a conversation with Db2.
- Db2 creates a Db2 thread to process SQL requests. A *thread* is the Db2 structure that describes the connection of an application and traces application progress.
- The SQL statement CALL tells the Db2 server that the application is going to run a stored procedure. The calling application provides the necessary arguments.
- Db2 processes information about the request and loads the stored procedure program.
- The stored procedure executes SQL statements.

One of the SQL statements opens a cursor that has been declared WITH RETURN. This action causes a result set to be returned to the workstation application.

- The stored procedure assigns values to the output parameters and exits. Control returns to the Db2 stored procedures region and goes from there to the Db2 subsystem.
- Control returns to the calling application, which receives the output parameters and the result set.

The application can call other stored procedures, or it can execute additional SQL statements. Db2 receives and processes the COMMIT or ROLLBACK request. The commit or rollback operation covers all SQL operations that the application or the stored procedure executes during the unit of work, unless the procedure is defined with the AUTONOMOUS option. Autonomous procedures execute in a separate unit of work from the calling application.

If the application involves IMS or CICS, similar processing occurs. This processing is based on the IMS or CICS synchronization model, rather than on an SQL COMMIT or ROLLBACK statement.

Figure 47. Processing with stored procedures

Related concepts

[Stored procedures \(Db2 Application programming and SQL\)](#)

Related tasks

[Implementing Db2 stored procedures \(Db2 Administration Guide\)](#)

Chapter 9. Db2 sample tables

Much of the Db2 information refers to or relies on the Db2 sample tables. As a group, the tables include information that describes employees, departments, projects, and activities, and they make up a sample application that exemplifies many of the features of Db2.

GUIP

The sample storage group, databases, table spaces, tables, and views are created when you run the installation sample jobs DSNTEJ1 and DSNTEJ7. Db2 sample objects that include LOBs are created in job DSNTEJ7. All other sample objects are created in job DSNTEJ1. The CREATE INDEX statements for the sample tables are not shown here; they, too, are created by the DSNTEJ1 and DSNTEJ7 sample jobs.

Authorization on all sample objects is given to PUBLIC in order to make the sample programs easier to run. You can review the contents of any table by executing an SQL statement, for example SELECT * FROM DSN8C10.PROJ. For convenience in interpreting the examples, the department and employee tables are listed in full.

GUIP

Related concepts

[Phase 1: Creating and loading sample tables \(Db2 Installation and Migration\)](#)

Activity table (DSN8C10.ACT)

The activity table describes the activities that can be performed during a project.

GUIP

The activity table resides in database DSN8D12A and is created with the following statement:

```
CREATE TABLE DSN8C10.ACT
  (ACTNO    SMALLINT      NOT NULL,
   ACTKWD   CHAR(6)       NOT NULL,
   ACTDESC  VARCHAR(20)   NOT NULL,
   PRIMARY KEY (ACTNO)
)
IN DSN8D12A.DSN8S12P
CCSID EBCDIC;
```

GUIP

Content of the activity table

The following table shows the content of the columns in the activity table.

Table 36. Columns of the activity table

Column	Column name	Description
1	ACTNO	Activity ID (the primary key)
2	ACTKWD	Activity keyword (up to six characters)
3	ACTDESC	Activity description

The activity table has the following indexes.

Table 37. Indexes of the activity table

Name	On column	Type of index
DSN8C10.XACT1	ACTNO	Primary, ascending

Table 37. Indexes of the activity table (continued)

Name	On column	Type of index
DSN8C10.XACT2	ACTKWD	Unique, ascending

Relationship to other tables

The activity table is a parent table of the project activity table, through a foreign key on column ACTNO.

Department table (DSN8C10.DEPT)

The department table describes each department in the enterprise and identifies its manager and the department to which it reports.

GUIP

The department table resides in table space DSN8D12A.DSN8S12D and is created with the following statement:

```
CREATE TABLE DSN8C10.DEPT
  (DEPTNO CHAR(3) NOT NULL,
   DEPTNAME VARCHAR(36) NOT NULL,
   MGRNO CHAR(6) ,
   ADMRDEPT CHAR(3) NOT NULL,
   LOCATION CHAR(16) ,
   PRIMARY KEY (DEPTNO) )
IN DSN8D12A.DSN8S12D
CCSID EBCDIC;
```

Because the department table is self-referencing, and also is part of a cycle of dependencies, its foreign keys must be added later with the following statements:

```
ALTER TABLE DSN8C10.DEPT
  FOREIGN KEY RDD (ADMRDEPT) REFERENCES DSN8C10.DEPT
  ON DELETE CASCADE;

ALTER TABLE DSN8C10.DEPT
  FOREIGN KEY RDE (MGRNO) REFERENCES DSN8C10.EMP
  ON DELETE SET NULL;
```

GUIP

Content of the department table

The following table shows the content of the columns in the department table.

Table 38. Columns of the department table

Column	Column name	Description
1	DEPTNO	Department ID, the primary key.
2	DEPTNAME	A name that describes the general activities of the department.
3	MGRNO	Employee number (EMPNO) of the department manager.
4	ADMRDEPT	ID of the department to which this department reports; the department at the highest level reports to itself.
5	LOCATION	The remote location name.

The following table shows the indexes of the department table.

Table 39. Indexes of the department table

Name	On column	Type of index
DSN8C10.XDEPT1	DEPTNO	Primary, ascending
DSN8C10.XDEPT2	MGRNO	Ascending
DSN8C10.XDEPT3	ADMRDEPT	Ascending

The following table shows the content of the department table.

Table 40. DSN8C10.DEPT: department table

DEPTNO	DEPTNAME	MGRNO	ADMRDEPT	LOCATION
A00	SPIFFY COMPUTER SERVICE DIV.	000010	A00	-----
B01	PLANNING	000020	A00	-----
C01	INFORMATION CENTER	000030	A00	-----
D01	DEVELOPMENT CENTER	-----	A00	-----
E01	SUPPORT SERVICES	000050	A00	-----
D11	MANUFACTURING SYSTEMS	000060	D01	-----
D21	ADMINISTRATION SYSTEMS	000070	D01	-----
E11	OPERATIONS	000090	E01	-----
E21	SOFTWARE SUPPORT	000100	E01	-----
F22	BRANCH OFFICE F2	-----	E01	-----
G22	BRANCH OFFICE G2	-----	E01	-----
H22	BRANCH OFFICE H2	-----	E01	-----
I22	BRANCH OFFICE I2	-----	E01	-----
J22	BRANCH OFFICE J2	-----	E01	-----

The LOCATION column contains null values until sample job DSNTJ6 updates this column with the location name.

Relationship to other tables

The department table is self-referencing: the value of the administering department must be a valid department ID.

The department table is a parent table of the following :

- The employee table, through a foreign key on column WORKDEPT
- The project table, through a foreign key on column DEPTNO

The department table is a dependent of the employee table, through its foreign key on column MGRNO.

Employee table (DSN8C10.EMP)

The sample employee table identifies all employees by an employee number and lists basic personnel information.

GUPI The employee table resides in the partitioned table space DSN8D12A.DSN8S12E. Because this table has a foreign key that references DEPT, that table and the index on its primary key must be created first. Then EMP is created with the following statement:

```
CREATE TABLE DSN8C10.EMP
(EMPNO      CHAR(6)                NOT NULL,
 FIRSTNME   VARCHAR(12)            NOT NULL,
 MIDINIT    CHAR(1)                NOT NULL,
 LASTNAME   VARCHAR(15)            NOT NULL,
 WORKDEPT   CHAR(3)
 PHONENO    CHAR(4)                CONSTRAINT NUMBER CHECK
      (PHONENO >= '0000' AND
       PHONENO <= '9999')
 HIREDATE   DATE
 JOB        CHAR(8)
 EDLEVEL    SMALLINT
 SEX        CHAR(1)
 BIRTHDATE  DATE
 SALARY     DECIMAL(9,2)
 BONUS      DECIMAL(9,2)
 COMM       DECIMAL(9,2)
 PRIMARY KEY (EMPNO)
 FOREIGN KEY RED (WORKDEPT) REFERENCES DSN8C10.DEPT
      ON DELETE SET NULL
)
EDITPROC DSN8EAE1
IN DSN8D12A.DSN8S12E
CCSID EBCDIC;
```

GUPI

Content of the employee table

The following table shows the type of content of each of the columns in the employee table. The table has a check constraint, NUMBER, which checks that the four-digit phone number is in the numeric range 0000 to 9999.

Table 41. Columns of the employee table

Column	Column name	Description
1	EMPNO	Employee number (the primary key)
2	FIRSTNME	First name of employee
3	MIDINIT	Middle initial of employee
4	LASTNAME	Last name of employee
5	WORKDEPT	ID of department in which the employee works
6	PHONENO	Employee telephone number
7	HIREDATE	Date of hire
8	JOB	Job held by the employee
9	EDLEVEL	Number of years of formal education
10	SEX	Sex of the employee (M or F)
11	BIRTHDATE	Date of birth
12	SALARY	Yearly salary in dollars
13	BONUS	Yearly bonus in dollars

Table 41. Columns of the employee table (continued)

Column	Column name	Description
14	COMM	Yearly commission in dollars

The following table shows the indexes of the employee table.

Table 42. Indexes of the employee table

Name	On column	Type of index
DSN8C10.XEMP1	EMPNO	Primary, partitioned, ascending
DSN8C10.XEMP2	WORKDEPT	Ascending

The following table shows the first half (left side) of the content of the employee table. (Table 44 on page 336 shows the remaining content (right side) of the employee table.)

Table 43. Left half of DSN8C10.EMP: employee table. Note that a blank in the MIDINIT column is an actual value of " " rather than null.

EMPNO	FIRSTNME	MIDINIT	LASTNAME	WORKDEPT	PHONENO	HIREDATE
000010	CHRISTINE	I	HAAS	A00	3978	1965-01-01
000020	MICHAEL	L	THOMPSON	B01	3476	1973-10-10
000030	SALLY	A	KWAN	C01	4738	1975-04-05
000050	JOHN	B	GEYER	E01	6789	1949-08-17
000060	IRVING	F	STERN	D11	6423	1973-09-14
000070	EVA	D	PULASKI	D21	7831	1980-09-30
000090	EILEEN	W	HENDERSON	E11	5498	1970-08-15
000100	THEODORE	Q	SPENSER	E21	0972	1980-06-19
000110	VINCENZO	G	LUCCHESSI	A00	3490	1958-05-16
000120	SEAN		O'CONNELL	A00	2167	1963-12-05
000130	DOLORES	M	QUINTANA	C01	4578	1971-07-28
000140	HEATHER	A	NICHOLLS	C01	1793	1976-12-15
000150	BRUCE		ADAMSON	D11	4510	1972-02-12
000160	ELIZABETH	R	PIANKA	D11	3782	1977-10-11
000170	MASATOSHI	J	YOSHIMURA	D11	2890	1978-09-15
000180	MARILYN	S	SCOUTTEN	D11	1682	1973-07-07
000190	JAMES	H	WALKER	D11	2986	1974-07-26
000200	DAVID		BROWN	D11	4501	1966-03-03
000210	WILLIAM	T	JONES	D11	0942	1979-04-11
000220	JENNIFER	K	LUTZ	D11	0672	1968-08-29
000230	JAMES	J	JEFFERSON	D21	2094	1966-11-21
000240	SALVATORE	M	MARINO	D21	3780	1979-12-05
000250	DANIEL	S	SMITH	D21	0961	1969-10-30
000260	SYBIL	P	JOHNSON	D21	8953	1975-09-11

Table 43. Left half of DSN8C10.EMP: employee table. Note that a blank in the MIDINIT column is an actual value of " " rather than null. (continued)

EMPNO	FIRSTNME	MIDINIT	LASTNAME	WORKDEPT	PHONENO	HIREDATE
000270	MARIA	L	PEREZ	D21	9001	1980-09-30
000280	ETHEL	R	SCHNEIDER	E11	8997	1967-03-24
000290	JOHN	R	PARKER	E11	4502	1980-05-30
000300	PHILIP	X	SMITH	E11	2095	1972-06-19
000310	MAUDE	F	SETRIGHT	E11	3332	1964-09-12
000320	RAMLAL	V	MEHTA	E21	9990	1965-07-07
000330	WING		LEE	E21	2103	1976-02-23
000340	JASON	R	GOUNOT	E21	5698	1947-05-05
200010	DIAN	J	HEMMINGER	A00	3978	1965-01-01
200120	GREG		ORLANDO	A00	2167	1972-05-05
200140	KIM	N	NATZ	C01	1793	1976-12-15
200170	KIYOSHI		YAMAMOTO	D11	2890	1978-09-15
200220	REBA	K	JOHN	D11	0672	1968-08-29
200240	ROBERT	M	MONTEVERDE	D21	3780	1979-12-05
200280	EILEEN	R	SCHWARTZ	E11	8997	1967-03-24
200310	MICHELLE	F	SPRINGER	E11	3332	1964-09-12
200330	HELENA		WONG	E21	2103	1976-02-23
200340	ROY	R	ALONZO	E21	5698	1947-05-05

(Table 43 on page 335 shows the first half (right side) of the content of employee table.)

Table 44. Right half of DSN8C10.EMP: employee table

(EMPNO)	JOB	EDLEVEL	SEX	BIRTHDATE	SALARY	BONUS	COMM
(000010)	PRES	18	F	1933-08-14	52750.00	1000.00	4220.00
(000020)	MANAGER	18	M	1948-02-02	41250.00	800.00	3300.00
(000030)	MANAGER	20	F	1941-05-11	38250.00	800.00	3060.00
(000050)	MANAGER	16	M	1925-09-15	40175.00	800.00	3214.00
(000060)	MANAGER	16	M	1945-07-07	32250.00	600.00	2580.00
(000070)	MANAGER	16	F	1953-05-26	36170.00	700.00	2893.00
(000090)	MANAGER	16	F	1941-05-15	29750.00	600.00	2380.00
(000100)	MANAGER	14	M	1956-12-18	26150.00	500.00	2092.00
(000110)	SALESREP	19	M	1929-11-05	46500.00	900.00	3720.00
(000120)	CLERK	14	M	1942-10-18	29250.00	600.00	2340.00
(000130)	ANALYST	16	F	1925-09-15	23800.00	500.00	1904.00
(000140)	ANALYST	18	F	1946-01-19	28420.00	600.00	2274.00
(000150)	DESIGNER	16	M	1947-05-17	25280.00	500.00	2022.00
(000160)	DESIGNER	17	F	1955-04-12	22250.00	400.00	1780.00

Table 44. Right half of DSN8C10.EMP: employee table (continued)

(EMPNO)	JOB	EDLEVEL	SEX	BIRTHDATE	SALARY	BONUS	COMM
(000170)	DESIGNER	16	M	1951-01-05	24680.00	500.00	1974.00
(000180)	DESIGNER	17	F	1949-02-21	21340.00	500.00	1707.00
(000190)	DESIGNER	16	M	1952-06-25	20450.00	400.00	1636.00
(000200)	DESIGNER	16	M	1941-05-29	27740.00	600.00	2217.00
(000210)	DESIGNER	17	M	1953-02-23	18270.00	400.00	1462.00
(000220)	DESIGNER	18	F	1948-03-19	29840.00	600.00	2387.00
(000230)	CLERK	14	M	1935-05-30	22180.00	400.00	1774.00
(000240)	CLERK	17	M	1954-03-31	28760.00	600.00	2301.00
(000250)	CLERK	15	M	1939-11-12	19180.00	400.00	1534.00
(000260)	CLERK	16	F	1936-10-05	17250.00	300.00	1380.00
(000270)	CLERK	15	F	1953-05-26	27380.00	500.00	2190.00
(000280)	OPERATOR	17	F	1936-03-28	26250.00	500.00	2100.00
(000290)	OPERATOR	12	M	1946-07-09	15340.00	300.00	1227.00
(000300)	OPERATOR	14	M	1936-10-27	17750.00	400.00	1420.00
(000310)	OPERATOR	12	F	1931-04-21	15900.00	300.00	1272.00
(000320)	FIELDREP	16	M	1932-08-11	19950.00	400.00	1596.00
(000330)	FIELDREP	14	M	1941-07-18	25370.00	500.00	2030.00
(000340)	FIELDREP	16	M	1926-05-17	23840.00	500.00	1907.00
(200010)	SALESREP	18	F	1933-08-14	46500.00	1000.00	4220.00
(200120)	CLERK	14	M	1942-10-18	29250.00	600.00	2340.00
(200140)	ANALYST	18	F	1946-01-19	28420.00	600.00	2274.00
(200170)	DESIGNER	16	M	1951-01-05	24680.00	500.00	1974.00
(200220)	DESIGNER	18	F	1948-03-19	29840.00	600.00	2387.00
(200240)	CLERK	17	M	1954-03-31	28760.00	600.00	2301.00
(200280)	OPERATOR	17	F	1936-03-28	26250.00	500.00	2100.00
(200310)	OPERATOR	12	F	1931-04-21	15900.00	300.00	1272.00
(200330)	FIELDREP	14	F	1941-07-18	25370.00	500.00	2030.00
(200340)	FIELDREP	16	M	1926-05-17	23840.00	500.00	1907.00

Relationship to other tables

The employee table is a parent table of:

- The department table, through a foreign key on column MGRNO
- The project table, through a foreign key on column RESPEMP

The employee table is a dependent of the department table, through its foreign key on column WORKDEPT.

Employee photo and resume table (DSN8C10.EMP_PHOTO_RESUME)

The sample employee photo and resume table complements the employee table.



Each row of the photo and resume table contains a photo of the employee, in two formats, and the employee's resume. The photo and resume table resides in table space DSN8D12L.DSN8S12B. The following statement creates the table:

```
CREATE TABLE DSN8C10.EMP_PHOTO_RESUME
  (EMPNO      CHAR(06) NOT NULL,
   EMP_ROWID  ROWID NOT NULL GENERATED ALWAYS,
   PSEG_PHOTO BLOB(500K),
   BMP_PHOTO  BLOB(100K),
   RESUME     CLOB(5K),
   PRIMARY KEY (EMPNO))
IN DSN8D12L.DSN8S12B
CCSID EBCDIC;
```

Db2 requires an auxiliary table for each LOB column in a table. The following statements define the auxiliary tables for the three LOB columns in DSN8C10.EMP_PHOTO_RESUME:

```
CREATE AUX TABLE DSN8C10.AUX_BMP_PHOTO
  IN DSN8D12L.DSN8S12M
  STORES DSN8C10.EMP_PHOTO_RESUME
  COLUMN BMP_PHOTO;

CREATE AUX TABLE DSN8C10.AUX_PSEG_PHOTO
  IN DSN8D12L.DSN8S12L
  STORES DSN8C10.EMP_PHOTO_RESUME
  COLUMN PSEG_PHOTO;

CREATE AUX TABLE DSN8C10.AUX_EMP_RESUME
  IN DSN8D12L.DSN8S12N
  STORES DSN8C10.EMP_PHOTO_RESUME
  COLUMN RESUME;
```



Content of the employee photo and resume table

The following table shows the content of the columns in the employee photo and resume table.

Table 45. Columns of the employee photo and resume table

Column	Column name	Description
1	EMPNO	Employee ID (the primary key).
2	EMP_ROWID	Row ID to uniquely identify each row of the table. Db2 supplies the values of this column.
3	PSEG_PHOTO	Employee photo, in PSEG format.
4	BMP_PHOTO	Employee photo, in BMP format.
5	RESUME	Employee resume.

The following table shows the indexes for the employee photo and resume table.

Table 46. Indexes of the employee photo and resume table

Name	On column	Type of index
DSN8C10.XEMP_PHOTO_RESUME	EMPNO	Primary, ascending

The following table shows the indexes for the auxiliary tables that support the employee photo and resume table.

Table 47. Indexes of the auxiliary tables for the employee photo and resume table

Name	On table	Type of index
DSN8C10.XAUX_BMP_PHOTO	DSN8C10.AUX_BMP_PHOTO	Unique
DSN8C10.XAUX_PSEG_PHOTO	DSN8C10.AUX_PSEG_PHOTO	Unique
DSN8C10.XAUX_EMP_RESUME	DSN8C10.AUX_EMP_RESUME	Unique

Relationship to other tables

The employee photo and resume table is a parent table of the project table, through a foreign key on column RESPEMP.

Project table (DSN8C10.PROJ)

The sample project table describes each project that the business is currently undertaking. Data that is contained in each row of the table includes the project number, name, person responsible, and schedule dates.

The project table resides in database DSN8D12A. Because this table has foreign keys that reference DEPT and EMP, those tables and the indexes on their primary keys must be created first. Then PROJ is created with the following statement:

GUI

```
CREATE TABLE DSN8C10.PROJ
  (PROJNO CHAR(6) PRIMARY KEY NOT NULL,
   PROJNAME VARCHAR(24) NOT NULL WITH DEFAULT
   'PROJECT NAME UNDEFINED',
   DEPTNO CHAR(3) NOT NULL REFERENCES
   DSN8C10.DEPT ON DELETE RESTRICT,
   RESPEMP CHAR(6) NOT NULL REFERENCES
   DSN8C10.EMP ON DELETE RESTRICT,
   PRSTAFF DECIMAL(5, 2) ,
   PRSTDATE DATE ,
   PRENDATE DATE ,
   MAJPROJ CHAR(6))
IN DSN8D12A.DSN8S12P
CCSID EBCDIC;
```

Because the project table is self-referencing, the foreign key for that constraint must be added later with the following statement:

```
ALTER TABLE DSN8C10.PROJ
  FOREIGN KEY RPP (MAJPROJ) REFERENCES DSN8C10.PROJ
  ON DELETE CASCADE;
```

GUI

Content of the project table

The following table shows the content of the columns of the project table.

Table 48. Columns of the project table

Column	Column name	Description
1	PROJNO	Project ID (the primary key)
2	PROJNAME	Project name

Table 48. Columns of the project table (continued)

Column	Column name	Description
3	DEPTNO	ID of department responsible for the project
4	RESPEMP	ID of employee responsible for the project
5	PRSTAFF	Estimated mean number of persons that are needed between PRSTDATE and PRENDATE to complete the whole project, including any subprojects
6	PRSTDATE	Estimated project start date
7	PRENDATE	Estimated project end date
8	MAJPROJ	ID of any project of which this project is a part

The following table shows the indexes for the project table:

Table 49. Indexes of the project table

Name	On column	Type of index
DSN8C10.XPROJ1	PROJNO	Primary, ascending
DSN8C10.XPROJ2	RESPEMP	Ascending

Relationship to other tables

The table is self-referencing: a non-null value of MAJPROJ must be a valid project number. The table is a parent table of the project activity table, through a foreign key on column PROJNO. It is a dependent of the following tables:

- The department table, through its foreign key on DEPTNO
- The employee table, through its foreign key on RESPEMP

Project activity table (DSN8C10.PROJACT)

The sample project activity table lists the activities that are performed for each project.

The project activity table resides in database DSN8D12A. Because this table has foreign keys that reference PROJ and ACT, those tables and the indexes on their primary keys must be created first. Then PROJACT is created with the following statement:

GUI

```
CREATE TABLE DSN8C10.PROJACT
  (PROJNO    CHAR(6)                NOT NULL,
   ACTNO     SMALLINT               NOT NULL,
   ACSTAFF   DECIMAL(5,2)           ,
   ACSTDATE  DATE                  NOT NULL,
   ACENDATE  DATE                  ,
   PRIMARY KEY (PROJNO, ACTNO, ACSTDATE),
   FOREIGN KEY RPAP (PROJNO) REFERENCES DSN8C10.PROJ
                                     ON DELETE RESTRICT,
   FOREIGN KEY RPAA (ACTNO) REFERENCES DSN8C10.ACT
                                     ON DELETE RESTRICT)
IN DSN8D12A.DSN8S12P
CCSID EBCDIC;
```

GUI

Content of the project activity table

The following table shows the content of the columns of the project activity table.

Table 50. Columns of the project activity table

Column	Column name	Description
1	PROJNO	Project ID
2	ACTNO	Activity ID
3	ACSTAFF	Estimated mean number of employees that are needed to staff the activity
4	ACSTDATE	Estimated activity start date
5	ACENDATE	Estimated activity completion date

The following table shows the index of the project activity table:

Table 51. Index of the project activity table

Name	On columns	Type of index
DSN8C10.XPROJAC1	PROJNO, ACTNO, ACSTDATE	primary, ascending

Relationship to other tables

The project activity table is a parent table of the employee to project activity table, through a foreign key on columns PROJNO, ACTNO, and EMSTDATE. It is a dependent of the following tables:

- The activity table, through its foreign key on column ACTNO
- The project table, through its foreign key on column PROJNO

Related reference

Activity table (DSN8C10.ACT)

The activity table describes the activities that can be performed during a project.

Project table (DSN8C10.PROJ)

The sample project table describes each project that the business is currently undertaking. Data that is contained in each row of the table includes the project number, name, person responsible, and schedule dates.

Employee-to-project activity table (DSN8C10.EMPPROJACT)

The sample employee-to-project activity table identifies the employee who performs an activity for a project, tells the proportion of the employee's time that is required, and gives a schedule for the activity.

GUI

The employee-to-project activity table resides in database DSN8D12A. Because this table has foreign keys that reference EMP and PROJACT, those tables and the indexes on their primary keys must be created first. Then EMPPROJACT is created with the following statement:

```
CREATE TABLE DSN8C10.EMPPROJACT
(EMPNO      CHAR(6)                NOT NULL,
 PROJNO     CHAR(6)                NOT NULL,
 ACTNO      SMALLINT               NOT NULL,
 EMPTIME    DECIMAL(5,2)           ,
 EMSTDATE   DATE                   ,
 EMENDATE   DATE                   ,
 FOREIGN KEY REPAPA (PROJNO, ACTNO, EMSTDATE)
 REFERENCES DSN8C10.PROJACT
 ON DELETE RESTRICT,
 FOREIGN KEY REPAE (EMPNO) REFERENCES DSN8C10.EMP
 ON DELETE RESTRICT)
IN DSN8D12A.DSN8S12P
CCSID EBCDIC;
```

Content of the employee-to-project activity table

The following table shows the content of the columns in the employee-to-project activity table.

Table 52. Columns of the employee-to-project activity table

Column	Column name	Description
1	EMPNO	Employee ID number
2	PROJNO	Project ID of the project
3	ACTNO	ID of the activity within the project
4	EMPTIME	A proportion of the employee's full time (between 0.00 and 1.00) that is to be spent on the activity
5	EMSTDATE	Date the activity starts
6	EMENDATE	Date the activity ends

The following table shows the indexes for the employee-to-project activity table:

Table 53. Indexes of the employee-to-project activity table

Name	On columns	Type of index
DSN8C10.XEMPPROJACT1	PROJNO, ACTNO, EMSTDATE, EMPNO	Unique, ascending
DSN8C10.XEMPPROJACT2	EMPNO	Ascending

Relationship to other tables

The employee-to-project activity table is a dependent of the following tables:

- The employee table, through its foreign key on column EMPNO
- The project activity table, through its foreign key on columns PROJNO, ACTNO, and EMSTDATE.

Related reference

[Employee table \(DSN8C10.EMP\)](#)

The sample employee table identifies all employees by an employee number and lists basic personnel information.

[Project activity table \(DSN8C10.PROJACT\)](#)

The sample project activity table lists the activities that are performed for each project.

Unicode sample table (DSN8C10.DEMO_UNICODE)

The Unicode sample table is used to verify that data conversions to and from EBCDIC and Unicode are working as expected.

The table resides in database DSN8D12A, and is defined with the following statement:

```
CREATE TABLE DSN8C10.DEMO_UNICODE
  (LOWER_A_TO_Z      CHAR(26)
   UPPER_A_TO_Z      CHAR(26)
   ZERO_TO_NINE      CHAR(10)
   X00_TO_XFF        VARCHAR(256)    FOR BIT DATA)
  IN DSN8D81E.DSN8S81U
  CCSID UNICODE;
```

Content of the Unicode sample table

The following table shows the content of the columns in the Unicode sample table:

Table 54. Columns of the Unicode sample table

Column	Column Name	Description
1	LOWER_A_TO_Z	Array of characters, 'a' to 'z'
2	UPPER_A_TO_Z	Array of characters, 'A' to 'Z'
3	ZERO_TO_NINE	Array of characters, '0' to '9'
4	X00_TO_XFF	Array of characters, x'00' to x'FF'

This table has no indexes.

Relationship to other tables

This table has no relationship to other tables.

Relationships among the sample tables

Relationships among the sample tables are established by foreign keys in dependent tables that reference primary keys in parent tables.

The following figure shows relationships among the sample tables. You can find descriptions of the columns with the descriptions of the tables.

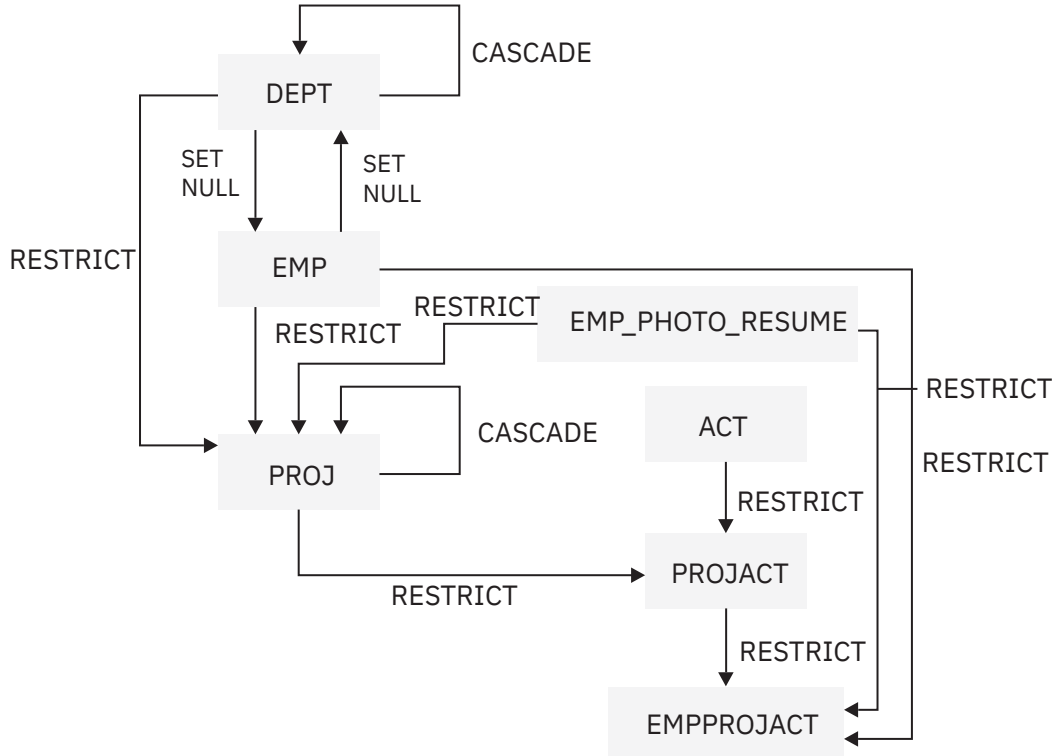


Figure 65. Relationships among tables in the sample application

Related reference

[Activity table \(DSN8C10.ACT\)](#)

The activity table describes the activities that can be performed during a project.

Department table (DSN8C10.DEPT)

The department table describes each department in the enterprise and identifies its manager and the department to which it reports.

Employee photo and resume table (DSN8C10.EMP_PHOTO_RESUME)

The sample employee photo and resume table complements the employee table.

Employee table (DSN8C10.EMP)

The sample employee table identifies all employees by an employee number and lists basic personnel information.

Employee-to-project activity table (DSN8C10.EMPPROJECT)

The sample employee-to-project activity table identifies the employee who performs an activity for a project, tells the proportion of the employee's time that is required, and gives a schedule for the activity.

Project activity table (DSN8C10.PROJECT)

The sample project activity table lists the activities that are performed for each project.

Project table (DSN8C10.PROJ)

The sample project table describes each project that the business is currently undertaking. Data that is contained in each row of the table includes the project number, name, person responsible, and schedule dates.

Unicode sample table (DSN8C10.DEMO_UNICODE)

The Unicode sample table is used to verify that data conversions to and from EBCDIC and Unicode are working as expected.

Views on the sample tables

Db2 creates a number of views on the sample tables for use in the sample applications.

The following table indicates the tables on which each view is defined and the sample applications that use the view. All view names have the qualifier DSN8C10.

Table 55. Views on sample tables

View name	On tables or views	Used in application
VDEPT	DEPT	Organization Project
VHDEPT	DEPT	Distributed organization
VEMP	EMP	Distributed organization Organization Project
VPROJ	PROJ	Project
VACT	ACT	Project
VPROJECT	PROJECT	Project
VEMPPROJECT	EMPPROJECT	Project
VDEPMG1	DEPT EMP	Organization
VEMPDPT1	DEPT EMP	Organization

Table 55. Views on sample tables (continued)

View name	On tables or views	Used in application
VASTRDE1	DEPT	
VASTRDE2	VDEPMG1 EMP	Organization
VPROJRE1	PROJ EMP	Project
VPSTRDE1	VPROJRE1 VPROJRE2	Project
VPSTRDE2	VPROJRE1	Project
VFORPLA	VPROJRE1 EMPPROJACT	Project
VSTAFAC1	PROJACT ACT	Project
VSTAFAC2	EMPPROJACT ACT EMP	Project
VPHONE	EMP DEPT	Phone
VEMPLP	EMP	Phone

GUI

The following SQL statement creates the view named VDEPT.

```
CREATE VIEW DSN8C10.VDEPT
  AS SELECT ALL      DEPTNO ,
                     DEPTNAME,
                     MGRNO ,
                     ADMRDEPT
  FROM DSN8C10.DEPT;
```

The following SQL statement creates the view named VHDEPT.

```
CREATE VIEW DSN8C10.VHDEPT
  AS SELECT ALL      DEPTNO ,
                     DEPTNAME,
                     MGRNO ,
                     ADMRDEPT,
                     LOCATION
  FROM DSN8C10.DEPT;
```

The following SQL statement creates the view named VEMP.

```
CREATE VIEW DSN8C10.VEMP
  AS SELECT ALL      EMPNO ,
                     FIRSTNME,
                     MIDINIT ,
                     LASTNAME,
```

```

        WORKDEPT
FROM DSN8C10.EMP;

```

The following SQL statement creates the view named VPROJ.

```

CREATE VIEW DSN8C10.VPROJ
AS SELECT ALL
        PROJNO, PROJNAME, DEPTNO, RESPEMP, PRSTAFF,
        PRSTDATE, PRENDATE, MAJPROJ
FROM DSN8C10.PROJ ;

```

The following SQL statement creates the view named VACT.

```

CREATE VIEW DSN8C10.VACT
AS SELECT ALL      ACTNO      ,
                   ACTKWD     ,
                   ACTDESC    ,
FROM DSN8C10.ACT ;

```

The following SQL statement creates the view named VPROJACT.

```

CREATE VIEW DSN8C10.VPROJACT
AS SELECT ALL
        PROJNO,ACTNO, ACSTAFF, ACSTDATE, ACENDATE
FROM DSN8C10.PROJACT ;

```

The following SQL statement creates the view named VEMPPROJACT.

```

CREATE VIEW DSN8C10.VEMPPROJACT
AS SELECT ALL
        EMPNO, PROJNO, ACTNO, EMPTIME, EMSTDATE, EMENDATE
FROM DSN8C10.EMPPROJACT ;

```

The following SQL statement creates the view named VDEPMG1.

```

CREATE VIEW DSN8C10.VDEPMG1
(DEPTNO, DEPTNAME, MGRNO, FIRSTNME, MIDINIT,
LASTNAME, ADMRDEPT)
AS SELECT ALL
        DEPTNO, DEPTNAME, EMPNO, FIRSTNME, MIDINIT,
        LASTNAME, ADMRDEPT
FROM DSN8C10.DEPT LEFT OUTER JOIN DSN8C10.EMP
ON MGRNO = EMPNO ;

```

The following SQL statement creates the view named VEMPDPT1.

```

CREATE VIEW DSN8C10.VEMPDPT1
(DEPTNO, DEPTNAME, EMPNO, FRSTINIT, MIDINIT,
LASTNAME, WORKDEPT)
AS SELECT ALL
        DEPTNO, DEPTNAME, EMPNO, SUBSTR(FIRSTNME, 1, 1), MIDINIT,
        LASTNAME, WORKDEPT
FROM DSN8C10.DEPT RIGHT OUTER JOIN DSN8C10.EMP
ON WORKDEPT = DEPTNO ;

```

The following SQL statement creates the view named VASTRDE1.

```

CREATE VIEW DSN8C10.VASTRDE1
(DEPT1NO,DEPT1NAM,EMP1NO,EMP1FN,EMP1MI,EMP1LN,TYPE2,
DEPT2NO,DEPT2NAM,EMP2NO,EMP2FN,EMP2MI,EMP2LN)
AS SELECT ALL
        D1.DEPTNO,D1.DEPTNAME,D1.MGRNO,D1.FIRSTNME,D1.MIDINIT,
        D1.LASTNAME, '1',
        D2.DEPTNO,D2.DEPTNAME,D2.MGRNO,D2.FIRSTNME,D2.MIDINIT,
        D2.LASTNAME
FROM DSN8C10.VDEPMG1 D1, DSN8C10.VDEPMG1 D2
WHERE D1.DEPTNO = D2.ADMRDEPT ;

```

The following SQL statement creates the view named VASTRDE2.

```

CREATE VIEW DSN8C10.VASTRDE2
(DEPT1NO,DEPT1NAM,EMP1NO,EMP1FN,EMP1MI,EMP1LN,TYPE2,

```



```

DEPT2NO,DEPT2NAM,EMP2NO,EMP2FN,EMP2MI,EMP2LN)
AS SELECT ALL
    D1.DEPTNO,D1.DEPTNAME,D1.MGRNO,D1.FIRSTNME,D1.MIDINIT,
    D1.LASTNAME,'2',
    D1.DEPTNO,D1.DEPTNAME,E2.EMPNO,E2.FIRSTNME,E2.MIDINIT,
    E2.LASTNAME
FROM DSN8C10.VDEPMG1 D1, DSN8C10.EMP E2
WHERE D1.DEPTNO = E2.WORKDEPT;

```

The following figure shows the SQL statement that creates the view named VPROJRE1.

```

CREATE VIEW DSN8C10.VPROJRE1
(PROJNO,PROJNAME,PROJDEP,RESPEMP,FIRSTNME,MIDINIT,
LASTNAME,MAJPROJ)
AS SELECT ALL
    PROJNO,PROJNAME,DEPTNO,EMPNO,FIRSTNME,MIDINIT,
    LASTNAME,MAJPROJ
FROM DSN8C10.PROJ, DSN8C10.EMP
WHERE RESPEMP = EMPNO ;

```

Figure 66. VPROJRE1

The following SQL statement creates the view named VPSTRDE1.

```

CREATE VIEW DSN8C10.VPSTRDE1
(PROJ1NO,PROJ1NAME,RESP1NO,RESP1FN,RESP1MI,RESP1LN,
PROJ2NO,PROJ2NAME,RESP2NO,RESP2FN,RESP2MI,RESP2LN)
AS SELECT ALL
    P1.PROJNO,P1.PROJNAME,P1.RESPEMP,P1.FIRSTNME,P1.MIDINIT,
    P1.LASTNAME,
    P2.PROJNO,P2.PROJNAME,P2.RESPEMP,P2.FIRSTNME,P2.MIDINIT,
    P2.LASTNAME
FROM DSN8C10.VPROJRE1 P1,
    DSN8C10.VPROJRE1 P2
WHERE P1.PROJNO = P2.MAJPROJ ;

```

The following SQL statement creates the view named VPSTRDE2.

```

CREATE VIEW DSN8C10.VPSTRDE2
(PROJ1NO,PROJ1NAME,RESP1NO,RESP1FN,RESP1MI,RESP1LN,
PROJ2NO,PROJ2NAME,RESP2NO,RESP2FN,RESP2MI,RESP2LN)
AS SELECT ALL
    P1.PROJNO,P1.PROJNAME,P1.RESPEMP,P1.FIRSTNME,P1.MIDINIT,
    P1.LASTNAME,
    P1.PROJNO,P1.PROJNAME,P1.RESPEMP,P1.FIRSTNME,P1.MIDINIT,
    P1.LASTNAME
FROM DSN8C10.VPROJRE1 P1
WHERE NOT EXISTS
    (SELECT * FROM DSN8C10.VPROJRE1 P2
    WHERE P1.PROJNO = P2.MAJPROJ) ;

```

The following SQL statement creates the view named VFORPLA.

```

CREATE VIEW DSN8C10.VFORPLA
(PROJNO,PROJNAME,RESPEMP,PROJDEP,FRSTINIT,MIDINIT,LASTNAME)
AS SELECT ALL
    F1.PROJNO,PROJNAME,RESPEMP,PROJDEP, SUBSTR(FIRSTNME, 1, 1),
    MIDINIT, LASTNAME
FROM DSN8C10.VPROJRE1 F1 LEFT OUTER JOIN DSN8C10.EMPPROJACT F2
ON F1.PROJNO = F2.PROJNO;

```

The following SQL statement creates the view named VSTAFAC1.

```

CREATE VIEW DSN8C10.VSTAFAC1
(PROJNO, ACTNO, ACTDESC, EMPNO, FIRSTNME, MIDINIT, LASTNAME,
EMPTIME,STDATE,ENDATE, TYPE)
AS SELECT ALL
    PA.PROJNO, PA.ACTNO, AC.ACTDESC,' ',' ',' ',' ',' ',
    PA.ACSTAFF, PA.ACSTDATE,
    PA.ACENDATE,'1'
FROM DSN8C10.PROJACT PA, DSN8C10.ACT AC
WHERE PA.ACTNO = AC.ACTNO ;

```

The following SQL statement creates the view named VSTAFAC2.

```
CREATE VIEW DSN8C10.VSTAFAC2
  (PROJNO, ACTNO, ACTDESC, EMPNO, FIRSTNME, MIDINIT, LASTNAME,
   EMPTIME, STDATE, ENDATE, TYPE)
AS SELECT ALL
   EP.PROJNO, EP.ACTNO, AC.ACTDESC, EP.EMPNO, EM.FIRSTNME,
   EM.MIDINIT, EM.LASTNAME, EP.EMPTIME, EP.EMSTDATE,
   EP.EMENDATE, '2'
FROM DSN8C10.EMPPROJACT EP, DSN8C10.ACT AC, DSN8C10.EMP EM
WHERE EP.ACTNO = AC.ACTNO AND EP.EMPNO = EM.EMPNO ;
```

The following SQL statement creates the view named VPHONE.

```
CREATE VIEW DSN8C10.VPHONE
  (LASTNAME,
   FIRSTNAME,
   MIDDLEINITIAL,
   PHONENUMBER,
   EMPLOYEENUMBER,
   DEPTNUMBER,
   DEPTNAME)
AS SELECT ALL
   LASTNAME,
   FIRSTNME,
   MIDINIT,
   VALUE(PHONENO, ' ', ' '),
   EMPNO,
   DEPTNO,
   DEPTNAME
FROM DSN8C10.EMP, DSN8C10.DEPT
WHERE WORKDEPT = DEPTNO;
```

The following SQL statement creates the view named VEMPLP.

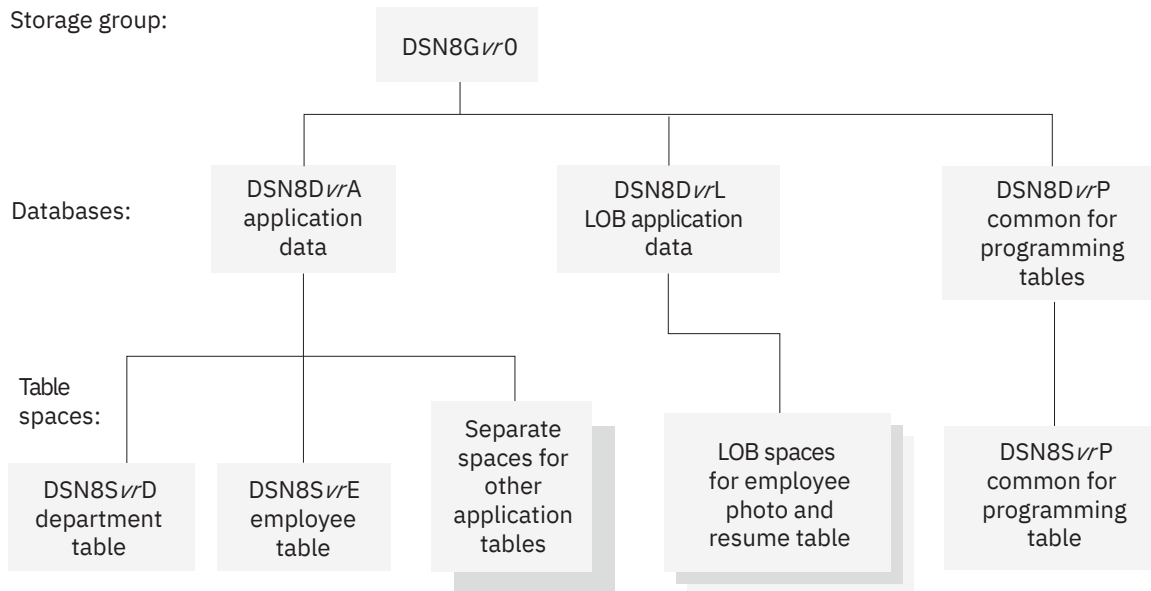
```
CREATE VIEW DSN8C10.VEMPLP
  (EMPLOYEENUMBER,
   PHONENUMBER)
AS SELECT ALL
   EMPNO,
   PHONENO
FROM DSN8C10.EMP ;
```



Storage of sample application tables

Normally, related data is stored in the same database.

The following figure shows how the sample tables are related to databases and storage groups. Two databases are used to illustrate the possibility.



vr is a 2-digit version identifier.

Figure 67. Relationship among sample databases and table spaces

In addition to the storage group and databases that are shown in the preceding figure, the storage group DSN8G12U and database DSN8D12U are created when you run DSNTSJ2A.

Storage group for sample application data

Sample application data is stored in storage group DSN8G120. The default storage group, SYSDEFLT, which is created when Db2 is installed, is not used to store sample application data.

GUI

The storage group that is used to store sample application data is defined by the following statement:

```
CREATE STOGROUP DSN8G120
  VOLUMES (DSNV01)
  VCAT DSNCL11;
```

GUI

Databases for sample application data

Sample application data is stored in several different databases. The default database that is created when Db2 is installed is not used to store the sample application data.

GUI DSN8D12P is the database that is used for tables that are related to programs. The other databases are used for tables that are related to applications. The databases are defined by the following statements:

```
CREATE DATABASE DSN8D12A
  STOGROUP DSN8G120
  BUFFERPOOL BP0
  CCSID EBCDIC;

CREATE DATABASE DSN8D12P
  STOGROUP DSN8G120
  BUFFERPOOL BP0
  CCSID EBCDIC;

CREATE DATABASE DSN8D12L
  STOGROUP DSN8G120
  BUFFERPOOL BP0
  CCSID EBCDIC;
```

```

CREATE DATABASE DSN8D12E
STOGROUP DSN8G120
BUFFERPOOL BP0
CCSID UNICODE;

CREATE DATABASE DSN8D12U
STOGROUP DSN8G12U
CCSID EBCDIC;

```



Table spaces for sample application data

The table spaces that are not explicitly defined are created implicitly in the DSN8D12A database, using the default space attributes.



The following SQL statements explicitly define a series of table spaces.

```

CREATE TABLESPACE DSN8S12D
IN DSN8D12A
USING STOGROUP DSN8G120
PRIQTY 20
SECQTY 20
ERASE NO
LOCKSIZE PAGE LOCKMAX SYSTEM
BUFFERPOOL BP0
CLOSE NO
CCSID EBCDIC;

CREATE TABLESPACE DSN8S12E
IN DSN8D12A
USING STOGROUP DSN8G120
PRIQTY 20
SECQTY 20
ERASE NO
NUMPARTS 4
(PART 1 USING STOGROUP DSN8G120
PRIQTY 12
SECQTY 12,
PART 3 USING STOGROUP DSN8G120
PRIQTY 12
SECQTY 12)
LOCKSIZE PAGE LOCKMAX SYSTEM
BUFFERPOOL BP0
CLOSE NO
COMPRESS YES
CCSID EBCDIC;

```

```

CREATE TABLESPACE DSN8S12B
IN DSN8D12L
USING STOGROUP DSN8G120
PRIQTY 20
SECQTY 20
ERASE NO
LOCKSIZE PAGE
LOCKMAX SYSTEM
BUFFERPOOL BP0
CLOSE NO
CCSID EBCDIC;

```

```

CREATE LOB TABLESPACE DSN8S12M
IN DSN8D12L
LOG NO;

CREATE LOB TABLESPACE DSN8S12L
IN DSN8D12L
LOG NO;

CREATE LOB TABLESPACE DSN8S12N
IN DSN8D12L
LOG NO;

```

```
CREATE TABLESPACE DSN8S12C
  IN DSN8D12P
  USING STOGROUP DSN8G120
    PRIQTY 160
    SECQTY 80
  SEGSIZE 4
  LOCKSIZE TABLE
  BUFFERPOOL BP0
  CLOSE NO
  CCSID EBCDIC;
```

```
CREATE TABLESPACE DSN8S12P
  IN DSN8D12A
  USING STOGROUP DSN8G120
    PRIQTY 160
    SECQTY 80
  SEGSIZE 4
  LOCKSIZE ROW
  BUFFERPOOL BP0
  CLOSE NO
  CCSID EBCDIC;
```

```
CREATE TABLESPACE DSN8S12R
  IN DSN8D12A
  USING STOGROUP DSN8G120
    PRIQTY 20
    SECQTY 20
  ERASE NO
  LOCKSIZE PAGE LOCKMAX SYSTEM
  BUFFERPOOL BP0
  CLOSE NO
  CCSID EBCDIC;
```

```
CREATE TABLESPACE DSN8S12S
  IN DSN8D12A
  USING STOGROUP DSN8G120
    PRIQTY 20
    SECQTY 20
  ERASE NO
  LOCKSIZE PAGE LOCKMAX SYSTEM
  BUFFERPOOL BP0
  CLOSE NO
  CCSID EBCDIC;
```

```
CREATE TABLESPACE DSN8S81Q
  IN DSN8D81P
  USING STOGROUP DSN8G810
    PRIQTY 160
    SECQTY 80
  SEGSIZE 4
  LOCKSIZE PAGE
  BUFFERPOOL BP0
  CLOSE NO
  CCSID EBCDIC;
```

```
CREATE TABLESPACE DSN8S81U
  IN DSN8D81E
  USING STOGROUP DSN8G810
    PRIQTY 5
    SECQTY 5
  ERASE NO
  LOCKSIZE PAGE LOCKMAX SYSTEM
  BUFFERPOOL BP0
  CLOSE NO
  CCSID UNICODE;
```



SYSDUMMYx tables

Db2 for z/OS provides a set of SYSDUMMYx catalog tables.



The last character of the table name identifies the SYSDUMMYx table that is associated with each encoding scheme as follows:

- [SYSIBM.SYSDUMMY1](#) uses the EBCDIC encoding scheme.
- [SYSIBM.SYSDUMMYE](#) uses the EBCDIC encoding scheme.
- [SYSIBM.SYSDUMMYA](#) uses the ASCII encoding scheme.
- [SYSIBM.SYSDUMMYU](#) uses the UNICODE encoding scheme.

Although the SYSDUMMYx tables are implemented as catalog tables, they are similar to sample tables, and are used in some examples in the Db2 for z/OS documentation.

You can use any of the SYSDUMMYx tables when you need to write a query, but no data from a table is referenced. In any query, you must specify a table reference in the FROM clause, but if the query does not reference any data from the table, it does not matter which table is referenced in the FROM clause. Each of the SYSDUMMYx tables contains one row, so a SYSDUMMYx table can be referenced in a SELECT INTO statement without the need for a predicate to limit the result to a single row of data.

For example, when you want to retrieve the value of a special register or global variable, you can use a query that references a SYSDUMMYx table.

```
SELECT CURRENT PATH -- Retrieve the value of a special register
INTO :myvar
FROM SYSIBM.SYSDUMMY1;
SELECT TEMPORAL_LOGICAL_TRANSACTION_TIME -- Retrieve the value of a special register
INTO :myvar
FROM SYSIBM.SYSDUMMY1;
```

Sometimes when Db2 for z/OS processes an SQL statement, the statement is rewritten, and a reference to a SYSDUMMYx table is added. For example, some of the internal rewrites that result in adding a reference to a SYSDUMMYx table are for processing the search condition of a trigger, or the SQL PL control statements IF, REPEAT, RETURN, or WHILE. In these situations, the privilege set must include the SELECT privilege on the SYSDUMMYx table that is referenced.



Related concepts

[Objects with different CCSIDs in the same SQL statement \(Db2 Internationalization Guide\)](#)

Related tasks

[Avoiding character conversion for LOB locators \(Db2 Application programming and SQL\)](#)